Jet Ranger Arduino and C# Program Summary

Master document for the Bell 206 Jet Ranger cockpit: Arduino sketches, Windows programs and PCBs

Regenerated 7 November 2026

Contents

1 Project overview....8
1.1 How it fits together....8
1.2 System block diagrams....9
1.3 Arduino sketches....12
1.4 Windows programs....13
1.5 PCBs....13
1.6 Where things stand....15
2 Colour codes and cabling....16
2.1 Reading the diagrams....16
2.2 Colours in the programs....16
2.3 Cabling colour codes....17
2.3.1 Round power connectors....17
2.3.2 PCB connectors....17
2.4 OLED connectors....17
3 Network and communication....18
3.1 Devices and addresses....18
3.2 UDP ports....18
3.3 Packet format....19
3.4 Board life cycle....20
3.5 Power rules in the bridges....21
3.6 Backlight....22
3.6.1 Start-up hold....22
3.7 Remote reset....23
3.8 Debug and monitoring....24
4 Radio controller (172.16.1.101)....25
4.1 Program flow....25
4.2 Pin usage....26
4.3 Data from the PC (UDP 13136)....27
4.4 Commands to the PC (UDP 27001)....27
4.5 Auto Pilot and Audio panel switches and knobs....27
4.6 Auto Pilot and Audio panel LEDs (MAX7219)....29
4.7 Huey Warning Panel switches and LEDs....30
4.8 Auto Pilot display....30
4.9 When the USB HID board is missing....31
4.10 DME display....31
4.11 ADF controller display....31
4.12 Backlight....32
4.13 Start-up hold....32
4.14 Tuning steps....32
4.15 Power and blanking....32
4.16 Diagnostics....32

4.17 PCB features the sketch does not use yet....33
4.18 Known issues....33
5 Servo controller (172.16.1.102)....34
5.1 Program flow....34
5.2 Servos and calibration....35
5.3 Warning lamps....36
5.4 Debounced switch inputs....36
5.5 On-demand commands....37
5.6 Backlight....37
5.7 Pin usage....37
5.8 Known issues....37
6 Upper panel (now on the ADF controller)....38
7 USB HID board (172.16.1.11)....39
7.1 Program flow....39
7.2 How a switch becomes a joystick button....40
7.3 Auto Pilot, Audio and ADF panel buttons....40
7.4 Switch mapping in use....40
7.5 Pin usage and network....41
7.6 Known issues....41
8 Stepper controller (172.16.1.105)....42
8.1 Gauges and calibration....42
8.2 Program flow....43
8.3 Packets....43
8.4 Start-up announcement....43
8.5 Backlight....43
8.6 On-demand swings....44
8.7 Pin usage....44
8.8 Network....44
8.9 Known issues....44
9 Dual stepper controller (172.16.1.106)....45
9.1 Gauges and calibration....45
9.2 OLED displays....46
9.3 Start-up hold....46
9.4 Backlight....47
9.5 Program flow....47
9.6 Packets....47
9.7 On-demand commands....47
9.8 Pin usage....47
9.9 Network....48
9.10 Known issues....48
10 ADF and Upper controller (172.16.1.107)....49
10.1 Program flow....50
10.2 Pin usage....50

10.3 ADF keypad....51
10.4 Upper panel matrix and keypad....51
10.5 Panel brightness (pot switched off)....52
10.6 Start-up hold....52
10.7 Panel blanking and backlight....52
10.8 Power monitoring....53
10.9 What it sends....53
10.10 Panel switches....53
10.11 Landing gear lamps....54
10.12 What it receives....54
10.13 OLED....54
10.14 Notes....55
11 FSUIPC bridge (FSUIPCWinformsAutoCS)....56
11.1 Program flow....56
11.2 FSUIPC offsets....57
11.3 Packets sent....57
11.4 Aircraft information for the GPS interface....58
11.5 Commands from the Radio controller (UDP 27001)....58
11.6 Notes and quirks....59
11.7 Panel emulators....59
11.8 Packet monitor....59
12 MSFS bridge (MSFSWinformsAutoCS)....60
12.1 Files....60
12.2 Program flow....61
12.3 Packets....61
12.4 Aircraft information for the GPS interface....62
12.5 SimVars requested....62
12.6 Autopilot and audio panel LEDs....62
12.7 Auto Pilot display values....62
12.8 Commands from the panel (UDP 27001)....63
12.9 Aircraft profiles....64
12.9.1 FlyInside B206....64
12.9.2 CowaSim Jet Ranger....65
12.10 Warning lamps....65
12.11 Known gaps....66
12.12 Panel emulators....66
12.13 Packet monitor....66
13 DCS bridge (DCSBiosBridge)....67
13.1 DCS-BIOS....67
13.2 How it works....67
13.3 Window and command line....68
13.4 The Huey profile....68
13.5 Gauges....69

13.6 Testing without DCS....69
13.7 Known gaps....69
13.8 Packet monitor....70
14 X-Plane bridge (XPlaneBridge)....71
14.1 How it works....71
14.2 The common values....71
14.3 Aircraft profiles....72
14.4 Window and command line....72
14.5 Testing without X-Plane....73
14.6 Known gaps....73
14.7 Packet monitor....73
15 Health Monitor (JetRangerHealthMonitor)....74
15.1 Program flow....74
16 Radio and Upper controller emulators....76
16.1 JetRangerRadioControllerEmulator....76
16.2 JetRangerUpperControllerEmulator....76
17 StepperVSITester....77
17.1 What it sends....77
18 SwingTester....78
18.1 How it works....78
18.2 Window....78
18.3 COM ports window....79
18.4 Brightness....79
18.5 Warning lamps....79
18.6 Keep-alive....80
19 Debug listener (UDP reflector)....81
20 Lowrance AirMap GPS interface (LowranceAirMapGps)....82
20.1 Behaviour....82
20.2 NMEA sentences....83
20.3 GPS date and time....83
20.4 Senders....83
20.5 Files....84
20.6 Status....84
21 GPS bench tester (LowranceAirMapGpsBenchTester)....85
21.1 Window....85
21.2 Testing....85
22 Start-up script (Start Jet Ranger.ps1)....86
22.1 What it does....86
22.2 Options....86
22.3 Settings file....87
22.4 Per-simulator start-up....87
22.5 Tested....87
23 Panel emulators (JetRangerPanelEmulator)....88

23.1 How the bridges reach the emulators....88
23.2 What each window does....89
23.3 Running it....93
23.4 Tests....93
23.5 Not done yet....94
24 PCB notes....95
24.1 Boards in use....95
24.2 Backlight driver....97
24.3 Warning Mute, Fuel Valve and TOT Clear inputs....98
24.4 Pin and copper check....98
24.5 KiCad: changing the revision number....98
Appendix A: UDP token reference....99
A.1 Gauge tokens....99
A.2 Display and status tokens....99
A.3 Lamp tokens (Servo .102)....100
A.4 Control tokens (C,<COMMAND>:<value>)....100
A.5 Messages from the ADF controller to the PC (UDP 27003)....100
A.6 Aircraft information to the GPS interface (UDP 27002)....101
A.7 Messages from the Radio controller to the bridge (UDP 27001)....101
Appendix B: Building and flashing....102
B.1 Libraries....102
B.2 Compile and upload....102
B.3 Firmware versions....102
B.4 COM port to board mapping....103
B.5 Sketch settings to know about....104
Appendix C: Calibration....105
Appendix D: How this document is built....106
Appendix E: PCB schematics and layouts....107
E.1 Jet Ranger Radio Auto Pilot....108
E.2 Jet Ranger COM and NAV....110
E.3 Jet Ranger Front Panel Servo....112
E.4 Jet Ranger 13 Stepper....114
E.5 Jet Ranger OLED Dual Stepper....116
E.6 Jet Ranger X27 Small Stepper....118
E.7 Jet Ranger Upper ADF Controller....120
E.8 Jet Ranger Audio Panel....122
E.9 Jet Ranger Auto Pilot Panel....124
E.10 Jet Ranger ADF Panel....126
E.11 Annunciator....128
E.12 Jet Ranger Altimeter....129
E.13 Large Guage (Simmeters Large Guage Replacement)....131
E.14 Simkits Large Dual Gauge....133
E.15 Simkits Large Guage....135

E.16 Simkits Small Dual Gauge....137
E.17 Simkits Small Guage....139
Appendix F: Installing the controller PCBs....141
F.1 Jet Ranger Radio Auto Pilot....141
F.2 Jet Ranger Front Panel Servo....143
F.3 Jet Ranger 13 Stepper....145
F.4 Jet Ranger OLED Dual Stepper....147
F.5 Jet Ranger Upper ADF Controller....149
Appendix G: Instrument map....151
G.1 Stepper gauges and the Stepper controller....152
G.2 Dual Stepper gauges and the Dual Stepper controller....153
G.3 Attitude indicator and the Servo controller....154
G.4 Huey Warning Panel, key, and the top of the Radio and ADF boards....155
G.5 Dual Stepper displays and the Servo annunciator lamps....156
G.6 Radio panels (COM and NAV, Auto Pilot, Audio) and the ADF panel....157
G.7 Gauges and the connectors they use....158
G.8 Lamps and displays and the connectors they use....158
Appendix H: Cockpit systems wiring....159
H.1 Power supply....159
H.2 USB wiring....160
H.3 CKAS 6DOF platform....161
H.4 Lighting....162
H.5 PC displays....163
H.6 Sound system....164

1 Project overview

A home-built Bell 206 Jet Ranger cockpit: real instrument gauges, warning lamps, radio and overhead switch panels and small displays, driven from a flight simulator by Arduino Mega boards on a private Ethernet network. The simulator can be Microsoft Flight Simulator 2020 or 2024 (through SimConnect), Prepar3D / FSX (through FSUIPC), DCS World (through DCS-BIOS, first the UH-1H Huey) or X-Plane (through its UDP datarefs).

This document ties together the three parts of the repository: the **Arduino sketches**, the **Windows (C#) programs** and the **KiCad PCBs**. It is rebuilt from each project's own `PROGRAM_SUMMARY.md` and from the code, and it covers only the projects that are in use.

1.1 How it fits together

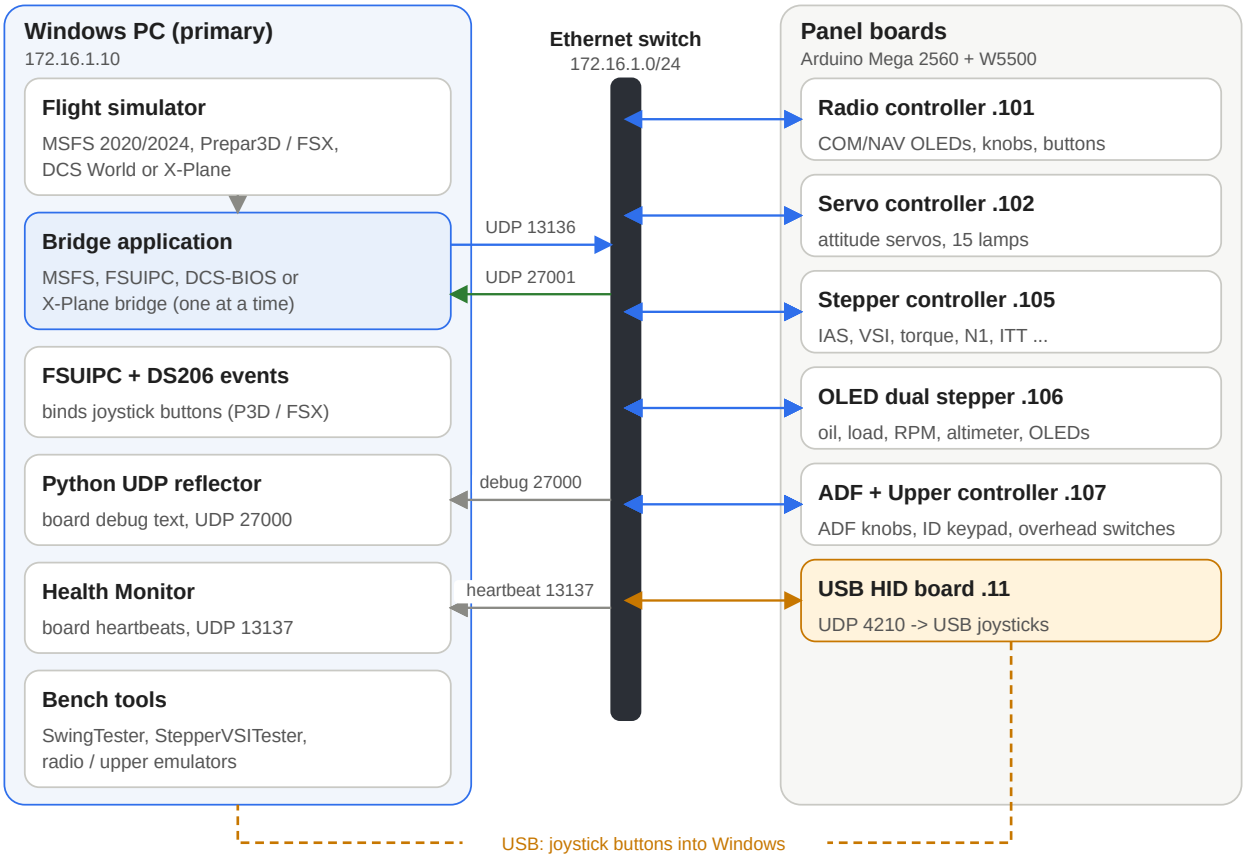


Figure 1 — The whole system: simulator, PC bridge, Ethernet and the panel boards

- The **simulator** supplies the flight data.
- A **bridge program** on the PC reads the data, converts it to real units and sends one short UDP text packet to each board about ten times a second.
- Each **Arduino board** turns the real units into stepper steps, servo positions, lamp states or display text with calibration tables in its own sketch.
- Switches and knobs go back the other way: the radio board sends commands to the bridge (UDP 27001), the ADF controller sends its frequency, radar follow ID and power status to the PC (UDP 27003, not read by the bridges yet), and the switch boards send joystick button changes to a USB HID board (UDP 4210) which Windows sees as game controllers.
- Every board sends a **heartbeat** every 10 seconds to the Health Monitor (UDP 13137) and its **debug text** to a listener on the PC (UDP 27000).

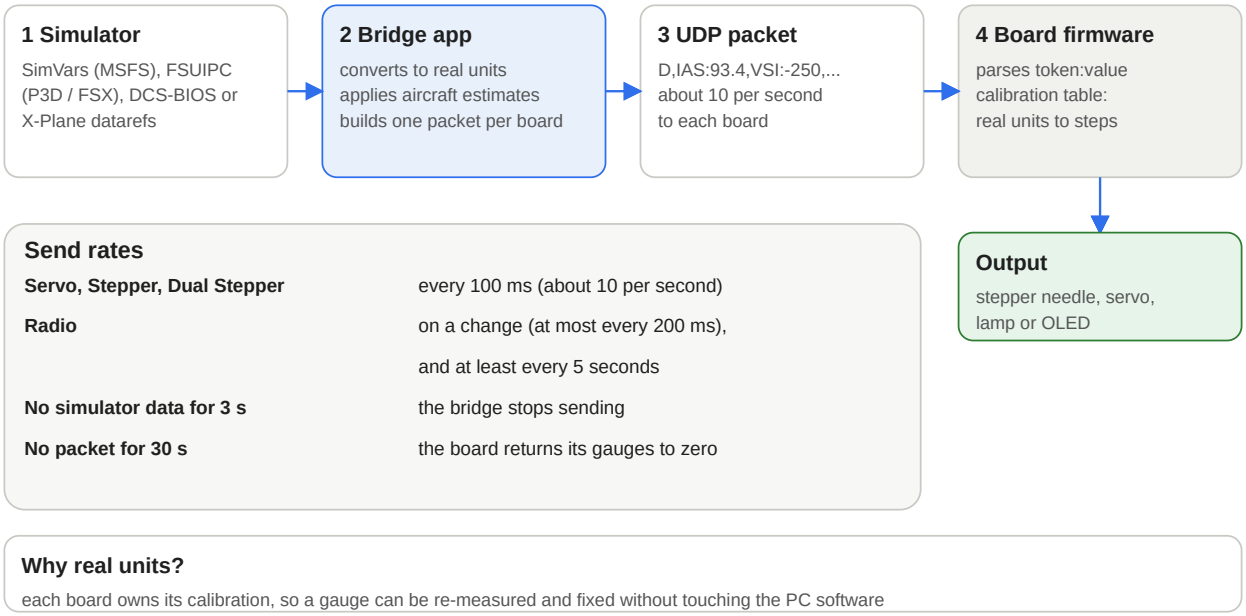


Figure 2 — From simulator value to moving needle

1.2 System block diagrams

The cockpit's power, USB, CKAS platform, lighting, display and sound systems are shown below at about a third of a page each. The same drawings at full size, with a description of each, are in Appendix H (Cockpit systems wiring).

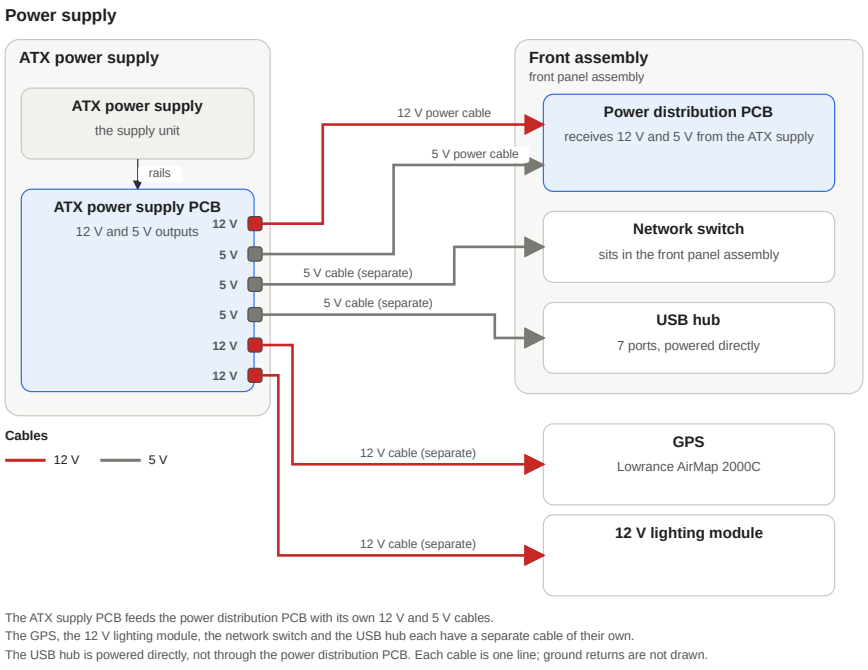
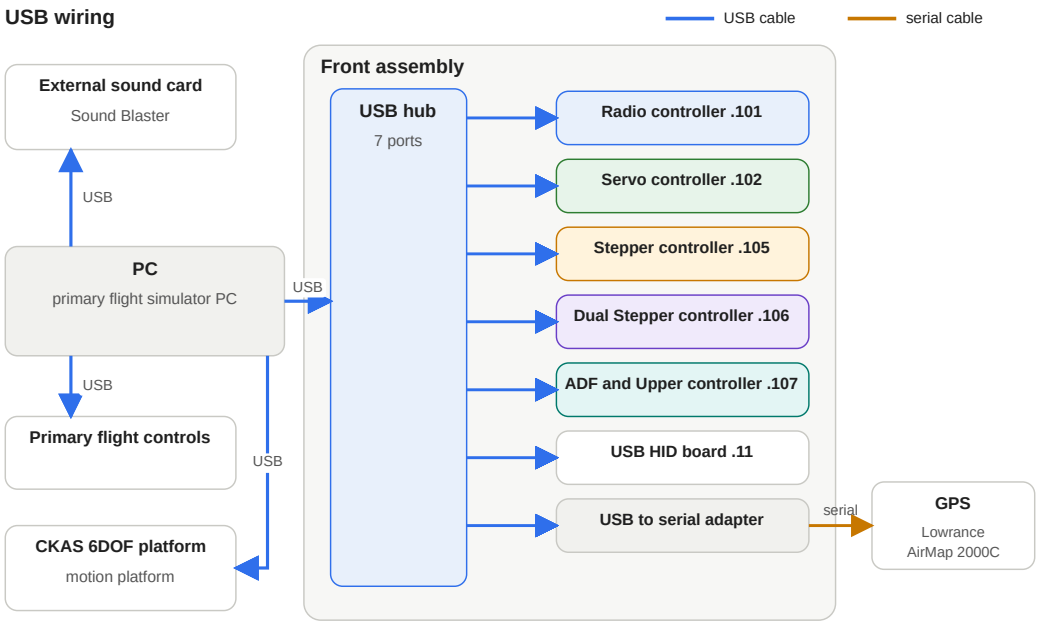
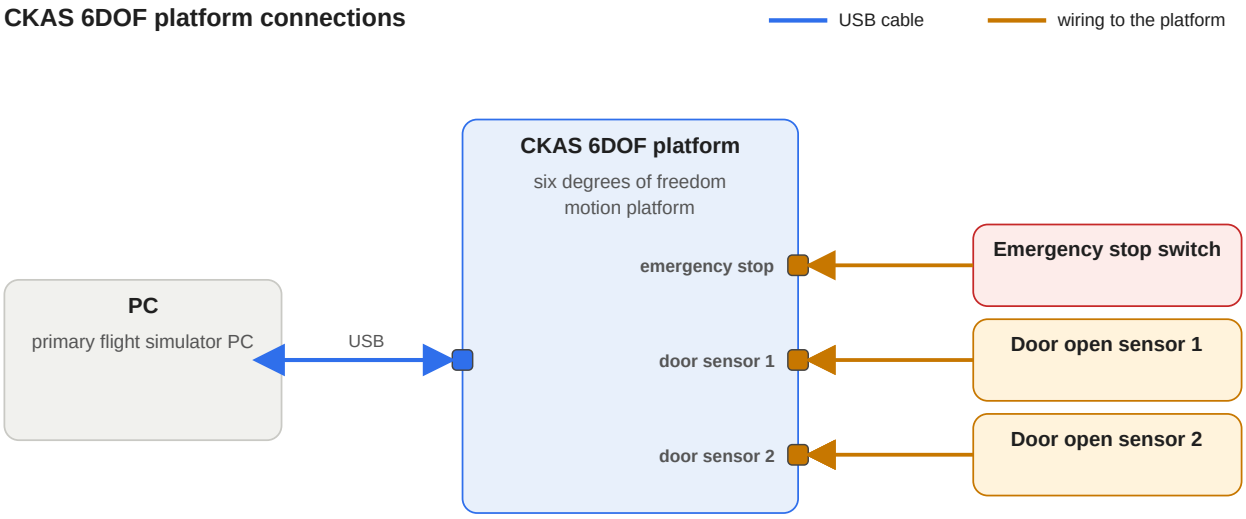


Figure 3 — Power supply (the full-size drawings are in Appendix H)



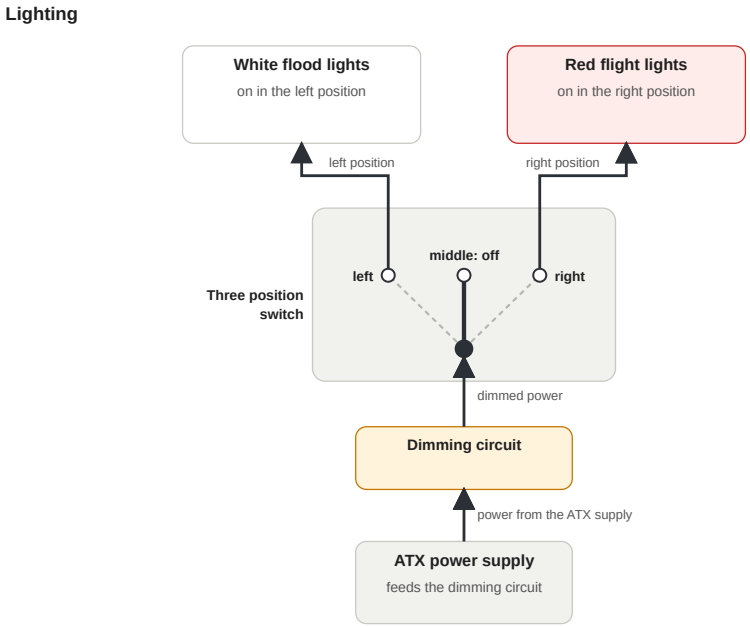
The sound card, the flight controls and the CKAS platform connect to the PC directly, not through the hub.

Figure 4 — USB wiring



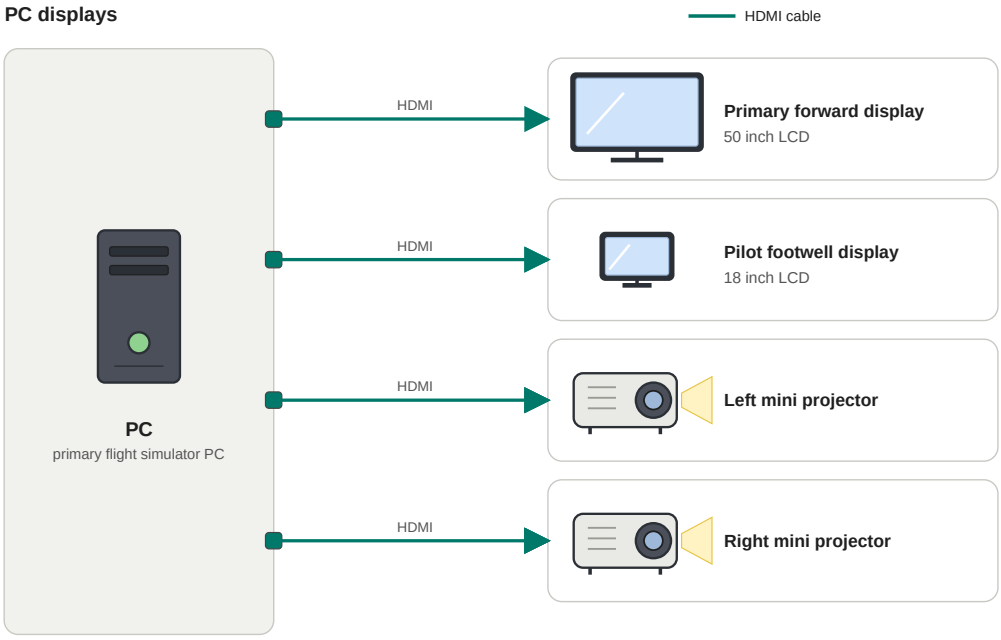
The emergency stop switch and the two door open sensors connect directly to the platform.
Not drawn: the platform's own power, the cable types, and what the platform does when an input trips.

Figure 5 — CKAS 6DOF platform



The switch is drawn in the middle (off) position. The supply voltage, the dimming circuit's design and the wiring of the lights are not drawn.

Figure 6 — Lighting



All four displays are connected to the PC by HDMI.

Figure 7 — PC displays

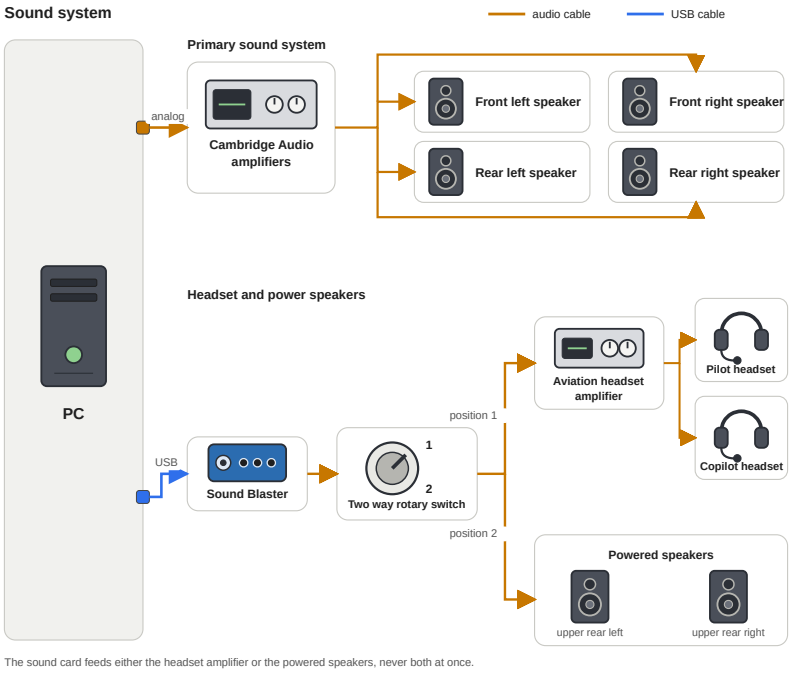


Figure 8 — Sound system

1.3 Arduino sketches

All sketches are in `Jet Ranger Arduino Sketches/`. Each is an Arduino Mega 2560 with a W5500 Ethernet module unless noted.

Sketch	IP	Role
`JET_RANGER_RADIO_CONTROLLER`	.101	COM and NAV frequency displays, eight tuning knobs, four frequency-swap switches; sends tuning and swap commands to the PC
`JET_RANGER_SERVO_CONTROLLER`	.102	Attitude indicator pitch and bank servos, 15 warning lamps, three front-panel switches (TOT clear, warning mute, fuel valve)
`JET_RANGER_ADF_CONTROLLER`	.107	The combined ADF and Upper board: ADF major and minor knobs, aircraft ID keypad for radar following, seven panel switches (gear, flaps, speed brakes, brakes, anti-ice, hyd boost, caution dim; joystick buttons and bridge commands), landing gear lamps, 12 V and 5 V rail monitoring with a status every 30 seconds, character OLED, and (merged in from the old Upper controller on 31 October 2026) the overhead switch matrix (72 positions) and the Upper keypad, forwarded to the USB HID board. The brightness pot on the PCB is switched off for the moment
`JET_RANGER_UPPER_CONTROLLER`	(.103, retired)	The old overhead switch matrix and keypad board; its code now runs inside the ADF controller sketch. The folder is kept for reference
`UDP_TO_HID`	.11	Leonardo-class board: UDP in, four 32-button joysticks and a keyboard out over USB
`JET_RANGER_STEPPER_CONTROLLER`	.105	Stepper gauges: airspeed, vertical speed, radar altitude, torque, gas producer, turbine out temperature, fuel quantity
`JET_RANGER_OLED_DUAL_STEPPER_CONTROLLER`	.106	Stepper gauges: engine and transmission oil, engine and rotor RPM, electrical load, fuel load, altimeter; four OLEDs (barometer, altimeter, clock, temperature)

Common to the three gauge boards (Servo, Stepper, Dual Stepper):

- bench-measured calibration tables with linear interpolation from real units to steps or servo positions, plus a raw bypass (`<CODE>RAW`) for calibration work;
- start-up self-test swings, also callable on demand (`C,SWINGTQ:1` and so on);
- a 30 second no-data watchdog that returns every gauge to zero;
- remote reset (`C,RESET:1`).

1.4 Windows programs

All programs are in `Jet Ranger/ C# Code/` (note the leading space in the folder name).

Program	Purpose
`FSUIPCWinformsAutoCS`	Bridge for Prepar3D / FSX using FSUIPC offsets; also sends position, altitude, airspeed, heading and the simulator date and time to the GPS interface
`MSFSWinformsAutoCS`	Bridge for MSFS 2020 / 2024 using SimConnect, with estimates for aircraft whose dials do not read the standard variables (FlyInside B206, CowaSim Jet Ranger); also feeds the GPS interface
`DCSBiosBridge`	Bridge for DCS World through the DCS-BIOS export stream, output only: Huey caution lamps (Huey Warning Panel LEDs), radar altitude and, for every aircraft, airspeed, altitude, heading and position, plus needle-position gauges for the UH-1H that still need calibrating; profile files per aircraft
`XPlaneBridge`	Bridge for X-Plane 11 and 12 over the UDP dataref interface (no plugin), output only: a generic profile for any aircraft plus Robinson R22 and Sikorsky S-76 profiles; not yet run against a real X-Plane
`JetRangerHealthMonitor`	Traffic-light dashboard for the 10 second board heartbeats
`StepperVSITester`	Bench tester that sends real and raw values to the stepper boards and the servo board
`SwingTester`	Runs every gauge's swing individually or in a fixed order, lights the OLEDs and warning lamps, tests the Auto Pilot and Audio panel LEDs one by one, resets boards, and sets the backlight and LED brightness
`LowranceAirMapGps`	Makes the PC a GPS source for a Lowrance AirMap 2000C: NMEA over a serial port, fed by the bridges over UDP, defaulting to Brisbane Airport
`LowranceAirMapGpsBenchTester`	Bench sender for the GPS interface: typed values, 8-way cursor, country / city / airport selection, Fly mode and a date / time clock
`JetRangerRadioControllerEmulator`, `JetRangerUpperControllerEmulator`	Stand-ins for the radio and upper boards' **inputs** (knobs, keys) when they are disconnected
`JetRangerPanelEmulator`	Stand-ins for the five controller boards' **outputs**: windows for the Radio, Servo, Stepper, Dual Stepper and ADF and Upper controllers that draw the gauges, lamps, LEDs and OLEDs from the same packets; every bridge can send to them (`--emulators`) instead of, or as well as, the boards

Python helpers in `Jet Ranger Python Code/`: a SimConnect exporter, a UDP test receiver and a wake-on-LAN script. The UDP debug listener (port 27000) is a small Python reflector that prints every board's debug text and can forward a copy to a second listener.

Run **only one bridge at a time**: the FSUIPC and MSFS bridges both listen on UDP 27001, and all four send to the same boards. The DCS and X-Plane bridges read their simulator and send only; they do not listen on 27001.

1.5 PCBs

KiCad projects are in `Jet Ranger/PCBs/`. The schematic and board layout of every one are in Appendix E, top-side pictures with the external connectors labelled, for installing the controller boards, are in Appendix F,

and a map of every gauge, lamp and display with the connector that drives it is in Appendix G. The boards used by the sketches and programs above are:

PCB	Revision	Used by
Jet Ranger Radio Auto Pilot	1.4	Radio controller
Jet Ranger COM and NAV	no revision set	Radio panel (four swap switches, encoders)
Jet Ranger Front Panel Servo	1.4	Servo controller
Jet Ranger Upper ADF Controller	1.0	ADF and Upper controller (the reference PCB for the ADF controller; replaces the Jet Ranger Upper Controller 1.4 and Jet Ranger ADF Controller 1.6 boards)
Jet Ranger 13 Stepper	1.6	Stepper controller
Jet Ranger OLED Dual Stepper	1.5	Dual stepper controller
Jet Ranger X27 Small Stepper	not set	One per stepper gauge
Jet Ranger Audio Panel, Auto Pilot Panel, ADF Panel	1.1	Switch panels for the Radio Auto Pilot and ADF controllers
Annunciator, Altimeter, Large Guage, Simkits gauge boards		Other panels and gauges

On 18 October 2026 the six controller boards of the time (Servo, Upper, 13 Stepper, Dual Stepper, Radio Auto Pilot and ADF Controller) were revised to fix the **backlight MOSFET**: the schematic symbol had source and drain swapped and the part is now an IRF540N. All of them drive their panel backlights from Mega pin D5 through this transistor (see the PCB notes chapter). The Upper and ADF boards were then combined into one, the Jet Ranger Upper ADF Controller (revision 1.0, 26 October 2026), so there are now five controller boards.



Figure 9 — The active PCBs, their revision and the sketches they belong to

1.6 Where things stand

All six controllers (Radio, Servo, Stepper, Dual Stepper, ADF and Upper, USB HID) were flashed on 6 November 2026 with firmware version 1.0: builds Radio 5, Servo 4, Stepper 3, Dual Stepper 5, ADF and Upper 6 and USB HID 2. Each reports its version at start-up and in its heartbeat, and the Health Monitor shows them (see Appendix B).

On 6 November 2026 the start-up backlight was changed: every controller holds its backlight, OLEDs and indicator lamps on from reset until the 13 Stepper controller announces on UDP port 13138 that start-up is complete (see the Network chapter). It is tested on the bench with the boards flashed and shown in the Health Monitor; it has not yet been watched through a full cockpit power cycle.

Bench-verified with UDP debug logs from the boards and real swing timings: every swing command, the OLED lamp test, the temperature OLED, the keep-alive, the debounced servo inputs, the radio matrix, the remote reset on every board except the old Upper controller (whose code now runs on the ADF controller), and the full SwingTester sequence. The FSUIPC bridge has been run against Prepar3D v6.1 and the MSFS bridge against MSFS 2024 with the FlyInside B206 and the CowaSim Jet Ranger. The DCS bridge (added 31 October 2026) has run against live DCS with the Huey caution lamps; its gauge values and the X-Plane bridge (also 31 October 2026) are tested only with synthetic data from fake simulators.

Backlight control (Servo, Stepper and Dual Stepper boards, and later the Radio controller) and the GPS interface were added on 18 October 2026: the backlight follows the main bus, the last commanded brightness and the 30 second no-data watchdog, and the GPS interface feeds a Lowrance AirMap 2000C from the simulator. Both are tested on the bench with packets; the GPS interface has not been tested against the real GPS, and the FSUIPC and MSFS bridges' GPS output has not been run against a simulator.

Open items:

- The three servo-board switches reach the HID board but are not sent to the PC itself (the bridges treat an unrecognised string on port 27001 as a radio frequency, so they would need updating first).
- The Upper switch matrix and Upper keypad now run on the combined ADF and Upper board (merged on 31 October 2026) and have not been bench-tested there. Its Warning Mute and TOT Test connectors are read by the matrix (indexes 88 and 89).
- The bridges and the SwingTester still send to the retired Upper address 172.16.1.103; the entries should be removed.
- The 16 Huey needle-position gauges in the DCS bridge are on first-guess scales and need calibrating in flight; the X-Plane bridge has not met a real X-Plane (check units, the R22 and S-76 rotor speed scales and the approximate Huey LEDs); neither sends inputs, port 27001 commands or the landing gear lamps.
- The estimates in the MSFS bridge for the FlyInside and CowaSim aircraft come from a few dial readings; check them at other power settings.
- The brightness pot input on A0 of the combined board is switched off in the sketch (`USE_BRIGHTNESS_POT` is 0); its backlight follows `BPWR`, `C,B` and the no-data timeout.
- The Radio controller emulator still sends a bare frequency for every radio, so only COM1 tunes from it; the real board sends labelled COM2, NAV1 and NAV2 values, which both bridges understand.
- The Dual Stepper's RAM use is high (79 percent); be careful adding code to it.
- Calibration tables, swing times and waits come from bench measurements; recheck them after any mechanical change to a gauge.

2 Colour codes and cabling

This chapter explains the colours used in the diagrams and programs, and the colour codes used for cables and connectors.

2.1 Reading the diagrams

Each diagram box is shaded to say what kind of item it is. The same shades are used in every diagram.

Colour	What it means in the diagrams
White	An ordinary item: a board, program, step or detail
Blue	The subject of the figure or the main step: the bridge, the Health Monitor, the code that decides
Green	Running or healthy, power on, or an output: connected, the gauge or backlight being driven
Amber	Waiting, a warning or a special path: no data, the default position, the USB HID path
Red	Lost, off, failed or a reset
Gray	Supporting or inactive: notes, items that stay off
Purple	Tells boards or groups apart where the figure says so (for example the Dual Stepper in the Swing ALL sequence, the Upper controller in the PCB map)
Teal	The same, for the ADF controller in the PCB map
Dark	Column headers and the Ethernet switch

Arrows follow the same idea: blue is data going to a board over UDP, green is commands or events coming back, amber is the USB HID path, gray is an ordinary link, and a dashed line is an optional or recovery path.

2.2 Colours in the programs

Where	Colour	Meaning
Health Monitor light	Green	A heartbeat was heard within the last 15 seconds
Health Monitor light	Orange	15 seconds without a heartbeat (connection warning)
Health Monitor light	Red	30 seconds without a heartbeat (connection lost)
Bridge status bar	Red	Disconnected from the simulator
Bridge status bar	Orange	Connected but no data (the simulator is paused or loading)
Bridge status bar	Green	Connected, with the aircraft name
Board status LEDs (pins 14 green, 15 red)	Green and red	Alternate every 200 ms while the sketch runs; red flashes fast while Ethernet starts
SwingTester	Gold	The gauge row being swung, or a warning lamp that is on
GPS bench tester	Pink	A box whose value could not be read (it is not used)
Upper controller emulator	Green	A switch that is on
GPS interface	Green	The position comes from aircraft data over UDP
GPS interface	Orange	No data for 30 seconds: the Brisbane Airport default is being sent

2.3 Cabling colour codes

2.3.1 Round power connectors

Colour	Supply
Red	12 V
Gray	5 V

2.3.2 PCB connectors

Colour	Use	Example
Uncoloured	Digital input	
Blue	Analog	
Red	12 V	Magnetic coils
Green	12 V backlighting	
Yellow	OLED	
Orange	LEDs	

The two 12 V colours are different on purpose: red is 12 V for coils, green is 12 V for backlighting.

2.4 OLED connectors

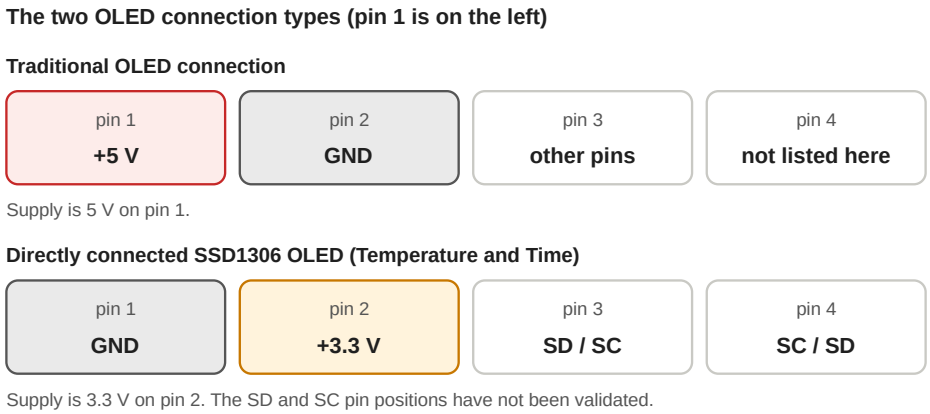


Figure 10 — The two OLED connection types

The OLED connectors are **not the same** on every display:

- **Traditional OLED connection:** +5 V on pin 1 and GND on pin 2.
- **Directly connected SSD1306 OLEDs** (the Temperature and Time displays): GND on pin 1 and +3.3 V on pin 2.

The SD and SC (data and clock) pin positions of the directly connected displays have not been validated. Do not plug one type of cable into the other type of connector: pins 1 and 2 are reversed and the supply voltages differ.

3 Network and communication

3.1 Devices and addresses

All devices sit on one wired `172.16.1.0/24` network with fixed addresses; there is no DHCP. The firmware sends its debug and heartbeat traffic to `172.16.1.10`, so the primary PC must hold that address (or `reflectorIP` must be changed in each sketch and the sketch uploaded again).

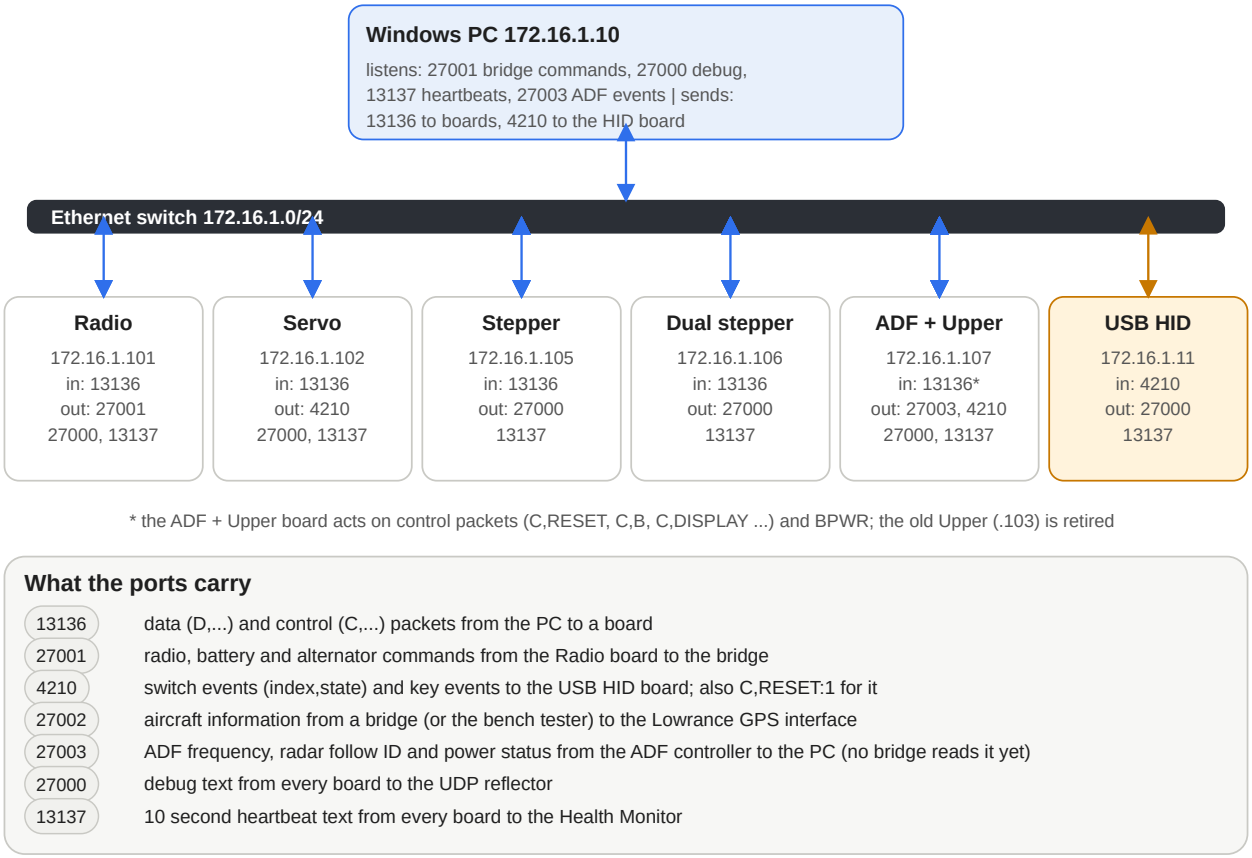


Figure 11 — Who talks to whom

Device	IP	MAC	Heartbeat text
Primary PC (simulator, bridge, listeners)	172.16.1.10		
USB HID board	172.16.1.11	DE:AD:BE:EF:FE:ED	`JOYSTICK`
Radio controller	172.16.1.101	A8:61:0A:9E:83:01	`COMM_NAV`
Servo controller	172.16.1.102	A8:61:0A:9E:83:02	`SERVO`
Stepper controller	172.16.1.105	A8:61:0A:67:83:69	`STEPPER`
Dual stepper controller	172.16.1.106	A8:61:0A:67:83:6A	`DUAL_STEPPER`
ADF and Upper controller	172.16.1.107	A8:61:0A:67:83:6B	`ADF_PANEL` and `UPPER_INPUT`

Every live board has its own IP and MAC address. The old Upper controller (172.16.1.103, MAC A8:61:0A:9E:83:03) was merged into the ADF controller on 31 October 2026 and no longer exists; the merged board sends both heartbeat names so the Health Monitor still shows an Upper light. The bridges and the SwingTester still send some packets to .103, which go nowhere.

3.2 UDP ports

Port	Direction	Purpose
13136	PC to boards	Data (`D,`) and control (`C,`) packets

Port	Direction	Purpose
27001	Radio and ADF controllers to PC bridge (FSUIPC and MSFS bridges only)	Standby frequencies, radio swaps, and the Auto Pilot, Audio and ADF panel controls (`AP_MASTER`, `HEADING_BUG_INC:5`, `GEAR_DOWN`, ...)
27002	Bridge (or bench tester) to the GPS interface	Aircraft position, altitude, airspeed, heading and simulator UTC time, every 500 ms (both on the PC)
27003	ADF controller to the PC	ADF frequency, radar follow ID and power status every 30 seconds (no bridge reads it yet; not 27001, where an unknown string is taken as a COM1 standby frequency)
4210	Servo, Radio and ADF and Upper controllers to USB HID board	Button changes (`index,state`), keys (`KEY,key,state`), reset
5010	DCS (DCS-BIOS) to the DCS bridge	The DCS-BIOS export stream, multicast 239.255.50.10, on the PC
49000	X-Plane and the X-Plane bridge	The bridge sends dataref requests to X-Plane's receive port and X-Plane answers to the bridge's own port; X-Plane's beacon is multicast 239.255.1.1 port 49707
13136 on 127.0.0.101 to .107	Bridge to the panel emulators	The same board packets, to the emulators on the PC (see the Panel emulators chapter)
13137	Boards to Health Monitor	Heartbeat text every 10 seconds
13138	Stepper controller to the other controllers	The text `STARTUPDONE`, sent to the Radio, Servo, Dual Stepper and ADF and Upper controllers when the Stepper controller has finished starting up (repeated every 10 seconds, 18 times); ends their start-up hold
27000	Boards to debug listener	Debug text, each line prefixed with the board name
7788	Local on the boards	Source port opened at start-up for outgoing packets

3.3 Packet format

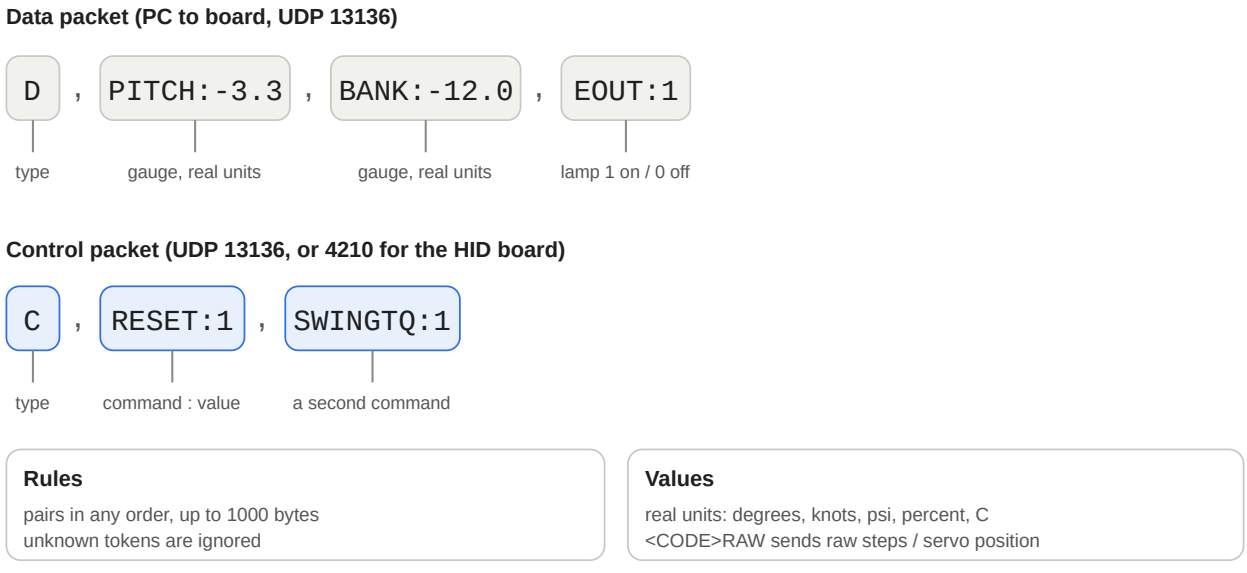


Figure 12 — Data and control packets

A packet is plain ASCII text, up to 1000 bytes.

- `D,CODE:value,CODE:value,...` is a **data** packet. It can carry any number of pairs, in any order. Unknown codes are ignored, so the PC can send only the values that changed.

- `C,COMMAND:value` is a **control** packet (swing tests, lamp test, keep-alive, reset). The value is usually ignored.
- Gauge codes take **real units**: degrees, knots, feet per minute, psi, percent, degrees C, US gallons. The board converts them with its own calibration table.
- A calibrated code usually has a raw twin with `RAW` on the end (for example `IASRAW:300`). It moves the needle to a raw step or servo position and skips the table, which is how new calibration points are measured.
- Lamp codes take `0` (off) or `1` (on).

Example sent to the servo board: `D,PITCH:5,BANK:-10,LOWF:0` sets 5 degrees pitch, minus 10 degrees bank and switches the Low Fuel lamp off.

To test without the simulator, send a packet from any PC on the subnet:

```
$u = New-Object System.Net.Sockets.UdpClient
$b = [Text.Encoding]::ASCII.GetBytes('D,PITCH:5,RLOW:1')
$u.Send($b, $b.Length, '172.16.1.102', 13136)
```

3.4 Board life cycle

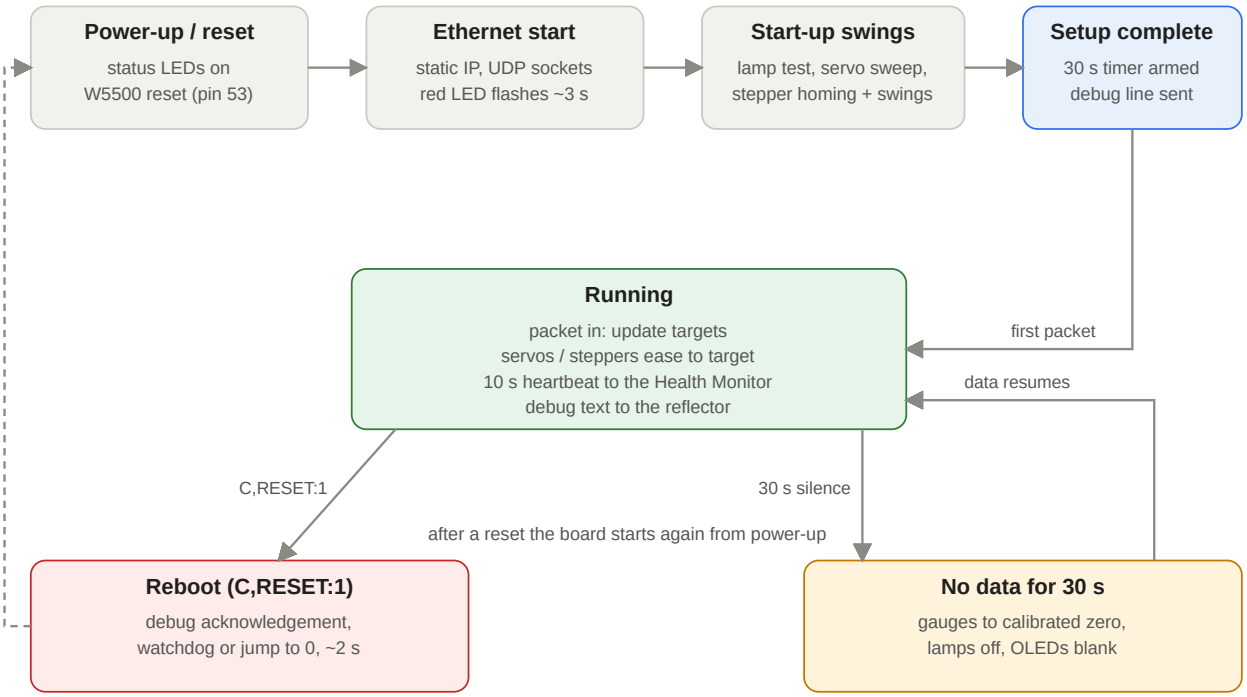


Figure 13 — What every gauge board does from power-up to shutdown

1. **Start-up:** status LEDs flash, the Ethernet module is reset, the board gets its static address and opens its sockets; the red LED flashes for a few seconds while the link settles.
2. **Self-test:** the gauge boards sweep each gauge (the Servo board also lights all lamps) and park at the calibrated zero.
3. **Running:** every 5 ms the board checks UDP 13136, stores each value as a new target and moves the needle towards it with smooth acceleration.
4. **Heartbeat:** every 10 seconds the board sends its heartbeat text to the Health Monitor, followed by its 5 V rail and its firmware version (`SERVO V5:5.02 FW:1.0.2`, where `FW:1.0.2` is version 1.0, build 2; the ADF controller also adds its 12 V rail, `ADF_PANEL V5:5.03 V12:12.41 FW:1.0.3`). The Health Monitor matches the first word and shows the voltages and the firmware version in its voltage table. The 5 V rail is the ADC reference (AVcc) measured against the chip's 1.1 V band-gap.
5. **No-data watchdog:** if no packet (data or control) arrives for 30 seconds, every gauge returns to its calibrated zero. The Servo board also switches its lamps off, the Dual Stepper blanks its OLEDs and

the Radio blanks its displays. The timer starts at the end of `setup()` so that a long self-test cannot use it up. Normal operation resumes with the next packet.

All sketches call `Ethernet.setRetransmissionTimeout(10)` after `Ethernet.begin()`. The W5500 default of 200 ms times 8 retries can block a UDP send for 1.6 seconds if the destination does not answer ARP (for example the PC is off); with 10 ms the worst case is about 80 ms, so one dead destination cannot stall gauge movement or button scanning.

3.5 Power rules in the bridges

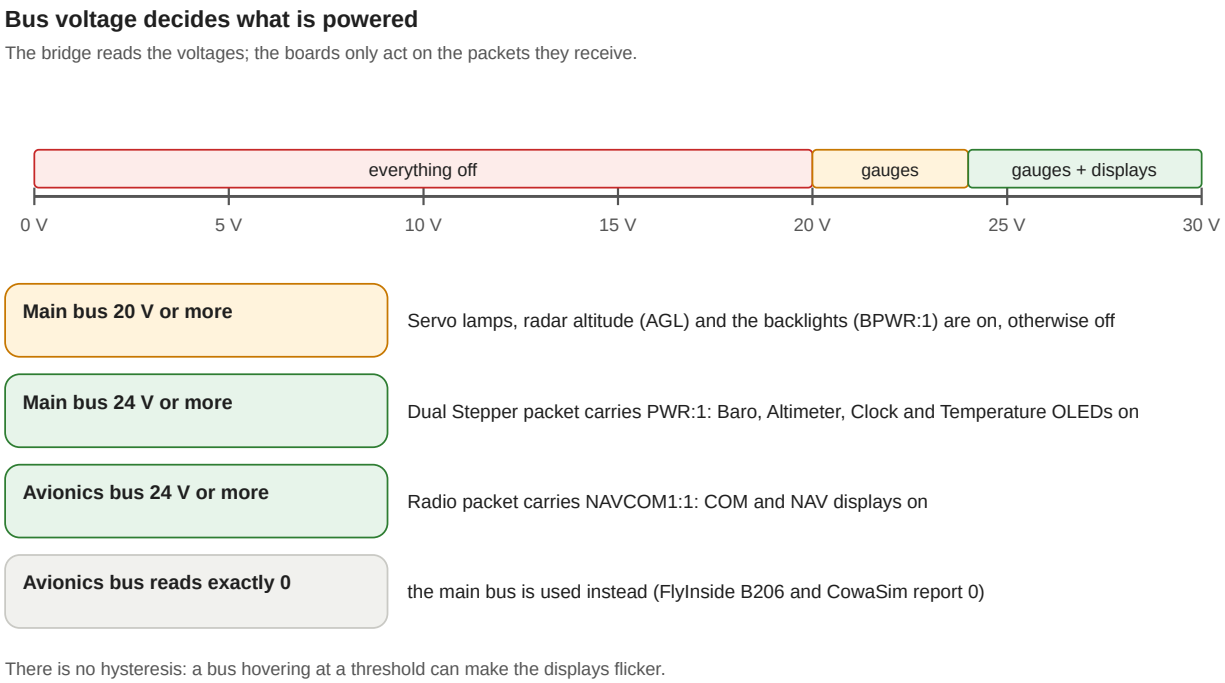


Figure 14 — How bus voltages switch the panel

- **Main bus below 20 V:** the Servo board's lamps are sent as off, the Stepper's radar altitude (`AGL`) is sent as zero and `BPWR:0` switches the Radio, Servo, Stepper, Dual Stepper and ADF and Upper backlights off.
- **Main bus below 24 V:** the bridge sends `PWR:0` to the Dual Stepper, which blanks its four OLEDs; `PWR:1` turns them back on.
- **Avionics bus below 24 V:** the bridge sends `NAVCOM1:0` to the Radio, which blanks its COM and NAV displays together. If the aircraft reports the avionics bus as exactly 0 (the FlyInside B206 does) the main bus is used instead.

There is no hysteresis on these thresholds. The 24 V figure is carried over from the FSUIPC bridge and has not been verified for every MSFS aircraft.

3.6 Backlight

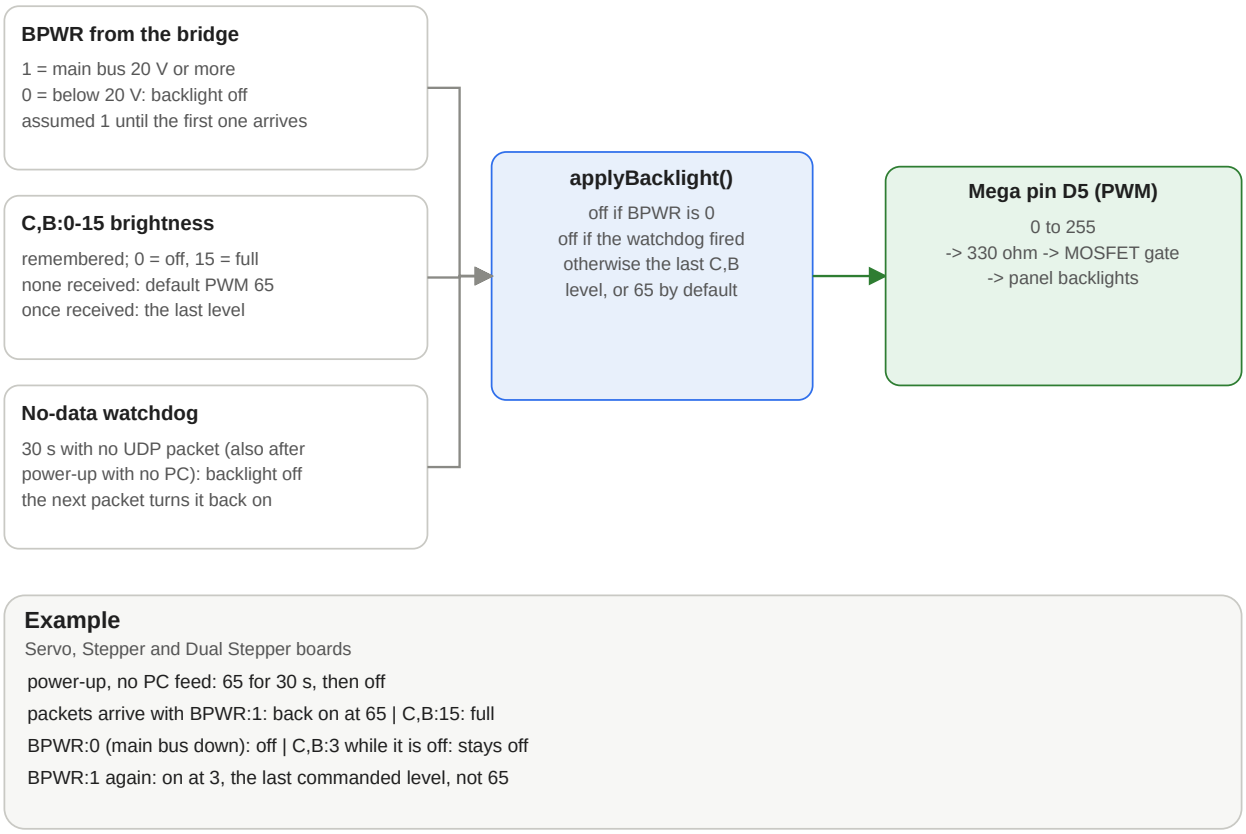


Figure 15 — How the backlight level is decided

The Radio, Servo, Stepper, Dual Stepper and ADF and Upper boards drive their panel backlights from Mega pin D5 (PWM). On the Radio the no-data rule is the same 30 second timeout that blanks its displays, which a knob or key press also restarts; on the ADF and Upper board a switch, key or knob also restarts it (the Upper panel's switch matrix and keypad count too). Three things decide the level:

- **`BPWR`** (from the bridges, first in the Servo and Stepper packets, after `PWR` in the Dual Stepper packet, and on its own to the ADF and Upper board): 1 while the main bus is 20 V or more, 0 below that. At 0 the backlight is off. Until the first `BPWR` arrives the board assumes 1.
- **`C,B:<0-15>`** sets the brightness (0 off, 15 full, mapped to PWM 0 to 255; larger values are clamped). It is remembered. With no brightness command received the default level, PWM 65, is used; once one has been received every return of power restores the **last commanded level**. A command sent while the bus is down is applied when power returns.
- **The no-data watchdog:** after 30 seconds with no UDP packet, including after power-up with no PC feed, the backlight goes off along with the gauges. The next packet turns it back on at the commanded (or default) level, provided the main bus is up.

The SwingTester's brightness slider sends `C,B` to every board except the USB HID board, and its *Backlight power* buttons send `BPWR` to the Radio, Servo, Stepper, Dual Stepper and ADF controllers (and still to the retired Upper address .103). The ADF controller can send `C,B` from its brightness pot to the other boards, but the pot is switched off for the moment (see the ADF controller chapter). The Radio also uses `C,B` for the intensity of its panel LEDs. The ADF controller also blanks its OLED and lamps together with the Radio's displays (the Radio sends it `C,DISPLAY:0/1`), see the ADF controller chapter.

3.6.1 Start-up hold

Start-up hold. From reset every controller keeps its backlight **on** (at the commanded or default level), and keeps its OLEDs and indicator lamps on, whatever the no-data watchdog or `BPWR` says. The hold ends when the 13 Stepper controller, which is the last to finish its start-up, announces that start-up is complete: it sends

the text `STARTUPDONE` to UDP port 13138 of the Radio, Servo, Dual Stepper and ADF and Upper controllers, and repeats it every 10 seconds, 18 times, so a board that was not listening yet or was reset still hears it. When the hold ends the backlight goes off and stays off until the bridge reports the main bus is up (`BPWR:1`). If the announcement never comes (the Stepper controller is off) each board ends the hold by itself 180 seconds after its reset. Port 13138 is separate from the data port, so the announcement never counts as data for the 30 second watchdog.

What stays on during the hold:

Controller	Stays on until the hold ends
Radio	Backlight, the four OLED displays (COM, NAV, Auto Pilot, DME) and all panel LEDs (lit)
Servo	Backlight and all 15 caution and warning lamps (lit)
Stepper	Backlight only (it ends its own hold at the end of its start-up and sends the announcement)
Dual Stepper	Backlight and the four OLEDs
ADF and Upper	Backlight, the OLED and the three landing gear lamps (lit)

When the hold ends the OLEDs and lamps go back to their normal rules (PWR, `NAVCOM1`, `C,DISPLAY` and the data). A board that has had no data or input for more than 2 seconds then also expires its own 30 second no-data timer, so its OLEDs go out together with the backlight instead of lingering. The USB HID board has no backlight and is not part of this.

3.7 Remote reset

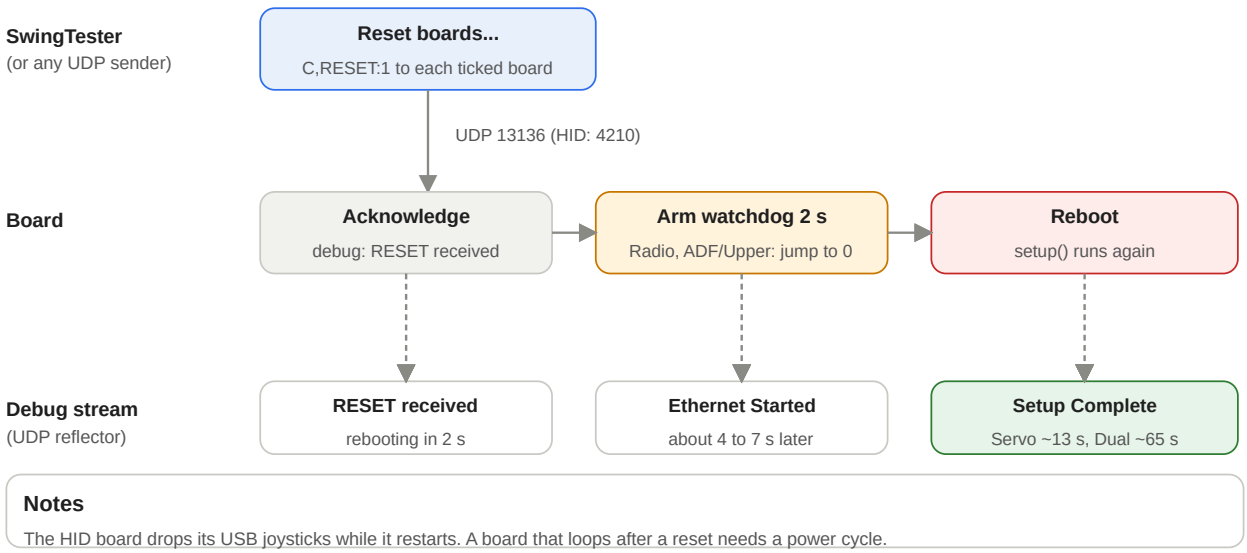


Figure 16 — C,RESET:1 on each board

The control packet `C,RESET:1` reboots a board about two seconds later.

- **Servo, Stepper, Dual Stepper and USB HID boards** arm the AVR watchdog for two seconds and spin until it fires. A short `.init3` function clears the watchdog first thing after every reset. The HID board listens on port 4210 (it also accepts the plain text `RESET`); its USB connection drops while it restarts, so Windows briefly loses the four joysticks and the keyboard.
- **Radio and ADF and Upper controllers** use a software reset (a jump to the reset vector after switching off interrupts and peripherals). A watchdog reset hung the Radio's bootloader, which restarts the watchdog at its shortest timeout and never reaches the sketch until it is reflashed.

If a board ever reboot-loops after a reset packet, power-cycle it and reflash. The SwingTester's *Reset boards...* button sends the command to the boards you tick.

> **Network note:** data that arrives in bursts with gaps of several seconds meant the PC's link to the board switch was dropping in and out. The cause on this cockpit was the 5 V rail feeding the network switch dipping below 4.75 V; the switch then dropped its ports. Check that supply first, then the cable and the switch port.

3.8 Debug and monitoring

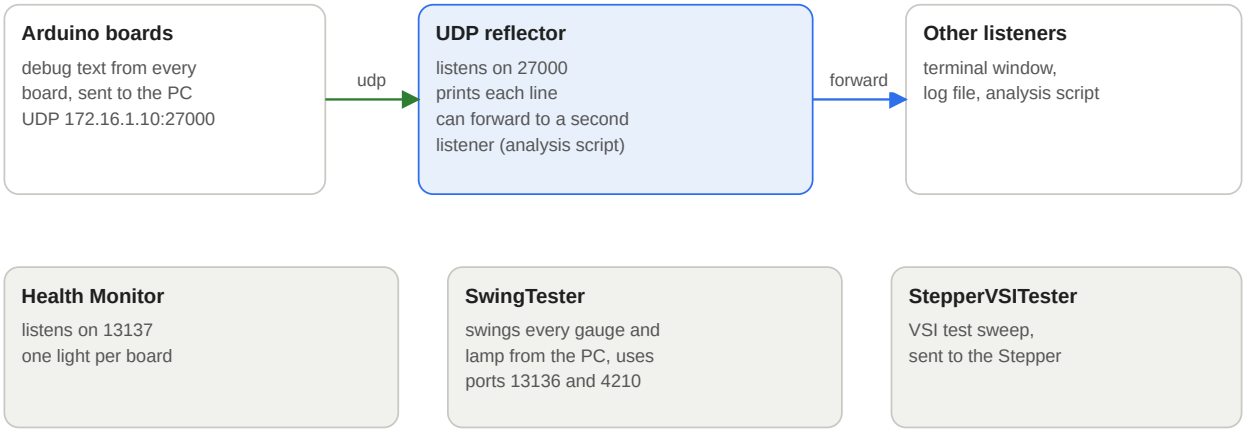


Figure 17 — Debug and test tools

Boards send debug text to `172.16.1.10:27000`. Only one program can own a UDP port, so if the reflector is running do not start a second listener on 27000; use the reflector's forwarding instead. The Health Monitor owns 13137 and each bridge owns 27001.

4 Radio controller (172.16.1.101)

`JET_RANGER_RADIO_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module. It shows the COM and NAV frequencies on two character OLEDs, reads eight rotary encoders (COM1, COM2, NAV1 and NAV2, each with a major and a minor knob) and a 4 by 5 button matrix, and keeps the radio panel in step with the simulator. It also serves the **Auto Pilot and Audio panels**: it reads their switches and the Auto Pilot's five knobs (sent as joystick buttons and as simulator controls) and drives their 19 LEDs through a MAX7219. The same MAX7219 also drives the 20 caution LEDs of the **Huey Warning Panel**, whose three switches the sketch reads too (that panel is wired by hand at the moment; see Huey Warning Panel switches and LEDs).

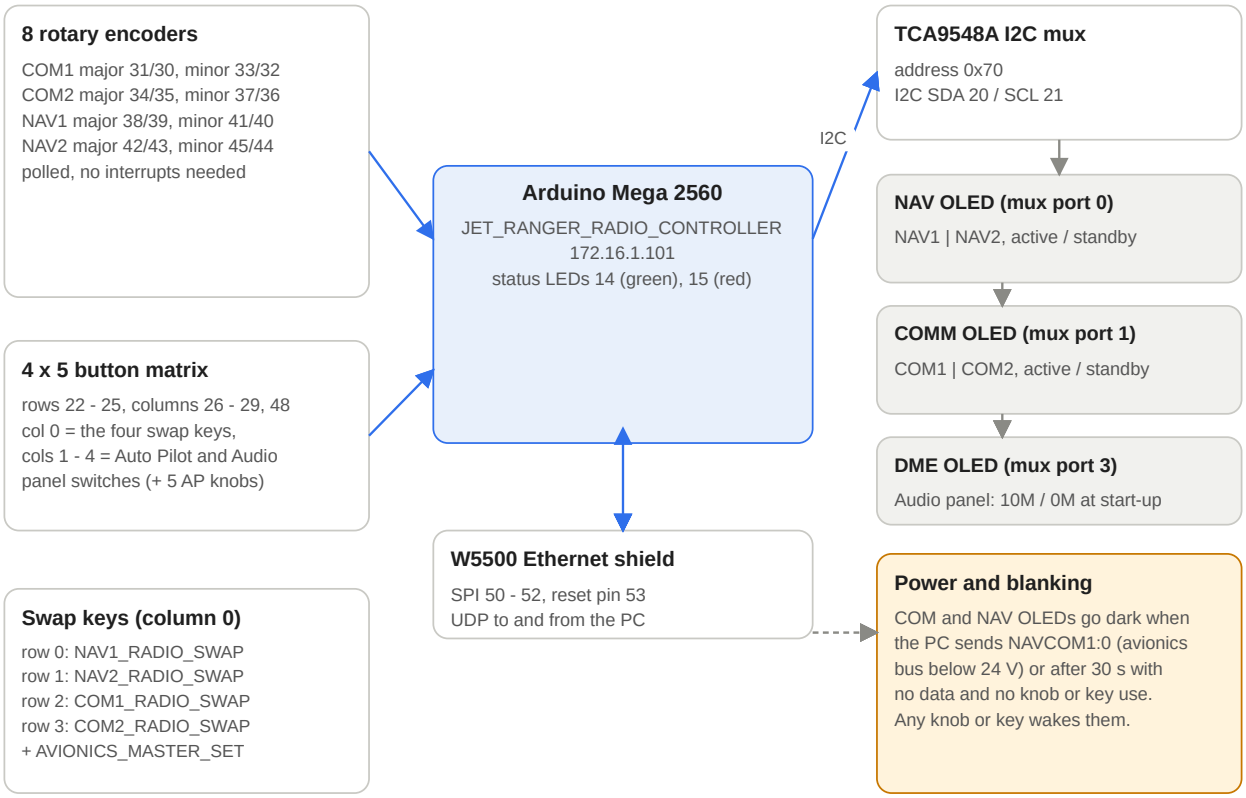


Figure 18 — Radio controller blocks

4.1 Program flow

1. **Setup:** reset the W5500 through pin 53, bring up Ethernet and open the UDP sockets, flash the red LED for 3 seconds while the link settles, send an "Ethernet Started" debug line, scan the I2C bus (results to the debug monitor, see Diagnostics), start the COM and NAV OLEDs through the TCA9548A I2C multiplexer, start the DME display (the Audio panel's OLED) with the text `10M` and `0M`, start the eight COM and NAV encoders (with acceleration), the five Auto Pilot knobs and the matrix scanner.
2. **Main loop:**
    1. toggle the green and red status LEDs every 200 ms;
    2. every 10 seconds send the `COMM_NAV` heartbeat;
    3. poll the eight encoders; a change recalculates the standby frequency, sends it to the PC and redraws the OLEDs;
    4. scan the matrix and act on key presses (below);
    5. read any packet on port 13136 and update the active and standby frequencies, the main bus voltage and the power state.

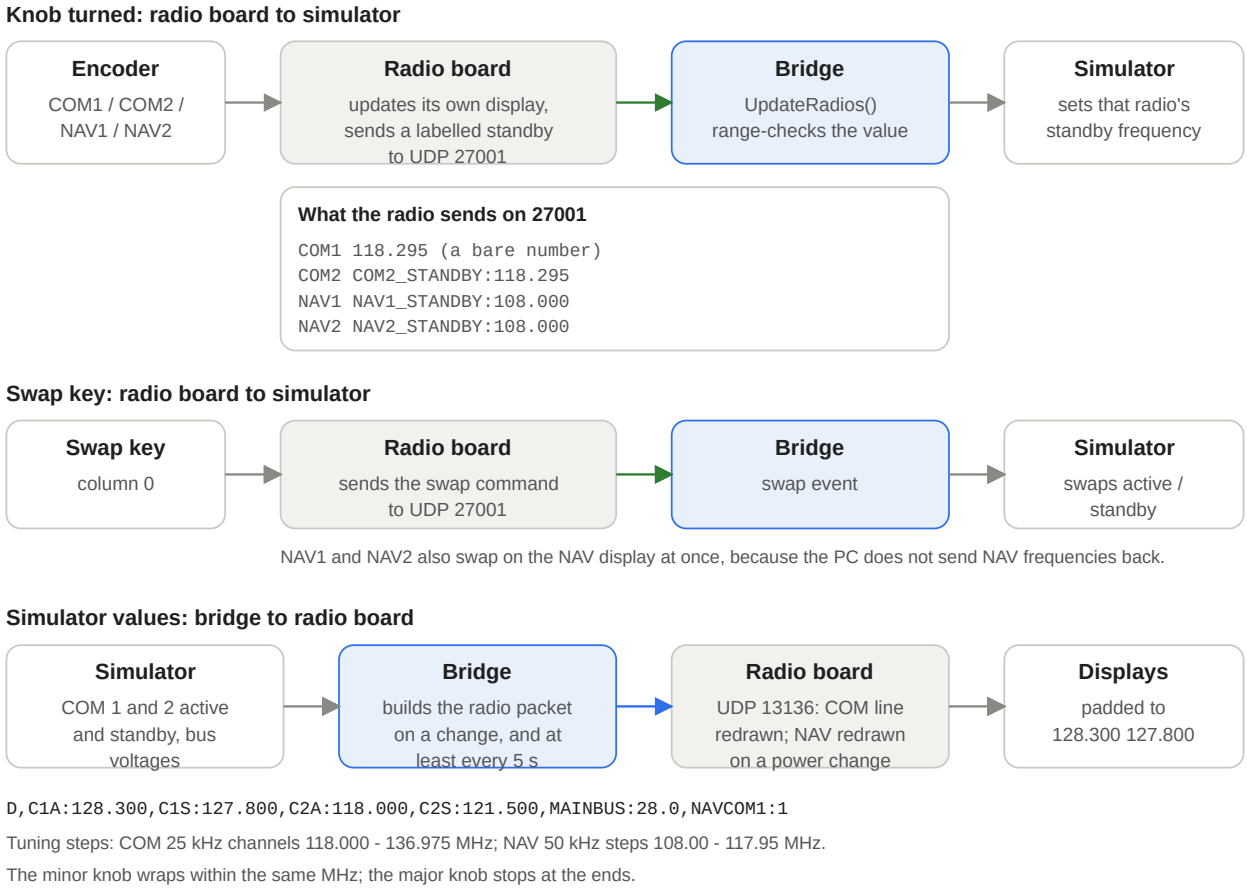


Figure 19 — Where each input and output goes

4.2 Pin usage

Pin	Function
5	`BACKLIGHT_PWM`: the AUDIO, AUTOPILOT and COMM_NAV panel backlights (IRF540N MOSFET)
14	Green status LED
15	Red status LED (flashes while the Ethernet link comes up)
53	W5500 manual reset
50 to 52 (and 10)	SPI to the W5500
SDA, SCL	TCA9548A mux (0x70): port 0 NAV OLED, port 1 COM OLED, both at 0x3C
31, 30	COM1 major encoder (A, B)
33, 32	COM1 minor encoder
34, 35	COM2 major encoder
37, 36	COM2 minor encoder
38, 39	NAV1 major encoder
41, 40	NAV1 minor encoder
42, 43	NAV2 major encoder
45, 44	NAV2 minor encoder
22 to 25	Matrix rows (`ROW0` to `ROW3`)
26, 27, 28, 29, 48	Matrix columns (`COL0` COM and NAV panel, `COL1` and `COL2` Auto Pilot panel, `COL3` and `COL4` Audio panel)
11, 12	Auto Pilot CRS knob (`CRS_A`, `CRS_B`)
A4, A5	Auto Pilot ALT knob (`ALT_A`, `ALT_B`)
46, 47	Auto Pilot VS knob (`VS_A`, `VS_B`)

Pin	Function
7, 6	Auto Pilot IAS knob (`IAS_A`, `IAS_B`)
8, 9	Auto Pilot HDG knob (`HDG_A`, `HDG_B`)
A15, A14, A13	MAX7219 `DIN`, `CLK` and `LOAD` (the panel LEDs)
A8, A9, A10	Huey Warning Panel switches: brightness / dim, test and reset (to ground when on)

4.3 Data from the PC (UDP 13136)

Token	Meaning	Example
`C1A`, `C1S`	COM1 active and standby frequency	`C1A:119.500`
`C2A`, `C2S`	COM2 active and standby frequency	`C2S:121.900`
`MAINBUS`	Main bus voltage	`MAINBUS:27.8`
`NAVCOM1`	Radio power: 1 = on, 0 = COM and NAV displays blank and the panel LEDs off	`NAVCOM1:1`
`BPWR`	Backlight power: 1 = main bus 20 V or more, 0 = backlight off (the radio assumes 1 until the first one arrives)	`BPWR:1`
`LED_xxx`	One of the 19 Auto Pilot and Audio panel LEDs (see below): 1 = on, 0 = off	`LED_HDG:1`
`LED_H_xxx`	One of the 20 Huey Warning Panel LEDs (see below): 1 = on, 0 = off. Sent by the DCS and X-Plane bridges, not by the FSUIPC and MSFS bridges	`LED_H_ENG_OIL:1`
`APHDG`, `APCRS`, `APIAS`, `APVS`, `APALT`	The five values on the Auto Pilot display (see below), up to 5 characters each	`APHDG:090`
`C,B:n`	Brightness 0 to 15 (control packet): the panel backlight level (0 off, 15 full) and the MAX7219 intensity of the panel LEDs	`C,B:5`
`C,LEDTEST:n`	Control packet: 1 lights all 39 panel LEDs (Auto Pilot, Audio and Huey), 0 returns them to the simulator's state	`C,LEDTEST:1`

The bridges send COM frequencies with two decimals; the radio pads every frequency to `xxx.xxx` (7 characters) so that the 16-column line (7 + 2 + 7) never slides COM2 or NAV2 behind the faceplate. The PC does not send NAV frequencies back.

4.4 Commands to the PC (UDP 27001)

Turning a knob changes the standby frequency on the OLED and sends it to the PC. COM1 is sent as the bare number (for example `118.295`); the others carry the radio's name: `COM2_STANDBY:118.295`, `NAV1_STANDBY:108.000`, `NAV2_STANDBY:108.000`. Nothing is sent during the first 1.5 seconds after power-up, so the knobs' start positions do not change the simulator.

The four frequency-swap switches are on the Jet Ranger COM and NAV PCB in a diode matrix that shares column 0 (D26); the rows are D22 to D25.

Switch	Matrix (row, col)	Action
NAV1 FREQ (SW6)	0, 0	Swaps NAV1 active and standby on the NAV OLED at once (the PC sends no NAV frequencies back) and sends `NAV1_RADIO_SWAP`
NAV2 FREQ (SW7)	1, 0	The same for NAV2, sends `NAV2_RADIO_SWAP`
COMM1 (SW4)	2, 0	Sends `COM1_RADIO_SWAP`; the display changes when the simulator returns the swapped frequencies
COMM2 (SW5)	3, 0	Sends `COM2_RADIO_SWAP`, then `AVIONICS_MASTER_SET`

Keys in columns 1 to 4 are the Auto Pilot and Audio panel switches (next section); any other key is only logged to the debug listener.

4.5 Auto Pilot and Audio panel switches and knobs

The Jet Ranger Auto Pilot Panel and Audio Panel PCBs plug into the Radio Auto Pilot controller. Each has eight switches in a diode matrix (row to the switch, diode to the column), and the Auto Pilot panel also has five rotary encoders wired straight to the Mega. Every switch and knob is sent **two ways**:

1. as a **joystick button** on the USB HID board (UDP `172.16.1.11:4210`, `index,state`), in the buttons nothing else uses: **Joystick 3** (indexes 64 to 95) for the Auto Pilot panel and **Joystick 4** (96 to 127) for the Audio and ADF panels. A switch holds its button down while it is pressed. A knob click is sent as a 40 ms press (the `HID_PULSE_MS` release goes out from the main loop), clockwise on one button and anticlockwise on the next;
2. as a **command to the PC bridge** (UDP 27001), the name of the simulator control, which the FSUIPC and MSFS bridges recognise (see their chapters). A switch sends its command when pressed; a knob sends one command for each click, with a `:N` repeat count when it is turned quickly (`HEADING_BUG_INC:5`).

Nothing is sent during the first 1.5 seconds after power-up (the quiet period). Windows numbers joystick buttons from 1, so index 64 is Joystick 3 button 1. These indexes correspond to Upper matrix positions (now on the ADF and Upper controller) that are wired but have no switch on them (the Upper's assigned switches are all below index 40).

Panel	Control	Matrix or pin	Joystick button (index)	Command to the bridge (UDP 27001)
Auto Pilot	AP ENGAGE	row 0, col 1	Joystick 3, button 1 (64)	`AP_MASTER`
Auto Pilot	HDG	row 1, col 1	Joystick 3, button 2 (65)	`AP_HDG_HOLD`
Auto Pilot	CRS	row 2, col 1	Joystick 3, button 3 (66)	`AP_NAV1_HOLD`
Auto Pilot	ALT	row 3, col 1	Joystick 3, button 4 (67)	`AP_ALT_HOLD`
Auto Pilot	APPR	row 0, col 2	Joystick 3, button 5 (68)	`AP_APR_HOLD`
Auto Pilot	FD	row 1, col 2	Joystick 3, button 6 (69)	`TOGGLE_FLIGHT_DIRECTOR`
Auto Pilot	BC	row 2, col 2	Joystick 3, button 7 (70)	`AP_BC_HOLD`
Auto Pilot	IAS	row 3, col 2	Joystick 3, button 8 (71)	`AP_PANEL_SPEED_HOLD`
Auto Pilot	CRS knob clockwise, anticlockwise	D11, D12	Joystick 3, buttons 9, 10 (72, 73)	`VOR1_OBI_INC`, `VOR1_OBI_DEC`
Auto Pilot	ALT knob	A4, A5	Joystick 3, buttons 11, 12 (74, 75)	`AP_ALT_VAR_INC`, `AP_ALT_VAR_DEC`
Auto Pilot	VS knob	D46, D47	Joystick 3, buttons 13, 14 (76, 77)	`AP_VS_VAR_INC`, `AP_VS_VAR_DEC`
Auto Pilot	IAS knob	D7, D6	Joystick 3, buttons 15, 16 (78, 79)	`AP_SPD_VAR_INC`, `AP_SPD_VAR_DEC`
Auto Pilot	HDG knob	D8, D9	Joystick 3, buttons 17, 18 (80, 81)	`HEADING_BUG_INC`, `HEADING_BUG_DEC`
Audio	COM1	row 0, col 3	Joystick 4, button 1 (96)	`COM1_TRANSMIT_SELECT`
Audio	BOTH	row 1, col 3	Joystick 4, button 2 (97)	`COM_RECEIVE_ALL_TOGGLE`
Audio	NAV2	row 2, col 3	Joystick 4, button 3 (98)	`RADIO_VOR2_IDENT_TOGGLE`
Audio	DME	row 3, col 3	Joystick 4, button 4 (99)	`RADIO_DME1_IDENT_TOGGLE`
Audio	COM2	row 0, col 4	Joystick 4, button 5 (100)	`COM2_TRANSMIT_SELECT`
Audio	NAV1	row 1, col 4	Joystick 4, button 6 (101)	`RADIO_VOR1_IDENT_TOGGLE`
Audio	MKR	row 2, col 4	Joystick 4, button 7 (102)	`MARKER_SOUND_TOGGLE`
Audio	ADF	row 3, col 4	Joystick 4, button 8 (103)	`RADIO_ADF_IDENT_TOGGLE`
ADF	Landing gear switch	ADF controller D31	Joystick 4, button 9 (104)	`GEAR_SET:1` when closed, `GEAR_SET:0` when open
ADF	Flaps switch	ADF controller D32	Joystick 4, button 10 (105)	`FLAPS_SET:1` when closed, `FLAPS_SET:0` when open
ADF	Speed brakes switch	ADF controller D33	Joystick 4, button 11 (106)	`SPOILERS_SET:1` when closed, `SPOILERS_SET:0` when open
ADF	Brakes button	ADF controller D34	Joystick 4, button 12 (107)	`PARKING_BRAKES` when pressed
ADF	ADF major knob clockwise, anticlockwise	ADF controller D6, D7	Joystick 4, buttons 13, 14 (108, 109)	none (the knob tunes the ADF frequency, `ADF_FREQ`)

Panel	Control	Matrix or pin	Joystick button (index)	Command to the bridge (UDP 27001)
ADF	ADF minor knob clockwise, anticlockwise	ADF controller D8, D9	Joystick 4, buttons 15, 16 (110, 111)	none
ADF	Anti-ice switch	ADF controller D36	Joystick 4, button 17 (112)	`ANTI_ICE_SET:1` when closed, `ANTI_ICE_SET:0` when open
ADF	Hyd boost switch	ADF controller D35	Joystick 4, button 18 (113)	none (bind the joystick button in FSUIPC for the Dodo)
ADF	Caution dim switch	ADF controller D37	Joystick 4, button 19 (114)	none (bind the joystick button in FSUIPC for the Dodo)
Huey	TEST switch	A9	Joystick 4, button 20 (115)	none (bind the joystick button in the simulator)
Huey	RESET switch	A10	Joystick 4, button 21 (116)	none
Huey	BRT / DIM switch	A8	Joystick 4, button 22 (117)	none

The Radio controller sends the first 18 Auto Pilot rows, the 8 Audio rows and the 3 Huey rows; the ADF rows come from the ADF and Upper controller (see its chapter). If a knob turns the wrong way, swap its A and B pins in the `RotaryEncoder` lines for the five Auto Pilot knobs. Which row and column each switch is on comes from the panel schematics: the matrix columns are the Radio Auto Pilot PCB nets `COL1` to `COL4`, and `COL0` is the COM and NAV panel's four swap switches. Each key is sampled once per millisecond and debounced by integration: a counter steps towards down or up with every sample and the key changes state only when it reaches an end (about 12 ms), and each sample takes two readings a few microseconds apart so a noise spike is skipped. This replaced a 50 ms timer that restarted on every reading that differed from the last, which let noise on a column line hold a release back (the MAX7219 LED digit lines run beside the column lines in the panel cables). The knobs use the same polling encoder library, with acceleration, as the COM and NAV knobs.

4.6 Auto Pilot and Audio panel LEDs (MAX7219)

One MAX7219 on the Radio Auto Pilot PCB (U2, `DOUT` unconnected, ISET resistor 10 kohm) drives 39 LEDs: DIN on A15, CLK on A14, LOAD on A13 (LedControl library, scan limit 7 since the Huey panel was added, intensity 8 until a `C,B` arrives; the scan limit was 3 before, and with all eight digits scanned each LED is lit for half as long, so the Auto Pilot and Audio LEDs are dimmer than they were). Its segment pins A to G and DP are the PCB nets `ANNODE_00` to `ANNODE_07` and its digit pins 0 to 7 are `CATHODE_00` to `CATHODE_07`; each panel LED sits between one segment (anode) and one digit (cathode). The Auto Pilot panel uses digits 0 and 1 (connector `AUTOPILOT`) and the Audio panel digits 2 and 3 (connector `AUDIO`). LedControl's `setLed(0, digit, column)` uses column 0 for DP and 1 to 7 for A to G; in the chip's own digit registers (address = digit + 1) bit 7 is DP, bit 6 is A and bit 0 is G.

LED	Token	MAX7219 digit	Segment		LED	Token	MAX7219 digit	Segment
CRS	`LED_CRS`	0	A		COM1	`LED_COM1`	2	A
HDG	`LED_HDG`	0	B		BOTH	`LED_BOTH`	2	B
IAS	`LED_IAS`	0	C		NAV2	`LED_NAV2`	2	C
ALT	`LED_ALT`	0	D		DME	`LED_DME`	2	D
APPR	`LED_APPR`	1	A		OUTER	`LED_OUTER`	2	E
FD	`LED_FD`	1	B		INNER	`LED_INNER`	2	F
BC	`LED_BC`	1	C		COM2	`LED_COM2`	3	A
AP ENGAGE	`LED_APENG`	1	D		NAV1	`LED_NAV1`	3	B
					MKR	`LED_MKR`	3	C
					ADF	`LED_ADF`	3	D
					MIDDLE	`LED_MID`	3	E

The first two columns are the Auto Pilot panel and the others the Audio panel. The bridges send each LED as a `LED_xxx:0/1` token in the radio data packet. `HandleOutputValuePair()` handles these before its per-pair debug line (19 extra debug packets per radio packet would flood the debug stream), `applyPanelLeds()` writes only the LEDs that changed, and the LEDs follow the radio power: they are lit only while `NAVCOM1` is 1 and the no-data timeout has not blanked the displays. `C,LEDTEST:1` lights all 39 for checking the wiring, and `C,B:0-15` (for example from the ADF controller's pot or the SwingTester slider) sets the intensity: 0 shuts the

MAX7219 down, 1 to 15 set the intensity. The SwingTester has a toggle for each LED, an All ON / OFF, a chase and the board test.

4.7 Huey Warning Panel switches and LEDs

The Huey Warning Panel (20 caution LEDs, three switches) connects to the Radio controller through four connectors added in PCB revision 1.4 (31 October 2026): **J20** the three switches (A8 to A10), **J21** the panel backlight, **J22** the LED anodes (`HUEY_ANODES`) and **J23** the LED cathodes (`HUEY_CATHODES`). The panel itself is hand wired and its PCB is not part of this documentation. The sketch supports it as follows (the Radio on COM20 was flashed with this build on 31 October 2026, 40,552 bytes of flash and 5,949 bytes of RAM, 72 percent; it has not been checked on the panel yet):

- **LEDs:** the 20 LEDs are on the same MAX7219 as the other panels, on its digits 4 to 7 and the same segment lines. They are the tokens `LED_H_xxx` in `panelLedTable[]`, handled exactly like the Auto Pilot and Audio LEDs (`applyPanelLeds()`, the radio-power rule, the `C,B` intensity and `C,LEDTEST`).
- **Switches:** TEST (A9), RESET (A10) and BRT / DIM (A8) are read with the internal pull-ups (a closed switch is on) and a 30 ms debounce. TEST and RESET are the two positions of one switch, so one of them is always closed. Each is a joystick button on the USB HID board, held while closed: Joystick 4 buttons 20 to 22 (indexes 115 to 117). Nothing is sent during the start-up quiet period, and afterwards the buttons are brought into line with the switches. They are resent when the HID board comes back.
- **Lamp test:** while TEST is closed all 20 Huey LEDs are lit (only while the radio has power), without any command from the PC.
- **Bridges:** the FSUIPC and MSFS bridges do not send `LED_H_xxx`; the DCS bridge sends all 20 from the UH-1H caution panel (tested live) and the X-Plane bridge sends ten approximations from generic annunciators, otherwise the LEDs only light from the SwingTester, the lamp test or a test packet. For the FSUIPC and MSFS bridges, which simulator state drives each lamp is not decided.

LED	Token	MAX7219 digit	Segment		LED	Token	MAX7219 digit	Segment
ENG OIL	`LED_H_ENG_OIL`	4	A		AUX FUEL LOW	`LED_H_AUX_FUEL_LOW`	5	A
ENG ICE	`LED_H_ENG_ICE`	4	B		XMSN OIL PRESS	`LED_H_XMSN_OIL_PRESS`	5	B
ENG ICE DETECT	`LED_H_ENG_ICE_DETECT`	4	C		XMSN OIL HOT	`LED_H_XMSN_OIL_HOT`	5	C
ENG CHIP	`LED_H_ENG_CHIP`	4	D		HYD PRESS	`LED_H_HYD_PRESS`	5	D
LEFT FUEL BOOST	`LED_H_LEFT_FUEL_BOOST`	4	E		ENG INLET AIR	`LED_H_ENG_INLET_AIR`	5	E
ENG FUEL PUMP	`LED_H_ENG_FUEL_PUMP`	4	F		INST INVERTER	`LED_H_INST_INVERTER`	5	F
RIGHT FUEL BOOST	`LED_H_RIGHT_FUEL_BOOST`	4	G		DC GENERATOR	`LED_H_DC_GENERATOR`	5	G
20 MINUTE FUEL	`LED_H_20_MINUTE_FUEL`	4	DP		EXT POWER	`LED_H_EXT_POWER`	5	DP
FUEL FILTER	`LED_H_FUEL_FILTER`	6	G		CHIP DETECTOR	`LED_H_CHIP_DETECTOR`	7	G
GOV EMER	`LED_H_GOV_EMER`	6	DP		IFF	`LED_H_IFF`	7	DP

4.8 Auto Pilot display

The Auto Pilot panel's character OLED (the PCB net `AP_OLED`, TCA9548A channel 2; `AP_OLED_Port`) sits behind a mask, so each of the five autopilot values has its own window on the 16-column, two-row display:

	Left hand end	Centre	Right hand end
Top row	HDG		VS
Bottom row	CRS	IAS	ALT

HDG and CRS start at column 0, IAS ends at column 8 so it is centred, and VS and ALT end at column 15. At start-up the display shows heading `090`, course `110`, airspeed `80`, vertical speed `0` and altitude `3200`:

```
090             0
110    80    3200
```

(`showAutoPilot()` builds the two lines and `placeLeft()` and `placeRight()` position each value.) The bridges then send the real values as the tokens `APHDG`, `APCRS`, `APIAS`, `APVS` and `APALT` in the radio data packet: heading and course as three digits (north is 360), airspeed in knots, vertical speed in feet per minute (signed) and altitude in feet, each at most 5 characters. `setAutoPilotValueFromToken()` stores them (handled before the per-pair debug line, like the LED tokens) and `updateAP()` redraws the display when one changes; like the COM and NAV displays it goes blank while the radio has no power (`NAVCOM1:0`, or the no-data timeout) and returns with it. The FSUIPC bridge reads the autopilot heading (`0x07CC`), altitude (`0x07D4`), airspeed (`0x07E2`) and vertical speed (`0x07F2`) values and the NAV1 OBS (`0x0C4E`) for the course; the MSFS bridge reads `AUTOPILOT HEADING LOCK DIR`, `NAV OBS:1`, `AUTOPILOT AIRSPEED HOLD VAR`, `AUTOPILOT VERTICAL HOLD VAR` and `AUTOPILOT ALTITUDE LOCK VAR`. The SwingTester's **Auto Pilot display** group sends test values.

4.9 When the USB HID board is missing

Every Audio and Auto Pilot switch and knob click is sent to the USB HID board (172.16.1.11). If the board is switched off or unplugged, the send fails after the W5500's ARP timeout (about 80 ms with the fast retransmission timeout, 1.6 seconds with the chip's default), and that wait used to be paid on every press and release, so the panel felt slow. Now a failed send to the HID board pauses joystick-button sends for **30 seconds** (`HID_RETRY_MS`), so only one press or release in that time pays the wait. The monitor shows `USB HID board 172.16.1.11 not answering (send gave up after N ms)` with the real wait, which also shows whether the fast timeout is working (N should be about 80), and `USB HID board is answering again` on recovery, when the switches that are held down are sent again. Debug sends to a host that does not answer are paused for 10 seconds in the same way (`DEBUG_RETRY_MS`).

4.10 DME display

The Audio panel's character OLED (the PCB net `AUDIO_OLED`, TCA9548A channel 3; the Auto Pilot panel's OLED is channel 2) is the DME display. `DME_OLED_Port` selects the channel. At start-up the sketch starts it with the same OLED sequence as the COM and NAV displays and writes `10M` on the top line and `0M` on the bottom line (`showDme()` pads each line to 16 characters so a shorter text replaces a longer one). Nothing else writes to it yet. It blanks and returns with the COM, NAV and Auto Pilot displays: `updateDme()` clears it while the radio has no power (the avionics bus is below 24 V) or the no-data timeout has fired, and redraws `10M` / `0M` when they come back (it is called wherever `updateAP()` is).

4.11 ADF controller display

The radio also tells the ADF controller (172.16.1.107, port 13136) when its displays blank, so the ADF panel's OLED and landing gear lamps go dark and come back with them. `updateDme()` calls `sendAdfDisplay()`, which sends `C,DISPLAY:0` or `C,DISPLAY:1` on every change and again every 10 seconds (`ADF_DISPLAY_RESEND_MS`, so a restarted ADF controller catches up). The ADF controller may be missing: a failed send pauses these sends for 30 seconds (`ADF_RETRY_MS`) and the monitor shows `ADF controller 172.16.1.107 not answering (send gave up after N ms)` and `ADF controller is answering again`, the same way as the USB HID board, so at most one send in 30 seconds pays the ARP wait.

4.12 Backlight

Mega pin D5 (`BACKLIGHT_PWM`) drives an IRF540N through 330 ohm (10 kohm gate pull-down), which switches the backlights of the **AUDIO, AUTOPILOT and COMM_NAV** panels, using the same rules as the Servo, Stepper and Dual Stepper boards (see the network chapter): the backlight is on only while `BPWR` is 1 (main bus 20 V or more; the radio assumes 1 until the first `BPWR` arrives), at the last brightness commanded with `C,B:0-15` (0 off, 15 full, mapped to PWM 0 to 255) or the default PWM 65 until one has been received. `applyBacklight()` also keeps it off while the displays are blanked by the no-data timeout, which is the same 30 seconds with no UDP packet **and no knob or key use** (a knob or key restarts it, so the radio can be used with no PC), and brings it back at the same level when data or local input resumes. The same `C,B` also sets the panel LEDs' intensity. The bridges send `BPWR` in the radio packet, and the SwingTester's *Backlight power* buttons and brightness slider test it from the bench. The ADF controller's pot sends `C,B` to the radio like any other board.

4.13 Start-up hold

From reset the Radio keeps the backlight, the four OLED displays and every panel LED **on** until the 13 Stepper controller announces that start-up is complete on UDP port 13138 (or 180 seconds after reset); see the network chapter. `radioPowered()` returns true while the hold is on, so the displays and `applyPanelLeds()` treat the radio as powered. When the hold ends `refreshAfterStartupHold()` redraws the displays and the LEDs to their normal state, and the backlight goes off until `BPWR:1`.

4.14 Tuning steps

The knobs follow standard channel spacing. COM runs 118.000 to 136.975 MHz in 25 kHz channels (major knob 118 to 136; minor knob 000, 025, 050, 075). NAV runs 108.00 to 117.95 MHz in 50 kHz steps (major 108 to 117). Each minor knob wraps round at its ends. The constants `COM_MINOR_STEP_KHZ` and `NAV_MINOR_STEP_KHZ` set the steps, and the wrap-around limits follow from them. 8.33 kHz channels are not offered. FSUIPC holds COM frequencies to 10 kHz, so in the simulator a channel such as 118.025 may appear as 118.02 or 118.03.

4.15 Power and blanking

- **Avionics power:** when the avionics bus is below 24 V the bridge sends `NAVCOM1:0` and the COM and NAV displays blank together; they light together again when the bus rises.
- **No data for 30 seconds:** if no packet arrives and no knob or key is used, the COM, NAV, Auto Pilot and DME displays blank once, and the ADF controller is told to blank too. A packet, a knob movement or a key press restores them, and they blank again after another 30 seconds of silence.
- **Quiet period:** for 1.5 seconds (`matrixQuietMs`) after the matrix scanner starts it records state but raises no events, so a floating input cannot send false button presses at boot.

4.16 Diagnostics

- **I2C scan:** at start-up, and whenever the control packet `C,I2CSCAN:1` arrives, the radio scans the Mega's own I2C bus and then each of the eight TCA9548A channels and sends every address that answers to the debug stream: `I2C main bus: 0x70 (TCA9548A)`, `I2C TCA port 0: 0x3C (OLED)`, one line for each channel with a device, `I2C TCA ports with nothing connected: ...` and `I2C scan done, N device(s) answered`. The scan uses a 5 ms I2C timeout so a stuck bus (SDA or SCL held low) is reported as `bus timeout` instead of hanging the board, and it takes a second or two, during which the knobs and keys are not read. It shows at once whether an OLED is missing or wired to the wrong mux channel.
- The control packet `C,MATRIX:1` prints the raw matrix (H or L for each of the five columns, with all rows high and then each row driven low) to the debug stream, bypassing debounce and the quiet period. A column that reads L with nothing pressed is a hardware fault on that Mega pin or board. A faulty or badly seated Mega did exactly this on the bench: all four swap switches share column 0, so none could register, and the board also dropped off USB and the network after flashing.

- Remote reset is a software reset (see the network chapter). Bench-tested: "Ethernet Started" about 4 seconds after the command and "Setup Complete" about 5 seconds.

4.17 PCB features the sketch does not use yet

The Jet Ranger Radio Auto Pilot PCB (v1.4) carries more than the sketch handles. What is still unused:

Mega pins	PCB net	Status in the sketch
A0 to A3	`COMM1_VOL`, `COMM2_VOL`, `NAV1_VOL`, `NAV2_VOL` (volume pots)	Not read
D2 to D4	`I2C_SEL_A2`, `A1`, `A0` (mux address)	Not driven

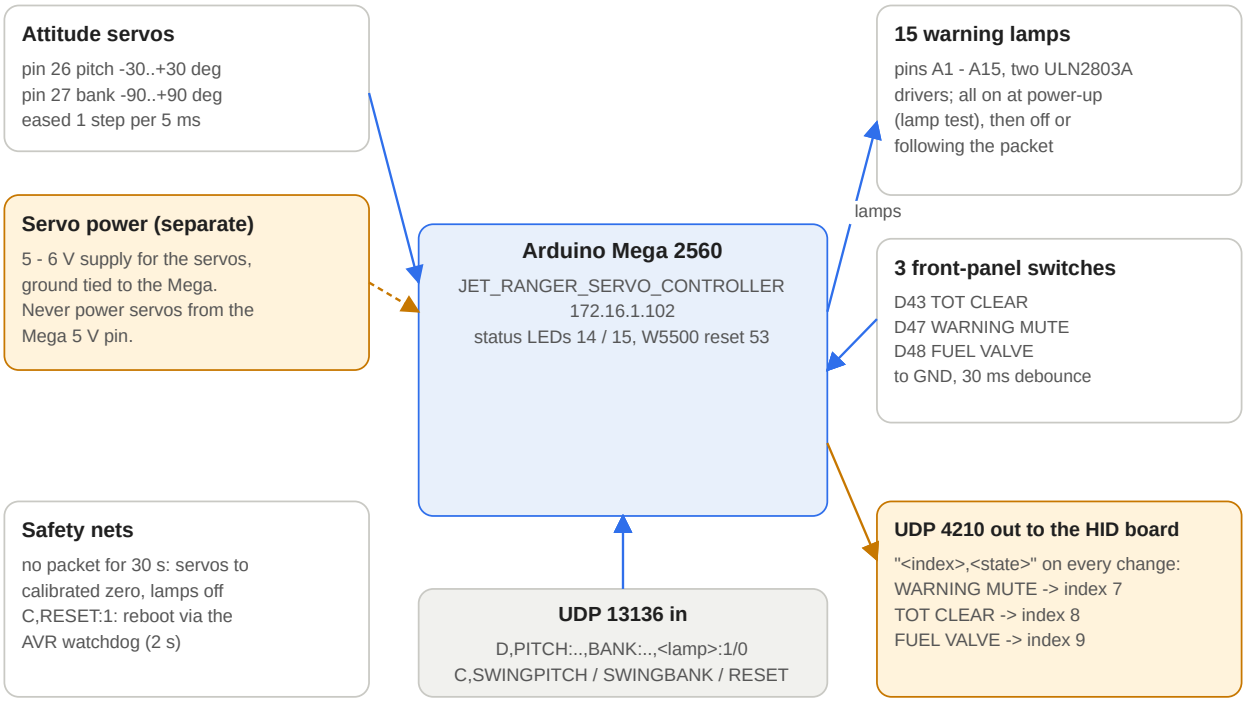
On the same PCB the COM1 major, COM1 minor, COM2 minor and NAV1 minor encoders have their A and B pins the other way round from the sketch (the sketch uses 31, 30 and 33, 32 for COM1, 37, 36 for COM2 minor and 41, 40 for NAV1 minor, where the PCB labels A as 30, 32, 36 and 40). It only matters if one of those knobs tunes backwards.

4.18 Known issues

- Only COM1 and COM2 frequencies arrive from the PC; there are no NAV frequency tokens yet.
- The Radio controller emulator sends a bare frequency for every radio (see the emulator chapter).
- The mux address pins (D2 to D4) are not driven by the sketch; the mux module's own pull-downs set address 0x70.

5 Servo controller (172.16.1.102)

`JET_RANGER_SERVO_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module, on the Jet Ranger Front Panel Servo PCB. It drives the attitude indicator's two servos (pitch and bank), switches the 15 caution and warning lamps and reads three front-panel switches. The other gauges that were once servo-driven (airspeed, vertical speed, torque, temperatures, pressures, fuel and RPM) now run on the two stepper boards.



Joystick 1 buttons 8, 9 and 10 in Windows (the firmware counts from 0). FSUIPC needs them bound to DS206 events.

Calibration: PITCH_DEG_TABLE -30 deg 179, 0 deg 113, +30 deg 70; BANK_DEG_TABLE -90 deg 5, 0 deg 93, +90 deg 179 (servo positions).

Figure 20 — Servo controller blocks

5.1 Program flow

1. **Setup:** set the status LED pins; light all 15 lamps as a lamp test; reset the W5500, start Ethernet and open the sockets (red LED flashes for 3 seconds); sweep pitch and then bank from minimum to maximum and back (about 2 seconds each) and park both at their calibrated zero; switch all lamps off; send the starting state of the three switches to the HID board; log "Setup Complete" and arm the 30 second watchdog.
2. **Main loop:**
    1. toggle the status LEDs every 200 ms and send the `SERVO` heartbeat every 10 seconds;
    2. every 5 ms check UDP 13136 and store each value as a new **target** (servo targets, lamp states);
    3. every 5 ms nudge each servo one step towards its target, so the attitude indicator moves smoothly; a servo that has not moved for 1 second is detached to stop buzz and re-attached on the next movement;
    4. service the three debounced switch inputs.

3. **Watchdog:** after 30 seconds with no packet both servos are eased back to their calibrated zero and all lamps go out. The remembered lamp states are reset as well, so a stale `1` cannot keep a lamp dark when the simulator next sends `1`.

5.2 Servos and calibration

Instrument	Mega pin	Token	Range
Attitude pitch	26	`PITCH`	-30 to +30 degrees
Attitude bank (roll)	27	`BANK`	-90 to +90 degrees

Both tokens take attitude in degrees with one decimal place allowed (for example `D,PITCH:4.5,BANK:-12`). The board converts degrees to a servo position (0 to 180) with two tables, interpolating in a straight line between points and holding the end value beyond them.

Instrument	Attitude	Servo position
Pitch (`PITCH_DEG_TABLE`)	-30 degrees	179
Pitch	0 (level)	113
Pitch	+30 degrees	70
Bank (`BANK_DEG_TABLE`)	-90 degrees	5
Bank	0 (level)	93
Bank	+90 degrees	179

`PITCHRAW` and `BANKRAW` take a raw servo position and skip the tables. The set-up sweep uses `aServMinPosition[]` and `aServMaxPosition[]` (bank 5 to 179, pitch 166 to 70) and parks at `aServZeroPosition[]` (bank 93, pitch 113). The PCB still has 16 servo headers from the earlier design; only pins 26 and 27 are used.

Calibrating a servo:

1. Send raw positions (`D,PITCHRAW:113`) until the instrument shows a known attitude.
2. Note the position for at least both ends and level.
3. Enter them in the table and set `aServZeroPosition[]` to the level position.
4. Upload, then check with real values (`D,PITCH:0`, `D,BANK:30`).

Do not drive a servo hard against its mechanical stop: a stalled servo draws high current and gets hot. Never power servos from the Mega's 5 V pin; use a separate 5 to 6 V supply with a common ground.

5.3 Warning lamps

The 15 warning lamps on the Servo board

Green: the bridge drives it from the simulator. Grey: needs aircraft-specific variables, so it stays off.

ENGINE OUT pin A1 token EOUT engine speed < 60 %	**ROTOR RPM LOW** pin A2 token RLOW rotor <= 95 %	**XMSN OIL PRESS** pin A3 token TOPW pressure < 40 psi
XMSN OIL TEMP pin A4 token TOTW always off	**BATTERY HOT** pin A5 token BHOT always off	**BATTERY TEMP** pin A6 token BTMP always off
ENGINE CHIP pin A7 token EC always off	**TAIL ROTOR CHIP** pin A8 token TRC always off	**XMSN CHIP** pin A9 token TC always off
BAGGAGE DOOR pin A10 token BD always off	**AFT FUEL FILTER** pin A11 token FFLTR always off	**FUEL PUMP** pin A12 token FPMP always off
GENERATOR FAIL pin A13 token GENF engine speed < 10 %	**LOW FUEL** pin A14 token LOWF fuel <= 17 gal	**SC FAIL** pin A15 token SCF always off

All lamps go out when the main bus is below 20 V. Sent as <token>:1 / <token>:0 in every Servo packet.
The lamp test at power-up lights all 15; a no-data timeout turns them all off.

Figure 21 — The 15 lamps and their Mega pins

Token	Lamp	Pin
`EOUT`	Engine out	A1
`RLOW`	Rotor RPM low	A2
`TOPW`	Transmission oil pressure	A3
`TOTW`	Transmission oil temperature	A4
`BHOT`	Battery hot	A5
`BTMP`	Battery temperature	A6
`EC`	Engine chip	A7
`TRC`	Tail rotor chip	A8
`TC`	Transmission chip	A9
`BD`	Baggage door	A10
`FFLTR`	Aft fuel filter	A12
`FPMP`	Fuel pump	A11
`GENF`	Generator fail	A13
`LOWF`	Low fuel	A14
`SCF`	SC fail (standby compass)	A15

A lamp pin is HIGH when lit (through a driver for lamps above 20 mA). The power-up lamp test lights every lamp, so it cannot show a mis-wired pin: send one lamp at a time (for example `D,RLOW:1`) to check each one.

5.4 Debounced switch inputs

Three front-panel switches are wired between a Mega pin and GND on the Front Panel Servo PCB and read with the internal pull-up (closed = pin LOW). A new reading must stay unchanged for 30 ms (`InputDebounceMs`) before it counts.

Pin	Switch	PCB net	HID index	Joystick 1 button in Windows
D43	TOT CLEAR (SW1)	`/SW_TOT_CLEAR`	8	9

Pin	Switch	PCB net	HID index	Joystick 1 button in Windows
D47	WARNING MUTE (SW3)	`/SW_WARNING_MUTE`	7	8
D48	FUEL VALVE (SW2)	`/SW_FUEL_VALVE`	9	10

Every accepted change is logged to the debug stream (for example `Jet Ranger Servo: WARNING MUTE CLOSED -> HID button 7`) and sent to the USB HID board at `172.16.1.11:4210` as `index,state` with 1 = closed. The HID board counts buttons from 0 and Windows from 1. The starting state of each switch is sent at the end of `setup()`; there is no periodic resend, so a HID board that boots later stays out of step until the switch next changes. Nothing is sent to the PC itself.

5.5 On-demand commands

- `C,SWINGPITCH:1` and `C,SWINGBANK:1` run one sweep (the value is ignored); the board pauses for about 2 seconds.
- `C,RESET:1` reboots the board through the AVR watchdog (bench-tested: "Ethernet Started" 6.9 seconds later, "Setup Complete" about 12.8 seconds after the command).

5.6 Backlight

Pin 5 (the PCB's `BACKLIGHT_PWM`) drives the panel backlights through the board's IRF540N MOSFET. The level follows `BPWR`, the last `C,B:0-15` brightness command and the 30 second no-data watchdog (see the network chapter): with no command the default PWM 65 is used, and once one has been received the last commanded level returns whenever power returns. After 30 seconds with no packet, including after power-up with no PC feed, the backlight goes off with the lamps and returns with the next packet. `setup()` sets pin 5 as an output and applies the default level; each change is logged (`Brightness 15 -> PWM 255`, `Backlight power off`, `Backlight restored - UDP data resumed`). Compiled at 24,468 bytes of flash (9%) and 30% RAM; flashed to the Servo board on COM16.

Start-up hold. From reset the backlight and all 15 warning lamps stay **on** (`lampWrite()` writes HIGH to every lamp pin) until the 13 Stepper controller announces on UDP port 13138 that start-up is complete, or 180 seconds after reset (see the network chapter). Then the backlight goes off until `BPWR:1`, and `refreshAfterStartupHold()` puts every lamp back to the state the bridge last sent for it.

5.7 Pin usage

Pin	Function
14	Green status LED
15	Red status LED
26, 27	Pitch and bank servos
5	Backlight PWM (`BACK_LIGHTS`, to the MOSFET gate through 330 ohm)
43, 47, 48	TOT clear, warning mute and fuel valve switches
53	W5500 reset
A1 to A15	Warning lamps
50 to 52	SPI to the Ethernet module

5.8 Known issues

- `powerAvailable` is set from `NAVCOM1` but nothing uses it, so the attitude indicator and lamps stay live when the simulator's power is off (the bridge itself sends all lamps off and `BPWR:0` below 20 V).
- Several lamps log "Transmission Temperature changed" to the debug stream whichever lamp changed.
- The `#define` comments for pins 26 and 27 still describe the old servo layout.
- The PCB labels 10 of the 15 lamp outputs on different Mega pins from the sketch (A1/A2, A5/A6, A7 to A10, A11/A12). Test each lamp on its own and correct whichever side is wrong.

6 Upper panel (now on the ADF controller)

The overhead switch matrix and the small numeric keypad used to have their own board, `JET_RANGER_UPPER_CONTROLLER` (172.16.1.103, the **Jet Ranger Upper Controller** PCB). On 31 October 2026 that sketch was **merged into the ADF controller sketch** and runs on the combined **Jet Ranger Upper ADF Controller** PCB at **172.16.1.107**; the pins, scan method, backlight rules and known issues are in the ADF and Upper controller chapter. What stays the same for everything downstream:

- each matrix switch is still sent to the USB HID board as `index,state` with `index = row x 11 + column`, so the joystick button numbers (0 to 95, Joystick 1 to 3) and the table below are unchanged. The matrix is now 9 rows by 8 columns (72 positions, indexes 0 to 95) instead of 16 by 11 (176 positions);
- the keypad is still sent as `KEY,key,state`;
- the remote reset (`C,RESET:1`), `C,B` brightness, `C,KEEPALIVE:1` and `D,BPWR` are handled by the ADF controller's code, which has the same rules;
- the Health Monitor's Upper light is fed by an `UPPER_INPUT` heartbeat that the merged board sends next to its `ADF_PANEL` one.

The old sketch (`JET_RANGER_UPPER_CONTROLLER.ino`) is retired but still in the repository for reference. Its DCS-BIOS code (`CreateDcsBiosMessage()`, every case empty) was not carried over, so nothing here is sent to DCS World. The old Upper board's remote reset and backlight code were never bench-tested because the board was not connected.

7 USB HID board (172.16.1.11)

`UDP_TO_HID` runs on an Arduino with native USB (Leonardo class, board option `arduino:avr:leonardo`) and a W5500 Ethernet module. It is a network-to-USB bridge: it receives button and key commands over UDP from the Servo controller, the Radio controller (Auto Pilot and Audio panels) and the ADF and Upper controller (the ADF panel switches and knobs, the Upper switch matrix and the Upper keypad) and re-emits them to Windows as **four virtual 32-button joysticks (128 buttons) and a keyboard**. The joysticks have buttons only: no axes and no hat switches.

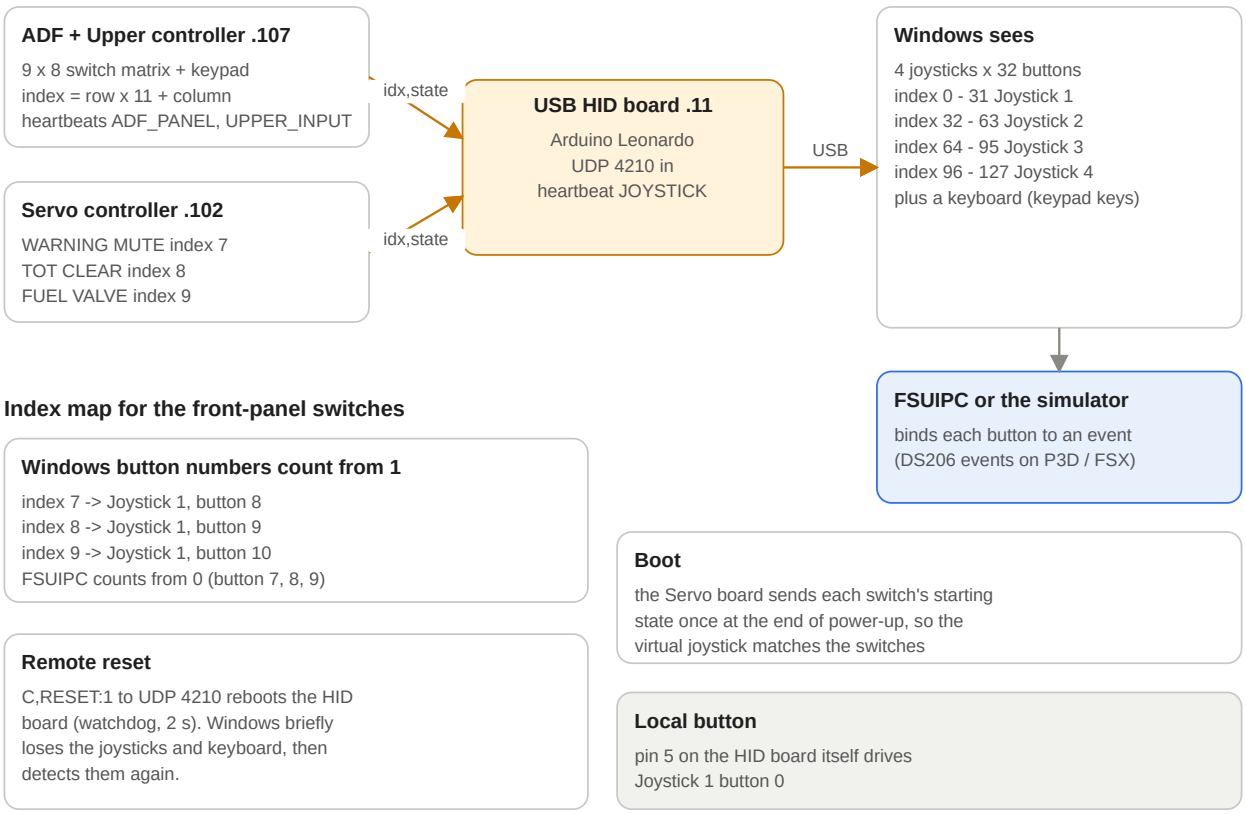


Figure 22 — From switch to joystick button

7.1 Program flow

1. **Setup:** start the four joysticks and the keyboard, bring up Ethernet on the static address, open the command, log and heartbeat sockets and log a boot message.
2. **Main loop:**
   1. send the `JOYSTICK` heartbeat every 10 seconds;
   2. read the one local push button on pin 5 and mirror it on Joystick 1, button 0;
   3. read any UDP command on port 4210, log the sender and payload and call `processCommand()`.

`processCommand()` understands three formats:

Payload	Effect
`index,state`	Sets button `index mod 32` of joystick `index div 32` (0 to 31 Joystick 1, 32 to 63 Joystick 2, 64 to 95 Joystick 3, 96 to 127 Joystick 4) pressed (1) or released (0)
`KEY,key,state`	Presses or releases a keyboard key. A single character is sent as its ASCII value; longer names (`ENTER`, `F5`, `LCTRL`) map to key codes
`C,RESET:1` (or `RESET`)	Reboots the board through the AVR watchdog; Windows briefly loses the joysticks and keyboard

7.2 How a switch becomes a joystick button

1. The Upper matrix (on the ADF and Upper controller) numbers each position as `index = row x 11 + column` (0 to 95: 9 rows by 8 columns).
2. When a switch changes it sends `index,state` to `172.16.1.11:4210` with state 1 = switch on.
3. The HID board holds that button down while the switch is on.
4. Keypad keys are sent as `KEY,<key>,<state>` and typed as keyboard keys 0 to 9, `*` and `#`.
5. The Servo controller sends the same format for its three front-panel switches (indexes 7, 8 and 9).

Index range	Windows device
0 to 31	Joystick 1
32 to 63	Joystick 2
64 to 95	Joystick 3
96 to 127	Joystick 4
128 and above	Not sent to Windows (out of range); nothing sends these any more

7.3 Auto Pilot, Audio and ADF panel buttons

The Radio controller (Auto Pilot and Audio panels) and the ADF controller send their switches and knob clicks in **Joystick 3** (indexes 64 to 95, Auto Pilot panel) and **Joystick 4** (96 to 127, Audio and ADF panels). Nothing else uses those buttons: the Upper matrix's assigned switches are all below index 40, and the Servo's are 7 to 9. The Upper matrix positions in rows 6 to 8 (indexes 66 to 73, 77 to 84 and 88 to 95) fall inside Joystick 3 and are wired but have no switch on them (88 and 89 are the Warning Mute and TOT Test connectors), so if switches are ever fitted at 66 to 73 or 77 to 81 they would clash with the Auto Pilot buttons. The matrix no longer reaches the Audio and ADF range (96 and above). Windows numbers buttons from 1 (button = index mod 32 + 1). A knob click is a 40 ms press, clockwise and anticlockwise on adjacent buttons.

Index	Windows button	Source
64 to 71	Joystick 3, buttons 1 to 8	Auto Pilot panel switches: ENGAGE, HDG, CRS, ALT, APPR, FD, BC, IAS
72 to 81	Joystick 3, buttons 9 to 18	Auto Pilot knobs, clockwise then anticlockwise: CRS, ALT, VS, IAS, HDG
96 to 103	Joystick 4, buttons 1 to 8	Audio panel switches: COM1, BOTH, NAV2, DME, COM2, NAV1, MKR, ADF
104 to 107	Joystick 4, buttons 9 to 12	ADF panel: landing gear, flaps, speed brakes, brakes
108 to 111	Joystick 4, buttons 13 to 16	ADF knobs, clockwise then anticlockwise: major, minor
112 to 114	Joystick 4, buttons 17 to 19	ADF panel: anti-ice, hyd boost, caution dim

Each of these is also sent to the bridge as a simulator control, so binding the joystick buttons in FSUIPC or MSFS is optional (see the Radio and ADF controller chapters for the full table).

7.4 Switch mapping in use

From `UpperSwitchMapping.ods`. Button numbers are as Windows and the simulators show them (1 to 32); the firmware counts from 0. A latching switch holds its button down while on, so bind both press and release. The suggested events are the closest match in `DODSIM206V2.EVT`; the bindings actually saved in FSUIPC or MSFS are not in the repository. All other matrix positions are wired but unassigned.

Index	Source	Switch	Joystick 1 button	Suggested DodoSim event
0 to 4	Upper 0, 0-4	Heater rotary HTR 0 to 4	1 to 5	
5	Upper 0, 5	Air vent	6	`HEAT_VENT_SWITCH_SET`
6	Upper 0, 6	Air heat	7	`HEAT_VENT_SWITCH_SET`
7	Servo D47	Warning mute	8	`DS206.WARNING_MUTE_BUTTON_SET`
8	Servo D43	TOT clear (TOT light test)	9	`DS206.TOT_TEST_BUTTON_SET`
9	Servo D48	Fuel valve	10	`DS206.FUEL_VALVE_SWITCH_SET`
11	Upper 1, 0	Forward fuel boost breaker	12	`FORE_FUEL_PUMP_CB_SET`
12	Upper 1, 1	Aft fuel boost breaker	13	`AFT_FUEL_PUMP_CB_SET`
13	Upper 1, 2	Generator field breaker	14	`GENERATOR_FIELD_CB_SET`

Index	Source	Switch	Joystick 1 button	Suggested DodoSim event
14	Upper 1, 3	Generator breaker	15	`GENERATOR_RESET_CB_SET`
15	Upper 1, 4	Directional gyro / attitude indicator	16	`DIR_GYRO_SWITCH_SET`
16	Upper 1, 5	Battery	17	`BATTERY_SWITCH_SET`
17	Upper 1, 6	Generator (on / start)	18	`GENERATOR_SWITCH_SET`
22	Upper 2, 0	Landing light	23	`LANDING_LIGHT_SWITCH_SET`
23	Upper 2, 1	Instrument lights	24	`PANEL_LIGHTS_SWITCH_SET`
24	Upper 2, 2	Caution light breaker	25	`CAUTION_LT_CB_SET`
25	Upper 2, 3	Anti-collision light	26	`COLLISION_LIGHTS_SET`
26	Upper 2, 4	Position lights	27	`POSITION_LIGHTS_SET`
27	Upper 2, 5	Pitot heat	28	`PITOT_HEAT_SWITCH_SET`
28	Upper 2, 6	Avionics	29	`AVIONICS_SWITCH_SET`
39	Upper 3, 6	Strobe light	Joystick 2, 8	none yet

7.5 Pin usage and network

Item	Value
Pin 5	Local push button, Joystick 1 button 0
Static IP, MAC	`172.16.1.11`, `DE:AD:BE:EF:FE:ED`
Listen port	4210 (button, key and reset commands)
Log to	172.16.1.10:27000 (boot, button and key events, malformed commands)
Heartbeat to	172.16.1.10:13137, `JOYSTICK` every 10 seconds

The Leonardo is flashed on COM8 with `--fqbn arduino:avr:leonardo`. Bench-tested remote reset: "Joystick controller booted" about 4 seconds after the command.

7.6 Known issues

- The board's own test button on pin 5 also drives Joystick 1 button 1, the same button as HTR 0.
- The Servo controller's three switches use indexes 7, 8 and 9, which Windows shows as buttons 8, 9 and 10. If Windows buttons 7, 8 and 9 were wanted, button 7 is already the Air heat switch (index 6), so the indexes would have to move.
- After a reset the USB connection drops; re-check any game that has the joysticks open.

8 Stepper controller (172.16.1.105)

`JET_RANGER_STEPPER_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module, on the Jet Ranger 13 Stepper PCB. It drives X27-style stepper gauges directly from the Mega's pins (no driver chips) and takes **real units**, converting them to needle steps with calibration tables in the sketch. The oil, transmission, RPM, load and altimeter gauges are on the Dual Stepper board (next chapter).

The sketch started as an A-10C front instrument board and still carries some DCS-BIOS code and an unused SARI roll stepper claim on pins 30 and 32; the Jet Ranger path is the UDP protocol.

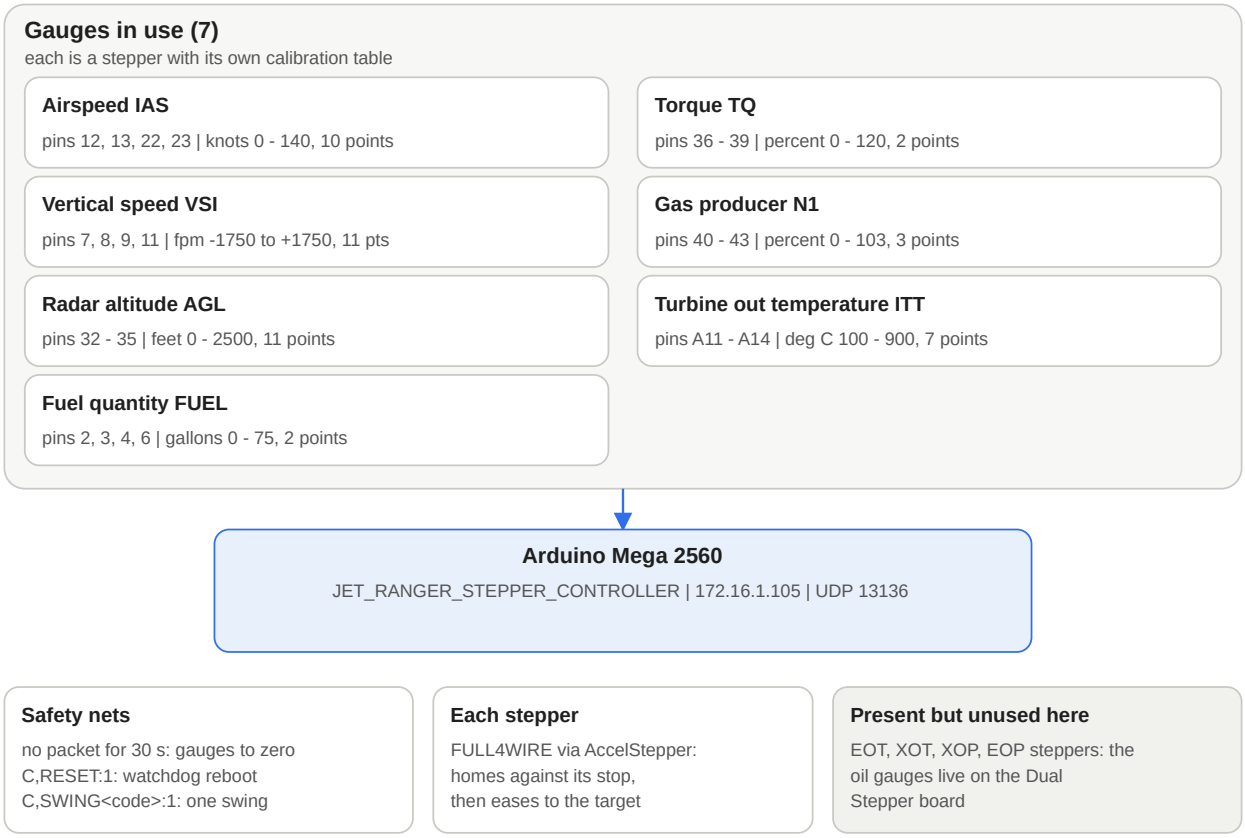


Figure 23 — Stepper controller blocks

8.1 Gauges and calibration

Gauge	Token	Mega pins	Unit and range	Calibration
Airspeed	`IAS`	12, 13, 22, 23	knots 0 to 140, one decimal allowed	`IAS_KT_TABLE`, 10 points (0 to 635 steps)
Vertical speed	`VSI`	7, 8, 9, 11	ft/min -1750 to +1750	`VSI_FPM_TABLE`, 11 points (-311 to +315 steps)
Radar altitude	`AGL`	32 to 35	feet 0 to 2500	`AGL_FT_TABLE`, 11 points (0 to 469 steps)
Torque	`TQ`	36 to 39	percent 0 to 120	`TQ_PCT_TABLE`, 2 points (0 to 600 steps)
Gas producer (N1)	`N1`	40 to 43	percent 0 to 103	`GP_PCT_TABLE`, 3 points (0 percent is step 15)
Turbine out temperature	`ITT`	A11 to A14	degrees C 100 to 900	`EGT_C_TABLE`, 7 points (100 C is step 0)
Fuel quantity	`FUEL`	2, 3, 4, 6	US gallons 0 to 75	`FA_GAL_TABLE`, 2 points (0 gal is step 53)

Notes:

- The first IAS row (0 kt = step 0) is an assumed zero. The VSI and IAS steppers are wired C, D, A, B.

- Every token has a raw twin (`IASRAW:300`) that moves the needle by raw steps and skips the table.
- Four more steppers are declared with placeholder linear scales and no start-up swing: engine oil temperature (`OILT`, pins 48, A0 to A2), transmission oil temperature (`XMSNT`, A3 to A6), transmission oil pressure (`XMSNP`, A7 to A10) and engine oil pressure (`OILP`, 44 to 47). The oil gauges are really driven by the Dual Stepper; wire each physical gauge to one board only.
- `RPME` and `RPMR` packets sent to this board are ignored.
- **Zero trims:** each gauge has a `<CODE>ZeroOffset` value (steps) to nudge its true zero without touching its table. Current values: `AGLZeroOffset` 10, `TQZeroOffset` 23, all others 0. The trim is applied in each gauge's real-value setter, so a `<CODE>:0` packet, the watchdog reset and the end of each swing all land on the same trimmed position. The `*RAW` codes do not apply it.

8.2 Program flow

1. **Setup:** flash the status LEDs, bring up Ethernet (10 ms retransmission timeout), ramp the backlight, then run each gauge's boot swing in this order, each behind its own `SwingXXX` gate: fuel quantity, torque, turbine out temperature, gas producer, VSI, airspeed, radar altitude. All seven gates are currently `true` (`SwingLoops` is 2). The 30 second watchdog is armed at the end of `setup()`.
2. **Main loop:** toggle the status LEDs, call `.run()` on every stepper, send the `STEPPER` heartbeat every 10 seconds, and every 5 ms check UDP 13136 and store new targets. Steppers move with smooth acceleration (maximum speed 9000, acceleration 1000).
3. **Watchdog:** after 30 seconds with no packet every gauge is driven to its calibrated zero through its own setter (trim included).

8.3 Packets

The bridges send this board its own packet:
`D,BPWR:1,IAS:93.4,VSI:500,AGL:850,FUEL:42,TQ:65,ITT:610,N1:92`. `AGL` is zero below 20 V on the main bus. The fractional part of `IAS` is interpolated between table rows, so the needle moves smoothly rather than in whole-knot jumps.

8.4 Start-up announcement

The Stepper controller is the last to finish its start-up. When `setup()` ends it ends its own start-up hold and sends the text `STARTUPDONE` to UDP port 13138 of the Radio (.101), Servo (.102), Dual Stepper (.106) and ADF and Upper (.107) controllers (`announceStartupDone()`), and repeats it every 10 seconds, 18 times, in case a board was not listening yet or was reset. Those boards then turn their start-up backlight, OLEDs and lamps over to the normal rules (see the network chapter). The debug line is `Start-up complete: told the other controllers to turn their backlighting off`.

8.5 Backlight

Pin 5 (the PCB's `BACKLIGHT_PWM`) drives the panel backlights through the board's IRF540N MOSFET (`BACK_LIGHTS`). `applyBacklight()` sets the level from `BPWR`, the last `C,B:0-15` brightness command and the 30 second no-data watchdog (see the network chapter): the default is PWM 65; once a brightness command has been received the last commanded level returns whenever power returns; `BPWR:0` or 30 seconds with no packet (including after power-up with no PC feed) switches it off, and the next packet switches it back on. The backlight still starts at 128 during setup. The old DCS-BIOS brightness handlers still call `analogWrite()` directly but never run. Each change is logged (`Brightness 15 -> PWM 255`, `Backlight power off`, `Backlight restored - UDP data resumed`). Compiled at 31,544 bytes of flash (12%) and 50% RAM; flashed to the Stepper on COM4.

8.6 On-demand swings

Send `C,<COMMAND>:1` (the value is ignored) to run one gauge's swing without rebooting. The swing blocks the loop for about 10 seconds and re-homes the stepper, so the needle then rests at its calibrated zero until the next packet.

Command	Gauge
`SWINGFUEL`	Fuel quantity
`SWINGTQ`	Torque
`SWINGITT`	Turbine out temperature
`SWINGN1`	Gas producer
`SWINGVSI`	Vertical speed
`SWINGIAS`	Airspeed
`SWINGAGL`	Radar altitude

The airspeed and radar altitude swings return to zero after the full-scale wind-up instead of 635 steps past it.

`C,B:0-15` sets the backlight brightness (above). `C,RESET:1` reboots the board through the AVR watchdog (bench-tested: "Ethernet Started" 6.6 seconds later, a normal boot, no loop).

8.7 Pin usage

Pin	Function
14, 15	Green and red status LEDs
53	W5500 reset
5	Backlight PWM (`BACK_LIGHTS`; the PCB routes `BACKLIGHT_PWM` to D5 and VSI coil B to D8)
7, 8, 9, 11	VSI coils
12, 13, 22, 23	Airspeed coils
32 to 35	Radar altitude coils
36 to 39	Torque coils
40 to 43	Gas producer coils
A11 to A14	Turbine out temperature coils
2, 3, 4, 6	Fuel quantity coils
48, A0 to A2	Engine oil temperature coils
A3 to A6	Transmission oil temperature coils
A7 to A10	Transmission oil pressure coils
44 to 47	Engine oil pressure coils
24 to 31	Unused (Turbine and Rotor speed moved to the Dual Stepper)

8.8 Network

Static IP `172.16.1.105`, MAC `A8:61:0A:67:83:69`, data port 13136, heartbeat `STEPPER` to 172.16.1.10:13137, debug to 172.16.1.10:27000. The StepperVSITester can send every token and raw token to this board without a simulator.

8.9 Known issues

- Pin 32 is claimed by both the radar altitude coil and the unused SARI roll stepper; pin A2 (56) is claimed by the engine oil temperature coil and the shared stepper enable pin. Neither SARI nor the enable pin is used.
- `VSIstepper.runToNewPosition(-X27_FULLWIRE_HOMING_STEPS)` expands to -635 + 5 = -630, not -640, because the macro is not parenthesised (about 1.5 percent).
- The placeholder oil and transmission gauges use a shared linear scale, not bench-measured tables.

9 Dual stepper controller (172.16.1.106)

`JET_RANGER_OLED_DUAL_STEPPER_CONTROLLER` runs on a third Mega 2560, on the Jet Ranger OLED Dual Stepper PCB. It drives stepper gauges for engine and transmission oil, engine and rotor RPM, fuel load, electrical load and the altimeter, plus **four small OLED displays**. Like the Stepper board it takes real units. It started as a copy of the Stepper sketch, which is why the two share their structure, their zero-trim approach and most of their code.

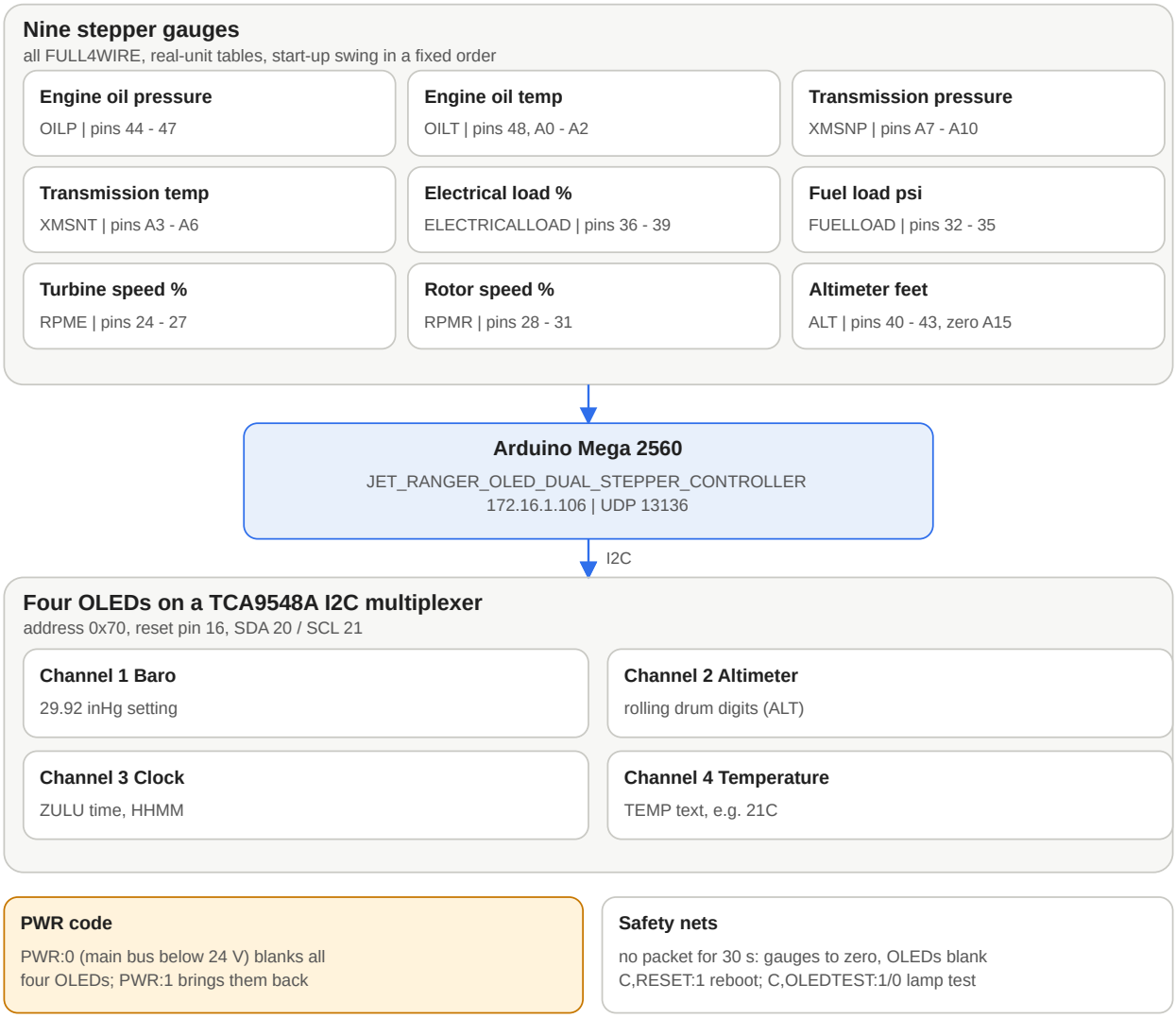


Figure 24 — Dual stepper controller blocks

9.1 Gauges and calibration

Gauge	Token	Mega pins	Unit	Calibration
Engine oil pressure	`OILP`	44 to 47	psi	`EOP_PSI_TABLE`: 0, 50, 100, 150 psi = 0, 74, 148, 220 steps
Engine oil temperature	`OILT`	48, A0 to A2	degrees C	`EOT_C_TABLE`: 0, 50, 100, 150 C = 0, 79, 156, 231 steps
Transmission oil pressure	`XMSNP`	A7 to A10	psi	`XOP_PSI_TABLE`: 0, 50, 70, 100, 150 psi = 0, 74, 104, 149, 220 steps
Transmission oil temperature	`XMSNT`	A3 to A6	degrees C	`XOT_C_TABLE`: 0, 50, 100, 150 C = 0, 75, 150, 225 steps
Electrical load	`ELECTRICALLOAD`	36 to 39	percent	`ELECTRICAL_LOAD_PCT_TABLE`: 0

Gauge	Token	Mega pins	Unit	Calibration
				percent = step 22, 100 percent = step 200
Fuel load	FUELLOAD	32 to 35	psi	FUEL_LOAD_PSI_TABLE: 0 psi = 0, 30 psi = 160 steps
Engine speed (N2)	RPME	24 to 27	percent, one decimal	TS_PCT_TABLE: 0, 55, 100, 110 percent = 0, 300, 535, 600 steps
Rotor speed	RPMR	28 to 31	percent, one decimal	RS_PCT_TABLE: the same values
Altimeter	ALT	40 to 43, zero sensor A15	feet	720 steps per 1000 ft

Turbine out temperature (ITT) and fuel quantity (FUEL) are also accepted, with the same tables as the Stepper board; wire each physical gauge to one board only. VSI, IAS, AGL, TQ and N1 are not handled here.

- Every token has a raw twin (FUELLOADRAW, RPMERAW, ALTRAW and so on).
- The 0 percent point of the RPM tables is assumed, and the 0 point of the electrical-load table is step 22, not zero.
- **Zero trims (steps):** TSoffset 10, RSoffset 0, EOTZeroOffset 24, EOPZeroOffset 0, XOTZeroOffset 16, XOPZeroOffset 5, FuelLoadZeroOffset 55, ElectricalLoadZeroOffset 24. The altimeter has none.
- The altimeter seeks its zero sensor on A15 by moving up to 10,080 steps (about 14 turns) before giving up, then sets position -25 and sweeps one 720 step revolution. The altimeter's own speed is 600 steps per second.

9.2 OLED displays

The diagram above shows the four displays on the multiplexer. Four 128 by 32 SSD1306 OLEDs share the Mega's I2C pins (20 SDA, 21 SCL) through a TCA9548A multiplexer at address 0x70. The firmware pulses the mux reset (pin 16) at start-up, calls Wire.begin() with a 25 ms timeout and probes the mux; if it does not answer the sketch logs "TCA mux NOT found - OLED init skipped" and starts the swings at once, then retries once after the swings.

Display	Mux channel	Token	Shows
Barometric setting	1	none yet	2992 at boot, not updated
Altimeter digits	2	ALT	Rolling-drum altitude, raw MSL (barometric correction ignored)
Clock	3	ZULU	Zulu time from HHMM, for example ZULU:1430 shows 14:30
Temperature	4	TEMP	The text as sent, for example TEMP:25C (up to 8 characters)

Updates are gated on change and throttled to every 300 ms. A boot-time I2C scan (OLED_I2C_SCAN) logs one line per channel to the debug stream, for example I2C scan ch1: 0x3c or I2C scan ch3: none; the latest scan found OLEDs on channels 1 to 4. The TCA9548A address pins (D17 to D19) are not driven by the sketch: the module's own pull-downs set address 0x70, so keep a module with pull-downs.

Power: PWR:0 (sent first in the bridge's packet while the main bus is below 24 V) blanks all four OLEDs and PWR:1 redraws them. The altimeter needle keeps following altitude. The lamp test ignores PWR.

Lamp test: C,OLEDTEST:1 fills all four OLEDs solid white and C,OLEDTEST:0 redraws their default content and clears the change gates so live data repaints them. It is ignored (with a debug line) if the mux was not found.

9.3 Start-up hold

From reset the backlight and the four OLEDs stay **on** (setOledsPower() ignores PWR:0 and BlankAllOleds() is not called by the no-data timer) until the 13 Stepper controller announces on UDP port 13138 that start-up is complete, or 180 seconds after reset. Then refreshAfterStartupHold() applies the last PWR state and, if no data has arrived for more than 2 seconds, expires the 30 second no-data timer so the OLEDs blank together with the backlight (this board's own timer starts only when its long setup ends, which is why the OLEDs used to stay on about 5 seconds longer than the other boards').

9.4 Backlight

Pin 5 (the PCB's `BACKLIGHT_PWM`) drives the panel backlights through the board's IRF540N MOSFET (`BACK_LIGHTS`). `applyBacklight()` sets the level from `BPWR`, the last `C,B:0-15` brightness command and the 30 second no-data watchdog (see the network chapter): the default is PWM 65; once a brightness command has been received the last commanded level returns whenever power returns; `BPWR:0` or 30 seconds with no packet (including after power-up with no PC feed) switches it off, and the next packet switches it back on. The backlight still starts at 128 during setup. The old DCS-BIOS brightness handlers still call `analogWrite()` directly but never run. Each change is logged (`Brightness 15 -> PWM 255`, `Backlight power off`, `Backlight restored - UDP data resumed`). Compiled at 58,312 bytes of flash (22%) and 79% RAM (1,720 bytes free); flashed to the Dual Stepper on COM5.

9.5 Program flow

1. **Setup:** initialise the OLEDs first so they are up while the needles swing, then run each gauge's swing in this order, each behind its own `SwingXXX` gate (all currently `true`, `SwingLoops` 1): engine oil pressure, engine oil temperature, transmission oil pressure, transmission oil temperature, electrical load, fuel load, engine speed, rotor speed, altimeter. Start-up takes about 65 seconds. The 30 second watchdog is armed at the end of `setup()`.
2. **Main loop:** run the steppers, send the `DUAL_STEPPER` heartbeat every 10 seconds and check UDP 13136 every 5 ms.
3. **Watchdog:** after 30 seconds without a packet every needle returns to its calibrated zero and all OLEDs blank.

9.6 Packets

Bridge packet:
`D,PWR:1,BPWR:1,ALT:1200,RPMR:100.0,RPME:100.0,ZULU:1430,OILT:60,OILP:55,XMSNT:50,XMSNP:35,FUELLOAD:17,ELECTRICALLOAD:50,TEMP:21C`. Rotor and engine speed carry one decimal so the fraction reaches the table interpolation.

9.7 On-demand commands

Command	Action
`SWINGOILP`, `SWINGOILT`, `SWINGXMSNP`, `SWINGXMSNT`	Oil and transmission gauges (about 5 to 6 seconds each)
`SWINGELECTRICALLOAD`, `SWINGFUELLOAD`, `SWINGRPME`, `SWINGRPMR`	Load and RPM gauges (about 6 to 7 seconds each)
`SWINGALT`	Altimeter (re-finds the zero sensor)
`OLEDTEST:1` / `OLEDTEST:0`	OLED lamp test on and off
`B:0` to `B:15`	Backlight brightness
`RESET:1`	Watchdog reboot (bench-tested: "Ethernet Started" 6.7 seconds later and a normal boot in about 56 seconds)

All are sent as `C,<COMMAND>:1`. The oil, transmission, load and RPM swings wind to the stop and return to the zero offset; the electrical-load swing returns to its 22 step zero.

9.8 Pin usage

Pin	Function
14, 15	Status LEDs
16	TCA9548A reset
20, 21	I2C SDA, SCL
53	W5500 reset
5	Backlight PWM (PCB `BACKLIGHT_PWM`)
24 to 27	Engine speed coils
28 to 31	Rotor speed coils

Pin	Function
32 to 35	Fuel load coils
36 to 39	Electrical load coils
40 to 43, A15	Altimeter coils and zero sensor
44 to 47, 48, A0 to A2	Oil pressure and temperature coils
A3 to A10	Transmission temperature and pressure coils
2, 3, 4, 6, A11 to A14	Fuel quantity and turbine out temperature coils

9.9 Network

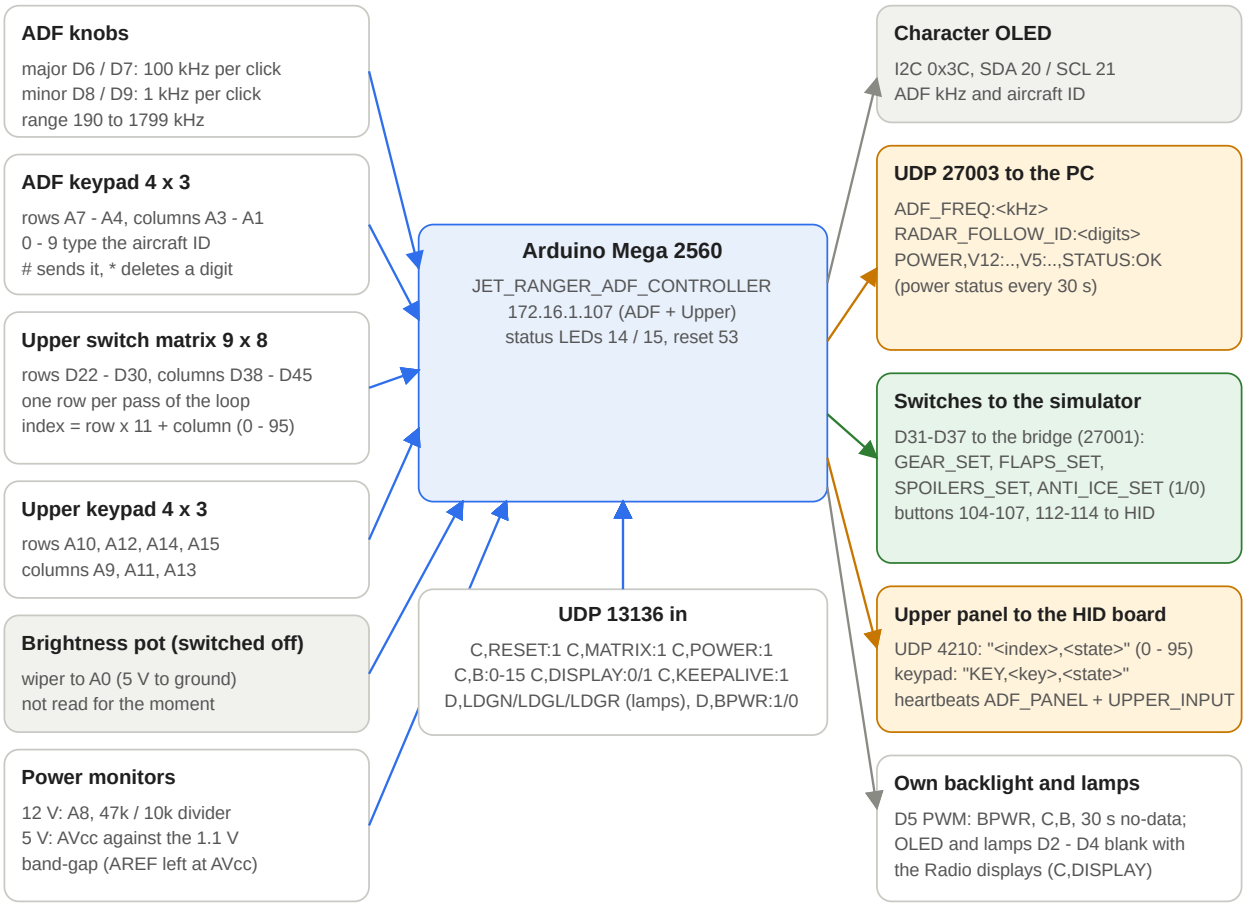
Static IP `172.16.1.106`, MAC `A8:61:0A:67:83:6A`, data port 13136, heartbeat `DUAL_STEPPER` (a prefix distinct from the Stepper's `STEPPER`, so the Health Monitor can tell them apart) to 172.16.1.10:13137, debug to 172.16.1.10:27000 including the I2C scan.

9.10 Known issues

- **RAM use is 79 percent** and the compiler warns "Low memory available". Avoid adding OLEDs or string-heavy code.
- The barometric OLED is initialised but never updated; nothing feeds it.
- The altimeter digits ignore barometric offset.
- Engine and rotor RPM tables assume 0 percent is 0 steps; `RSoffset` is not measured.
- The Clock OLED (channel 3) answered on the I2C scan but its display has not been confirmed by eye.
- **PCB version 1.4 (12 Oct 2026)** removes a short between Mega pins 2 and 32 (a copied net name joined fuel quantity coil A and fuel load coil A in copper). On a board older than 1.4 fit only one of those two gauges or cut the trace.

10 ADF and Upper controller (172.16.1.107)

`JET_RANGER_ADF_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module, on the **Jet Ranger Upper ADF Controller** PCB (revision 1.0, 26 October 2026), the combined ADF and Upper board, which is now the reference PCB for this sketch. On 31 October 2026 the former Upper controller sketch (`JET_RANGER_UPPER_CONTROLLER`, 172.16.1.103) was merged into it, so this one board (172.16.1.107) does the work of both old boards: it also scans the **overhead switch matrix** and the **Upper numeric keypad** and sends them to the USB HID board (see Upper panel matrix and keypad). The old Upper sketch is still in the repository but is retired, and its DCS-BIOS stubs (every case was empty) were not carried over. The sketch is built from the Radio controller sketch (same Ethernet start-up, debug stream, heartbeat, polling encoder library, matrix scanner, character OLED start-up and software reset) and does these jobs: it tunes the ADF with two knobs, takes the ID of an aircraft to follow on the radar from a keypad, sends the landing gear, flaps, speed brake, brakes, anti-ice, hyd boost and caution dim switches to the simulator (as joystick buttons and, for most of them, as bridge commands), drives the three landing gear lamps, blanks its display, lamps and backlight with the other panels (the Radio's displays, the main bus and a 30 second no-data timeout), and monitors the **12 V and 5 V** supply rails. The PCB also has a **panel brightness pot**, which the sketch can read but has switched off for the moment (see Panel brightness). The sketch compiles for `arduino:avr:mega` (29,296 bytes of flash, 2,557 bytes of RAM) and has been flashed to the board (COM26). On the bench so far: the 12 V reading (after the pin fix below), the landing gear lamps (from the SwingTester) and the ADF keypad, whose key labels were checked on this board. The Upper matrix and Upper keypad, the three newer ADF switches in their new positions, and the blanking and backlight rules have not been bench-tested on the combined board yet.



The toggle switches send their position as a value (every change and every 5 s); the brakes push button sends PARKING_BRAKES.
The Upper panel (switch matrix and keypad) was the separate Upper controller, 172.16.1.103, until it was merged into this sketch.

Figure 25 — ADF controller blocks

10.1 Program flow

1. **Setup:** set the status LEDs, set the board's own backlight (the pot is not read), reset the W5500, start Ethernet with the 10 ms retransmission timeout and open the sockets (red LED flashes for 3 seconds), start the I2C character OLED, start the two encoders (centred at 1000), the ADF keypad scanner, the Upper matrix and keypad, the panel switches and the landing gear lamps.
2. **Main loop:**
    1. toggle the status LEDs every 200 ms and send the `ADF_PANEL` heartbeat every 10 seconds, with the 5 V and 12 V readings and the firmware version (`ADF_PANEL V5:5.03 V12:12.41 FW:1.0.3`) for the Health Monitor, followed by the same readings under the old Upper board's name, `UPPER_INPUT`, so the Health Monitor's Upper light stays green;
    2. read both encoders. Each is kept near the middle of its range and recentred after every read, so the change since the last read is the distance from the middle; three counts make one click. The major knob changes the ADF frequency by 100 kHz per click and the minor knob by 1 kHz per click (the minor knob speeds up when it is turned quickly), clamped to 190 to 1799 kHz. 250 ms after the last movement the frequency is sent as `ADF_FREQ:<kHz>`, and each click is also a joystick button press;
    3. `scanMatrix()` scans the ADF keypad (4 by 3) with a 50 ms debounce. For the first 1.5 seconds after start-up it only records the state, so a floating column is not reported as key presses;
    4. `scanUpperMatrixRow()` scans **one row** of the Upper switch matrix per pass (a whole scan is 9 passes, so the encoders keep being polled between rows) and `scanUpperKeypad()` scans the Upper keypad; changes go to the USB HID board;
    5. the keys build an aircraft ID: digits 0 to 9 add to it (up to 8), `#` sends `RADAR_FOLLOW_ID:<digits>` and makes it the followed ID, `*` deletes the last digit, and `*` on an empty entry stops following (`RADAR_FOLLOW_ID:0`). An unfinished entry is dropped after 20 seconds;
    6. read the seven panel switches and send their changes (below);
    7. redraw the OLED when something changed (at most every 50 ms);
    8. send the power status (below);
    9. check the no-data timeout (30 seconds with no data packet and no local input blanks the panel, see Panel blanking and backlight);
    10. poll port 13136 and act on the control and data packets.

10.2 Pin usage

Pin	Function
D2, D3, D4	Landing gear lamps (nose, left, right), driven from data packets (high = lit)
D5	`BACKLIGHT_PWM`: this board's backlight MOSFET (IRF540N), driven from `BPWR`, `C,B` and the no-data timeout (see Panel blanking and backlight)
D6, D7	ADF major encoder A, B
D8, D9	ADF minor encoder A, B
D14, D15	Green and red status LEDs
D20, D21	I2C SDA and SCL (character OLED at 0x3C)
D22 to D30	Upper switch matrix rows `ROW0` to `ROW8` (connector J30; `ROW8` also feeds the Warning Mute J17 and TOT Test J18 switches)
D31 to D34	Landing gear, flaps, speed brakes and brakes switches (to ground when on)
D35	Hyd boost switch (`HYD_BOOST`, connector J20; to ground when on)
D36	Anti-ice switch (`ANTI_ICE`, connector J20; to ground when on)
D37	Caution dim switch (`CAUTION_DIM`, connector J20; to ground when on)
D38 to D45	Upper switch matrix columns `COL0` to `COL7` (connector J31), each with a 10 k pull-up to 5 V (R40 to R47)
D53	W5500 reset
A0	Panel brightness pot wiper (`GUAGE_BACKLIGHT_BRIGHT`), not read while the pot is switched off
A1 to A7	ADF keypad lines `ADF_KEYPAD_1` to `ADF_KEYPAD_7` (J41 pins 2 to 8): columns A3, A2, A1 and rows A7, A6, A5, A4
A8	12 V monitor (`12V-Monitor`)

Pin	Function
A9 to A15	Upper numeric keypad `UPPER_KEYPAD_1` to `UPPER_KEYPAD_7` (J32): rows A10, A12, A14, A15, columns A9, A11, A13

D46 to D48 are not connected and D49 is the `RESET_pin` header (J2). Pins that moved when the boards were combined: the ADF switches from D29 to D35 to D31 to D37 (hyd boost D35, anti-ice D36 and caution dim D37 are in a different order from the old ADF board), the ADF keypad from D22 to D28 to A1 to A7 and the 12 V monitor from A15 to A8. The ADF panel connectors J5 and J20 keep their pin-outs.

10.3 ADF keypad

The seven ADF keypad lines `ADF_KEYPAD_1` to `ADF_KEYPAD_7` come from pins 2 to 8 of connector J41 on A1 to A7, and reach the keypad through the ADF panel connector J5 (pins 18 to 24). On this PCB the keypad lines arrive **in reverse order**, so the sketch scans **A7, A6, A5 and A4 as the four rows and A3, A2 and A1 as the three columns** (the old ADF board used D22 to D25 as rows and D26 to D28 as columns). The key labels are unchanged from the old board, where the keys were pressed one by one and the labels in `keyMap[][]` moved to where each key was actually reported:

Scan row	Column 0 (A3)	Column 1 (A2)	Column 2 (A1)
0 (A7)	6	4	5
1 (A6)	9	7	8
2 (A5)	#	*	0
3 (A4)	3	1	2

With the first pin assignment on the combined board (rows A1 to A4, columns A5 to A7) keys 1, 2 and 3 did nothing and most of the others reported as the wrong key, which showed the reversal. After the swap every key reports correctly (checked on the bench, 31 October 2026). `C,MATRIX:1` prints the raw readings if the keys ever need checking again.

10.4 Upper panel matrix and keypad

These come from the former Upper controller. The Upper panel is the overhead switch panel: a diode-isolated switch matrix and a small numeric keypad.

- **Matrix:** 9 rows (`ROW0` to `ROW8`, D22 to D30) by 8 columns (`COL0` to `COL7`, D38 to D45). One row at a time is driven low and the columns are read with pull-ups; a switch closed to ground reads low. A change is sent to the USB HID board as `index,state` with `index = row x 11 + column`. The stride of 11 is the old board's 11 columns, kept so that the joystick button numbers (0 to 95, Joystick 1 to 3) and the existing FSUIPC and MSFS assignments are the same as before. Changes of one switch are locked out for 20 ms after each report. Switches that are closed at power-up are reported straight away, so the HID board's buttons follow the panel.
- **Scan method:** the old sketch drove 16 rows with direct port writes and scanned all of them on every pass. The combined board has 9 rows (D22 to D30, the same first nine rows as before, so the indexes do not change) and D31 to D37 are now ADF switches, so the scan uses `digitalWrite`, with an 80 microsecond settling time before the columns are read, and does **one row per pass of the main loop** so the ADF encoders keep being polled.
- **Smaller than before:** rows 9 to 15 and columns 8 to 10 of the old matrix no longer exist on the PCB, so the Upper matrix stops at index 95 (the old board had 176 positions). Indexes 96 and above belong to the Audio and ADF panels and the matrix no longer reaches them, but rows 6 to 8 (indexes 66 to 95) still fall inside Joystick 3, shared with the Auto Pilot panel's buttons 64 to 81, so a switch fitted at 66 to 73 or 77 to 81 would clash.
- **Warning Mute and TOT Test:** the PCB's Warning Mute (J17) and TOT Test (J18) switches sit on `ROW8` (D30) behind diodes D5 and D6 on `COL0` and `COL1`, so they are read by the matrix as indexes 88 and 89. The Servo board's own mute and TOT clear buttons (indexes 7 and 8) are unchanged, and nothing is assigned to 88 and 89 yet. The earlier warning that the Upper sketch might not read them no longer applies.
- **Keypad:** 4 rows (A10, A12, A14, A15) by 3 columns (A9, A11, A13), keys `2 1 3 / 0 * # / 8 7 9 / 5 4 6` as laid out in the sketch. A press is sent as `KEY,<key>,1` and the release as `KEY,<key>,0`, typed on the HID board's virtual keyboard. The old sketch waited in a loop until the key was released, which would have stopped the ADF knobs; the merged scan does not wait.

- **Activity:** every matrix or keypad change counts as local input for the no-data timeout (see Panel blanking and backlight).
- **Brightness:** the Upper panel's backlight is this board's backlight (D5, connector J33): the Upper controller no longer has its own.

10.5 Panel brightness (pot switched off)

The PCB has a brightness pot on A0 (5 V to ground, wiper to A0). The sketch can read it, smooth it (a running average over 8 samples every 20 ms), turn it into a **level 0 to 15** with hysteresis, set its own backlight from it and send `C,B:<level>` to the other panels. This is **switched off for the moment**: `USE_BRIGHTNESS_POT` at the top of the brightness code is 0, so the pot is not read, no `C,B` packets are sent, and this board's own backlight stays at the default level 4 (PWM 68, close to the other boards' default of 65) until a `C,B` command sets it (see Panel blanking and backlight). Set `USE_BRIGHTNESS_POT` to 1 and reflash to bring it back. With it on, the level goes to the Radio .101, Servo .102, Stepper .105 and Dual Stepper .106 (not the USB HID board; the Upper panel is on this board), one board per pass of the loop so a switched-off board costs at most one send timeout, once 5 seconds after start-up and then whenever it changes. The other panels' brightness meanwhile follows the SwingTester slider and their own defaults.

10.6 Start-up hold

From reset the backlight, the OLED and the three landing gear lamps stay **on** (`panelBlank()` returns false and every lamp is lit) until the 13 Stepper controller announces on UDP port 13138 that start-up is complete, or 180 seconds after reset. Then `refreshAfterStartupHold()` redraws the OLED and the lamps from the Radio's `C,DISPLAY`, the no-data timer and the bridge, and the backlight goes off until `BPWR:1`.

10.7 Panel blanking and backlight

The panel follows the same rules as the other boards (see the network chapter). The display, the landing gear lamps and the backlight are controlled like this:

- **Radio displays (`C,DISPLAY`):** the Radio controller sends `C,DISPLAY:0` when it blanks its displays (no avionics power, or no data or input for 30 seconds) and `C,DISPLAY:1` when they come back, on every change and again every 10 seconds. The ADF controller then clears its OLED and puts the landing gear lamps out (it remembers what they should show and restores it). It starts with the display on, so a Radio that is off or missing never blanks it.
- **Main bus (`BPWR`):** the bridges send `D,BPWR:1/0` to the ADF controller, like the other boards. At 0 the backlight is off; at 1 it returns at the same level. Until the first one arrives the backlight is on.
- **Brightness (`C,B:0-15`):** sets the backlight level (0 off, 15 full, mapped to PWM 0 to 255) while the pot is switched off; the default is level 4. If `USE_BRIGHTNESS_POT` is 1 the pot is the source and `C,B` is ignored.
- **No-data timeout:** 30 seconds (`NO_DATA_TIMEOUT_MS`) with no data packet from the bridge, no brightness command or keep-alive and no local input (a knob click, a key or a switch change) blanks the OLED, the lamps and the backlight until something arrives, so a panel with no PC feed goes dark after power-up. Any of those brings the panel back at the same level. The Radio's `C,DISPLAY` messages do not count as data.

Both bridges send `D,BPWR:x` to the ADF controller (172.16.1.107) as well as to the Servo, Stepper, Dual Stepper and Radio, and the SwingTester's brightness slider, backlight power buttons and keep-alive include it. The bridges and the SwingTester **still also send to the retired Upper address 172.16.1.103**, which no longer exists; those packets go nowhere and are harmless, but the entries should be removed (on the to-do list).

10.8 Power monitoring

- **12 V rail:** R3 47 kohm from the rail and R4 10 kohm to ground with C1 0.1 uF across R4, midpoint to **A8** (on the combined board; the old ADF-only board used A15, which is now an Upper keypad line: until the sketch was changed on 31 October 2026 it reported 5.33 V with 12.3 V on the board, and 2.12 V measured on A8 matched the divider). The sketch averages 16 readings and scales them by the measured 5 V: volts = pin volts x (47 + 10) / 10. At 12 V the pin sees about 2.1 V, and the circuit reads up to about 28 V.
- **5 V rail:** the ADC reference is AVcc, the 5 V rail the AREF pin follows (the AREF pin is only brought out to a header, and the sketch **leaves the reference at its default**: `analogReference()` is never called). The sketch measures it by converting the chip's internal 1.1 V band-gap against AVcc (ATmega2560 `ADMUX` selection `11110` with `MUX5` 0): AVcc in millivolts = 1125300 / reading. It waits 5 ms after switching the input and throws away the first conversion.
- **Status:** `OK`, or the rails outside their limits joined with `+`: `5V_LOW` (below 4.75 V), `5V_HIGH` (above 5.25 V), `12V_LOW` (below 10.8 V), `12V_HIGH` (above 15.0 V). `V12_CAL` and `V5_CAL` at the top of the sketch trim the readings against a meter.

A power status is sent to the PC (port 27003) 5 seconds after start-up, then **every 30 seconds**, and whenever `C,POWER:1` arrives. It is **not** sent to the debug reflector (UDP 27000) any more (firmware 1.0 build 2). The same two readings go out with every 10 second heartbeat, which the Health Monitor shows in its voltage table (the ADF is the only board with a 12 V measurement).

10.9 What it sends

To	Port	Content
172.16.1.10 (PC)	27003	`ADF_FREQ:<kHz>`, `RADAR_FOLLOW_ID:<digits>` (`0` = stop following) and `POWER,V12:12.41,V5:5.02,STATUS:OK`
172.16.1.10 (PC bridge)	27001	The gear, flaps, speed brake and anti-ice switch **positions** as `GEAR_SET:1` / `GEAR_SET:0`, `FLAPS_SET:1/0`, `SPOILERS_SET:1/0` and `ANTI_ICE_SET:1/0`, and `PARKING_BRAKES` when the brakes button is pressed
172.16.1.11 (USB HID board)	4210	The seven ADF switches and the two ADF knobs as joystick buttons `index,state`: indexes 104 to 107, 112 to 114 (the anti-ice, hyd boost and caution dim switches) and 108 to 111 (Joystick 4, buttons 9 to 19)
172.16.1.10	27000	Debug text, each line prefixed `Jet Ranger ADF:` (the power status is not sent here: it goes to port 27003 and, as `V5` and `V12`, in the heartbeat)
172.16.1.10	13137	`ADF_PANEL V5:5.03 V12:12.41 FW:1.0.3` heartbeat every 10 seconds, and the same readings again as `UPPER_INPUT V5:5.03 V12:12.41 FW:1.0.3` (the old Upper board's name)
172.16.1.11 (USB HID board)	4210	also the Upper switch matrix as `index,state` (0 to 95) and the Upper keypad as `KEY,key,state`

The events use **port 27003, not 27001**, because the FSUIPC and MSFS bridges treat any unrecognised string on 27001 as a COM1 standby frequency and would retune COM1. **Neither bridge reads port 27003 yet**, so the ADF frequency and the radar follow ID do not reach the simulator. The switch commands do go to 27001, which both bridges recognise.

10.10 Panel switches

Seven switches are read with pull-ups and a 30 ms debounce; a closed switch (to ground) is "on". For the first 1.5 seconds after power-up they are only read. Each change sends the joystick button (`index,1` closed or `index,0` open) and, for the ones with a command, the command to the bridge on UDP 27001:

Switch	Pin	Joystick 4 button (index)	Bridge command
Landing gear (toggle)	D31	9 (104)	`GEAR_SET:1/0`
Flaps (toggle)	D32	10 (105)	`FLAPS_SET:1/0`
Speed brakes (toggle)	D33	11 (106)	`SPOILERS_SET:1/0`
Brakes (push button)	D34	12 (107)	`PARKING_BRAKES` when pressed
Hyd boost (toggle)	D35	18 (113)	none

Switch	Pin	Joystick 4 button (index)	Bridge command
Anti-ice (toggle)	D36	17 (112)	`ANTI_ICE_SET:1/0`
Caution dim (toggle)	D37	19 (114)	none

The gear, flaps, speed brake and anti-ice switches are **toggle switches**, so each sends its **position as a value**: `GEAR_SET:1` when the gear switch is closed (gear down) and `GEAR_SET:0` when it is open, and the same for `FLAPS_SET` (flaps fully down or up), `SPOILERS_SET` (speed brakes fully out or in) and `ANTI_ICE_SET` (anti-ice on or off). The positions are sent again **every 5 seconds**, so the simulator always follows the switches, including the position they were in at power-up: about 5 seconds after the panel is switched on the aircraft's gear, flaps, speed brakes and anti-ice move to match. This replaced a scheme that sent an event on each change (`GEAR_DOWN` / `GEAR_UP` and so on). The brakes are a push button: its press sends `PARKING_BRAKES` (a toggle) and the joystick button is held while it is pressed. If a switch works the wrong way round in the simulator, swap its 1 and 0 in the sketch. Each ADF knob click is also a 40 ms joystick press (major clockwise and anticlockwise 108 and 109, minor 110 and 111). The full joystick and command table is in the Radio controller chapter.

The anti-ice, hyd boost and caution dim switches were added to the ADF PCB in its revision 1.5 (on the combined board they share the 6-pin connector J20: pin 3 ground, pin 4 hyd boost on D35, pin 5 anti-ice on D36, pin 6 caution dim on D37). Anti-ice also sends `ANTI_ICE_SET`: the FSUIPC bridge sends it as the FSUIPC control `ANTI_ICE_SET` (parameter 1 or 0) and the MSFS bridge as the SimConnect event `ANTI_ICE_SET`. **For the DodoSim 206 V2** the aircraft has its own events for these three switches, `ANTI_ICE_SWITCH_SET`, `HYDRAULICS_SWITCH_SET` (hyd boost) and `CAUTION_LIGHT_SWITCH_SET` (caution lights bright or dim), each taking 1 or 0, which FSUIPC binds to buttons through the `DodoSim206.evt` file. Its fallbacks for the plain simulator controls are toggles (Carb heat / engine anti-ice, and "Concorde nose & visor" retract and extend for hyd boost and caution dim), so a bridge has no clean position command for hyd boost or caution dim, and the Dodo may ignore the plain `ANTI_ICE_SET` event. The reliable route is to bind Joystick 4 buttons 17 to 19 (indexes 112 to 114) in FSUIPC's Buttons + Switches page to the three Dodo events, with 1 on press and 0 on release (the Dodo manual also advises turning on "Sync controls on load"), and to use either that binding or the bridge's `ANTI_ICE_SET`, not both.

10.11 Landing gear lamps

The three lamps on the ADF panel (nose D2, left D3, right D4, each LED with its own 330 ohm resistor to ground on the panel) are lit by a data packet to port 13136: `D,LDGN:1,LDGL:1,LDGR:0` (1 = lit, 0 = off). The SwingTester has a toggle for each lamp. Neither bridge sends them yet (the gear position SimVars and offsets are not read), so for now they are a bench test. They go out with the rest of the panel (see Panel blanking and backlight) and come back as last set.

10.12 What it receives

On UDP 13136: control packets (`C,<COMMAND>:<value>`) and data packets (`LDGN`, `LDGL` and `LDGR` for the lamps and `BPWR` for the backlight power; every other token is ignored). Every data packet, brightness command and keep-alive restarts the 30 second no-data timeout (see Panel blanking and backlight).

Command	Action
`C,RESET:1`	Software reset (jump to the reset vector), as on the Radio controller
`C,MATRIX:1`	Print the raw ADF keypad readings to the debug stream (idle line, then one line per row with that row driven low)
`C,POWER:1`	Send a power status now
`C,DISPLAY:0` / `C,DISPLAY:1`	Sent by the Radio controller: blank the OLED and lamps, or restore them, with the Radio's displays
`C,B:<0-15>`	Backlight brightness (ignored while `USE_BRIGHTNESS_POT` is 1)
`C,KEEPALIVE:1`	Counts as data: restarts the no-data timeout

10.13 OLED

The two-line, 16-character OLED is on the Mega's own I2C bus (no TCA9548A) and uses the Radio controller's start-up sequence. Line 1 is `ADF  340 kHz`. Line 2 is `ID: 1234_` while an ID is being typed, `FOLLOW 1234` for the followed ID, or `FOLLOW OFF`. The OLED is cleared while the panel is blanked.

10.14 Notes

- The knob directions have not been checked. If a knob tunes the wrong way, swap its two pins in the `RotaryEncoder` lines.
- The display code is the Radio controller's character OLED code. If the ADF panel's display is a graphics SSD1306 it needs different code.
- The rail limits (5 V: 4.75 to 5.25, 12 V: 10.8 to 15.0) are first guesses; check them against the real supply.
- The ADF frequency and the radar follow ID go to port 27003, which the bridges do not read, so they are not used in the simulator yet.
- The Upper matrix and keypad, the ADF switches at their new pins (D31 to D37), the display and lamp blanking, and the backlight and no-data rules have not been bench-tested on the combined board.
- **Remote reset** (`C,RESET:1`) is the same software reset as the Radio controller. The old Upper board's reset was never bench-tested.
- **No pause for a missing HID board:** unlike the Radio, the sketch has no pause after a failed send to the USB HID board, so with the HID board unplugged each switch or key change can stall the scan for up to about 80 ms (the W5500 retransmission timeout is 10 ms with 8 retries).
- **Retired address:** nothing answers at 172.16.1.103 any more. The bridges, the SwingTester and the Health Monitor's Upper light still know it (the light is kept green by the `UPPER_INPUT` heartbeat from this board).

11 FSUIPC bridge (FSUIPCWinformsAutoCS)

`FSUIPCWinformsAutoCS` is the bridge for Prepar3D and FSX. It reads flight data through **FSUIPC** (raw memory offsets), sends each panel board its own packet, and listens on UDP 27001 for radio, battery and alternator commands from the Radio controller, acting on them through the FSUIPC API. It is a Windows Forms application (internally named `FSUIPCTest`); its window lists the values being sent and the 21 annunciator bits.

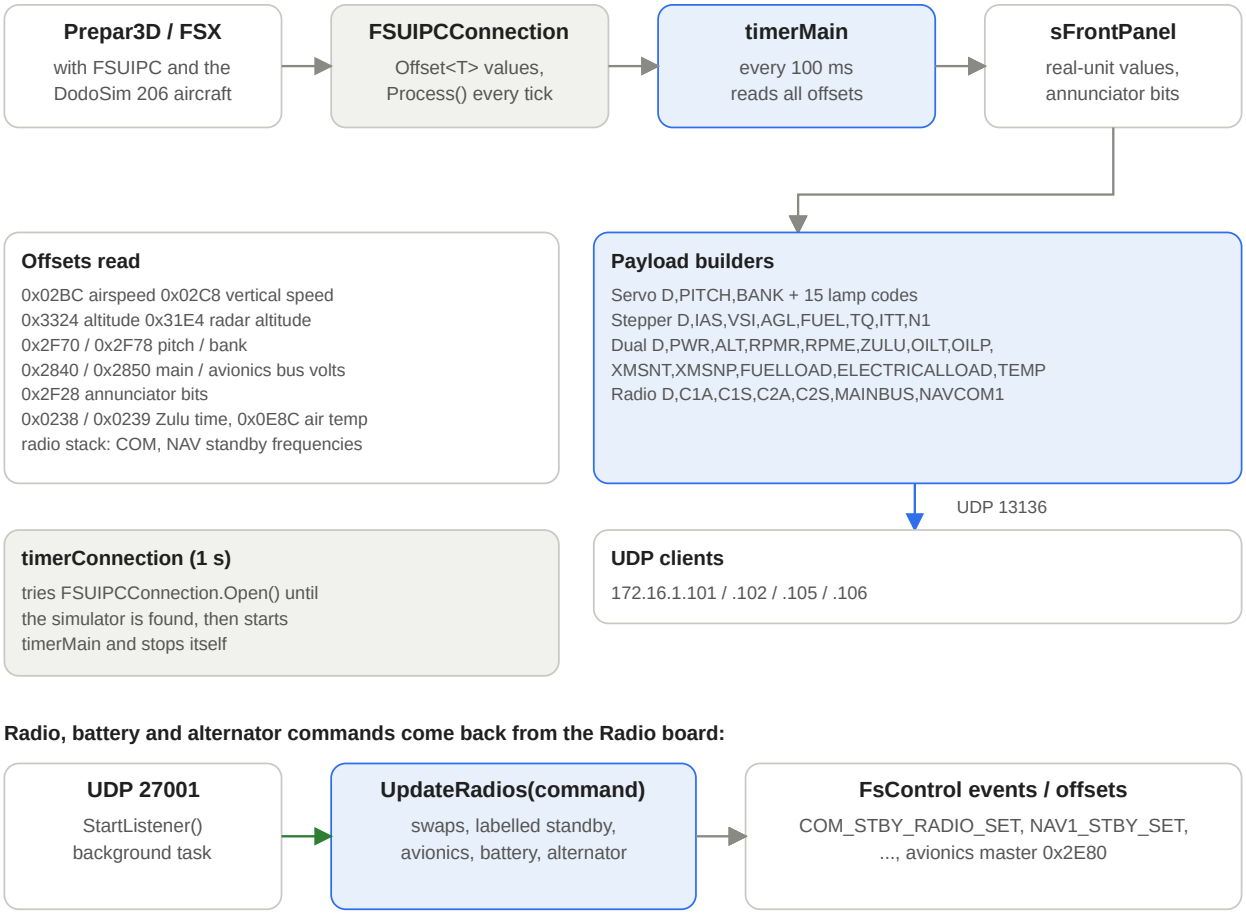


Figure 26 — FSUIPC bridge data flow

11.1 Program flow

1. **Constructor:** build the live annunciator list (one row per bit of the 0x2F28 field), connect the UDP clients and start `timerConnection`, which polls once a second for FSUIPC. Start the UDP listener on port 27001 in a background task. (The UDP client `.Connect` calls live in `Form1.Designer.cs`.)
2. **`timerConnection_Tick`:** keep trying `FSUIPCConnection.Open()`. Once it succeeds, stop and start `timerMain`.
3. **`timerMain_Tick`** (about 100 ms, so roughly 10 packets per second per board):
   1. `FSUIPCConnection.Process()` refreshes every declared offset and the radio-stack group;
   2. format the on-screen readout and update the annunciator list;
   3. build and send each board's packet (below);
   4. send the radio packet when a frequency or the bus voltage changed (at most every 200 ms) or every 5 seconds regardless.
4. **`UpdateRadios(command)`** handles port 27001 (below).

5. **chkAvionicsMaster_CheckedChanged** writes the avionics master offset when the operator toggles the checkbox, and closing the window stops both timers and the FSUIPC connection.

11.2 FSUIPC offsets

Offset	Meaning
0x02BC	Airspeed (knots x 128, so finer than 0.1 knot)
0x02C8	Vertical speed
0x0238, 0x0239	Zulu hours and minutes (combined into the HHMM ZULU)
0x0E8C	Outside air temperature, degrees C x 256 (sent as TEMP, for example 21C; a standard FSUIPC offset, not verified against DodoSim)
0x08F0	Turbine-out percent and temperature
0x08BA, 0x08B8	Engine oil pressure and temperature
0x0900, 0x0904	Transmission oil pressure and temperature
0x0B74, 0x0B78	Fuel percent quantity and fuel capacity
0x08F8	Fuel pressure
0x08F4	Torque percent
0x07BC, 0x07C4, 0x07C8, 0x07D0, 0x07DC, 0x0800, 0x0804	Autopilot master, NAV1 lock (CRS), heading lock, altitude lock, airspeed hold, approach hold and back course hold: the Auto Pilot panel LEDs
0x2EE0	Flight director active: the FD LED
0x3122	Radio audio switches (bit 7 COM1 transmit, 6 COM2 transmit, 5 COM receive both, 4 NAV1 sound, 3 NAV2 sound, 2 marker sound, 1 DME sound, 0 ADF1 sound): the Audio panel LEDs
0x0BAC, 0x0BAE, 0x0BB0	Inner, middle and outer marker: the INNER, MIDDLE and OUTER LEDs
0x07CC, 0x07D4, 0x07E2, 0x07F2	Autopilot heading (degrees x 65536 / 360), altitude (metres x 65536), airspeed (knots) and vertical speed (feet per minute): the values on the Auto Pilot panel's OLED
0x0C4E	NAV1 OBS setting (degrees): the CRS value on the Auto Pilot panel's OLED
0x0898	Gas producer
0x0908, 0x0896	Rotor RPM and engine N2
0x282C	Battery load amps
0x3324	Altitude
0x31E4	Radio altimeter
0x2F70, 0x2F78	Attitude pitch and bank
0x2F28	21-bit annunciator (warning lamp) bit field
0x2840, 0x2850	Main bus and avionics bus voltage
0x2E80	Avionics master (read and write)
0x0560, 0x0568	Latitude and longitude for the GPS interface (90 degrees = 10001750 x 65536 x 65536; 360 degrees = 65536^4)
0x0570	Altitude in metres for the GPS interface (64-bit with 32 bits of fraction, sent in feet)
0x0580	True heading for the GPS interface (360 degrees = 65536^2)
RadioStack group	COM1 and COM2, NAV1 and NAV2 active and standby, transponder, ADF

11.3 Packets sent

The Servo, Stepper and Dual Stepper boards each get their **own** packet in real units; the boards convert with their own calibration tables. Whole numbers are rounded.

Board	Packet contents
Servo .102	BPWR (1 while the main bus is 20 V or more), PITCH (degrees, one decimal, negated because FSUIPC pitch is nose-down positive), BANK (degrees, one decimal), the 15 lamp codes EOUT, TOPW,

Board	Packet contents
	BHOT, EC, BD, GENF, TOTW, RLOW, FFLTR, BTMP, TC, TRC, FPMP, LOWF, SCF as :1 or :0
Stepper .105	BPWR, IAS (knots, one decimal), VSI (ft/min), AGL (feet, power-gated), FUEL (US gal), TQ (%), ITT (degrees C), N1 (%)
Dual Stepper .106	PWR (first: 1, or 0 while the main bus is below 24 V), BPWR, ALT, RPMR, RPME (one decimal), ZULU, OILT, OILP, XMSNT, XMSNP, FUELLOAD (psi, from fuel pressure), ELECTRICALLOAD (%), TEMP
ADF and Upper .107 (and the retired Upper .103)	BPWR only (D,BPWR:1/0, sent from the same code as the others, adfClient and upperClient); it also feeds the 30 second no-data timeout. The Upper board was merged into the ADF board on 31 October 2026, so upperClient now sends to an address that no longer exists and should be removed
Radio .101	C1A, C1S, C2A, C2S, MAINBUS, NAVCOM1 (1 when the avionics bus, offset 0x2850, is 24 V or more) BPWR (backlight power, 1 while the main bus is 20 V or more) and the 19 LED_xxx tokens for the Auto Pilot and Audio panel LEDs (all 0 while NAVCOM1 is 0) and the five Auto Pilot display values APHDG, APCRS, APIAS, APVS and APALT (BuildPanelValuePayload(): heading and course as three digits, north is 360, airspeed in knots, vertical speed in feet per minute and altitude in feet). The offsets above are standard FSUIPC offsets; the DodoSim aircraft may not drive all of them

11.4 Aircraft information for the GPS interface

Every 500 ms SendGpsPacket() sends D,LAT:..,LON:..,ALT:..,IAS:..,HDG:..,UTC:yyyyMMddHHmmss to the Lowrance GPS interface (127.0.0.1:27002; change GpsHost and GpsPort in Form1.cs if it runs on another PC). Latitude and longitude are in degrees, altitude in feet, airspeed in knots (the value already read from 0x02BC) and heading in degrees true. UTC is the simulator's UTC date and time from the FSUIPC client library's FSUIPCConnection.UTCDateTime; if that read fails it is left out and the GPS interface uses the PC clock. Send errors, for example the GPS interface not running, are ignored. Built clean; **not yet run against Prepar3D**, so the offset conversions are unchecked against the simulator.

11.5 Commands from the Radio controller (UDP 27001)

Command	Action
COM1_RADIO_SWAP, COM2_RADIO_SWAP	FSUIPC controls COM_STBY_RADIO_SWAP and COM2_RADIO_SWAP
NAV1_RADIO_SWAP, NAV2_RADIO_SWAP	FSUIPC controls NAV1_RADIO_SWAP and NAV2_RADIO_SWAP
AVIONICS_MASTER_SET	Writes the avionics master offset 0x2E80
MASTER_BATTERY_ON, MASTER_BATTERY_OFF	FSUIPC only has a toggle, so the bridge reads the main bus voltage and toggles only when that would move the switch the requested way
ALTERNATOR_ON, ALTERNATOR_OFF	TOGGLE_MASTER_ALTERNATOR unconditionally (no alternator state is read)
COM2_STANDBY:, NAV1_STANDBY:, NAV2_STANDBY: plus a frequency	Tunes that radio's standby frequency
Auto Pilot, Audio and ADF panel controls: AP_MASTER, AP_HDG_HOLD, AP_NAV1_HOLD, AP_ALT_HOLD, AP_APR_HOLD, TOGGLE_FLIGHT_DIRECTOR, AP_BC_HOLD, AP_PANEL_SPEED_HOLD, VOR1_OBI_INC / _DEC, AP_ALT_VAR_INC / _DEC, AP_VS_VAR_INC / _DEC, AP_SPD_VAR_INC / _DEC, HEADING_BUG_INC / _DEC, COM1_TRANSMIT_SELECT, COM2_TRANSMIT_SELECT, COM_RECEIVE_ALL_TOGGLE, RADIO_VOR1_IDENT_TOGGLE, RADIO_VOR2_IDENT_TOGGLE, RADIO_DME1_IDENT_TOGGLE,	The FSUIPC control of the same name, sent with parameter 0 (TryPanelCommand(), checked first so none of these can fall through to the COM1 parser). A :N suffix repeats it N times (1 to 50) for a knob turned quickly

Command	Action
`RADIO_ADF_IDENT_TOGGLE`, `MARKER_SOUND_TOGGLE`, `GEAR_UP`, `GEAR_DOWN`, `FLAPS_UP`, `FLAPS_DOWN`, `SPOILERS_ON`, `SPOILERS_OFF`, `PARKING_BRAKES`	
`GEAR_SET:1/0`, `FLAPS_SET:1/0`, `SPOILERS_SET:1/0`, `ANTI_ICE_SET:1/0`	The positions of the ADF panel's toggle switches (1 = gear down, flaps fully down, speed brakes fully out, anti-ice on), sent on every change and every 5 seconds: `TryPositionCommand()` sends the FSUIPC control `GEAR_SET` (parameter 1 or 0), `FLAPS_SET` or `SPOILERS_SET` (16383 or 0) or `ANTI_ICE_SET` (1 or 0). The DodoSim 206 V2 may ignore the plain `ANTI_ICE_SET` control: bind the Dodo's own `ANTI_ICE_SWITCH_SET` event to the ADF panel's anti-ice joystick button in FSUIPC instead (see the ADF controller chapter)
A bare number from 118.00 to 136.975	COM1 standby: BCD-encoded and sent with `COM_STBY_RADIO_SET`. Any other unrecognised string ends up in this parser too

11.6 Notes and quirks

- Several unused annunciator bits (`SPARE_*`, `SIMCONNECT_FAIL`, `LOW_INLET_PRESSURE`) all share the short code `SCF`, so the last one in the packet wins for that lamp.
- `TURB_ENG_ITT_1` is assigned twice per tick (percent, then temperature), so the temperature wins.
- Run only one bridge at a time; the MSFS bridge also listens on UDP 27001.
- Built clean and run against Prepar3D v6.1 sending about 10 packets per second per board.

11.7 Panel emulators

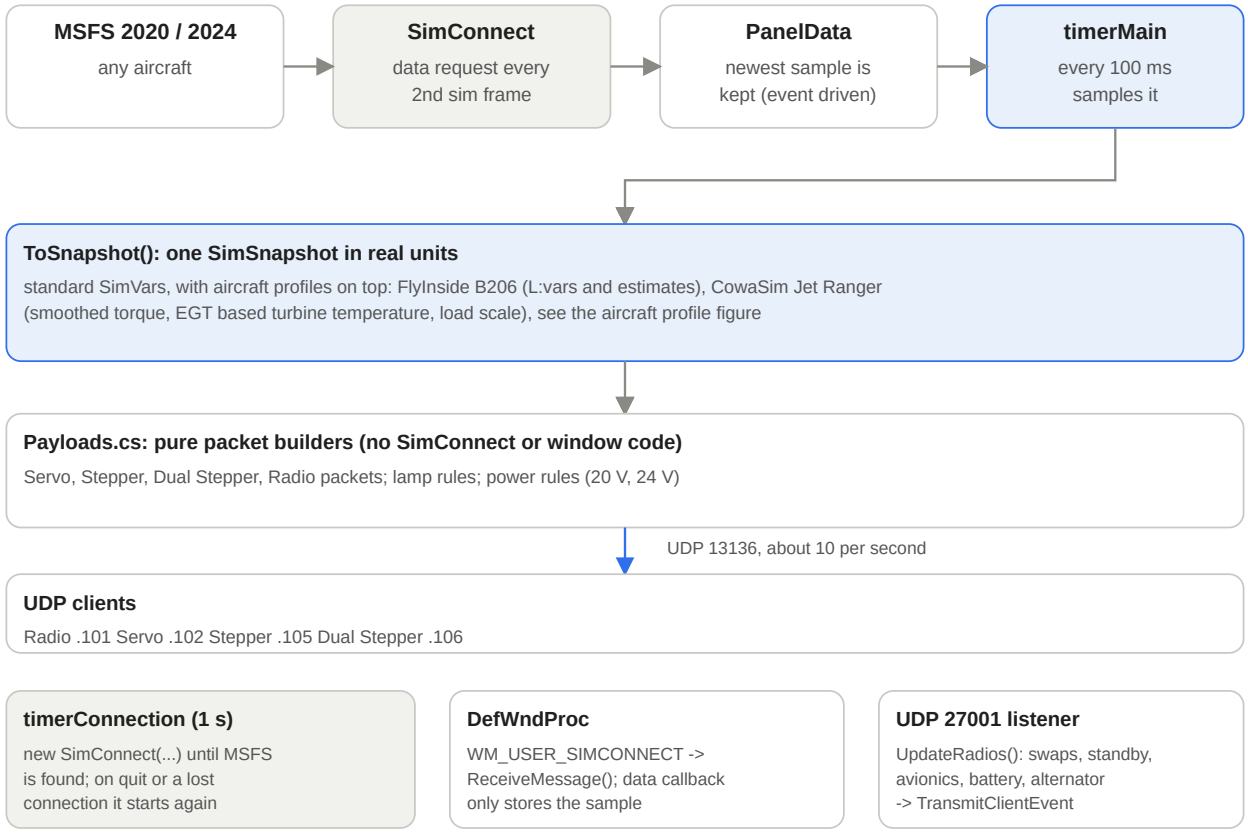
Panel emulators. The window has two check boxes under the Avionics Master box, **Send to the boards** (on) and **Send to the emulators** (off), and the command line takes `--emulators` (also send to the emulators), `--emulators-only` and `--emulator-net <a.b.c>`. Each board's packet then also goes to `JetRangerPanelEmulator` on 127.0.0.<the board's number> (see the Panel emulators chapter). No simulator data has gone through this bridge to the emulators yet.

11.8 Packet monitor

Packet monitor. Show packets... under the Avionics Master box opens a window that lists every payload as it is sent: one line per packet with the time, the target board (Radio .101, Servo .102, Stepper .105, Dual Stepper .106, ADF and Upper .107, the retired Upper .103, or the GPS interface), where it went (*board*, *emulator*, *board* + *emulator* or *local*) and the payload text. The **Target board** box filters the list to one board; **Pause** and **Clear** do what they say. Command line: `--show-packets` opens it at start, `--packet-board <radio, servo, stepper, dual, adf, gps or a last number such as 105>` opens it filtered. `BoardTargets` raises a `PacketSent` event for every send and `PacketMonitorForm` shows the queued packets ten times a second (the last 6,000 are kept for re-filtering), so a busy bridge is not slowed. Only packets that are really sent are shown. The window and the command line were checked; no simulator data has gone through this bridge's monitor yet.

12 MSFS bridge (MSFSWinformsAutoCS)

`MSFSWinformsAutoCS` is the bridge for Microsoft Flight Simulator 2020 and 2024. It has the same window, the same timers, the same UDP endpoints and the same real-unit packet per board as the FSUIPC bridge, but reads the data through **SimConnect**. For aircraft whose cockpit dials do not read the standard simulator variables it applies estimates fitted to dial readings (a FlyInside B206 profile and a CowaSim Jet Ranger profile).



No new data for 3 s (paused or loading): the bridge stops sending and the boards' 30 s watchdogs zero the gauges.

Run only one bridge at a time: FSUIPCWinformsAutoCS and MSFSWinformsAutoCS both listen on UDP 27001.

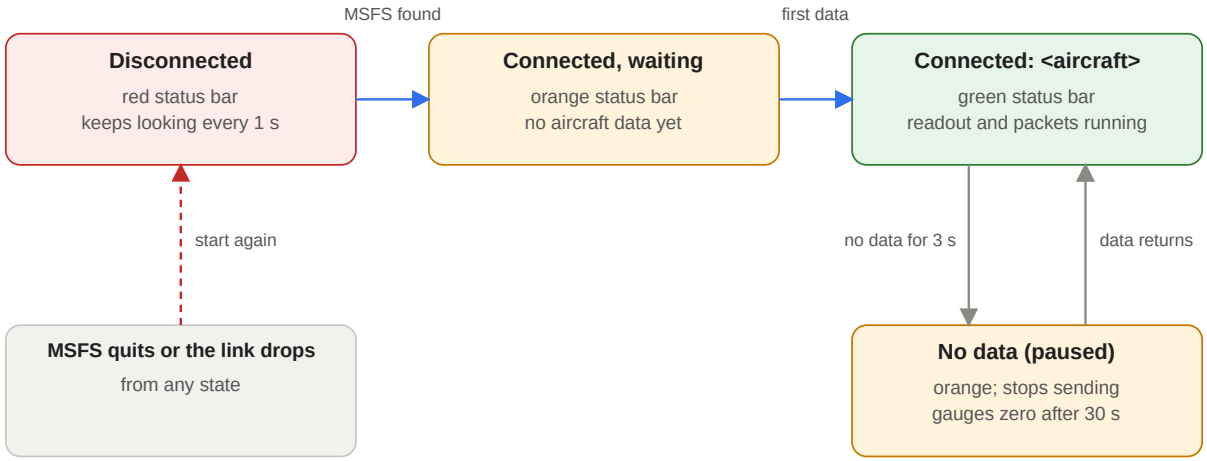
Figure 27 — MSFS bridge data flow

12.1 Files

File	Role
`Form1.cs`	SimConnect connection, 100 ms main timer, on-screen readout, lamp list, UDP listener and command handler
`Payloads.cs`	`SimSnapshot` (one sample of real-unit values) and `Payloads` (packet builders, lamp rules, power thresholds); no SimConnect or window code, so it can be tested on its own
`Form1.Designer.cs`, `Program.cs`	Window layout (identical to the FSUIPC bridge) and entry point
`lib\Microsoft.FlightSimulator.SimConnect.dll`, `lib\SimConnect.dll`	Managed wrapper and native library copied from the MSFS 2024 SDK so the project builds without the SDK path
`MSFSWinformsAutoCS.csproj`	`net10.0-windows10.0.19041.0`, WinForms

12.2 Program flow

1. **Constructor:** connect the UDP clients (port 13136), build the lamp list, start the 1 second `timerConnection` and the UDP listener on 27001.
2. **`timerConnection_Tick`:** try `new SimConnect("MSFS Winforms Bridge", Handle, WM_USER_SIMCONNECT, ...)`. While MSFS is not running this throws and the timer tries again. On success it registers the data definition, requests it every second sim frame, maps the client events, starts `timerMain` and stops itself.
3. **`DefWndProc`** passes `WM_USER_SIMCONNECT` to `ReceiveMessage()`; the data callback only stores the newest sample. If MSFS quits or the connection throws, the app closes SimConnect, stops `timerMain` and goes back to step 2.
4. **`timerMain_Tick`** (100 ms) converts the newest sample to a `SimSnapshot`, updates the readout, lamp list and Avionics Master box, then sends the packets: Radio when a frequency, the main bus voltage or the radio power flag changed (at most every 200 ms) or every 5 seconds; one packet each to the Servo, Stepper and Dual Stepper boards about every 100 ms.
5. **`UpdateRadios`** (UDP 27001) handles the Radio controller's commands as SimConnect client events.



A new aircraft is picked up while connected: the name in the status bar changes and the estimate models restart.

SimConnect errors are caught on every path, so a closed simulator never stops the bridge. FSUIPCWinformsAutoCS behaves the same way with FSUIPC.

Figure 28 — Connection states

If no new data has arrived for 3 seconds (paused or loading) the bridge stops sending, so the boards' 30 second watchdogs return the gauges to zero, and the status bar says so. The status bar also shows the aircraft title and follows it when a different aircraft is loaded.

12.3 Packets

Board	Packet
Servo .102	`BPWR` (1 while the main bus is 20 V or more), `PITCH` (degrees, one decimal, negated: the simulator reports nose-down positive), `BANK`, and the 15 lamp codes (all `0` while the main bus is below 20 V)
Stepper .105	`BPWR`, `IAS` (knots, one decimal), `VSI`, `AGL` (feet, 0 below 20 V), `FUEL` (US gal), `TQ`, `ITT` (degrees C), `N1`
Dual Stepper .106	`PWR` (1, or 0 while the main bus is below 24 V), `BPWR`, `ALT`, `RPMR`, `RPME`, `ZULU`, `OILT`, `OILP`, `XMSNT`, `XMSNP`, `FUELLOAD` (fuel pressure, psi), `ELECTRICALLOAD` (percent, below), `TEMP`
ADF and Upper .107 (and the retired Upper .103)	`BPWR` only (`Payloads.Adf()`, `D,BPWR:1/0`, sent to `adfClient` and `upperClient` with the other boards' packets); it also feeds the 30 second no-data timeout. The Upper board was merged into the ADF board on 31 October 2026, so `upperClient` now sends to an address that no longer exists and should be removed

Board	Packet
Radio .101	C1A, C1S, C2A, C2S, MAINBUS, NAVCOM1, BPWR (backlight power, 1 while the main bus is 20 V or more), the 19 LED_xxx tokens and the five Auto Pilot display values APHDG, APCRS, APIAS, APVS and APALT (see above) (NAVCOM1 is 1 when the avionics bus is 24 V or more; if the aircraft reports the avionics bus as exactly 0, as the FlyInside B206 does, the main bus is used instead)

ELECTRICALLOAD is the magnitude of ELECTRICAL TOTAL LOAD AMPS divided by 300 A (the sign goes negative while the battery supplies the bus, for example with the engine stopped); the CowaSim profile uses 139 A.

12.4 Aircraft information for the GPS interface

Every 500 ms the main timer sends D,LAT:..,LON:..,ALT:..,IAS:..,HDG:..,UTC:yyyyMMddHHmmss.fff (Payloads.Gps()) to the Lowrance GPS interface (127.0.0.1:27002; change GpsHost and GpsPort in Form1.cs if it runs on another PC). Three SimVars were added at the end of the data definition and the PanelData struct, in the same order: PLANE LATITUDE, PLANE LONGITUDE (degrees) and PLANE HEADING DEGREES TRUE, plus ZULU YEAR, ZULU MONTH OF YEAR and ZULU DAY OF MONTH. Altitude (PLANE ALTITUDE, feet MSL) and airspeed (AIRSPEED INDICATED) were already read. The simulator's UTC time is built from the already-read ZULU TIME (seconds since midnight) and the three date SimVars, and is left out if the date is invalid. The readout shows the position and heading. Nothing is sent while the data is stale, so the GPS interface falls back to Brisbane Airport after 30 seconds. The packet builder was checked with sample values and read back by the GPS interface's parser; **not yet run against MSFS**.

12.5 SimVars requested

PLANE ALTITUDE, AIRSPEED INDICATED, VERTICAL SPEED, PLANE ALT ABOVE GROUND, ATTITUDE INDICATOR PITCH and BANK DEGREES, ROTOR RPM PCT:1, GENERAL ENG PCT MAX RPM:1, ENG TORQUE PERCENT:1, TURB ENG MAX TORQUE PERCENT:1, ENG EXHAUST GAS TEMPERATURE:1, GENERAL ENG THROTTLE LEVER POSITION:1, ELECTRICAL TOTAL LOAD AMPS, TURB ENG ITT:1, ENG OIL TEMPERATURE:1, FUEL TOTAL QUANTITY, TURB ENG CORRECTED N1:1, ENG OIL PRESSURE:1, ENG TRANSMISSION PRESSURE:1, ENG TRANSMISSION TEMPERATURE:1, GENERAL ENG FUEL PRESSURE:1, ELECTRICAL MAIN BUS VOLTAGE, ELECTRICAL AVIONICS BUS VOLTAGE, AVIONICS MASTER SWITCH, AMBIENT TEMPERATURE, KOHLSMAN SETTING MB (shown in the readout only), ZULU TIME, the COM 1, COM 2, NAV 1 and NAV 2 active and standby frequencies, the two FlyInside L:vars and the six GPS SimVars above. PanelVars and the PanelData struct must list them in the same order; CheckDefinitionMatchesStruct() checks the count at connect.

12.6 Autopilot and audio panel LEDs

The 19 LEDs on the Auto Pilot and Audio panels are read through **two extra SimConnect requests** (PanelAp and PanelAudio, every 6th frame), separate from the main gauge data, so a SimVar the simulator does not know can only take these LEDs down, not the gauges. PanelAp: AUTOPILOT MASTER, AUTOPILOT NAV1 LOCK, AUTOPILOT HEADING LOCK, AUTOPILOT ALTITUDE LOCK, AUTOPILOT AIRSPEED HOLD, AUTOPILOT APPROACH HOLD, AUTOPILOT BACKCOURSE HOLD, AUTOPILOT FLIGHT DIRECTOR ACTIVE. PanelAudio: COM TRANSMIT:1 and :2, COM RECEIVE ALL, NAV SOUND:1 and :2, DME SOUND, ADF SOUND, MARKER SOUND, OUTER MARKER, MIDDLE MARKER, INNER MARKER. They become the LED_xxx tokens of the radio packet (Payloads.LedTokens(), all 0 while the radio bus is below 24 V). ADF SOUND could not be confirmed to exist in MSFS; if it does not, the audio LEDs stay off and the autopilot LEDs are unaffected.

12.7 Auto Pilot display values

The five values on the Auto Pilot panel's OLED are read through a third extra SimConnect request (PanelApValues, every 6th frame), again so an unknown SimVar cannot take the gauge data down: AUTOPILOT HEADING LOCK DIR, NAV OBS:1 (the course), AUTOPILOT AIRSPEED HOLD VAR, AUTOPILOT

`VERTICAL HOLD VAR` and `AUTOPILOT ALTITUDE LOCK VAR`. They become the tokens `APHDG`, `APCRS`, `APIAS`, `APVS` and `APALT` of the radio packet (`Payloads.ApValueTokens()`: heading and course as three digits with north as 360, the others as whole numbers).

12.8 Commands from the panel (UDP 27001)

The Auto Pilot, Audio and ADF panel controls (`AP_MASTER`, `AP_HDG_HOLD`, `HEADING_BUG_INC`, `COM1_TRANSMIT_SELECT`, `GEAR_DOWN`, `PARKING_BRAKES` and the others listed in the FSUIPC bridge chapter) are mapped to client events of the same name when SimConnect opens (`PanelEventNames`, event numbers from 1000) and sent by `TryPanelCommand()`, which is checked first; a `:N` suffix repeats the event N times. The ADF panel's toggle switches send their positions as `GEAR_SET:1/0`, `FLAPS_SET:1/0`, `SPOILERS_SET:1/0` and `ANTI_ICE_SET:1/0` (on every change and every 5 seconds), which `TryPositionCommand()` sends as the SimConnect events `GEAR_SET` (0 or 1), `FLAPS_SET` and `SPOILERS_SET` (0 or 16383) and `ANTI_ICE_SET` (0 or 1), event numbers from 2000, so the simulator follows the switch instead of toggling.

`COM1_RADIO_SWAP`, `COM2_RADIO_SWAP`, `NAV1_RADIO_SWAP`, `NAV2_RADIO_SWAP`; `COM2_STANDBY:`, `NAV1_STANDBY:`, `NAV2_STANDBY:` plus a range-checked frequency (sent as `COM2_STBY_RADIO_SET_HZ`, `NAV1_STBY_SET_HZ`, `NAV2_STBY_SET_HZ`); a bare number is the COM1 standby frequency (an out-of-range or unreadable value becomes 121.50); `AVIONICS_MASTER_SET`; `MASTER_BATTERY_ON` and `_OFF` (`MASTER_BATTERY_SET` 1 or 0); `ALTERNATOR_ON` and `_OFF` (`ALTERNATOR_SET`). Unlike FSUIPC, SimConnect has direct set events, so there is no voltage guard or blind toggle.

12.9 Aircraft profiles

Value	Standard SimVars	FlyInside B206	CowaSim Jet Ranger
Rotor, engine speed	**ROTOR RPM PCT,** GENERAL ENG PCT MAX RPM	**L:Aircraft.Rotor.RPM.Percent,** L:...Turbine.N2.Smooth	**standard**
Gas producer (N1)	**TURB ENG CORRECTED N1**	**0.92 x turbine N2**	**standard**
Torque (TQ)	**ENG TORQUE PERCENT**	**collective lever curve,** up to 120 %	**0.42 x SimVar,** smoothed 2.5 s
Turbine out temp (ITT)	**TURB ENG ITT**	**6 x N2 + 1.05 x** (torque - 21)	**EGT - 200 C**
Fuel load (psi)	**ENG FUEL PRESSURE**	**10 x SimVar**	**standard**
Electrical load (%)	**|total amps| / 300 A**	**|total amps| / 300 A**	**|total amps| / 139 A**
Transmission pressure	**SimVar (usually 0)**	**35 psi x rotor fraction**	**48 psi x rotor fraction**
Transmission temp	**SimVar (usually 0)**	**slow warm-up from air temp**	**0.92 x engine oil temp**
Recognised by	**otherwise**	**turbine L:var, or title** contains FlyInside	**title contains 206B3**

Green = the standard SimVar is used unchanged. Estimates come from a few dial readings and may be a few percent off.

Figure 29 — Which values come from where

Other aircraft use the standard SimVars unchanged. Estimates were fitted to a few dial readings taken from screenshots and may be a few percent off at power settings that were not checked.

12.9.1 FlyInside B206

The FlyInside B206 does not feed the standard rotor and engine speed SimVars (they read 0 and a slowly decaying idle while the cockpit shows 100 percent). Found with Developer Mode, Tools, Behaviors, Variables (Global tab, filter `Aircraft.`): the cockpit uses `L:Aircraft.Rotor.RPM.Percent` and `L:Aircraft.Engine.1.Turbine.N2.Smooth`. The aircraft is recognised by the turbine L:var being above zero or by `FlyInside` in the aircraft title. The bridge uses the L:vars when above zero, otherwise the standard SimVars. They feed `RPMR`, `RPME` and the `RLOW` and `EOUT` lamps.

Value	Estimate
Gas producer (`N1`)	0.92 x turbine N2 (no variable found; the standard N1 reads about 52 while the cockpit shows 92)
Torque (`TQ`)	`ENG TORQUE PERCENT:1` reads 0. With the turbine at 95 or more: 21 + 0.72 x collective lever up to lever 73.6 (dial 74), then about 1.74 per lever percent to 120 at lever 100, clamped 0 to 120. Below that speed it is estimated from `TURB ENG MAX TORQUE PERCENT:1` through two readings
Turbine out temperature (`ITT`)	6 x turbine N2 + 1.05 per torque percent above the idle torque of 21 (the standard value reads about 540 while the dial shows 600)
Fuel load (`FUELLOAD`)	10 x `GENERAL ENG FUEL PRESSURE:1` (the cockpit dial is a 0 to 3 scale marked x10)

Value	Estimate
Transmission pressure (XMSNP)	35 psi x the rotor speed fraction
Transmission temperature (XMSNT)	Starts at outside air temperature and moves slowly (5 minute warm-up, 10 minute cool-down) toward ambient + 0.8 per torque percent while the rotor is above 50 percent; restarts from ambient when the bridge restarts



Both curves were fitted to dial readings from screenshots; check them against the cockpit at other power settings.

FlyInside above lever 73.6 climbs at about 1.74 points per percent; the engine over-torques if full collective is held for 2 s.

Figure 30 — Torque curves

Holding full collective for more than about 2 seconds over-torques and breaks the engine in this aircraft (the dial swings to about 115 percent). The lever mapping depends on the controller; only two steady readings back the 0.72 slope.

12.9.2 CowaSim Jet Ranger

The CowaSim Jet Ranger (aircraft title `206B3 ...`, for example `206B3 C-GTOK`) reports the standard SimVars and most dials match them. Gas producer, engine oil, fuel pressure and rotor and engine speed need no change. Three dials do not read the standard values, and it has no transmission oil values:

- **Torque:** the dial is a smoothed copy of `ENG TORQUE PERCENT:1`, about 0.42 x the SimVar (dial about 38 at the steady idle value of 83, about 120 at a peak of about 260) with a smoothing time constant of about 2.5 seconds. Error is about 18 points on a signal swinging between 0 and 120.
- **Turbine out temperature:** the dial follows `ENG EXHAUST GAS TEMPERATURE:1`, not ITT (which reads over 2,000): EGT minus 200 C, within about 30 C over nine readings.
- **Electrical load:** the LOAD dial sat at about 50 percent while the total load was 69.5 A, so 139 A is 100 percent (one reading).
- **Transmission oil:** pressure is 48 psi x the rotor speed fraction (the dial read about 45 to 50 psi); temperature is 0.92 x the engine oil temperature (one reading, which did not follow a slow warm-up).

12.10 Warning lamps

Five lamps are driven; the other ten are aircraft specific (DodoSim supplies them through an FSUIPC offset, which MSFS aircraft do not have) and stay off. Add rules to `Payloads.LampState()` once an aircraft's variables are known.

Lamp	Rule
EOUT engine out	Turbine speed below 60 percent
RLOW rotor low	Rotor at or below 95 percent
LOWF low fuel	17 US gal or less
TOPW transmission oil pressure	Below 40 psi, only when a pressure is available (the FlyInside and CowaSim

Lamp	Rule
	estimates, or an aircraft whose SimVar reads above 0)
`GENF` generator fail	Engine speed below 10 percent

12.11 Known gaps

- The FlyInside and CowaSim estimates come from a few readings and have not been checked at other power settings.
- Some MSFS Jet Ranger add-ons do not report every SimVar (transmission pressure and temperature read 0 on two of them; one reports a fixed rotor speed). A missing avionics bus voltage (exactly 0) falls back to the main bus for the Radio power rule.
- Pitch sign and the 24 V thresholds are carried over from the FSUIPC bridge and are not verified for every MSFS aircraft.
- There is no hysteresis on the 20 V and 24 V thresholds.
- The FlyInside electrical load dial stays at about 50 percent whatever the load.
- The FlyInside transmission pressure estimate is 35 psi at normal rotor speed, which is below the 40 psi lamp threshold, so the `TOPW` lamp stays on with that aircraft.

12.12 Panel emulators

Panel emulators. The window has two check boxes under the Avionics Master box, **Send to the boards** (on) and **Send to the emulators** (off), and the command line takes `--emulators` (also send to the emulators), `--emulators-only` and `--emulator-net <a.b.c>`. Each board's packet then also goes to `JetRangerPanelEmulator` on 127.0.0.<the board's number> (see the Panel emulators chapter). No simulator data has gone through this bridge to the emulators yet.

12.13 Packet monitor

Packet monitor. Show packets... under the Avionics Master box opens a window that lists every payload as it is sent: one line per packet with the time, the target board (Radio .101, Servo .102, Stepper .105, Dual Stepper .106, ADF and Upper .107, the retired Upper .103, or the GPS interface), where it went (*board*, *emulator*, *board* + *emulator* or *local*) and the payload text. The **Target board** box filters the list to one board; **Pause** and **Clear** do what they say. Command line: `--show-packets` opens it at start, `--packet-board <radio, servo, stepper, dual, adf, gps or a last number such as 105>` opens it filtered. `BoardTargets` raises a `PacketSent` event for every send and `PacketMonitorForm` shows the queued packets ten times a second (the last 6,000 are kept for re-filtering), so a busy bridge is not slowed. Only packets that are really sent are shown. The window and the command line were checked; no simulator data has gone through this bridge's monitor yet.

13 DCS bridge (DCSBiosBridge)

`DCSBiosBridge` is the bridge for **DCS World**. It reads the **DCS-BIOS** export stream, converts the values an aircraft **profile** names into the panel's real units, and sends them to the boards in the same `D,TOKEN:value` packets (UDP 13136) as the other bridges, so the board firmware does not change. It is a Windows Forms program (`net10.0-windows`) and is **output only**: it sends nothing to the simulator (the panel's switches already reach DCS as joystick buttons through the USB HID board). The first aircraft is the UH-1H Huey, but a profile is a data file, so another aircraft needs no new code, and an aircraft with no profile of its own still drives the real-unit gauges from DCS-BIOS's common data.

Run only one bridge at a time. The bridge was run against live DCS for the Huey caution lamps (confirmed working); the gauge values added afterwards are tested with a synthetic stream and the real UH-1H and CommonData module files but have **not been flown against live DCS**.

13.1 DCS-BIOS

DCS-BIOS v0.11.8 was installed on this PC on 31 October 2026 from the official release (`DCS-BIOS_v0.11.8.zip`, GitHub DCS-Skunkworks/dcs-bios) into `Saved Games\DCS\Scripts`: the `DCS-BIOS` folder and a new `Export.lua` containing one line, `dofile(lfs.writedir() .. [[Scripts\DCS-BIOS\BIOS.lua]])`. The release also ships the generated module files (`DCS-BIOS\doc\json\*.json`, 56 of them, including `CommonData.json`).

13.2 How it works

1. **Export stream.** `ExportListener` joins UDP multicast 239.255.50.10 port 5010 on the loopback interface (the DCS-BIOS default, with address reuse on so other tools can listen too). `ExportParser` is the DCS-BIOS stream state machine (sync `0x55 0x55 0x55 0x55`, then blocks of start address, byte count and data, little endian) and keeps the 64 KB memory image in `ExportState`. A block may span datagrams.
2. **Aircraft.** The name of the aircraft is the 24 character string at address 0x0000 (`_ACFT_NAME`). `profiles\*.json` are matched on their `aircraft` names; the Profile box can also pick one by hand.
3. **Module definition.** The profile's `jsonFile` (for example `UH-1H.json`) is read from `Saved Games\DCS\Scripts\DCS-BIOS\doc\json\` (or `DCS.openbeta`, or `--json-dir`). `ModuleDoc` finds every control with an `identifier` and integer `outputs` (address, mask, shift_by, max_value); a value is `(word at address & mask) >> shift_by`.
4. **Profile values.** Each row maps a token to a control. `type` is `lamp` (1 when the value is at or above `threshold` of its maximum, optionally `invert`), `analog` (the control as a fraction of its maximum, or its raw value with `raw`, times `scale` plus `offset`, with `decimals`, or the calibration `points`), `digits` (drum digits, most significant first: the radar altimeter) or `coordinate` (degrees, minutes and fraction of a minute plus the N/S/E/W letter). A profile file whose name starts with an underscore (`_common.json`) is added to every aircraft's profile; the aircraft's own value for a token wins.
5. **Boards.** `Payloads.Build` sorts the tokens by board and builds one packet each: Radio (.101) `D,NAVCOM1:x,BPWR:x,LED_...`, Servo (.102), Stepper (.105), Dual Stepper (.106, with `PWR`), the GPS interface (127.0.0.1:27002) and `D,BPWR:x` to the ADF and Upper board (.107) always. The retired Upper address .103 is not used.
6. **Power rules** as in the other bridges: no bus voltage mapped (`MAINBUS`) means powered while data flows; `MAINBUS` at or above 20 V switches the backlights and the Servo lamps on, `AVIONICSBUS` (or `MAINBUS`) at or above 24 V powers the radio, and the Huey LEDs go dark with the radio.
7. **Cadence.** Radio packet on change (at most every 200 ms) or every 5 s; the other boards about every 90 ms; GPS twice a second. When the stream goes quiet for 3 seconds nothing is sent, so the boards' 30 second no-data watchdogs return the gauges to zero.

13.3 Window and command line

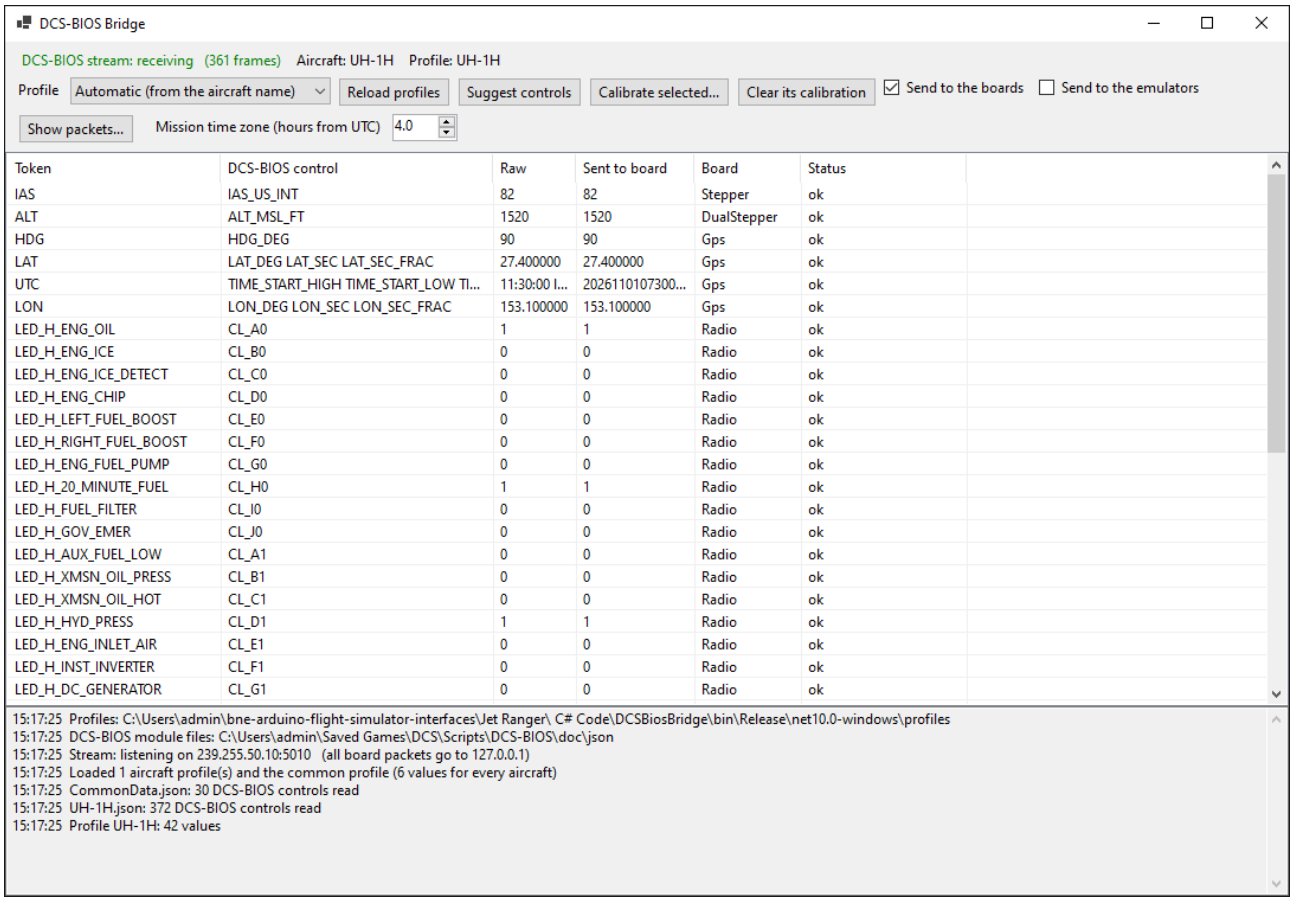


Figure 31 — DCSBiosBridge with a synthetic UH-1H stream: three caution lights on, the common values, the mission clock with its time zone box and the Huey LED rows (scroll for the gauges)

The window shows the stream status, the aircraft name and the profile in use; the **Profile** box (Automatic or one chosen by hand); **Reload profiles**; **Suggest controls** (lists the module's controls whose identifier or description match each unmapped token's `hint`, best first, in the log); **Calibrate selected...** and **Clear its calibration** (below); **Send to the boards**; the **Mission time zone** box (above); a table of tokens with their control, raw value, value sent, board and status; and a log. Options: `--board-host <ip>` (send every board packet there, for testing), `--json-dir`, `--profile-dir`, `--module <name>`, `--utc-offset <hours>`, `--no-send`, `--logfile`, and `--emulators`, `--emulators-only` and `--emulator-net <a.b.c>` (with the **Send to the emulators** box, see the Panel emulators chapter: the packets also go to `JetRangerPanelEmulator`), and `--show-packets` and `--packet-board <board>` (with the **Show packets...** button, see Packet monitor below).

13.4 The Huey profile

`profiles\UH-1H.json` maps the 20 `LED_H_*` tokens to the UH-1H *Caution Lights Panel* indicator lights `CL_A0` to `CL_J1` (DCS cockpit arguments 91 to 110), in the order ENG_OIL, ENG_ICE, ENG_ICE_DETECT, ENG_CHIP, LEFT_FUEL_BOOST, RIGHT_FUEL_BOOST, ENG_FUEL_PUMP, 20_MINUTE_FUEL, FUEL_FILTER, GOV_EMER, AUX_FUEL_LOW, XMSN_OIL_PRESS, XMSN_OIL_HOT, HYD_PRESS, ENG_INLET_AIR, INST_INVERTER, DC_GENERATOR, EXT_POWER, CHIP_DETECTOR, IFF. The aircraft names are `UH-1H` and `Bell47_2`. The LEDs light only while the Radio is powered (`NAVCOM1:1`). No bus voltage is mapped: `DC_VOLTAGE` and `DC_V` are 0 to 1 needle positions with no known voltage scale, so the aircraft counts as powered while data flows.

13.5 Gauges

Every aircraft (`profiles\_common.json`, DCS-BIOS CommonData). `IAS` (knots), `ALT` (feet above sea level, no barometric correction), `HDG` (true degrees) and `LAT` and `LON` (degrees, built from the latitude and longitude pieces; DCS-BIOS names the minutes `LAT_SEC`, south and west are negative). They go to the Stepper (`IAS`), the Dual Stepper (`ALT`) and the GPS interface (`LAT`, `LON`, `ALT`, `IAS`, `HDG` and `UTC`).

GPS date and time. The Lowrance GPS follows the simulator's clock: the bridge sends `UTC:yyyyMMddHHmmss.fff` in the 500 ms GPS packet. DCS-BIOS gives the mission start time (`TIME_START_HIGH` and `LOW`, seconds) and the model time (`TIME_MODEL_HIGH` and `LOW`, hundredths of a second since the start); their sum is the mission clock, which is the **theatre's local time**, so it follows the time of day set in the mission and time acceleration. DCS-BIOS has no time zone and no date, so the window has a **Mission time zone (hours from UTC)** box (also `--utc-offset <hours>`) that is taken off the clock, and the PC's UTC date is used. Set it to 4 for the Caucasus and the Persian Gulf, 3 for Syria, -8 for Nevada. The box starts at 0 and is not remembered between runs. That the start time is seconds since midnight of the mission's local day has not been checked against live DCS.

UH-1H. `AGL` (radar altitude, feet) is exact: it is read from the radar altimeter's four drum digits `RALT_DIGIT_1` to `4`. The other 16 gauge values are the cockpit needles' **positions** (0 to 1 of the dial, all DCS-BIOS exports for them) and run on a **first-guess scale**: the dial's whole sweep is mapped onto the panel gauge's own range.

Token	DCS-BIOS control	Provisional mapping of the dial
`VSI`	`VVI_P`	-1 to +1 = -1750 to +1750 ft/min
`TQ`	`TORQUE_PRESS`	0 to 1 = 0 to 120 percent
`N1`	`GAS_RPM`	0 to 1 = 0 to 103 percent
`ITT`	`EXHAUST_TEMP`	0 to 1 = 100 to 900 C
`FUEL`	`FUEL_QTY`	0 to 1 = 0 to 75 gallons
`OILP`, `OILT`	`ENG_OIL_PRESS`, `ENG_OIL_TEMP`	0 to 1 = 0 to 150 psi, 0 to 150 C
`XMSNP`, `XMSNT`	`TRANS_OIL_PRESS`, `TRANS_OIL_TEMP`	0 to 1 = 0 to 150 psi, 0 to 150 C
`RPMR`, `RPME`	`ROTOR_RPM`, `ENG_RPM`	0 to 1 = 0 to 110 percent
`ELECTRICALLOAD`	`MAIN_GEN_LOAD`	0 to 1 = 0 to 100 percent
`FUELLOAD`	`FUEL_PRESS`	0 to 1 = 0 to 30 psi
`PITCH`, `BANK`	`PADI_PITCH`, `PADI_ROLL`	-1 to +1 = -90 to +90 and -180 to +180 degrees (scale and sign unchecked)

They show **uncalibrated** in the Status column. To calibrate one, select its row, fly to a reading, press **Calibrate selected...** and type what the cockpit gauge reads (in the units the panel board uses); the bridge stores the needle position DCS-BIOS reports at that moment as a point of that value. Two or more points replace the guess (straight lines between the points, held beyond the ends) and the Status shows *calibrated (n points)*. The points are kept in `profiles\UH-1H.calibration.json` next to the profile; **Clear its calibration** removes a token's points. The airspeed, altitude and clock gauges of the Huey are not mapped (the real airspeed and altitude come from CommonData).

13.6 Testing without DCS

`python tests\test_bridge.py` starts the bridge with `--board-host 127.0.0.1`, sends a fake DCS-BIOS stream for an aircraft called TestModule (each frame split over two datagrams) and checks the board packets: lamp decoding with mask and shift, the radio power rule, `BPWR` following the bus, the aircraft's value beating the common one, `AGL` from the drum digits, calibration points against the plain scale, the GPS packet, and an aircraft with no profile. All 15 checks pass (including the GPS packet's `UTC:` for a mission clock of 11:30:00 at +4 hours). It stops only the bridge process it started.

13.7 Known gaps

- The 16 needle-position gauges are not calibrated against the live cockpit.
- No bus voltage for the Huey, and no profile for any other aircraft yet.

- No inputs: DCS-BIOS commands (UDP 7778) would set exact switch positions, and the radio commands on port 27001 and the autopilot values are not handled; port 27003 (ADF frequency, radar follow ID) is not read.
- The landing gear lamps are not sent.

13.8 Packet monitor

Packet monitor. Show packets... in the toolbar opens a window that lists every payload as it is sent: one line per packet with the time, the target board (Radio .101, Servo .102, Stepper .105, Dual Stepper .106, ADF and Upper .107, or the GPS interface), where it went (*board*, *emulator*, *board* + *emulator* or *local*) and the payload text. The **Target board** box filters the list to one board; **Pause** and **Clear** do what they say. Command line: `--show-packets` opens it at start, `--packet-board <radio, servo, stepper, dual, adf, gps or a last number such as 105>` opens it filtered. `BoardTargets` raises a `PacketSent` event for every send and `PacketMonitorForm` shows the queued packets ten times a second (the last 6,000 are kept for re-filtering), so a busy bridge is not slowed. Only packets that are really sent are shown. Tested with the bridge and its synthetic DCS-BIOS stream: all boards, the `--packet-board stepper` filter, and the filter box changed while the window was open.

14 X-Plane bridge (XPlaneBridge)

`XPlaneBridge` is the bridge for **X-Plane 11 and 12**. It reads **datarefs over X-Plane's built-in UDP interface** (no plugin), converts them with a **profile**, and sends them to the boards in the same `D,TOKEN:value` packets (UDP 13136) as the other bridges, so the board firmware does not change. It is a Windows Forms program (`net10.0-windows`) and is **output only**. Run only one bridge at a time.

It builds with no warnings and passes `tests\test_bridge.py` (a fake X-Plane answering the dataref subscriptions, 11 checks, with the generic profile and the Robinson R22 profile). It has **not been run against a real X-Plane yet**. The dataref names in the profiles were checked against X-Plane 12's `Resources\plugins\DataRefs.txt`.

14.1 How it works

1. **Dataref link (`XPlaneLink.cs`).** One UDP socket on an ephemeral port. For every dataref the profile needs, the bridge sends an `RREF` request (`RREF` and a zero byte, an int32 frequency in Hz, an int32 index and a 400 byte dataref path; an array element is written `name[0]`) to X-Plane, by default `127.0.0.1:49000`. X-Plane answers to the request's source address and port with `RREF,` followed by 8 byte records of an int32 index and a float32 value. The requests are repeated every 5 seconds, at 10 Hz, so the bridge can start before X-Plane and survives X-Plane restarting. On exit the bridge cancels them (frequency 0).
2. **Finding X-Plane.** With no `--xplane-host` the bridge listens for X-Plane's beacon (multicast 239.255.1.1 port 49707) and uses the host and port it announces; with no beacon it keeps trying `127.0.0.1:49000`. Use `--xplane-host` and `--xplane-port` when X-Plane is on another PC or its receive port was changed. X-Plane must be allowed to receive UDP data (Settings, Network).
3. **Profiles.** `profiles\_common.json` holds the values nearly every aircraft gives through the standard datarefs and is added to every aircraft profile (the aircraft's own value for a token wins). On its own, as *Generic (common values, any aircraft)* in the Profile box, it drives any aircraft. X-Plane gives no reliable aircraft name over this interface, so the **profile is chosen by hand** (or with `--module <name>`). A value has a `token`, a `dataref` (or `datarefs` for the clock), an optional `divisor` dataref (a ratio, for torque), a `type` (`analog`, `lamp` or `hhmm`), `scale`, `offset`, `decimals`, `suffix`, `threshold`, `invert`, `hint` and `note`.
4. **Boards, power rules and cadence** are those of the DCS bridge: one packet per board through `Payloads.Build`, `BPWR` to the ADF and Upper board (.107), the 20 V and 24 V thresholds when a `MAINBUS` or `AVIONICSBUS` voltage is mapped (otherwise the aircraft counts as powered while data flows), an unpowered aircraft sends `AGL:0` to the Stepper, and nothing is sent for 3 seconds after X-Plane's data stops, so the boards' 30 second watchdogs return the gauges to zero.

14.2 The common values

Boards	Tokens and datarefs
Stepper	`IAS` (`airspeed_kts_pilot`), `VSI` (`vvi_fpm_pilot`), `AGL` (`radio_altimeter_height_ft_pilot`), `N1` (`N1_percent[0]`), `ITT` (`ITT_deg_C[0]`), `TQ` (`torque_n_mtr[0]` over `POINT_max_TRQ[0]`, as a percent), `FUEL` (`m_fuel_total` in kg times 0.3303, as US gallons of jet fuel)
Dual Stepper	`ALT` (`altitude_ft_pilot`), `RPME` (`N2_percent[0]`), `OILP`, `OILT` (`oil_pressure_psi[0]`, `oil_temperature_deg_C[0]`), `FUELLOAD` (`fuel_pressure_psi[0]`), `ZULU` (zulu hours and minutes), `TEMP` (outside air temperature with the C suffix)
Servo	`PITCH` (`pitch_AHARS_deg_pilot`), `BANK` (`roll_AHARS_deg_pilot`), and the lamps `RLOW`, `GENF`, `LOWF`, `EC`, `FPMP` and `EOUT` (engine 1 not running) from X-Plane's generic annunciators
Radio	Ten Huey Warning Panel LEDs from the nearest generic annunciator (approximations: X-Plane has no Huey caution panel)
GPS interface	`LAT`, `LON` (6 decimals), `ALT`, `IAS`, `HDG` (`true_psi`), and `UTC`: the simulator's date and time from `sim/time/zulu_time_sec` with `sim/cockpit2/clock_timer/zulu_month` and `zulu_day` (X-Plane has no year, so the PC's UTC year is used), sent as

Boards	Tokens and datarefs
	`UTC:yyyyMMddHHmmss.fff` so the Lowrance GPS follows the simulator's clock

Some aircraft leave datarefs at 0, and that gauge then sits at zero; use **Search datarefs...** to find the right ones and put them in an aircraft profile.

14.3 Aircraft profiles

- **Robinson R22** (`profiles\Robinson R22.json`): rotor speed from `prop_speed_rpm[0]` (530 rpm = 100 percent) and the 14 V bus doubled to `MAINBUS`, so the boards' 20 V and 24 V thresholds work.
- **Sikorsky S-76** (`profiles\Sikorsky S-76.json`): rotor speed with 293 rpm = 100 percent and a 28 V bus.

The rotor speeds and bus scales are first guesses: check them against the cockpit gauges.

14.4 Window and command line

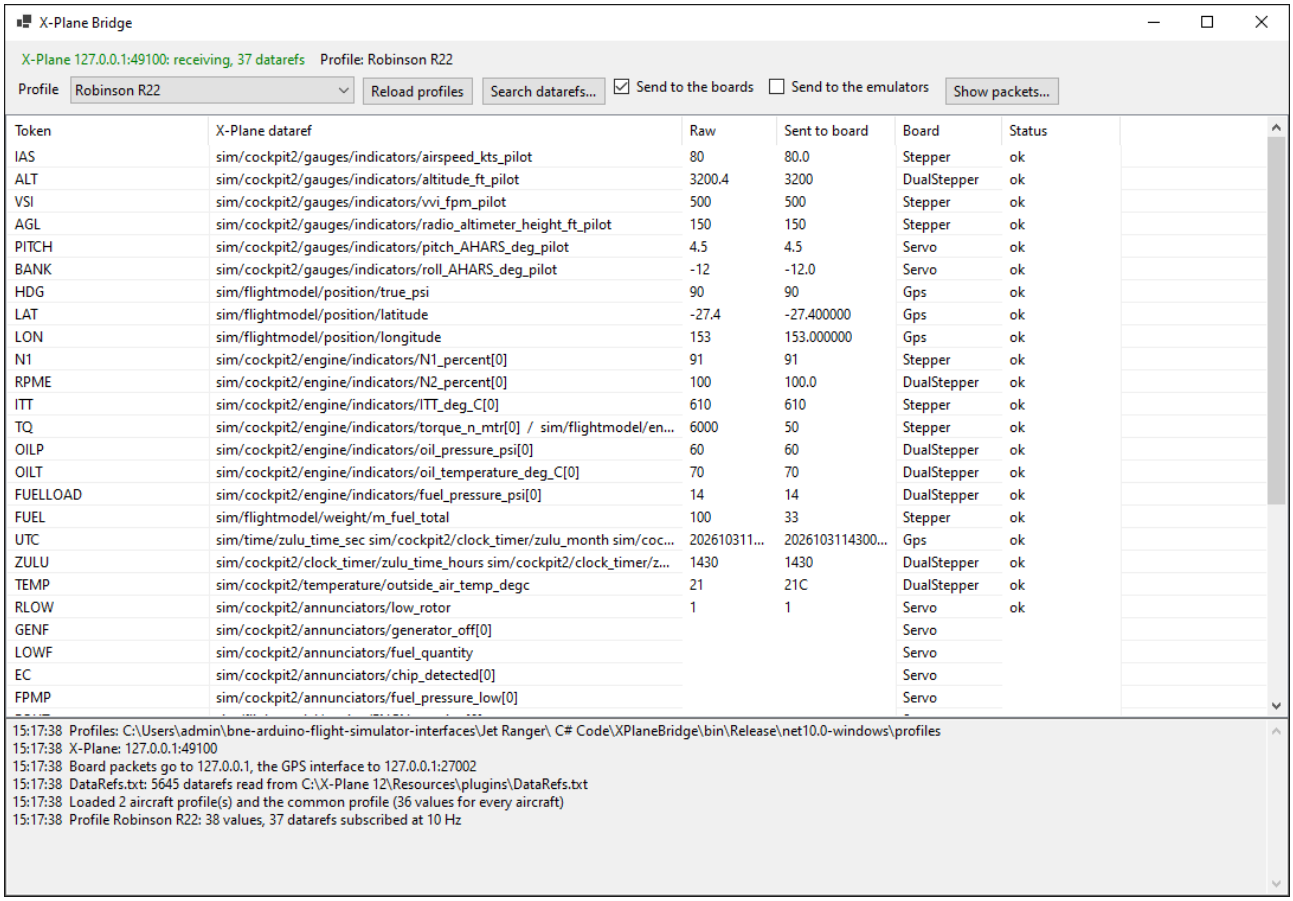


Figure 32 — XPlaneBridge against a fake X-Plane with the Robinson R22 profile (scroll for the Huey LEDs, rotor speed and bus voltage)

The window shows the X-Plane address and whether data is arriving, the **Profile** box, **Reload**, **Search datarefs...** (words matched against the names and descriptions in `DataRefs.txt`, best first, in the log), **Send to the boards**, a table of tokens with dataref, raw value, value sent and board, and a log. Options: `--board-host <ip>` (send every board packet there, for testing), `--xplane-host`, `--xplane-port`, `--datarefs <DataRefs.txt>` (default `C:\X-Plane 12\Resources\plugins\DataRefs.txt`), `--profile-dir`, `--module <name>`, `--no-send`, `--logfile`, and `--emulators`, `--emulators-only` and `--emulator-net <a.b.c>` (with the **Send to the emulators** box, see the Panel emulators chapter: the packets also go to `JetRangerPanelEmulator`), and `--show-packets` and `--packet-board <board>` (with the **Show packets...** button, see Packet monitor below).

14.5 Testing without X-Plane

`python tests\test_bridge.py` starts a fake X-Plane on UDP 49100, runs the bridge against it first with the generic profile and then with the Robinson R22 profile (switching the bus voltage off half way), and checks the packets on UDP 13136 and 27002. It stops only the processes it started.

14.6 Known gaps

- Not yet run against a real X-Plane.
- No inputs (the switches already reach X-Plane through the USB HID board as a joystick), no radio commands on port 27001, port 27003 is not read, and the landing gear lamps are not sent.
- Only the Robinson R22 and Sikorsky S-76 have aircraft profiles; the Huey LEDs are approximations.

14.7 Packet monitor

Packet monitor. Show packets... in the toolbar opens a window that lists every payload as it is sent: one line per packet with the time, the target board (Radio .101, Servo .102, Stepper .105, Dual Stepper .106, ADF and Upper .107, or the GPS interface), where it went (*board*, *emulator*, *board + emulator* or *local*) and the payload text. The **Target board** box filters the list to one board; **Pause** and **Clear** do what they say. Command line: `--show-packets` opens it at start, `--packet-board <radio, servo, stepper, dual, adf, gps or a last number such as 105>` opens it filtered. `BoardTargets` raises a `PacketSent` event for every send and `PacketMonitorForm` shows the queued packets ten times a second (the last 6,000 are kept for re-filtering), so a busy bridge is not slowed. Only packets that are really sent are shown. It shares its code with the DCS bridge's monitor, which was tested with a synthetic stream; this bridge was built and its window checked.

15 Health Monitor (JetRangerHealthMonitor)

A small Windows Forms dashboard (namespace `MegaHealthMonitor`) that watches for the 10 second heartbeat strings sent by the networked Arduino boards and shows one traffic light per board: Comm/Nav, Servo, Joystick, Upper Input, Stepper, Dual Stepper and ADF. A **Show Voltages** button adds a table of the last voltage reading from each board.

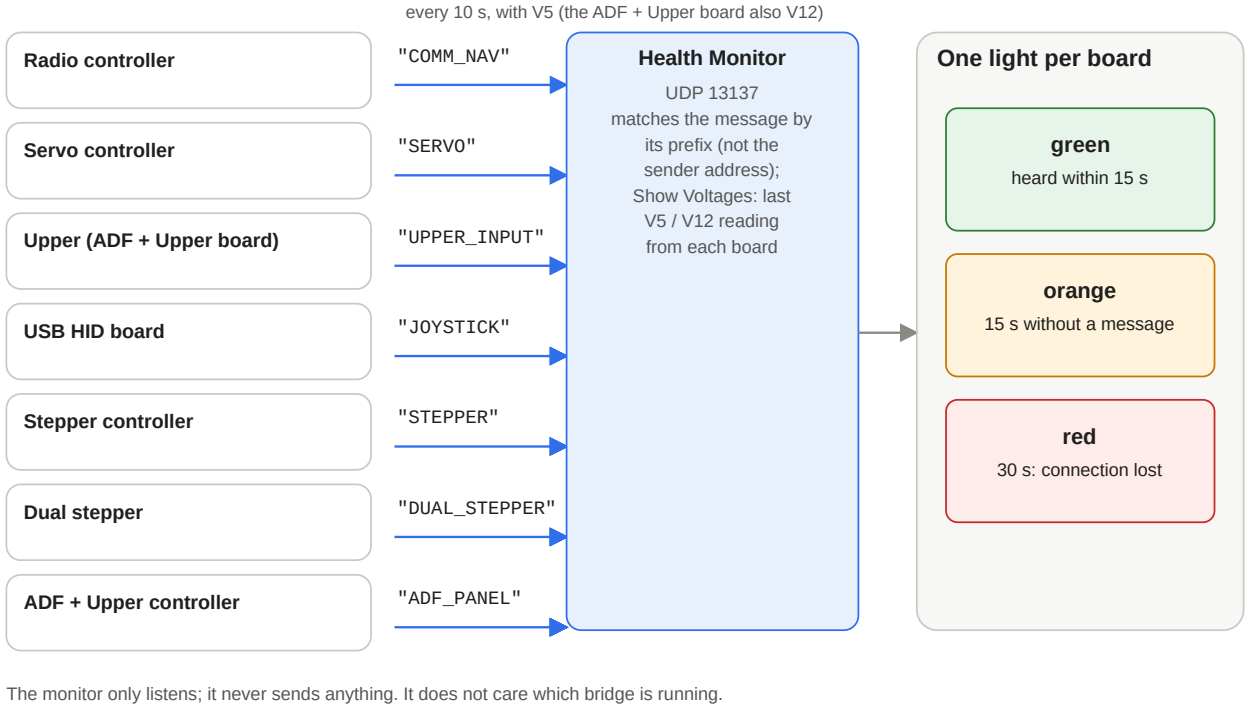


Figure 33 — Heartbeats in, one light per board out

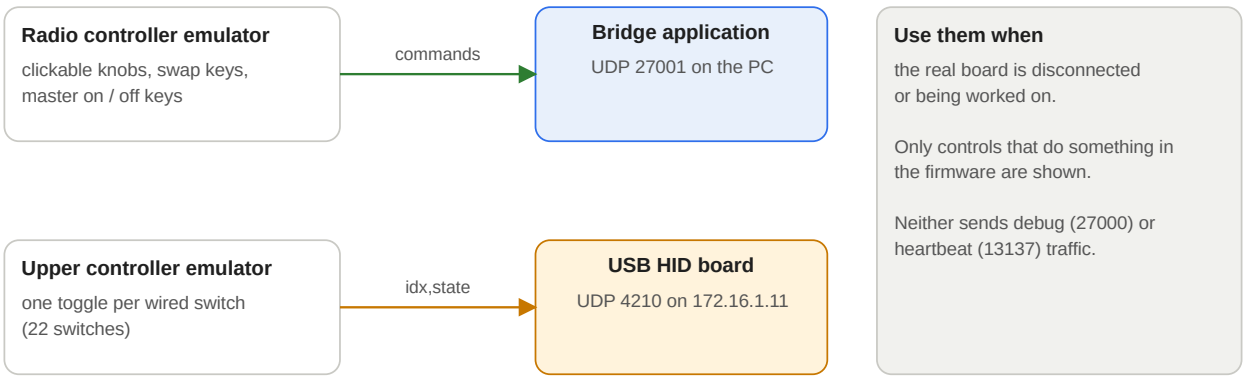
15.1 Program flow

1. `Form1_Load` starts a background task that binds a `UdpClient` to port **13137** and loops on `ReceiveAsync()`.
2. Each packet is matched by **prefix** (not by sender address): `COMM_NAV`, `SERVO`, `JOYSTICK`, `UPPER_INPUT`, `STEPPER`, `DUAL_STEPPER` or `ADF_PANEL`. A match updates that board's last-received time and turns its label green; each match can also be logged (sender and timestamp) to a list box.
3. A timer calls `CheckHealth()` about once a second: 30 seconds or more since the last packet turns the indicator **red** (connection lost), 15 seconds or more **orange** (connection warning), otherwise it stays green.
4. *Toggle logs* shows or hides the log list box and resizes the form.
5. **Voltages and firmware.** Every board appends its 5 V rail and its firmware version to its heartbeat after its name (`SERVO V5:5.02 FW:1.0.2`) and the ADF and Upper controller also its 12 V rail (`ADF_PANEL V5:5.03 V12:12.41 FW:1.0.3`; the same board also sends the readings under the old Upper name, `UPPER_INPUT V5:5.03 V12:12.41`, so the Upper light shows the same board). `RecordVoltages()` keeps the last reading and its time for each board, and the *Show Voltages* button shows a table with one row per board: the last 5 V reading, the 12 V reading (only the ADF measures it; the other rows say "not measured") the time of the last reading with "N s ago", and the **Firmware** column (`1.0 build 2`, from `FW:`; *not reported* for a board whose firmware does not send it, grey when the reading is stale). The window is 105 pixels wider with the fifth column. A reading outside its limits (5 V: 4.75 to 5.25, 12 V: 10.8 to 15.0) is orange, and a reading older than 35 seconds, or none yet, is grey. A board that has not been reflashed since the voltage was added shows "no reading". `ApplyLayout()` sizes the window for whichever of the log list and the voltage table are showing.

The monitor is a passive listener: it never sends anything and does not care which bridge is running. The Dual Stepper's prefix is deliberately distinct from `STEPPER` (the monitor uses `StartsWith`, so overlapping prefixes would light the wrong indicator). The monitor finds the voltages with a pattern (`V5:` and `V12:` followed by a number), so a heartbeat without them still lights the board's indicator.

16 Radio and Upper controller emulators

Two small Windows Forms panels stand in for the physical boards when they are not connected. Each shows only the controls that actually do something in the firmware.



The Radio emulator still sends a bare frequency for every radio, so only COM1 tunes from it.

(The real board sends labelled COM2_STANDBY:, NAV1_STANDBY: and NAV2_STANDBY: values, which both bridges understand.)

Figure 34 — The emulators and where they send

16.1 JetRangerRadioControllerEmulator

Shows the two character-OLED displays (COM and NAV), a clickable knob for each of the eight encoders (click the right half to turn one detent clockwise, the left half for counter-clockwise), the COM1 and COM2 *XFER* swap keys and the *MASTER ON* and *MASTER OFF* keys. It sends the same payloads the board would send to the bridge on UDP 27001 (default host `172.16.1.10`):

- each knob sends the new standby frequency as a bare ASCII number such as `121.010`;
- the XFER keys swap the local active and standby values and send `COM1_RADIO_SWAP`, or `COM2_RADIO_SWAP` followed by `AVIONICS_MASTER_SET`;
- the MASTER keys send `MASTER_BATTERY_ON` with `ALTERNATOR_ON`, or the two `_OFF` commands.

Limits: the emulator still sends a bare frequency for every radio, so only COM1 tunes correctly from it (the real board sends `COM2_STANDBY:`, `NAV1_STANDBY:` and `NAV2_STANDBY:`). It has no NAV swap keys and sends no debug (27000) or heartbeat (13137) traffic. Its knob model is 5 kHz per minor detent, not the 25 kHz and 50 kHz channel spacing the real board now uses.

16.2 JetRangerUpperControllerEmulator

Shows one toggle button per switch the Upper panel acts on (22 switches in groups: Cabin Heat, Air, Circuit Breakers, Instruments, Electrical, Lights; green means on) and sends `index,state` packets to the USB HID board (default `172.16.1.11:4210`) exactly as the real board would (the Upper matrix now runs on the combined ADF and Upper board, with the same indexes). Indexes 7 to 10, 18 to 21, 29 to 38 and 40 to 95 are wired in the matrix (or, for 7 to 9, on the Servo board) but drive nothing and are not shown; the combined board's matrix has no positions above 95.

17 StepperVSITester

A Windows Forms bench tool that sends real-unit values or raw step counts straight to the boards' UDP handlers without a simulator. It started as a vertical speed tester (hence the name) and now covers the whole Stepper and Dual Stepper boards plus the servo pitch and bank.

17.1 What it sends

Target	Controls
Stepper .105	`VSI`, `AGL`, `ITT`, `IAS`, `TQ`, `N1`, `FUEL` (real values, with sliders for VSI, AGL, ITT, IAS); `OILT`, `OILP`, `XMSNT`, `XMSNP` (also broadcast to .106)
Dual Stepper .106	`ALT`, `RPME`, `RPMR` (broadcast to both stepper boards), `FUELLOAD`, `ELECTRICALLOAD`, `ZULU` (hour and minute boxes), `TEMP` (text)
Servo .102	`PITCH`, `BANK` (degrees, one decimal) and `PITCHRAW`, `BANKRAW`
Raw step panel	A drop-down of raw codes (`IASRAW`, `VSIRAW`, `ALTRAW`, `OILTRAW`, `OILPRAW`, `XMSNTRAW`, `XMSNPRAW`, `ITTRAW`, `RPMERAW`, `RPMRRAW`, `N1RAW`, `FUELRAW`, `TQRAW`, `FUELLOADRAW`, `ELECTRICALLOADRAW`) sent to both stepper boards, with -1 step and +1 step buttons

Each row has a slider or text box, *Send* (or Enter) and *Zero*. Values are sent as typed, with no conversion; `RPME` and `RPMR` carry one decimal. Raw codes that a board does not have are silent no-ops there (for example `FUELLOADRAW` on the Stepper board).

Notes: the raw panel still lists `FLAPS`, `AOA`, `GFORCE`, `SPDMAX` and `N1RAW`; the first four no longer exist on any board, and `N1RAW` does nothing on the Dual Stepper. `RPME` and `RPMR` are no longer handled by the Stepper board, so only the Dual Stepper acts on them. Run it **instead of** a bridge, since both would otherwise fight over what drives the gauges.

18 SwingTester

A Windows Forms (.NET 10) bench tool that triggers each gauge's start-up swing over UDP, one gauge at a time or all of them in a fixed order. It also lights the warning lamps and, in a **Panel LEDs** group, each of the 39 Auto Pilot, Audio and Huey Warning Panel LEDs on its own (a toggle per LED sending `D,NAVCOM1:1,LED_xxx:1/0` to the Radio controller; the Huey LEDs are in two columns, *Huey Warning Panel* and *(continued)*, and use `LED_H_xxx` tokens), all on or off, in a chase (one at a time, with the on gap), or with the Radio's own `C,LEDTEST` board test. The **Auto Pilot display** group sends the five values (HDG, VS, CRS, IAS, ALT) to the Radio for the Auto Pilot panel's OLED (`D,NAVCOM1:1,APHDG:090,APVS:0,APCRS:110,APIAS:80,APALT:3200`), with a *Start-up values* button, and the **ADF panel landing gear lamps** group (nose, left, right, All ON and All OFF) lights the lamps on the ADF controller (`D,LDGN:1,LDGL:0,LDGR:0`); an `ADF (.107)` box holds the ADF controller's address. Build with `dotnet build -c Release` in its folder; the executable is `bin\Release\net10.0-windows\SwingTester.exe`.

18.1 How it works

It sends a control packet `C,<COMMAND>:1` to `<board IP>:13136`. The boards do not report completion, so after each command the app waits a per-row number of seconds (editable in the window, not saved between runs) before sending the next. A swing blocks the board's loop; too short a wait lets the next command queue in the board's UDP buffer and the order across different boards drifts.

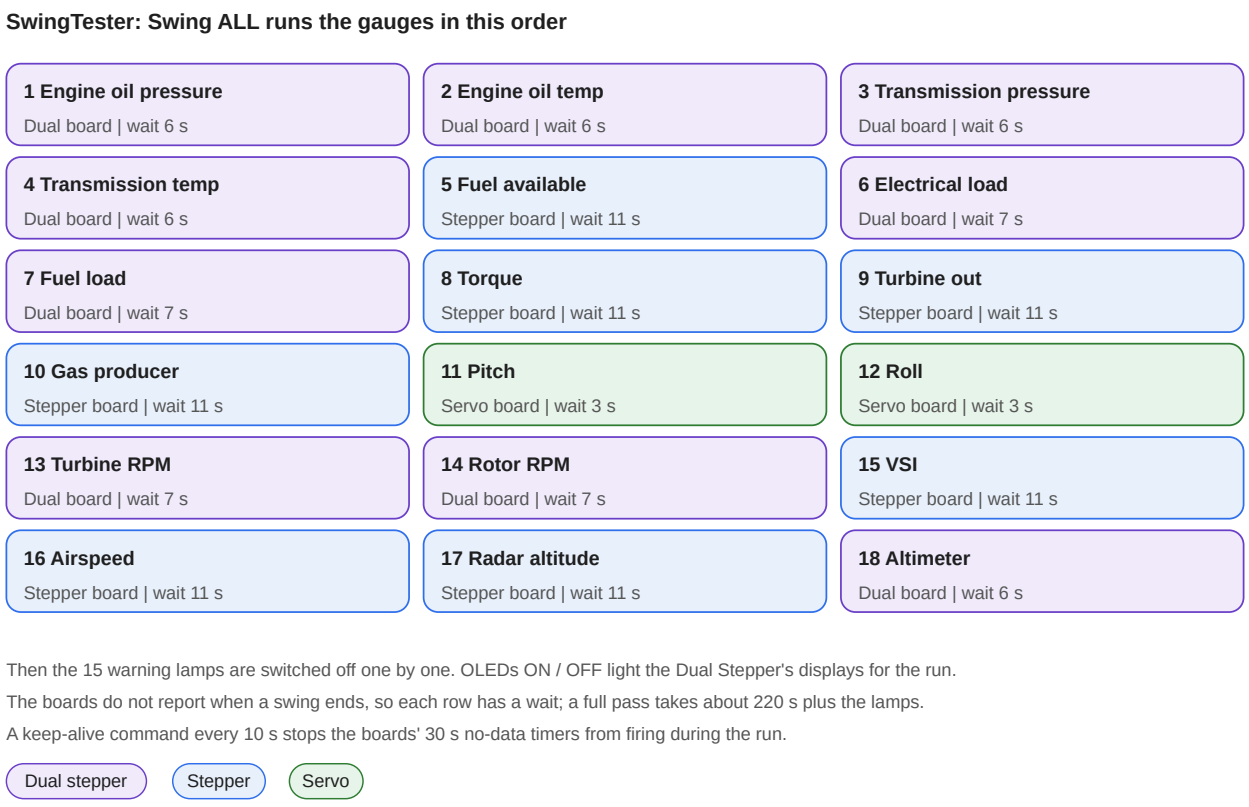


Figure 35 — Swing ALL sequence

The default waits are the swing times measured on the real boards plus about one second of margin: Dual Stepper 5.0 to 5.2 s (oil and transmission gauges, altimeter) and 6.4 to 6.5 s (load and RPM gauges); Stepper 9.5 to 10.7 s; Servo about 2.1 s. A full pass takes about 220 seconds.

18.2 Window

- The three board IPs at the top (defaults `172.16.1.105`, `.106`, `.102`).

- **Swing ALL (in order)**, **Stop** (cancels the remaining waits; a swing already started still finishes), **OLEDs ON** and **OLEDs OFF**, and the check box *Light OLEDs during Swing ALL* (on by default).
- One row per gauge (18 rows on the left) with its board, a seconds box and a **Swing** button; the running row is highlighted gold. The 15 warning-lamp buttons are on the right.
- **OLEDs ON** sends `C,OLEDTEST:1` to the Dual Stepper and **OLEDs OFF** sends `C,OLEDTEST:0`. The **Temp OLED** box (default `21C`) with **Send TEMP** sends `D,TEMP:<text>`.
- **Reset boards...** opens a dialog with a tick box per board (Radio .101, Servo .102, Upper .103, Stepper .105, Dual Stepper .106, USB HID .11; all ticked by default; the ADF and Upper board .107 is not in the list yet and the Upper .103 entry is the retired address) and sends `C,RESET:1` to each ticked board (the HID board on port 4210). The Dual Stepper takes about a minute to restart and the HID board briefly drops its USB devices. The button is disabled while a run is active.

18.3 COM ports window

The **COM ports...** button (next to *Reset boards...*) opens a window that lists the PC's serial ports (`PortsForm`): the port number, Windows' description, the board type guessed from the USB vendor and product ids (Arduino Mega 2560, Leonardo, Uno, Due, CH340 clone, FTDI, CP210x), the **USB serial number** (genuine Arduino boards have one; a clone usually has none, and the window then says so instead of showing Windows' made-up id), whether **another program has the port open** (for example the Arduino IDE serial monitor, which blocks flashing), the device status and the USB device id. The list comes from WMI (`Win32_PnPEntity`) and the device tree (`CM_Get_Parent` finds the USB device behind a composite function) and refreshes by itself when a device is plugged in or removed. **Refresh** rereads it and **Copy list** puts it on the clipboard as tab-separated text.

Restart selected USB device runs `pnputil /restart-device` on the selected board's USB device after a confirmation. Windows asks for administrator rights. This stops and starts the device's driver (the same as disabling and enabling it in Device Manager) so the COM port reappears; it cannot cut the power to the port. A board that appears as several devices (the Leonardo) is restarted as a whole. If the administrator prompt is declined or `pnputil` fails the window says so. The window needs the port closed in other programs first.

18.4 Brightness

A **Brightness (0-15)** slider sends `C,B:<level>` to the Radio (.101), Servo (.102), Upper (.103, retired), Stepper (.105), Dual Stepper (.106) and ADF (.107, the combined ADF and Upper board), every board except the USB HID board, each time it is moved; **Send to all boards** resends the current level, and the status line lists the boards sent to. The Radio, Servo, Stepper, Dual Stepper and ADF and Upper controller set their backlight from it (the Radio also sets its panel LEDs' intensity). **Backlight power ON** and **OFF** send `D,BPWR:1` and `D,BPWR:0` to the Radio, Servo, Stepper, Dual Stepper, ADF and Upper (and the retired .103) controller, the packet the bridges send from the main bus voltage, to test the main bus rule from the bench: OFF switches the backlight off and ON returns it to the last brightness commanded.

18.5 Warning lamps

The Servo board's lamps are driven by ordinary `D,<code>:1` and `D,<code>:0` packets to `172.16.1.102:13136`. There is a toggle per lamp (gold when on), **Lamps ON (in order)** (one lamp at a time, *on gap* default 1000 ms), **Lamps OFF (in order)** (*off gap* default 400 ms) and the check box *Light warning lamps during Swing ALL* (on by default). Turn-off order, also the button order: `SCF`, `GENF`, `FFLTR`, `BD`, `EC`, `BHOT`, `TOPW`, `EOUT`, `LOWF`, `FPMP`, `TRC`, `TC`, `BTMP`, `TOTW`, `RLOW`.

Swing ALL with the check box on first lights the 15 lamps one at a time (about 15 seconds), then runs the swings, then turns the lamps off one at a time (always at the end, also after Stop or an error). The Servo board only drives a lamp when the incoming value differs from the state it last remembered, so keep the bridge off while testing.

18.6 Keep-alive

Every sketch resets its gauges after 30 seconds with no packet. During a full run some boards get no command for 100 seconds or more, so while any run is active (Swing ALL, an individual swing or the lamp sequence) the app sends `C,KEEPALIVE:1` to all three boards every 10 seconds (and once at the start). The boards ignore the command name but count the packet as received data. The keep-alive stops when the run ends, so lamps or OLEDs left on manually still time out after 30 seconds.

19 Debug listener (UDP reflector)

Every board sends debug text to `172.16.1.10:27000`. A small Python reflector on the primary PC prints each line and can forward a copy to a second listener, so an analysis script can read the same stream. Only one program can own the port, so use the reflector's forwarding rather than starting a second listener. Keep the reflector running while boards are being flashed and restarted: the start-up lines (Ethernet Started, I2C scan, Setup Complete) are the quickest check that a flash worked.

Other helpers in `Jet Ranger Python Code/`: `Export_From_SimConnect.py` (a SimConnect exporter), `Test_Receiver.py` (a generic UDP JSON receiver) and `send_wol.py` (wake-on-LAN).

20 Lowrance AirMap GPS interface (LowranceAirMapGps)

`LowranceAirMapGps` makes the PC act as the GPS for a **Lowrance AirMap 2000C**. It is a Windows Forms program (.NET 10) that receives aircraft information over UDP from a bridge (Prepar3D / FSX, MSFS, DCS World or X-Plane) or the bench tester and sends NMEA 0183 sentences with the position, altitude, airspeed, heading and time over a serial port, twice a second.

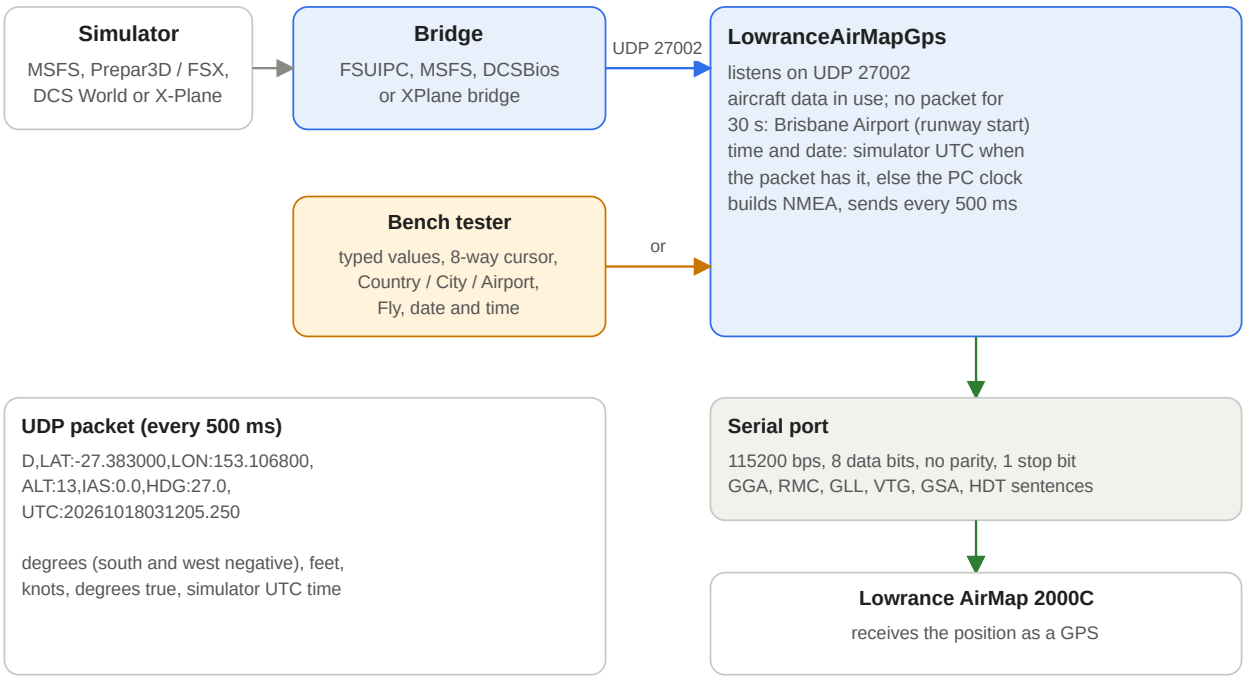


Figure 36 — From the simulator to the GPS

20.1 Behaviour

1. **Serial port:** 115200 bps, 8 data bits, no parity, 1 stop bit, no handshake. The chosen port is saved in `%AppData%\LowranceAirMapGps\settings.json`. If none is saved (or the saved port no longer exists) a dialog asks for one at start-up, and *Change serial port...* reopens it. If the port cannot be opened or a write fails the program retries every 3 seconds, so a USB serial adapter can be plugged in later.
2. **UDP input:** UDP port **27002** (`UdpPort` in the settings file). Two layouts are accepted: key/value `D,LAT:-33.9461,LON:151.1772,ALT:1500,IAS:110,HDG:90,UTC:20261018031205.250` (any subset of keys; missing keys keep the previous value) and CSV `lat,lon,altitude_ft,airspeed_kt,heading_deg[,utc]`. Units are degrees (south and west negative), feet, knots and degrees true. Packets that are not aircraft data or are out of range are ignored and do not reset the 30 second timer.
3. **Default position:** with no accepted packet for 30 seconds, or since start-up, the program sends Brisbane Airport: -27.3830, 153.1068, 13 ft, speed 0, heading 027 degrees true, the threshold of runway 01L (016 magnetic). The values can be changed in the settings file (`DefaultLatitude`, `DefaultLongitude`, `DefaultAltitudeFt`, `DefaultHeadingDeg`). They come from SkyVector and were not checked against the AIP.
4. **Output rate:** one set of sentences every 500 ms.

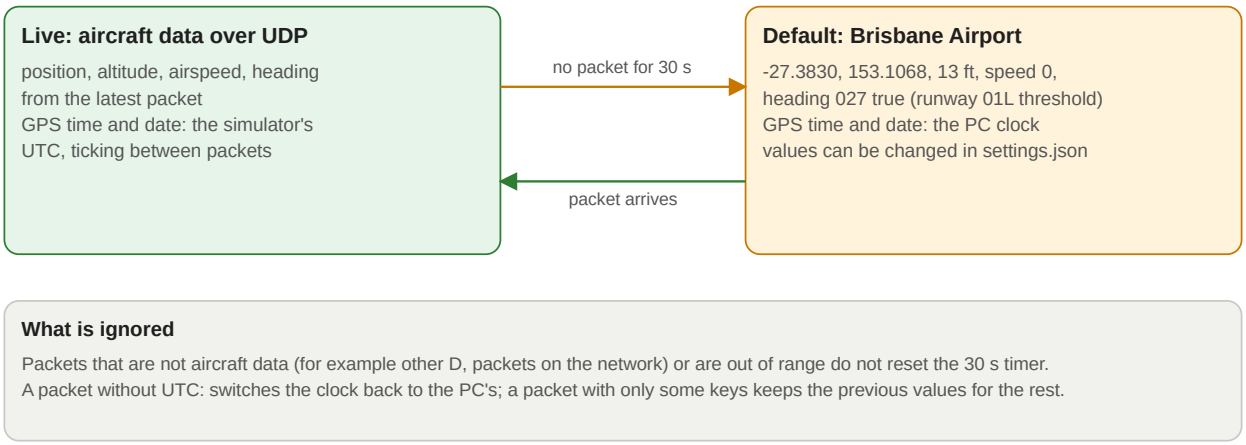


Figure 37 — Live data and the Brisbane default

20.2 NMEA sentences

Each update sends `$GPGGA` (position, altitude in metres), `$GPRMC` (position, speed in knots, track, date), `$GPGLL`, `$GPVTG` (track, speed in knots and km/h), `$GPGSA` (a fixed 3D fix with 8 satellites) and `$GPHDT` (heading), each with its `*CS` checksum and CR LF. NMEA has no airspeed sentence a GPS accepts, so the airspeed is sent as the GPS speed in RMC and VTG, and the heading as the track and as HDT.

20.3 GPS date and time

When a packet carries `UTC:` the program takes the offset between the simulator's UTC and the PC's UTC and adds it to the PC clock for every sentence, so the GPS time and date (the RMC date field and every sentence time) are the simulator's and tick smoothly between packets. The offset is only re-taken when the simulator time moves more than 1.5 seconds from where it predicts, for example when the user changes the time or the simulation rate is not 1x. The window shows *GPS time ... (simulator time)* or *(PC clock)*. The PC clock is used when no packet has arrived for 30 seconds or when a packet has no `UTC:`.

The FSUIPC, MSFS, DCS and X-Plane bridges all send `UTC:`. The DCS bridge's clock is the mission's local time, so it has a time zone box (see the DCS bridge chapter); the X-Plane bridge uses the simulator's zulu clock with the PC's year.

20.4 Senders

Every bridge feeds the GPS interface from the simulator on the same PC (UDP 127.0.0.1:27002, every 500 ms), and so does the bench tester. Run one sender at a time.

Sender	Position, altitude, speed, heading	Date and time (`UTC:`)
`FSUIPCWinformsAutoCS` (Prepar3D / FSX)	FSUIPC offsets	The simulator's zulu date and time
`MSFSWinformsAutoCS` (MSFS 2020 / 2024)	SimConnect variables	The simulator's zulu date and time
`DCSBiosBridge` (DCS World)	DCS-BIOS CommonData: latitude, longitude, altitude, airspeed and heading, for every aircraft	The mission clock (the theatre's local time) less the **Mission time zone** box, with the PC's date
`XPlaneBridge` (X-Plane)	Datarefs `latitude`, `longitude`, `altitude_ft_pilot`, `airspeed_kts_pilot` and `true_psi`, for every aircraft	X-Plane's zulu clock and date, with the PC's year
`LowranceAirMapGpsBenchTester`	Typed values, cursor, Fly	The typed date and time, or the PC clock

20.5 Files

File	Role
`MainForm.cs`	Window, serial port handling, UDP listener task, 500 ms timer and the 30 second fallback
`NmeaBuilder.cs`	Builds the sentences and checksums
`AircraftData.cs`	The `AircraftData` record and the UDP packet parser
`AppSettings.cs`	JSON settings: serial port, UDP port, default position
`SerialPortDialog.cs`	The serial port picker

20.6 Status

Built clean. The NMEA builder (checksums checked against two reference sentences) and the parser passed a scratch test program, and the running window was exercised with UDP packets from the bench tester: it switched to the packet position and time, and went back to Brisbane Airport 30 seconds after the last packet. **Not yet tested against a real GPS**, so the AirMap 2000C's acceptance of these sentences and of the 115200 bps rate is unverified. A second sender's packets simply overwrite the position.

21 GPS bench tester (LowranceAirMapGpsBenchTester)

A Windows Forms bench tool that sends aircraft information to the GPS interface (default `127.0.0.1:27002`, twice a second) so the GPS can be exercised without a simulator.

21.1 Window

- **Send to UDP** host and port, and a *Send twice a second* check box.
- **Country, City and Airport** drop-down lists (starting on Australia, Brisbane, YBBN). Choosing an airport puts the aircraft on the ground there (the airport's reference point and field elevation, speed 0; the heading is left as it is). The lists hold 10,133 airports with an ICAO code in 236 countries, from an embedded list trimmed from the public-domain OurAirports `airports.csv`. The positions are reference points, not runway thresholds.
- **Latitude, longitude, altitude (ft), speed (kt) and heading (degrees true)** boxes: type a value and press Enter (an unreadable value turns the box pink and is not used). They follow the cursor while it moves.
- **8-way cursor:** a 3 by 3 pad of buttons (N, NE, E, SE, S, SW, W, NW) moves the position by the **Cursor step** (metres, default 100). A press moves one step; holding repeats every 100 ms after 400 ms. The arrow keys do the same, two together giving a diagonal, while the window is active and no box is being edited. With *Point heading in direction of travel* ticked (the default) the cursor also sets the heading.
- **Fly:** while ticked the position advances along the heading at the current speed with altitude, speed and heading constant. Distance is the real time since the last 100 ms tick times the speed (1 kt = 0.514444 m/s), multiplied by **Time x** (1 to 100) so a long leg can be flown quickly. Nothing moves while the speed is 0; change the speed, heading or altitude while flying to change course.
- **Date / time (UTC)** with a *PC clock* button and a *Send date/time* check box (on by default). Type `yyyy-MM-dd HH:mm:ss` and press Enter; the clock then runs in real time from that value and is sent as the simulator time.
- **Back to the selected airport** returns to the selected airport.

Movement uses a great-circle destination-point calculation, so a step or a leg is the same distance on any bearing and stays accurate over long flights.

21.2 Testing

Run together with the GPS interface on one PC. The GPS interface showed the tester's position; a North click moved latitude about 0.0009 degrees (100 m) and set heading 0, holding the Left arrow moved the position west with heading 270, choosing Japan and then Osaka moved it to Kansai (RJBB), Fly at 120 kt, heading 90 and Time x10 moved the longitude about 2.8 km in 4.5 seconds, and with the clock set to 2027-03-05 12:30:00 the GPS interface's RMC sentence carried time 123001 and date 050327 and advanced in real time.

22 Start-up script (Start Jet Ranger.ps1)

`Start Jet Ranger.ps1` (with a double-clickable `Start Jet Ranger.bat` and a sample settings file, `Start Jet Ranger.sample.json`) is in the `Jet Ranger` folder. It starts the PC side of the cockpit for one simulator, in order, and checks the result. It is plain Windows PowerShell 5.1 with no extra modules. Run it through the `.bat`: this PC's execution policy refuses to run `.ps1` files directly, and the `.bat` starts PowerShell with `-ExecutionPolicy Bypass` for that run only (the same works by hand: `powershell -ExecutionPolicy Bypass -File "Start Jet Ranger.ps1" -Simulator DCS`).

22.1 What it does

1. **Finds the programs.** For each program it needs it takes the newest build (Release or Debug) under `Jet Ranger\ C# Code\<project>\bin` (the folder name starts with a space). A program that has not been built is reported by name and nothing is started.
2. **UDP debug reflector.** Reports whether something is listening on UDP 27000. It never starts a second listener on that port; only if the settings file names a `Reflector` command and nothing is listening does it run that command.
3. **One bridge.** Stops any other of the four bridge programs (`FSUIPCWinformsAutoCS`, `MSFSWinformsAutoCS`, `DCSBiosBridge`, `XPlaneBridge`) that is running: it closes the window, then ends the process if it is still there. No other program is ever stopped.
4. **Health Monitor and GPS interface.** Starts `JetRangerHealthMonitor` and `LowranceAirMapGps` unless they are already running.
5. **Simulator.** Only with `-StartSimulator` and a program set under `Simulators` in the settings file (a program path, or an address such as `steam://...`) does it start the simulator; otherwise it prints a reminder. The bridge is started straight afterwards, which is fine because every bridge keeps trying to connect.
6. **Bridge.** Starts the bridge for the simulator: `FSUIPCWinformsAutoCS` for P3D and FSX, `MSFSWinformsAutoCS` for MSFS, `DCSBiosBridge` for DCS (with `--utc-offset` from `-DcsTimeZone` or the settings file) and `XPlaneBridge` for X-Plane (with `--xplane-host` from `-XPlaneHost` or the settings file).
7. **Check.** After five seconds it reports whether the bridge, the Health Monitor and the GPS interface are running, and which process owns UDP 27002 (it should be `LowranceAirMapGps`; if nothing does, the GPS interface is waiting at its serial port window). It ends with what to look at in that simulator.

A program that is already running is left alone, so the script can be run again, or used to switch simulator: only the bridge changes.

22.2 Options

Option	Meaning
`-Simulator MSFS, P3D, FSX, DCS or XPlane`	The simulator; a menu is shown if it is left out
`-DcsTimeZone <hours>`	DCS bridge's Mission time zone (hours from UTC)
`-XPlaneHost <host>`	X-Plane's host, when it is not this PC
`-StartSimulator`	Also start the simulator from the settings file
`-NoGps`, `-NoHealthMonitor`, `-NoReflector`	Leave that part out
`-Emulators`	Also open the panel emulators and send the bridge's output to them as well as the boards
`-EmulatorsOnly`	Open the panel emulators and send to them only (no boards needed)
`-DryRun`	Print what would be done; start and stop nothing
`-Stop`	Stop the bridges, the GPS interface and the Health Monitor (not the reflector or the simulator)

22.3 Settings file

`Start Jet Ranger.json`, next to the script, is optional: `Reflector` (a command line that starts the UDP reflector), `Simulators` (a program per simulator for `-StartSimulator`), `DcsTimeZone` and `XPlaneHost`. `Start Jet Ranger.sample.json` shows the layout. The script does not guess simulator paths.

22.4 Per-simulator start-up

Simulator	Bridge	Before the bridge connects	The GPS interface is fed
MSFS 2020 / 2024	`MSFSWinformsAutoCS`	A Jet Ranger loaded and not paused	Position, altitude, airspeed and heading from SimConnect, and the zulu date and time
Prepar3D / FSX	`FSUIPCWinformsAutoCS`	FSUIPC running, DodoSim 206 loaded; buttons bound to the DS206 events	FSUIPC offsets and the simulator's UTC time
DCS World	`DCSBiosBridge`	DCS-BIOS installed, a mission running in the cockpit; set the Mission time zone	CommonData position, altitude, airspeed and heading, and the mission clock less the time zone
X-Plane	`XPlaneBridge`	Settings, Network: incoming UDP allowed; a profile chosen	`latitude`, `longitude`, altitude, airspeed and `true_psi` datarefs, and the zulu clock and date

The step-by-step version for each simulator is in the User Guide (*Starting each simulator*).

22.5 Tested

Dry runs for all four simulators and a live run (Health Monitor, GPS interface and the DCS bridge started; started again with nothing to do; switched to X-Plane, which stopped the DCS bridge; `-Stop` ended all of them) on 31 October 2026. The simulators and the real GPS were not involved. On the PC it was tried on, the saved GPS port (COM25) was not present, so the GPS interface showed its serial port window, which the script's check reports.

23 Panel emulators (JetRangerPanelEmulator)

`JetRangerPanelEmulator` is a Windows Forms program (.NET 10) that stands in for the **controller boards and their displays**: one window each for the Radio, Servo, Stepper, Dual Stepper and ADF and Upper controllers. Each window receives the packets its board would receive (UDP 13136, `D,TOKEN:value,...` data and `C,COMMAND:value` control) and draws what the panel would show: the dial gauges and their needles, the attitude indicator, the caution lamps, the LEDs, the character OLEDs and the landing gear lamps. The four bridges can send their output to the emulators instead of the boards, or as well as them, so the whole chain from simulator to panel can be tried with no cockpit hardware.

It is different from the older `JetRangerRadioControllerEmulator` and `JetRangerUpperControllerEmulator`: those stand in for a board's **inputs** (knobs, keys) and send commands to the bridge. This program shows a board's **outputs** and sends nothing.

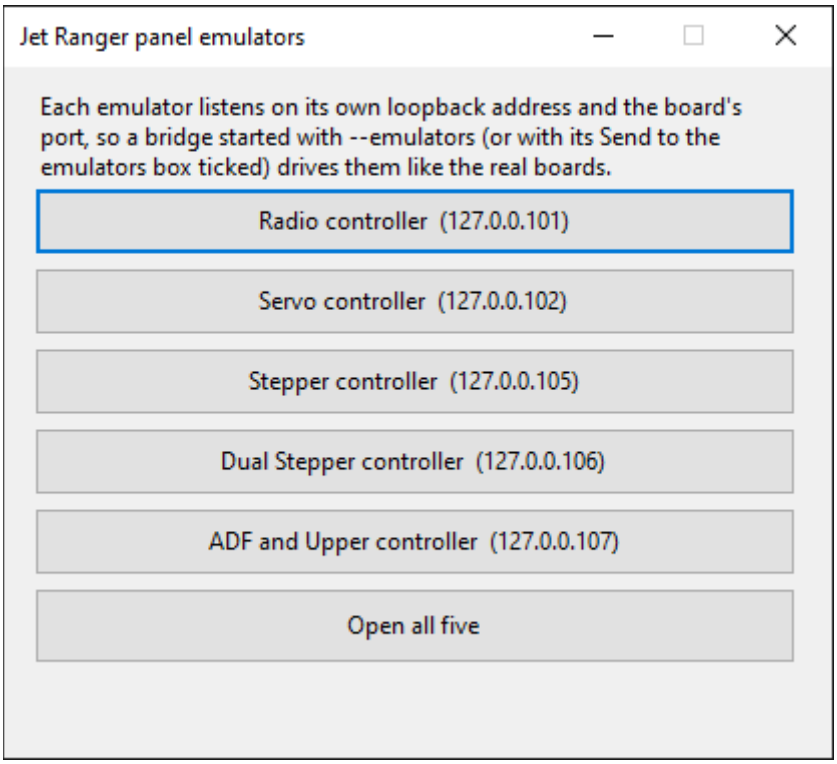


Figure 38 — The launcher

23.1 How the bridges reach the emulators

Each emulator listens on the board's own port, 13136, on a **loopback address with the board's last number**. Every address from 127.0.0.1 to 127.255.255.254 is loopback on Windows, so a bridge only has to swap `172.16.1` for `127.0.0` and send the very same packet. Nothing has to be sorted by content, and the emulators run on the bridge's PC.

Board	Real address	Emulator address
Radio controller	172.16.1.101	127.0.0.101
Servo controller	172.16.1.102	127.0.0.102
Stepper controller	172.16.1.105	127.0.0.105
Dual Stepper controller	172.16.1.106	127.0.0.106
ADF and Upper controller	172.16.1.107	127.0.0.107

In each bridge a small `BoardTargets` class holds, for every board's UDP client, a second client for its emulator, and sends to the board, the emulator or both. The bridge window has two check boxes, **Send to the boards** (on) and **Send to the emulators** (off), and the command line sets them:

Option	Meaning
`--emulators`	Send to the emulators as well as the boards

Option	Meaning
`--emulators-only`	Send to the emulators only: no packet goes to a real board
`--emulator-net <a.b.c>`	The first three numbers of the emulators' addresses, when they are not 127.0.0 (the emulator takes `--net` with the same meaning)

The GPS interface is not a board and is not affected. To see exactly what is being sent to each board or emulator, open the bridge's packet monitor (**Show packets...**, see the bridge chapters). The start-up script (`Start Jet Ranger.bat -Emulators` or `-EmulatorsOnly`) opens the emulators and starts the bridge with the option.

23.2 What each window does

Common to every window: the board's address in the title and status bar, the packet count and the last packet. `BPWR:0`, or no data, switches the **backlight** off (the scale and the lamp legends go dark; the needles stay visible); `C,B:0-15` sets its brightness (about 0.7 until one arrives); `C,RESET:1` forgets everything. After **30 seconds without data** the board blanks, the needles return to their calibrated zero and the lamps and OLEDs go out, as on the real boards (`--watchdog <seconds>` shortens it for tests). The needles move at a limited speed, like a stepper.

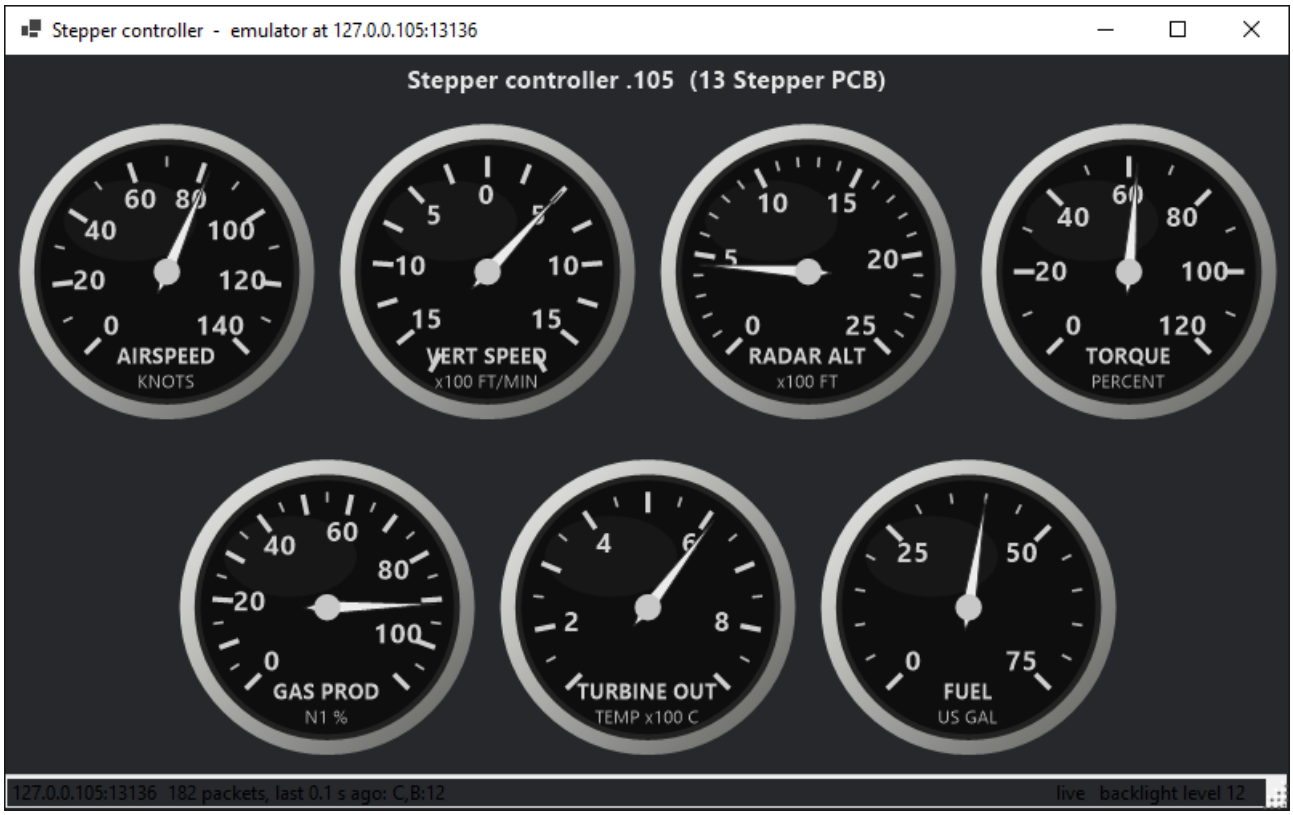


Figure 39 — Stepper controller emulator

Stepper .105. Airspeed, vertical speed, radar altitude, torque, gas producer, turbine out temperature and fuel quantity on their dials. `C,SWING<token>:1` (for example `C,SWINGIAS:1`) sweeps a needle to the end of its scale and back.



Figure 40 — Dual Stepper controller emulator

Dual Stepper .106. Engine and transmission oil pressure and temperature, rotor and engine speed, electrical load, fuel pressure, the altimeter (one turn per 1000 ft) and the four OLEDs: barometer (`2992`), altitude, the zulu clock from `ZULU:HHMM` and the temperature text. `PWR:0` blanks the OLEDs; `C,OLEDTEST:1` fills them white.

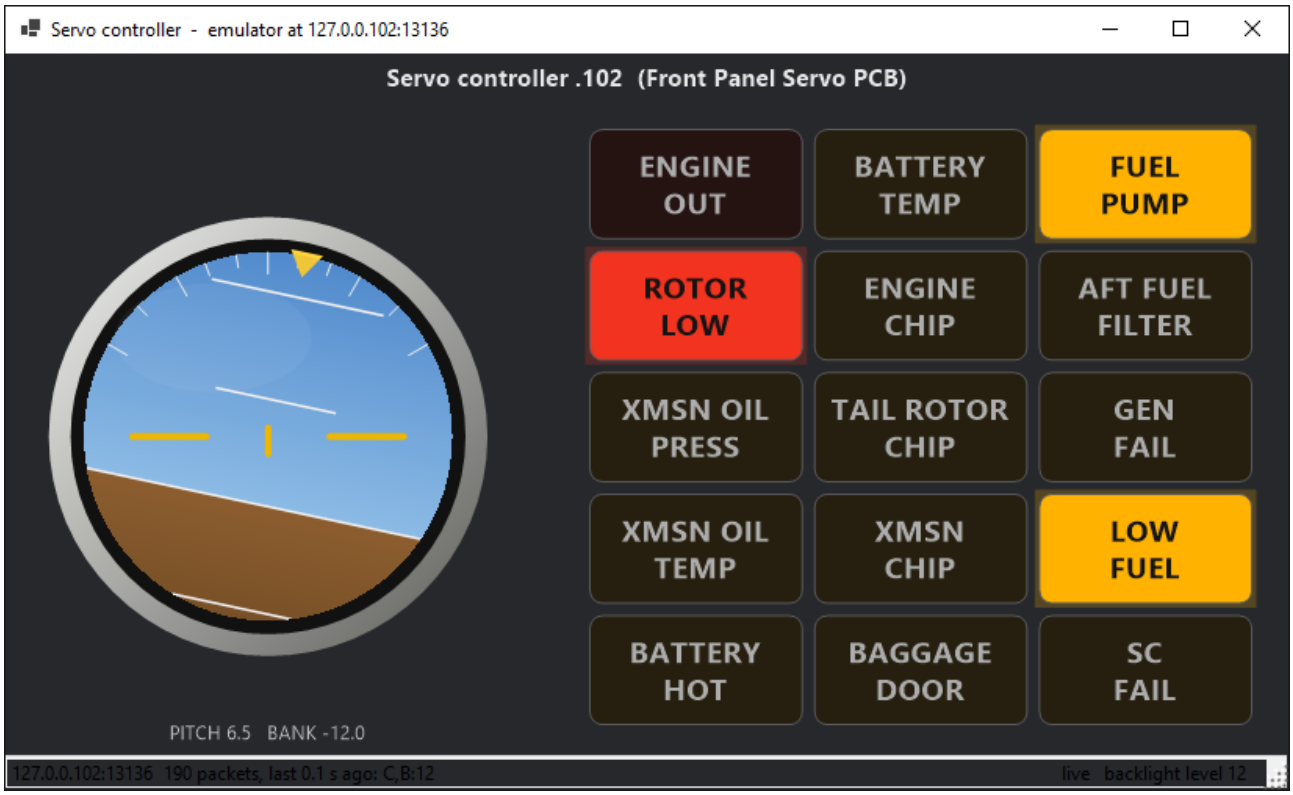


Figure 41 — Servo controller emulator

Servo .102. The attitude indicator (pitch -30 to +30 degrees, bank -90 to +90; the servos move at a limited speed) and the 15 caution and warning lamps, lit by their tokens (`RLOW:1` and so on). The direction in which the horizon turns for a positive bank is drawn as in a real indicator; it has not been checked against the servo's mounting.

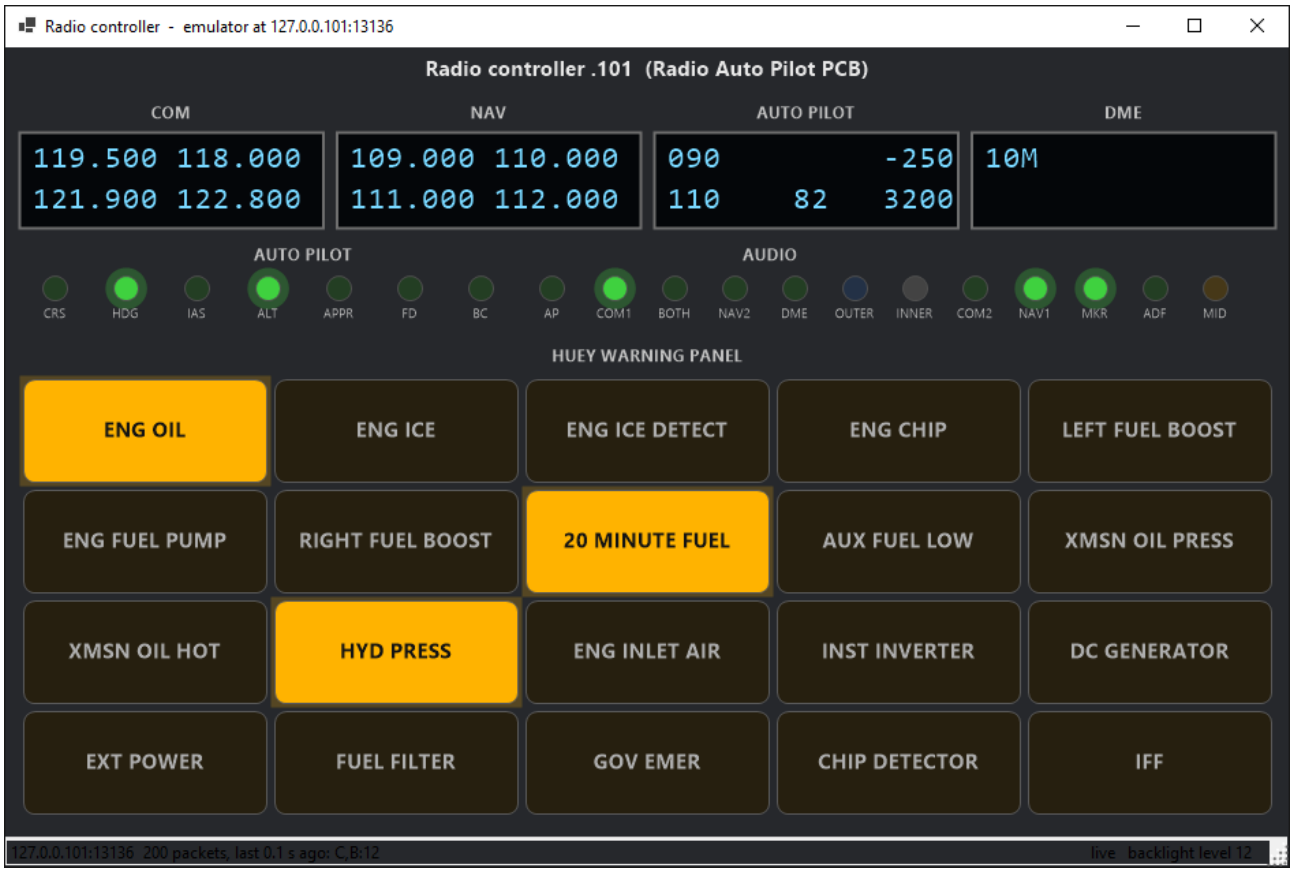


Figure 42 — Radio controller emulator

Radio .101. The COM and NAV displays (two radios on each, `xxx.xxx`), the Auto Pilot display (HDG, CRS, IAS, VS and ALT in their windows on a 16 column, two row display), the DME display, the 8 Auto Pilot and 11 Audio LEDs and the 20 Huey Warning Panel lamps. `NAVCOM1:0` blanks the displays and puts the LEDs out; `C,LEDTEST:1` lights all 39. The PC sends only COM1 and COM2, so the NAV displays keep the sketch's start-up values (NAV1 109.000 and 111.000, NAV2 110.000 and 112.000).



Figure 43 — ADF and Upper controller emulator

ADF and Upper .107. The three landing gear lamps (`LDGN`, `LDGL`, `LDGR`), the OLED (`ADF  340 kHz`, `FOLLOW OFF`; `C,DISPLAY:0` from the Radio blanks it) and the backlight. The Upper switch matrix and the keypads are inputs and are not shown.

The faces (ranges, scales, numerals and legends) are those of the instrument map, Appendix G.

The emulators do not copy the real boards' start-up hold (backlight, OLEDs and lamps on until the Stepper controller announces start-up on UDP 13138): they start dark and follow the data.

23.3 Running it

`JetRangerPanelEmulator.exe` opens the launcher, with a button for each emulator and **Open all five**. `--board radio|servo|stepper|dual|adf|all` opens those windows directly, `--net <a.b.c>` changes the addresses and `--dump <folder>` writes each board's state to `<folder>\<board>.json` four times a second (used by the tests). Each address can be listened on by one program at a time.

23.4 Tests

`python tests\test_emulator.py` starts the five emulators with a 3 second watchdog and sends them the bridges' packets: values and needles, swings, lamps, blanking (`PWR:0`, `NAVCOM1:0`, `C,DISPLAY:0`), brightness and `BPWR`, `C,LEDTEST:1`, `C,RESET:1` and the watchdog (29 checks). `python tests\test_bridges_to_emulators.py` runs the real DCS bridge, fed its synthetic DCS-BIOS stream, and then the real X-Plane bridge, against its fake X-Plane, both with `--emulators-only`, and checks what the emulators show (9 checks). The MSFS and FSUIPC bridges need their simulators for data: they were built, their windows and command lines were checked, and they use the same `BoardTargets` code, but no simulator data has gone through them to the emulators yet.

23.5 Not done yet

Input emulation on these windows (use the older Radio and Upper emulators), heartbeats to the Health Monitor (the emulators do not appear there), the real boards' start-up swings and lamp test, the NAV frequency knobs, and a side-by-side check against the real boards.

24 PCB notes

Notes from reading the KiCad projects in `Jet Ranger/PCBs` (schematics, PCB layouts and netlists), brought up to date on 18 October 2026 (and for the combined Upper ADF Controller on 31 October 2026). Matching boards to sketches is by name. The checks read the files only; no board was measured.

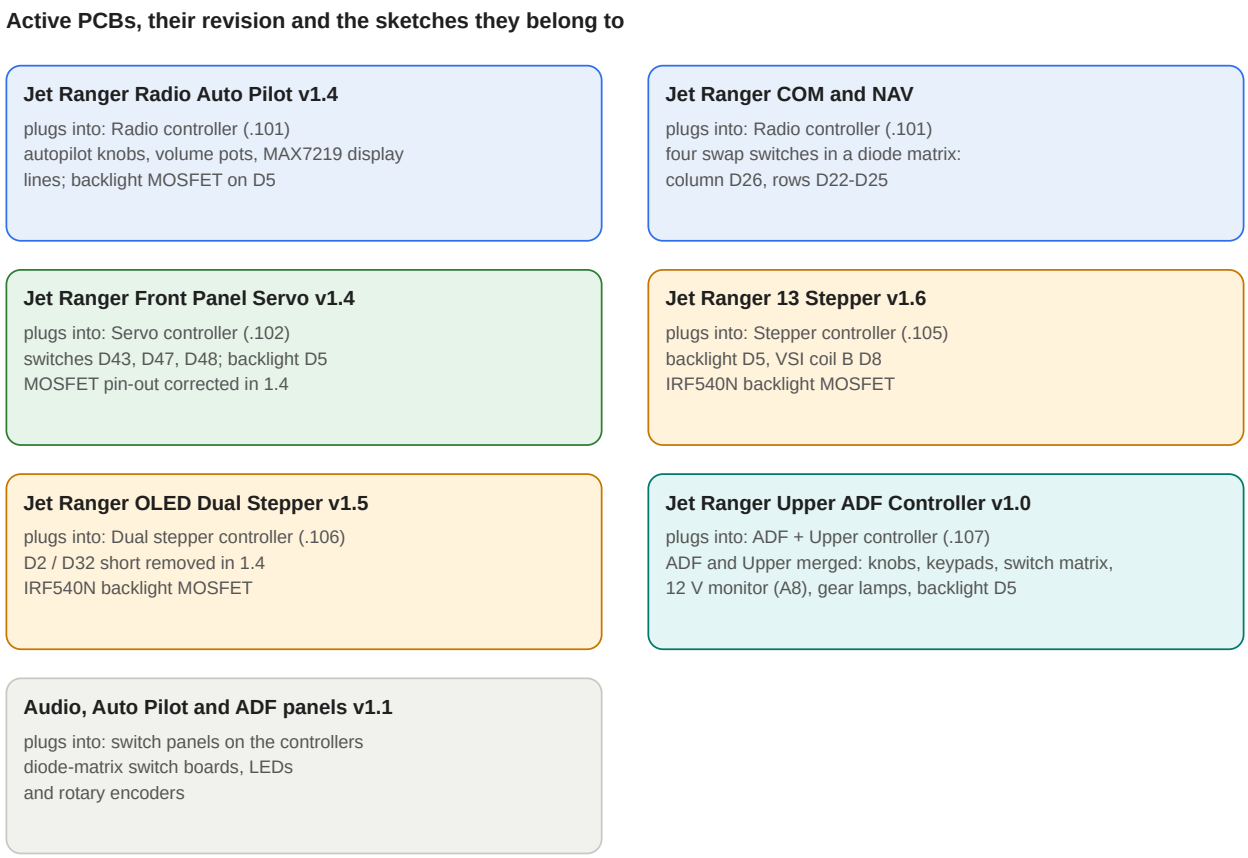


Figure 44 — Active PCBs, their revision and the sketches they belong to

24.1 Boards in use

PCB	Revision (title block)	Used by	Notes
Jet Ranger Front Panel Servo	1.4 (2026-10-18)	Servo controller	Mega shield, two ULN2803A lamp drivers, servo headers (16, 2 used), 15 lamp outputs, three switch inputs, backlight MOSFET. Change log: 1.3 had the MOSFET pinned wrongly (source and drain need swapping), 1.4 corrects it
Jet Ranger 13 Stepper	1.6 (2026-10-18)	Stepper controller	Mega shield, USR-ES1 Ethernet, 14 six-pin gauge sockets, 12 V lamp supply with polyfuses, backlight MOSFET. The change log lists up to 1.5 (1.4 moved the LEDs away from the USB connectors, 1.5 selected the IRF540N); the title block now says 1.6

PCB	Revision (title block)	Used by	Notes
Jet Ranger OLED Dual Stepper	1.5 (2026-10-18)	Dual stepper controller	Mega shield, USR-ES1, gauge sockets, TCA9548A sockets for the OLEDs, backlight MOSFET. No change log text; the schematic now uses the IRF540N
Jet Ranger X27 Small Stepper	not set	Each stepper gauge	One X27 or VID29 stepper, backlight LED, six-pin connector to the controller
Jet Ranger Radio Auto Pilot	1.4 (2026-10-31)	Radio controller	Radio and autopilot controller. Change log: 1.2 added the audio panel, 1.3 corrected the MOSFET mis-pinout, 1.4 added the Huey Warning Panel connections: J20 (switches A8 to A10), J21 (backlight), J22 (LED anodes) and J23 (LED cathodes)
Jet Ranger COM and NAV	not set	Radio panel	Eight rotary encoders, push buttons and the four frequency-swap switches in a diode matrix
Jet Ranger Upper ADF Controller	1.0 (2026-10-26)	ADF and Upper controller (`JET_RANGER_ADF_CONTROLLER`, .107); the reference PCB for the ADF controller	The Upper Controller (1.4) and ADF Controller (1.6) merged onto one Mega: the ADF major and minor knobs (D6 to D9), an I2C OLED, the ADF keypad (A1 to A7, J41), landing-gear, flaps, speed-brake and brake inputs (D31 to D34), hyd boost, anti-ice and caution dim inputs (D35 to D37 on J20), three landing-gear lamps (D2 to D4), the Upper switch matrix (rows D22 to D30 on J30, columns D38 to D45 on J31 with 10 k pull-ups), the Upper keypad (A9 to A15, J32), the Warning Mute (J17) and TOT Test (J18) switches on `ROW8`, the backlight MOSFET (D5, panel backlight on J33), a panel brightness pot (A0) and a 12 V monitor (A8, 47 k / 10 k divider). The ADF panel connectors J5 and J20 keep their pin-outs. The 1x16 breakout headers J9 to J14 were removed to make room. Change log: 1.0 merged the two boards, starting from ADF Controller 1.6 (earlier history: 1.2 added the audio panel, 1.3 corrected the MOSFET mis-pinout (IRF540 selected), 1.4 added the 12 V power rail monitors and the pot for panel brightness, 1.5 added the lower panel's anti-ice, hyd boost and caution dim inputs, 1.6 moved to a single connector for the lower panel). The layout is routed: KiCad's design rule check on 31 October 2026 found no unconnected items and four keepout warnings on mounting holes H1, H2, H4 and H5
Jet Ranger Audio Panel, Auto Pilot Panel, ADF Panel	1.1 (2026-10-17)	Switch panels	Diode-matrix switch boards: Audio (10 switches, 21 diodes), Auto Pilot (16 switch and encoder parts, 18 diodes, 2 LEDs), ADF (9 switch and encoder parts, 4 diodes)
Annunciator, Altimeter, Large Guage, Simkits gauge boards	not set	Other panels and gauges	Not covered by the sketches in this document

The X27 steppers run straight from the Mega's pins, so the stepper boards need no driver chips; the 12 V input feeds lamps and backlights. The schematic and board layout of every PCB are in Appendix E, and installation views of the controller boards in Appendix F.

24.2 Backlight driver

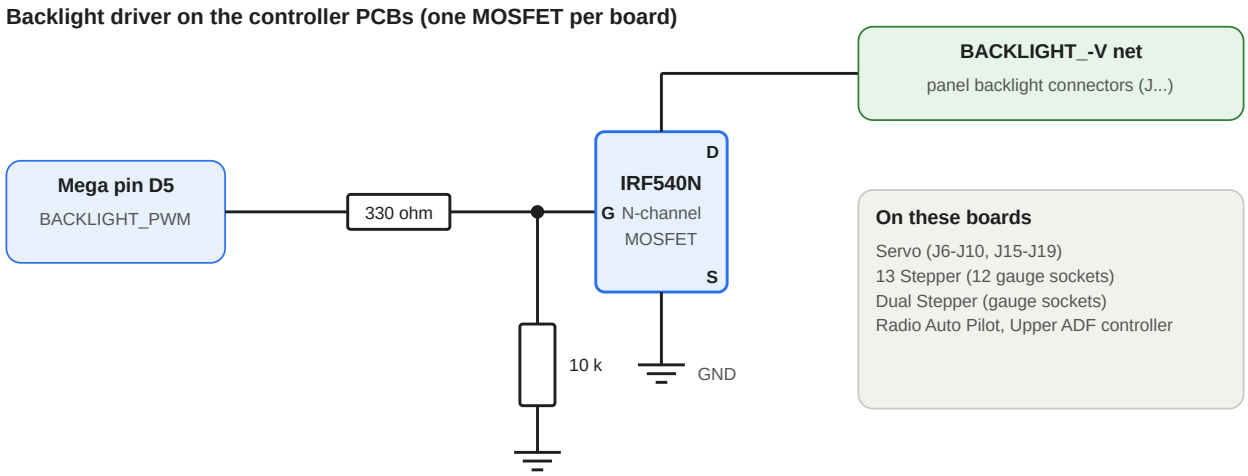


Figure 45 — The backlight MOSFET circuit

All the controller boards (Servo, 13 Stepper, Dual Stepper, Radio Auto Pilot and the combined Upper ADF Controller, and the Upper and ADF Controller boards it replaces) have the same backlight circuit, traced pad to pad in the PCB files: Mega pin **D5** is the net `BACKLIGHT_PWM`, which goes through a 330 ohm resistor to the gate of an **IRF540N** in a TO-220 package, with a 10 k resistor from the gate to ground. The drain is the backlight net (`/BACKLIGHT_PANELS` or `BACKLIGHT_-V`) that the panel or gauge backlight connectors share, and the source is ground, so the MOSFET switches the backlight low side and D5 PWM sets the brightness. `BACKLIGHT_PWM` is also brought out on pin 6 of an 8-pin header on each board.

Board	Backlight connectors on the MOSFET drain net
Front Panel Servo	Ten `BK_LIGHT` connectors (J6 to J10, J15 to J19)
13 Stepper	Pin 6 of 13 gauge sockets (airspeed, radar altitude, gas producer, transmission oil pressure, VSI, engine torque, turbine speed, rotor speed, fuel level, engine oil pressure and temperature, transmission oil temperature, EGT)
Dual Stepper	Gauge sockets for airspeed, VSI, fuel level, EGT, altimeter, engine, transmission, rotor and engine speed, and electrical and fuel load
Radio Auto Pilot	The AUDIO, AUTOPILOT and COMM_NAV connectors
Upper ADF Controller	The BACKLIGHT connector (J33, net `/BACKLIGHT_PANELS`) and the ADF connector (J5)

The earlier revisions used a generic N-channel MOSFET symbol (`Q_NMOS_GSD`) whose pin-out had source and drain swapped; the revisions dated 18 October 2026 (1.4 Servo, 1.3 Radio Auto Pilot, the then ADF Controller and Upper 1.4, 1.5 Stepper and Dual Stepper); the combined Upper ADF Controller started from the corrected ADF design use the IRF540N with the correct pin-out. A board made from an earlier revision should have the transistor checked.

The Radio, Servo, Stepper, Dual Stepper and ADF sketches drive D5 (see the network, Radio and ADF controller chapters); the ADF sketch now also covers the Upper panel's backlight. The combined PCB has a brightness input on A0 (`GUAGE_BACKLIGHT_BRIGHT`, connector J16); the ADF sketch can read it and send the level to the other panels, but that is switched off for the moment (`USE_BRIGHTNESS_POT` is 0).

24.3 Warning Mute, Fuel Valve and TOT Clear inputs

All KiCad schematic, PCB and netlist files under `Jet Ranger/PCBs` were searched for a Warning Mute (horn mute) input. Two boards have one.

- **Jet Ranger Front Panel Servo:** SW3 (`WARNING_MUTE`, a 2-pin Molex KK-254 connector) connects net `/SW_WARNING_MUTE` to Arduino Mega pin D47 and GND. The same board has SW1 `TOT_CLEAR` on D43 and SW2 `FUEL_VALVE` on D48, each also wired to GND. The Servo sketch reads these three pins as debounced inputs (internal pull-up, closed = LOW), logs every transition and sends it to the USB HID board.
- **Jet Ranger Upper ADF Controller:** J17 (`WARNING MUTE`, a 2-pin Molex KK-254 connector) connects `/ROW8` (Arduino D30, shared with J18 TOT TEST) on pin 2 and, on pin 1, a diode D5 whose other side goes to `/COL0`; J18 does the same through diode D6 to `/COL1`. They are part of the board's diode-isolated key matrix, which the ADF sketch now scans (rows D22 to D30), so they are read as matrix indexes 88 and 89. Nothing is assigned to them yet.

24.4 Pin and copper check

The KiCad boards were traced pad to pad through tracks and vias on 12 October 2026 and the Mega pin tables were compared again on 18 October 2026 against that check. This checks connections only; it is not a design rule check (clearances, pours and footprints are not checked).

- **No Mega pin assignment changed** between the two checks on the Servo, 13 Stepper, Dual Stepper and the old Upper boards, apart from the 13 Stepper D2 net being renamed (below). The Radio Auto Pilot and the old ADF Controller were first checked on 18 October. The combined Upper ADF Controller was checked on 31 October 2026 from its KiCad netlist against the merged ADF sketch: every pin used by the sketch matches the netlist (see the ADF and Upper controller chapter for the table).
- **Jet Ranger OLED Dual Stepper:** up to version 1.3, D2 and D32 were one net (`STEPPER_FL_A`), joining fuel quantity and fuel load coil A. Fixed in 1.4: D2 is `STEPPER_FA_A` and D32 `STEPPER_FL_A` in 1.5. Boards older than 1.4 still have the short: fit only one of those two gauges or cut the trace.
- **Jet Ranger 13 Stepper:** D2 carried the same copied name but nothing else used it; it is `STEPPER_FA_A` (label only). D32 is the radar altitude coil `STEPPER_RA_A`.
- **Both stepper PCBs:** `BACKLIGHT_PWM` is D5 and VSI coil B is D8. Both stepper sketches use D5 for `BACK_LIGHTS` (pin 8 used to be both the backlight and VSI coil B).
- **Jet Ranger Front Panel Servo:** the lamp outputs are still wired to different Mega pins from the sketch for 10 of the 15 lamps: the PCB has A1 Rotor RPM low, A2 Engine out, A5 Battery temp, A6 Battery hot, A7 Transmission chip, A8 Baggage door, A9 Engine chip, A10 Tail rotor chip, A11 Fuel pump and A12 Aft fuel filter; the sketch has them the other way round for A1/A2, A5/A6 and A11/A12 and in a different order for A7 to A10 (A3, A4 and A13 to A15 agree). The power-up lamp test cannot show this. Test each lamp with a single `D,<lamp>:1` and fix whichever side is wrong.
- **Jet Ranger Radio Auto Pilot:** the COM1 major, COM1 minor, COM2 minor and NAV1 minor encoders have A and B the other way round from the sketch (see the Radio controller chapter); the other four match. The volume pots and mux address pins are wired but not used by the sketch (the autopilot knobs, MAX7219 lines and fifth matrix column are now used).
- **Servo, 13 Stepper and Upper ADF Controller:** a net `GPS_TX` joins Mega pin D18 (TX1) to pin 5 of an 8-pin header (J11 on the Servo and 13 Stepper, J43 on the Upper ADF Controller). No sketch uses Serial1 on those boards. The Dual Stepper uses D16 to D19 for the OLED multiplexer instead.
- **Upper ADF Controller, 13 Stepper and OLED Dual Stepper:** pins otherwise match their sketches. The TCA9548A address pins (D17 to D19 on the Dual Stepper, D2 to D4 on the Radio Auto Pilot) are not driven; the module's pull-downs set address 0x70.

24.5 KiCad: changing the revision number

In the PCB Editor use *File, Page Settings*, set the *Title Block* revision field, then save the board. To show it on the board, place a text item on the `F.Silkscreen` layer (press T) containing `Rev ${REVISION}`; other variables such as `${TITLE}`, `${ISSUE_DATE}` and `${COMPANY}` work the same way. The schematic and the PCB each keep their own title block, so set both if they should match. Make a fresh Gerber zip in `MANUFACTURING` before ordering.

Appendix A: UDP token reference

From the current sketches. Gauge tokens take real units; lamp tokens take 0 (off) or 1 (on). `UDP_Interface_Values.xlsx` still lists the old 16-servo plan, so use these tables instead. Every gauge token also has a `RAW` twin (`PITCHRAW`, `IASRAW`) that takes a raw servo position or step count and skips the calibration table.

A.1 Gauge tokens

Token	Gauge	Board	Unit
`PITCH`	Attitude pitch	Servo .102	degrees, -30 to +30
`BANK`	Attitude bank	Servo .102	degrees, -90 to +90
`IAS`	Airspeed	Stepper .105	knots, one decimal allowed
`VSI`	Vertical speed	Stepper .105	ft/min
`AGL`	Radar altitude	Stepper .105	feet, 0 to 2500
`TQ`	Engine torque	Stepper .105	percent, 0 to 120
`N1`	Gas producer	Stepper .105	percent, 0 to 103
`ITT`	Turbine out temperature	Stepper .105 or Dual .106	degrees C
`FUEL`	Fuel quantity	Stepper .105 or Dual .106	US gallons
`OILT`	Engine oil temperature	Dual .106 (or Stepper)	degrees C
`OILP`	Engine oil pressure	Dual .106 (or Stepper)	psi
`XMSNT`	Transmission oil temperature	Dual .106 (or Stepper)	degrees C
`XMSNP`	Transmission oil pressure	Dual .106 (or Stepper)	psi
`RPME`	Engine speed (N2)	Dual .106	percent, one decimal
`RPMR`	Rotor speed	Dual .106	percent, one decimal
`ALT`	Altimeter	Dual .106	feet
`FUELLOAD`	Fuel load	Dual .106	psi
`ELECTRICALLOAD`	Electrical load	Dual .106	percent

A.2 Display and status tokens

Token	Board	Meaning
`ZULU`	Dual .106	Zulu time as HHMM, for example `1430`
`TEMP`	Dual .106	Temperature text up to 8 characters, for example `25C`
`PWR`	Dual .106	1 = all four OLEDs on, 0 = blank (main bus below 24 V). Sent first in the packet
`APHDG`, `APCRS`, `APIAS`, `APVS`, `APALT`	Radio .101	The values on the Auto Pilot panel's OLED: heading and course (three digits, north is 360), airspeed (knots), vertical speed (feet per minute, signed) and altitude (feet), up to 5 characters each
`LDGN`, `LDGL`, `LDGR`	ADF and Upper .107	The landing gear lamps (nose, left, right): 1 = lit
`C1A`, `C1S`, `C2A`, `C2S`	Radio .101	COM1 and COM2 active and standby frequency, MHz
`MAINBUS`	Radio .101	Main bus voltage
`NAVCOM1`	Radio .101	0 = avionics bus below 24 V (COM and NAV displays blank, panel LEDs off), 1 = on
`LED_CRS`, `LED_HDG`, `LED_IAS`, `LED_ALT`, `LED_APPR`, `LED_FD`, `LED_BC`, `LED_APENG`	Radio .101	Auto Pilot panel LEDs (CRS, HDG, IAS, ALT, APPR, FD, BC, AP ENGAGE): 1 = on
`LED_COM1`, `LED_BOTH`, `LED_NAV2`, `LED_DME`, `LED_OUTER`, `LED_INNER`, `LED_COM2`, `LED_NAV1`, `LED_MKR`, `LED_ADF`, `LED_MID`	Radio .101	Audio panel LEDs: 1 = on

Token	Board	Meaning
LED_H_ENG_OIL, LED_H_ENG_ICE, LED_H_ENG_ICE_DETECT, LED_H_ENG_CHIP, LED_H_LEFT_FUEL_BOOST, LED_H_ENG_FUEL_PUMP, LED_H_RIGHT_FUEL_BOOST, LED_H_20_MINUTE_FUEL, LED_H_AUX_FUEL_LOW, LED_H_XMSN_OIL_PRESS, LED_H_XMSN_OIL_HOT, LED_H_HYD_PRESS, LED_H_ENG_INLET_AIR, LED_H_INST_INVERTER, LED_H_DC_GENERATOR, LED_H_EXT_POWER, LED_H_FUEL_FILTER, LED_H_GOV_EMER, LED_H_CHIP_DETECTOR, LED_H_IFF	Radio .101	Huey Warning Panel LEDs: 1 = on (sent by the DCS bridge from the UH-1H caution panel, and ten of them approximately by the X-Plane bridge; not by the FSUIPC and MSFS bridges)
BPWR	Radio .101, Servo .102, Stepper .105, Dual .106, ADF and Upper .107 (the bridges still also send it to the retired Upper .103)	Backlight power: 1 = main bus 20 V or more, 0 = backlight off

A.3 Lamp tokens (Servo .102)

EOUT, RLOW, TOPW, TOTW, BHOT, BTMP, EC, TRC, TC, BD, FPMP, FFLTR, GENF, LOWF, SCF on pins A1 to A15 in that order (the fuel pump and aft fuel filter pins were swapped on 25 October 2026: FPMP is A11 and FFLTR is A12) (see the Servo controller chapter).

A.4 Control tokens (C,<COMMAND>:<value>)

Command	Meaning
SWING<CODE>	Run one gauge's start-up swing on demand, for example C,SWINGTQ:1; the value is ignored
OLEDTEST	1 fills the four Dual Stepper OLEDs solid white, 0 restores them
KEEPALIVE	Ignored command that still counts as received data and restarts the 30 second no-data timer
VERSION	Send the firmware version, build number and compile date and time to the debug stream now (see Firmware versions in Appendix B; the USB HID board answers on its logging port)
RESET	Reboot the board about 2 seconds later (port 13136; USB HID board on 4210)
MATRIX	Radio and ADF controllers: print the raw matrix or ADF keypad readings to the debug stream
LEDTEST	Radio controller: 1 lights all 39 Auto Pilot, Audio and Huey panel LEDs, 0 returns them to the simulator's state
POWER	ADF controller: send a power status (12 V and 5 V rails) now
DISPLAY	ADF and Upper controller: 0 blanks its OLED and landing gear lamps, 1 restores them (sent by the Radio controller with its own displays)
B	Backlight brightness 0 to 15 (0 off, 15 full): Radio, Servo, Stepper, Dual Stepper and ADF and Upper; the Radio also sets the intensity of its Auto Pilot and Audio panel LEDs from it (0 = LEDs off). The ADF controller can send it from its brightness pot, which is switched off for the moment

A.5 Messages from the ADF controller to the PC (UDP 27003)

ADF_FREQ:<kHz> (190 to 1799, sent 250 ms after the knobs stop), RADAR_FOLLOW_ID:<digits> (RADAR_FOLLOW_ID:0 = stop following) and POWER,V12:<volts>,V5:<volts>,STATUS:<OK, or 5V_LOW, 5V_HIGH, 12V_LOW, 12V_HIGH joined with +> every 30 seconds. No bridge reads this port yet.

A.6 Aircraft information to the GPS interface (UDP 27002)

`D,LAT:<deg>,LON:<deg>,ALT:<feet>,IAS:<knots>,HDG:<degrees true>,UTC:<yyyyMMddHHmmss[.fff]>`, every 500 ms from a bridge or the bench tester. Any subset of keys is accepted; a CSV form `lat,lon,altitude_ft,airspeed_kt,heading_deg[,utc]` also works. South and west are negative.

A.7 Messages from the Radio controller to the bridge (UDP 27001)

`COM1_RADIO_SWAP`, `COM2_RADIO_SWAP`, `NAV1_RADIO_SWAP`, `NAV2_RADIO_SWAP`, `AVIONICS_MASTER_SET`, `MASTER_BATTERY_ON`, `MASTER_BATTERY_OFF`, `ALTERNATOR_ON`, `ALTERNATOR_OFF`, `COM2_STANDBY:<MHz>`, `NAV1_STANDBY:<MHz>`, `NAV2_STANDBY:<MHz>` and a bare number for the COM1 standby frequency. The ADF controller also sends the positions of its toggle switches as values: `GEAR_SET:1/0` (gear down / up), `FLAPS_SET:1/0` (flaps fully down / up) and `SPOILERS_SET:1/0` (speed brakes fully out / in), on every change and every 5 seconds. The Radio controller also sends the Auto Pilot and Audio panel controls and the ADF controller the gear, flaps, speed brake and brake controls, each as the name of the simulator control with an optional `:N` repeat count: `AP_MASTER`, `AP_HDG_HOLD`, `AP_NAV1_HOLD`, `AP_ALT_HOLD`, `AP_APR_HOLD`, `TOGGLE_FLIGHT_DIRECTOR`, `AP_BC_HOLD`, `AP_PANEL_SPEED_HOLD`, `VOR1_OBI_INC`, `VOR1_OBI_DEC`, `AP_ALT_VAR_INC`, `AP_ALT_VAR_DEC`, `AP_VS_VAR_INC`, `AP_VS_VAR_DEC`, `AP_SPD_VAR_INC`, `AP_SPD_VAR_DEC`, `HEADING_BUG_INC`, `HEADING_BUG_DEC`, `COM1_TRANSMIT_SELECT`, `COM2_TRANSMIT_SELECT`, `COM_RECEIVE_ALL_TOGGLE`, `RADIO_VOR1_IDENT_TOGGLE`, `RADIO_VOR2_IDENT_TOGGLE`, `RADIO_DME1_IDENT_TOGGLE`, `RADIO_ADF_IDENT_TOGGLE`, `MARKER_SOUND_TOGGLE`, `GEAR_UP`, `GEAR_DOWN`, `FLAPS_UP`, `FLAPS_DOWN`, `SPOILERS_ON`, `SPOILERS_OFF` and `PARKING_BRAKES`.

Appendix B: Building and flashing

B.1 Libraries

Ethernet 2.0.2 with SPI, AccelStepper 1.64.0 (steppers), Servo and Ewma (Servo controller), U8g2 and Wire (OLEDs), DCS-BIOS 0.3.13 (sketches that still carry DCS-BIOS code), and the Joystick and Keyboard libraries for the HID board. Install them from the Arduino Library Manager.

B.2 Compile and upload

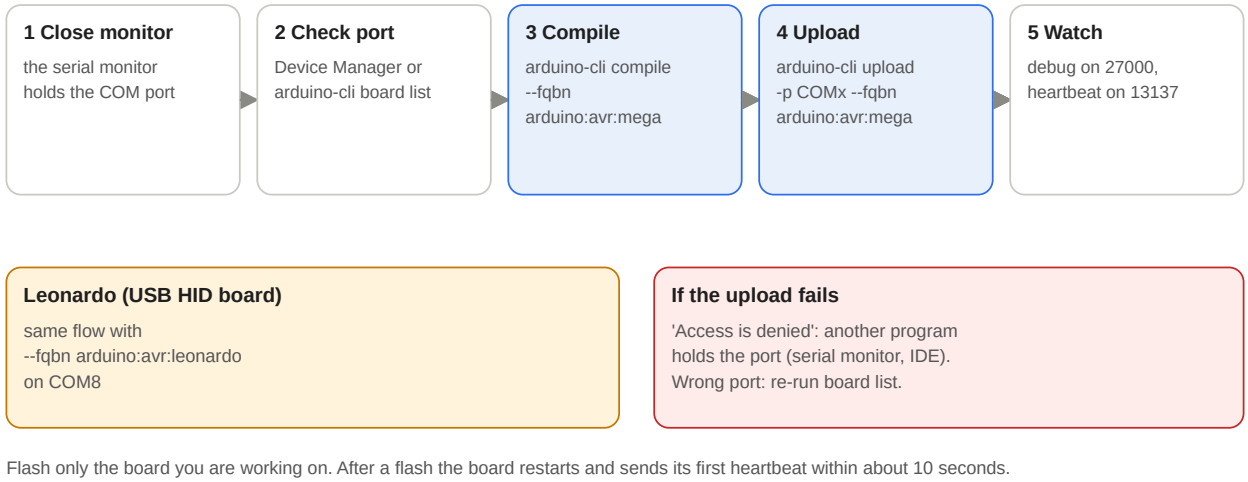


Figure 46 — Flashing a board

```
arduino-cli compile --fqbn arduino:avr:mega "Jet Ranger Arduino
Sketches\JET_RANGER_STEPPER_CONTROLLER"
arduino-cli upload -p COM4 --fqbn arduino:avr:mega "Jet Ranger Arduino
Sketches\JET_RANGER_STEPPER_CONTROLLER"
```

Use `--fqbn arduino:avr:leonardo` for the USB HID board. Close the Arduino IDE serial monitor first: it holds the port ("Access is denied"). Check the port with Device Manager or `arduino-cli board list`.

B.3 Firmware versions

Every controller sketch has a firmware version, a build number and the compile date and time. The three constants are at the top of the sketch, next to its debug function:

```
#define FIRMWARE_NAME "Radio Controller"
#define FIRMWARE_VERSION "1.0"
#define FIRMWARE_BUILD 2
```

The rule: raise `FIRMWARE_BUILD` by one every time a sketch is changed (and `FIRMWARE_VERSION` for a larger change), before it is compiled and flashed. The compile date and time (`__DATE__` and `__TIME__`) change on every compile by themselves, so a board that reports an older date than the compile you just made has old code on it.

The board sends one line to the debug stream (UDP 27000) straight after *Ethernet Started* in its start-up sequence, and again whenever it receives the control packet `C,VERSION:1`, so the version can be checked without resetting it (the USB HID board sends it to its logging port):

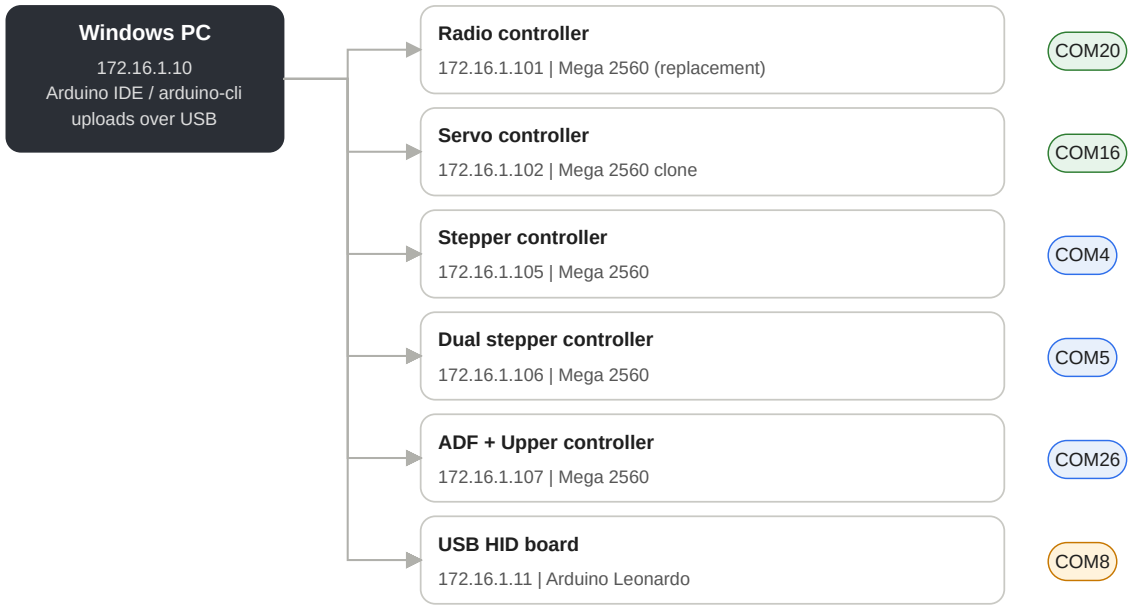
```
Radio Controller firmware 1.0 build 5 compiled Nov  6 2026 14:05:12
```

(each board's debug function puts its own name in front of the line). The start-up line is only seen if the debug listener is already running when the board starts; otherwise send `C,VERSION:1` to the board. The

version is also in every heartbeat to the Health Monitor, as `FW:1.0.2` (version 1.0, build 2), and the Health Monitor shows it in the **Firmware** column of its voltage table, so the code on every board can be checked at a glance.

Controller	Sketch	FIRMWARE_NAME	Version and build now (flashed 6 November 2026)
Radio controller .101	JET_RANGER_RADIO_CONTROLLER	Radio Controller	1.0 build 5
Servo controller .102	JET_RANGER_SERVO_CONTROLLER	Servo Controller	1.0 build 4
Stepper controller .105	JET_RANGER_STEPPER_CONTROLLER	Stepper Controller	1.0 build 3
Dual Stepper controller .106	JET_RANGER_OLED_DUAL_STEPPER_CONTROLLER	OLED Dual Stepper Controller	1.0 build 5
ADF and Upper controller .107	JET_RANGER_ADF_CONTROLLER	ADF and Upper Controller	1.0 build 6
USB HID board .11	UDP_TO_HID	USB HID Board	1.0 build 2

B.4 COM port to board mapping



COM numbers are the ones used at the last flash. They follow the USB socket and the board, so check them (see the guide) before an upload.
Each board also has a network connection: USB is for programming only, the cockpit runs over UDP.

Figure 47 — Which board is on which COM port

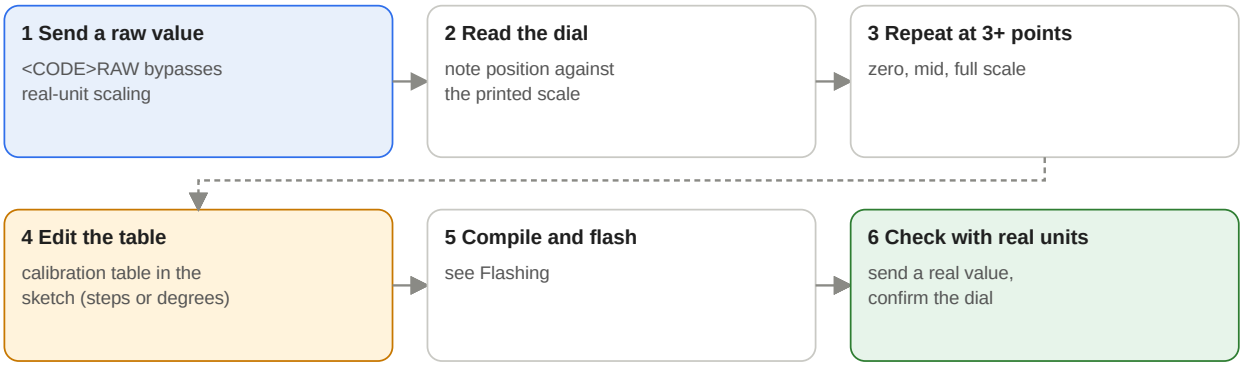
Board	IP	Hardware	COM port
Radio controller	.101	Mega 2560 (replacement board)	COM20
Servo controller	.102	Mega 2560 clone	COM16
Stepper controller	.105	Mega 2560	COM4
Dual stepper controller	.106	Mega 2560	COM5
ADF and Upper controller (combined)	.107	Mega 2560	COM26
USB HID board	.11	Arduino Leonardo	COM8

The Lowrance GPS cable is not one of these boards: the GPS interface asks for its serial port the first time it runs and remembers it. These are the ports used at the last flash of each board. They follow the USB socket and the board: the Servo clone has no USB serial number, so its COM number is tied to the socket it is plugged into, and a replaced Mega gets a new number. Check before every upload.

B.5 Sketch settings to know about

- Per-gauge `SwingXXX` gates and `SwingLoops` near the top of the Stepper and Dual Stepper sketches control which swings run at boot.
- Zero trims (`<CODE>ZeroOffset`, `TSoffset`, `RSoffset`) are set near the top of the same sketches.
- Every sketch has `reflectorIP` (172.16.1.10) for debug and heartbeat traffic.

Appendix C: Calibration



Typical tables: pitch -30 / 0 / +30 degrees, bank -90 / 0 / +90 degrees, one row per stepper or servo gauge.

Sending <CODE>RAW:n moves the motor to step n directly, so any dial can be calibrated without a simulator.

Figure 48 — The calibration loop

Between table points the boards interpolate in a straight line and hold the end value beyond the ends.

1. Send a **raw** value (`D,TQRAW:300`) to move the needle to a raw step or servo position.
2. Read the dial against its printed scale.
3. Repeat at three or more points (zero, mid scale, full scale).
4. Edit the calibration table in the sketch.
5. Compile and upload.
6. Send **real** values (`D,TQ:60`) and confirm the dial.

The StepperVSITester sends every raw and real token without a simulator. Do not drive a gauge hard against its stop: a stalled stepper or servo draws high current and gets hot. After any mechanical change to a gauge, recheck its table and zero trim.

Appendix D: How this document is built

This document and the User Guide are generated, not typeset by hand. The sources are Markdown files and vector diagrams in `Jet Ranger Documentation/DocBuild`:

Item	Purpose
`src/master/*.md`, `src/guide/*.md`	Text of this document and the User Guide
`diagrams_a.py` to `diagrams_e.py`, `svgdraw.py`, `make_diagrams.py`	The 35 diagrams, written as Python functions that draw SVG; `python make_diagrams.py` regenerates them
`build_docs.py`, `run_lo.ps1`	Builds the `.odt` and `.pdf` through LibreOffice's Python interface (`run_lo.ps1 build_docs.py master`)
`export_pcb_images.py`, `pcb_to_jpg.ps1`	The schematic and layout pictures of Appendix E, drawn from the KiCad files
`export_install_views.py`, `trim_to_jpg.ps1`	The installation views of Appendix F (top side, external connectors labelled) and the text of that appendix, drawn from the KiCad layout files
`make_instrument_map.py`	The instrument map of Appendix G (the A3 drawing as SVG, PDF and PNG, the readable crops used in the document, and the text of the appendix)
`pdf2png.ps1`, `montage.ps1`, `svg2png.py`	Helpers for checking pages and diagrams visually

To change a diagram edit its function and rebuild; to change text edit the Markdown and rebuild. Keep `PROGRAM_SUMMARY.md` files and these sources in step after a code change.

Appendix E: PCB schematics and layouts

The schematic (the top sheet of each project) and the board layout of every PCB project in `Jet Ranger/PCBs/`, exported from KiCad 9. The layout shows the front and back copper (red and blue), the silkscreen (yellow) and the board outline, viewed from the front. The images are bitmaps: zoom in on the PDF or the document to read the labels, or open the KiCad project for the full detail. The schematic and layout images are regenerated from the KiCad files, so they show the boards as they were on the day the document was built, including any changes not yet committed.

E.1 Jet Ranger Radio Auto Pilot

Revision 1.4 (31 October 2026; the title block shows 1.4, 2026-10-31). Mega shield for the Radio controller (172.16.1.101). Revision 1.4 added the Huey Warning Panel connectors J20 (switches), J21 (backlight), J22 (LED anodes) and J23 (LED cathodes). Carries the USR-ES1 Ethernet module, the TCA9548A I2C multiplexer for the OLEDs, the MAX7219 that lights the 19 Auto Pilot and Audio panel LEDs, the backlight MOSFET (D5), the COM and NAV panel connector (`COMM_NAV`), the `AUTOPILOT` and `AUDIO` panel connectors, five matrix columns and the Auto Pilot knob inputs.

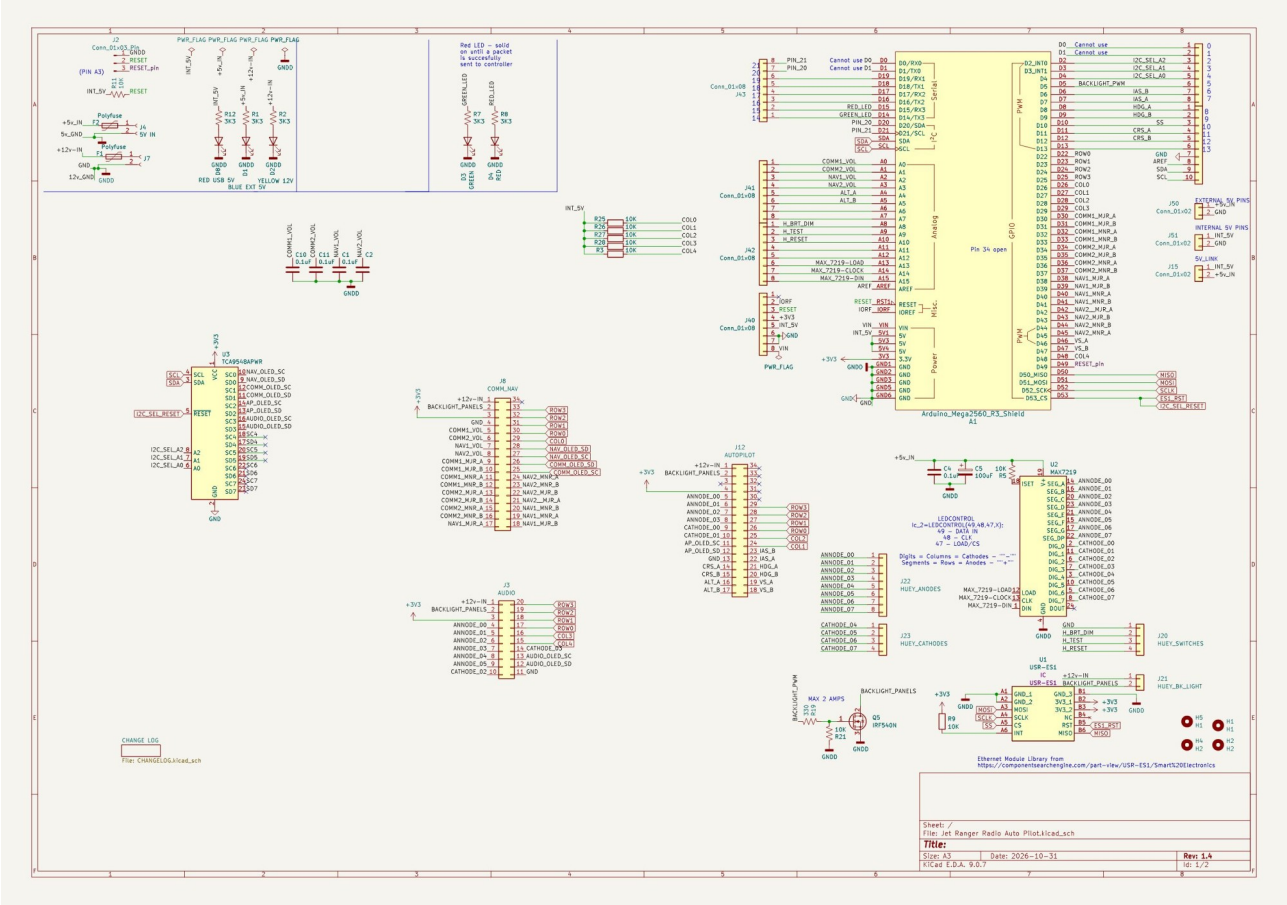


Figure 49 — Jet Ranger Radio Auto Pilot schematic

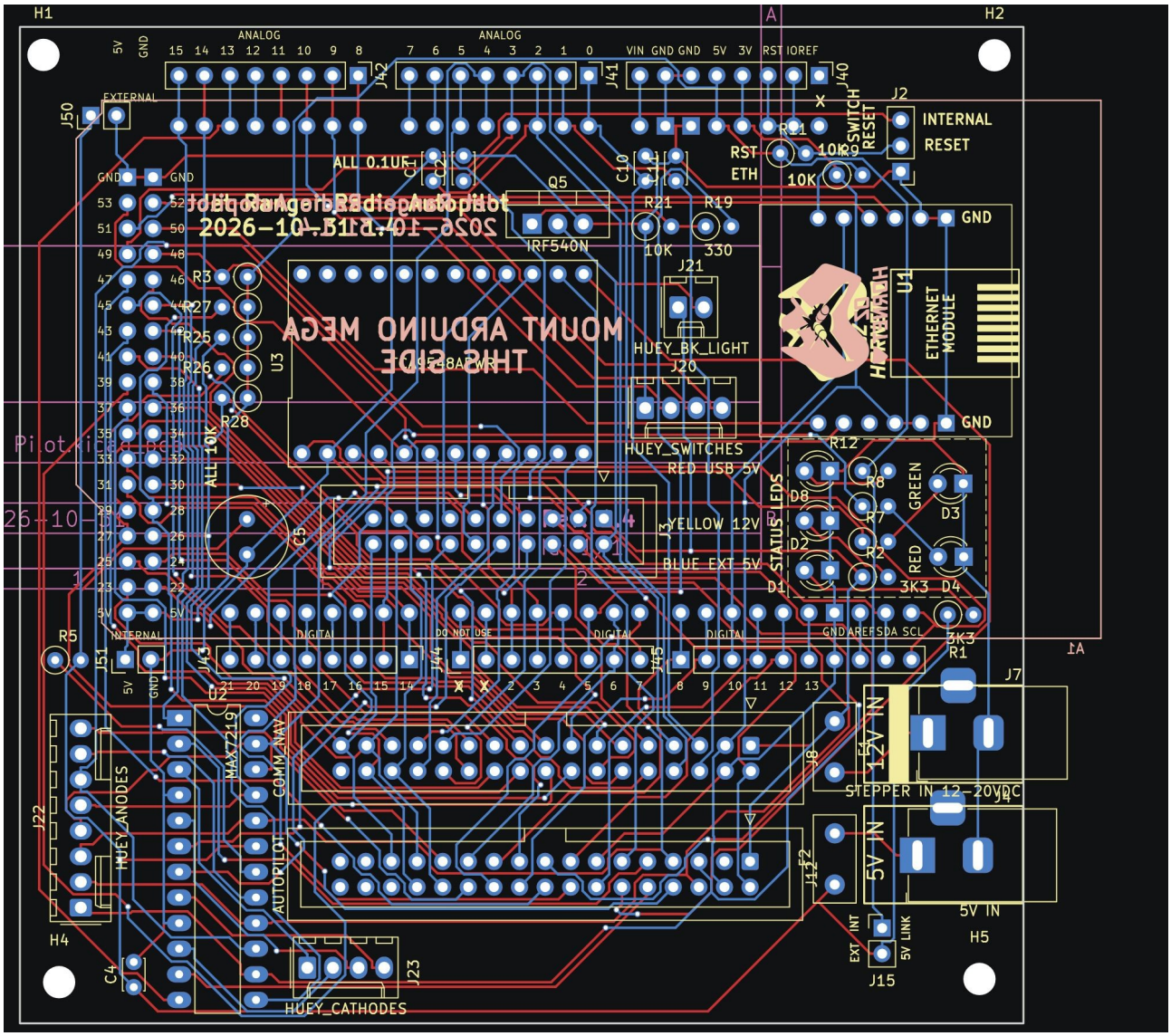


Figure 50 — Jet Ranger Radio Auto Pilot board layout

E.2 Jet Ranger COM and NAV

No revision set (title block dated 2024-08-10). Radio panel board behind the COM and NAV faceplate. Eight rotary encoders (a major and a minor knob for COM1, COM2, NAV1 and NAV2), the four frequency-swap switches in a diode matrix on column 0, four volume pots, and sockets for the two character OLEDs.

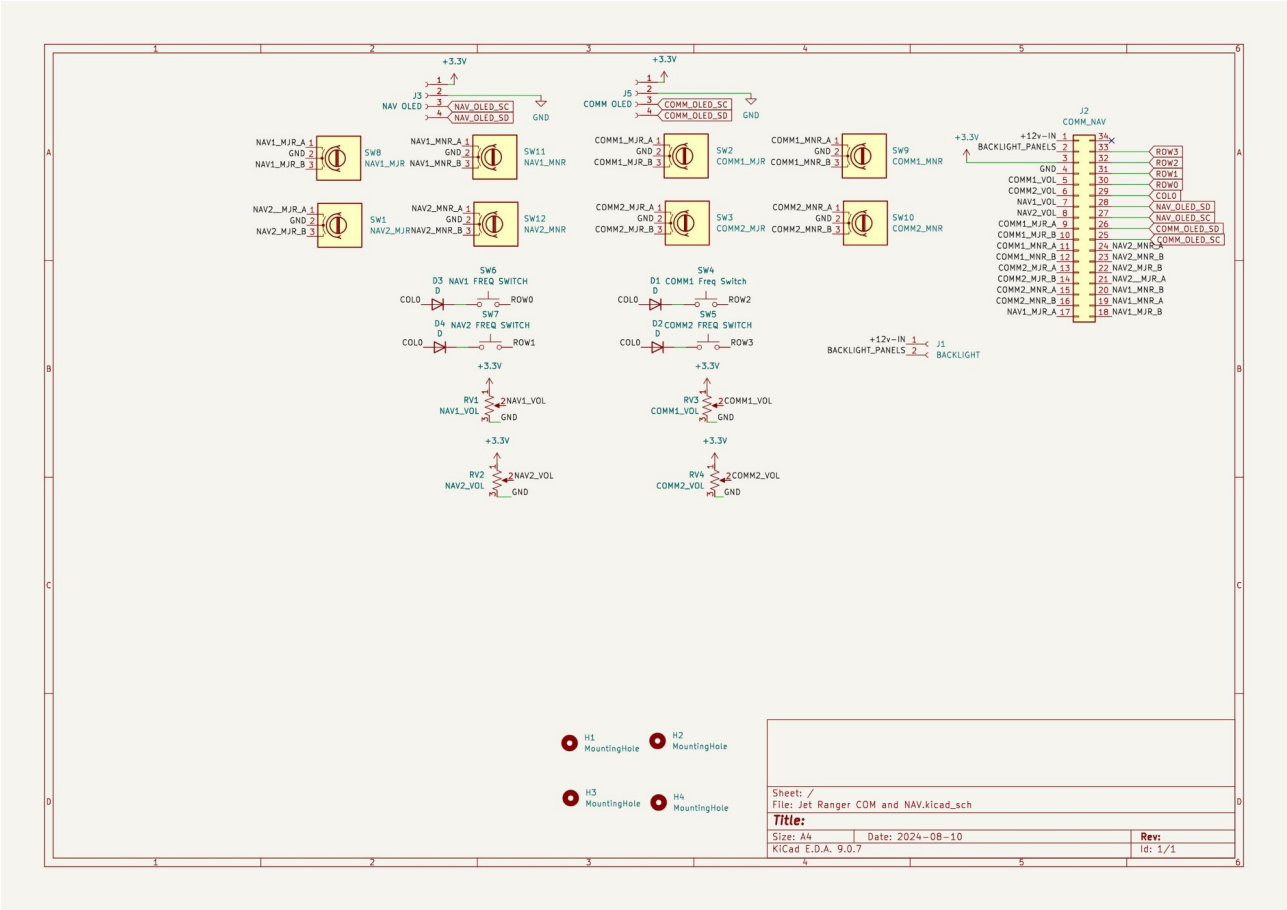


Figure 51 — Jet Ranger COM and NAV schematic

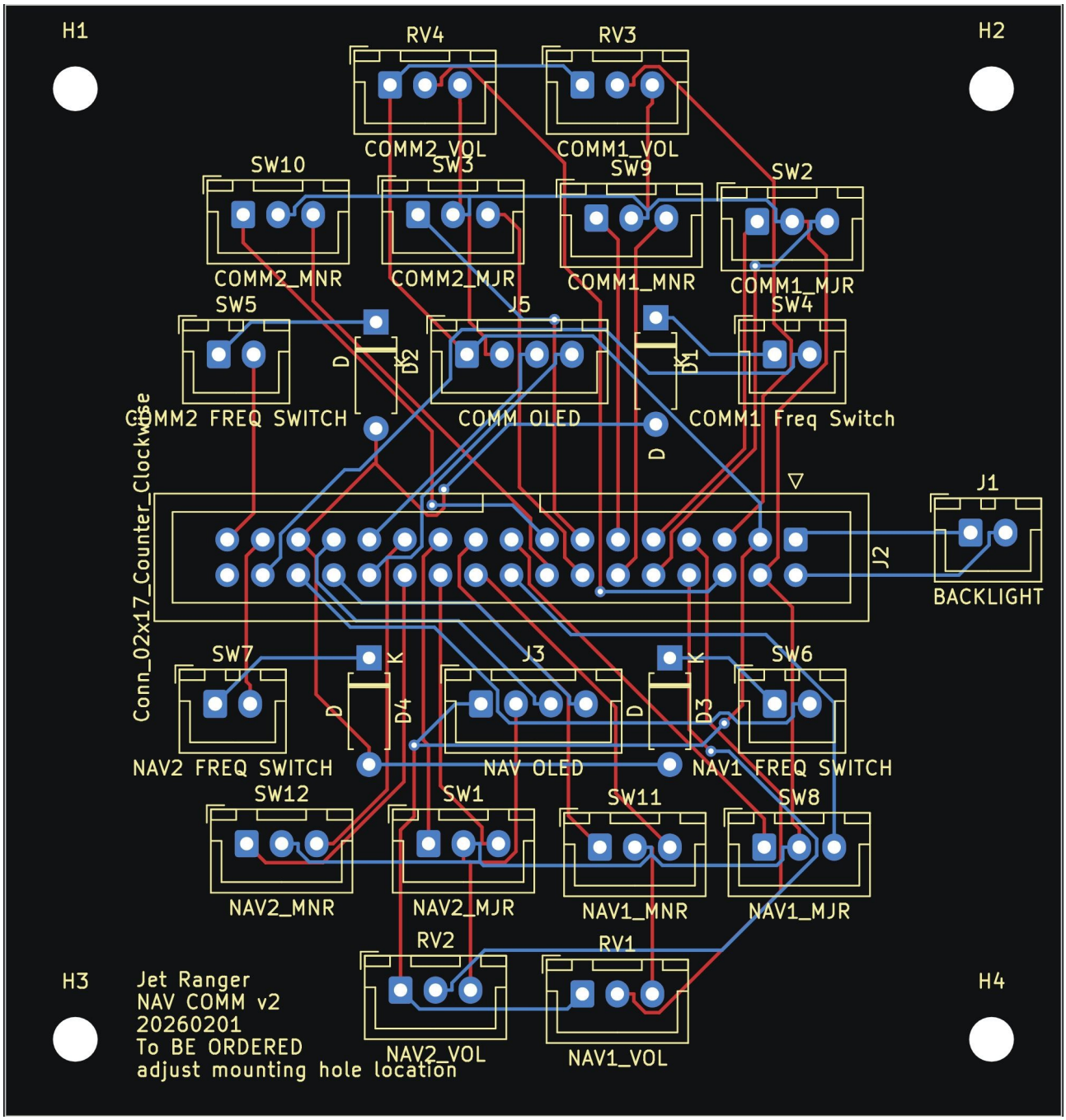


Figure 52 — Jet Ranger COM and NAV board layout

E.3 Jet Ranger Front Panel Servo

Revision 1.4 (2026-10-18). Mega shield for the Servo controller (172.16.1.102). ULN2803A lamp drivers for the 15 warning lamps, headers for the pitch and bank servos, three front-panel switch inputs, the USR-ES1 Ethernet module and the backlight MOSFET. Version 1.4 corrected the MOSFET pin-out.

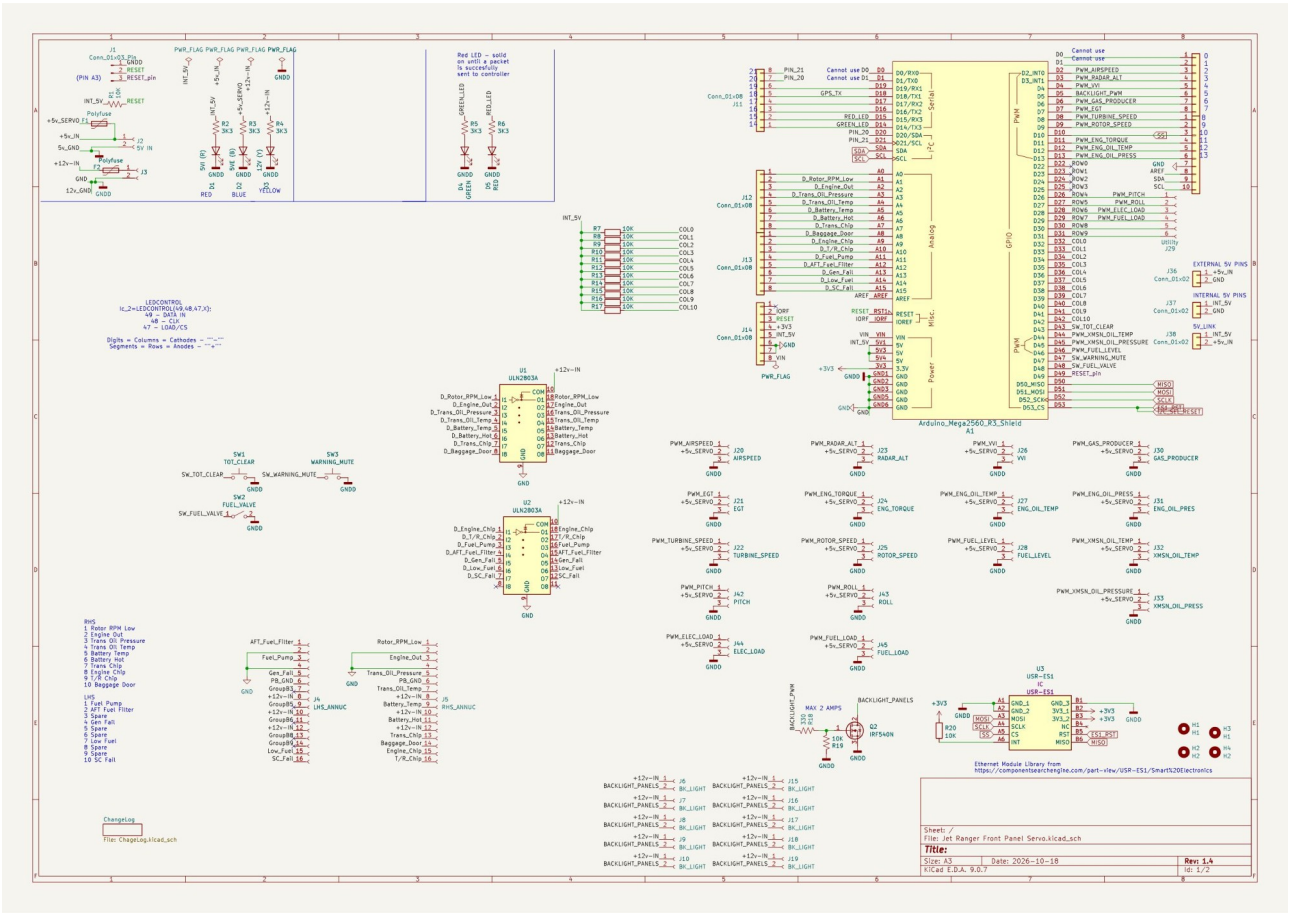


Figure 53 — Jet Ranger Front Panel Servo schematic

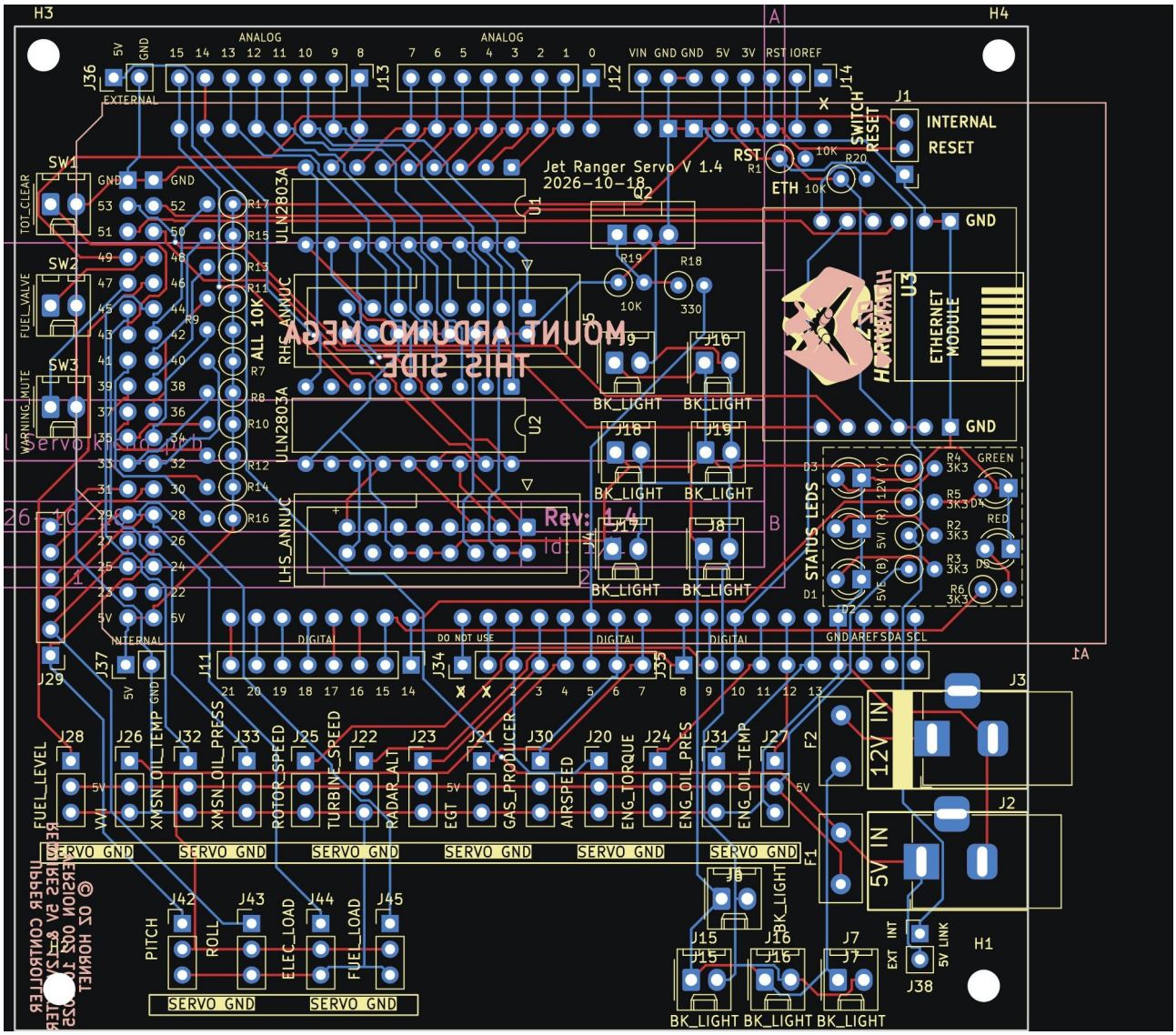


Figure 54 — Jet Ranger Front Panel Servo board layout

E.4 Jet Ranger 13 Stepper

Title block revision 1.6 (2026-10-18); the change log lists changes up to 1.5. Mega shield for the Stepper controller (172.16.1.105). Fourteen six-pin gauge sockets for the X27 steppers, a 12 V lamp supply with polyfuses, the USR-ES1 Ethernet module and the backlight MOSFET. 1.4 moved the LEDs away from the USB connectors and 1.5 selected the IRF540N.

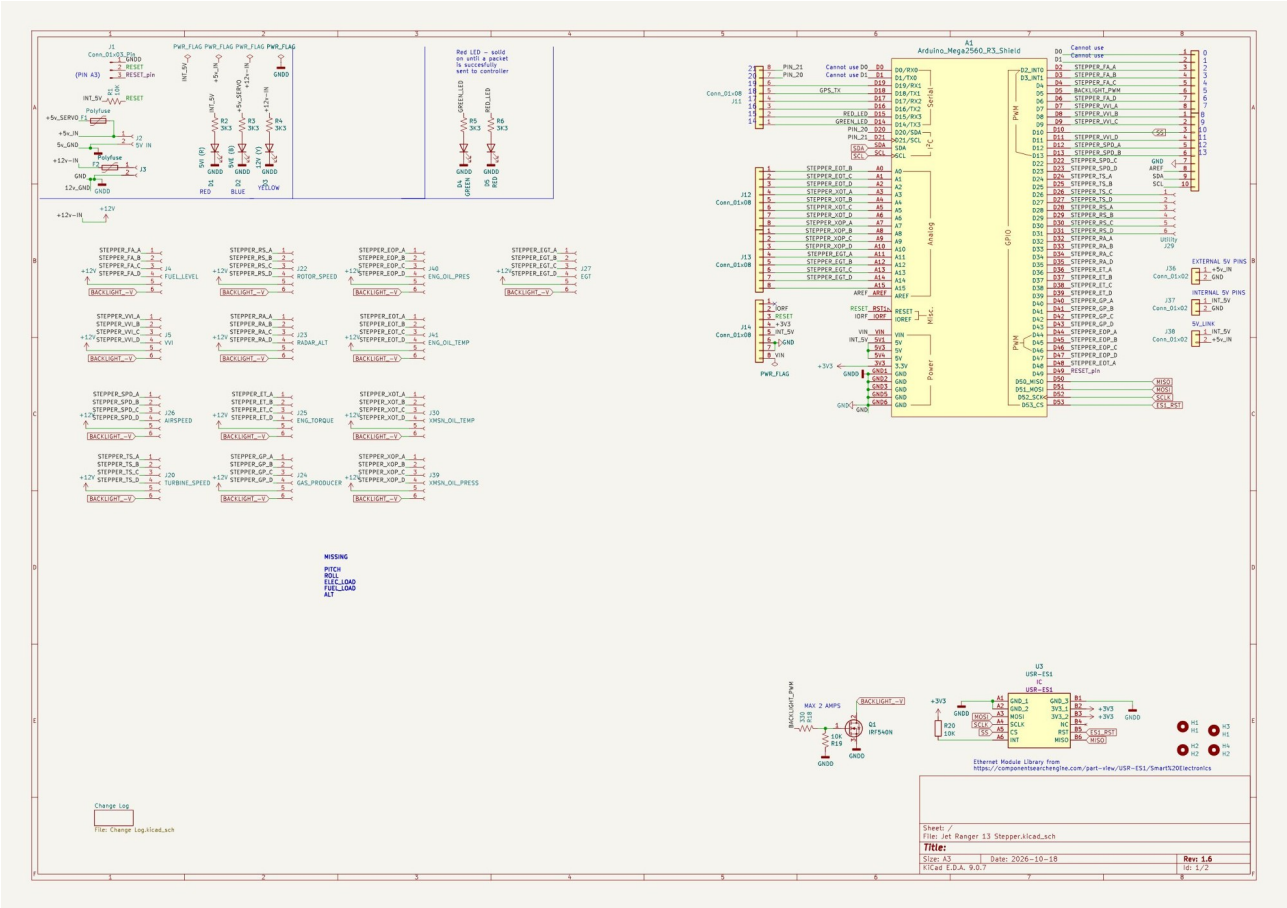


Figure 55 — Jet Ranger 13 Stepper schematic

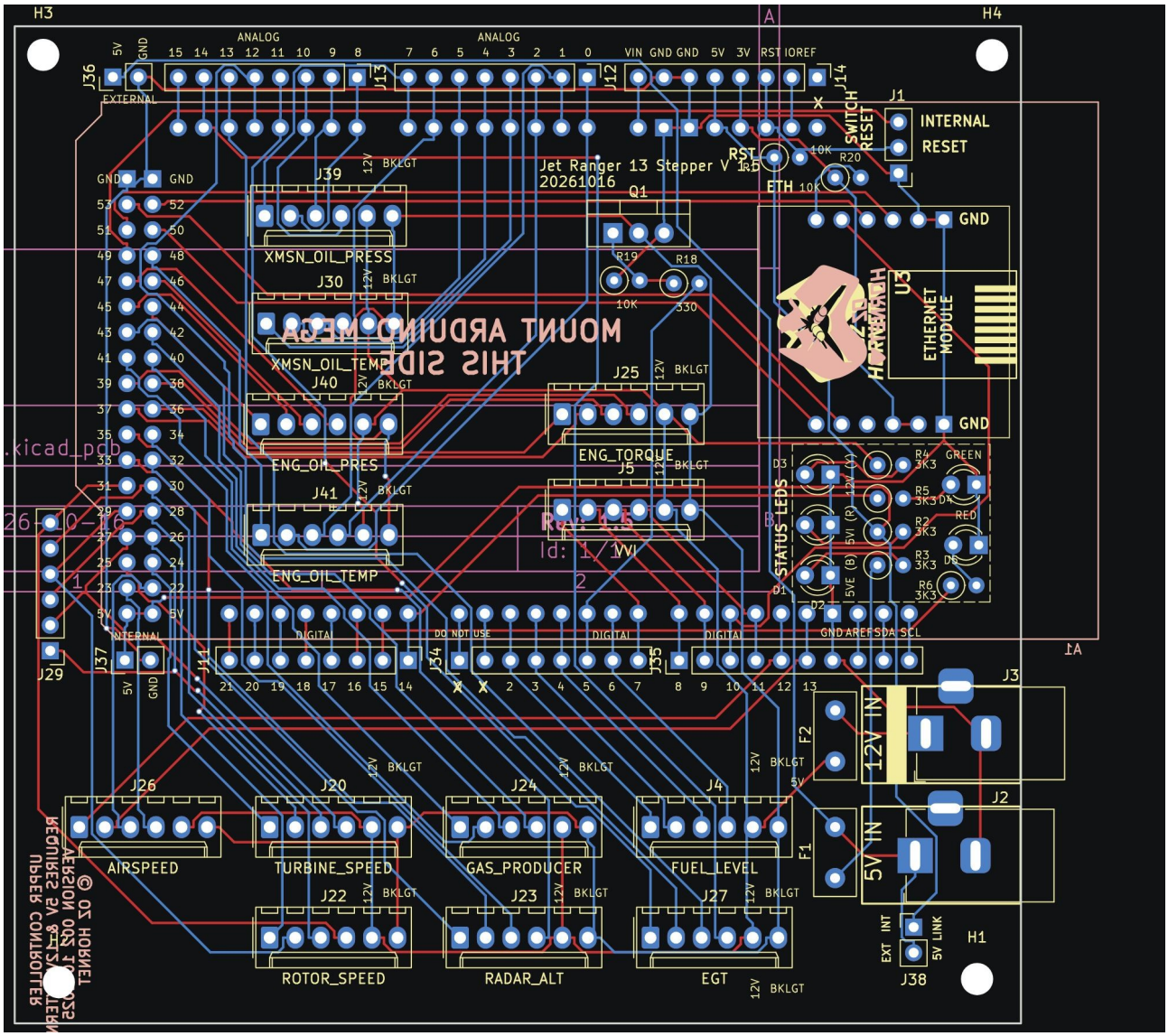


Figure 56 — Jet Ranger 13 Stepper board layout

E.5 Jet Ranger OLED Dual Stepper

Revision 1.5 (2026-10-18). Mega shield for the Dual Stepper controller (172.16.1.106). Six-pin gauge sockets for the oil, load, RPM and altimeter gauges, TCA9548A sockets for the four OLEDs (barometer, altimeter, clock, temperature), the USR-ES1 Ethernet module and the backlight MOSFET. 1.3 swapped power and ground on the four-pin OLED connections, 1.4 removed the short between Mega pins 2 and 32 and 1.5 corrected the MOSFET.

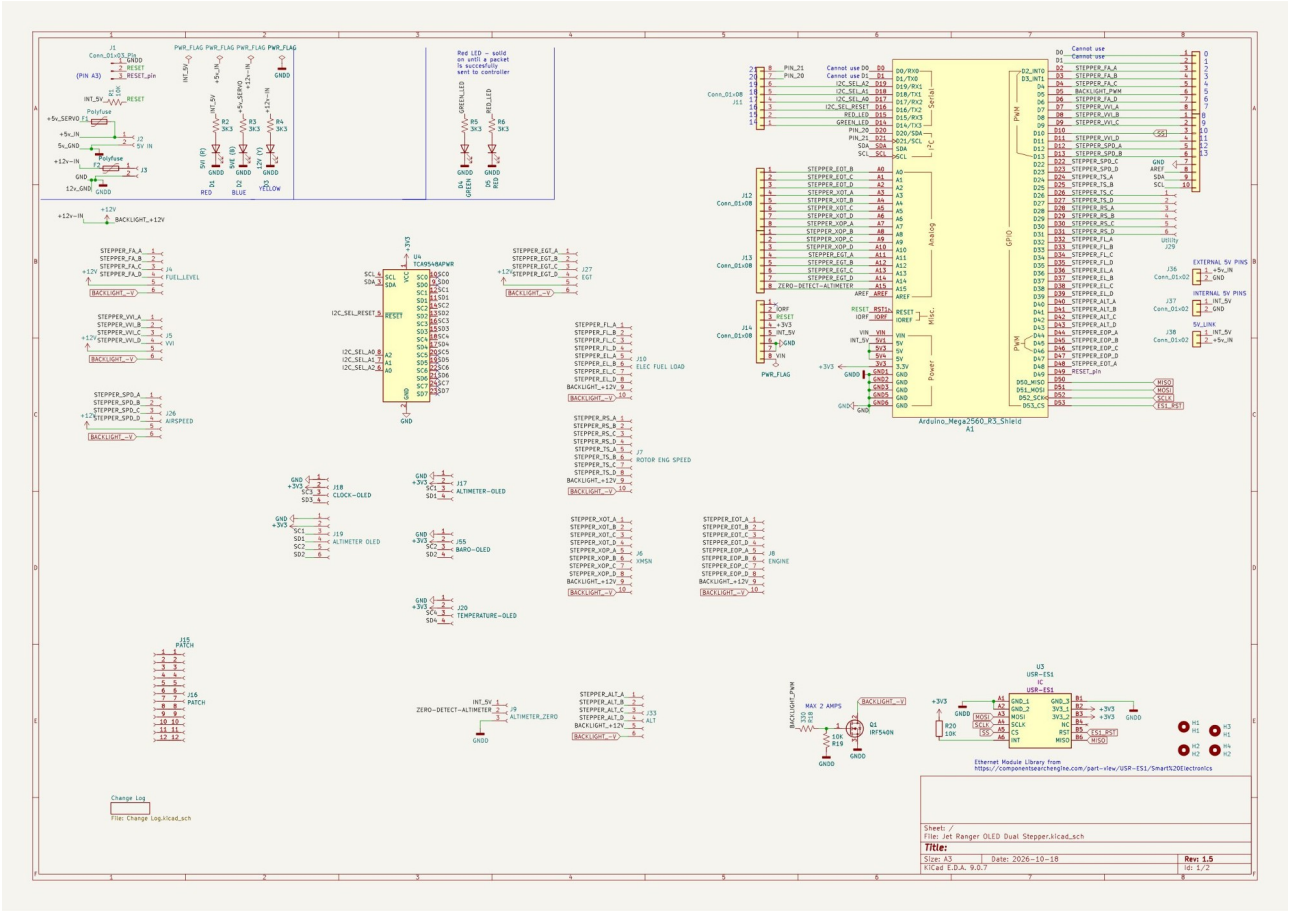


Figure 57 — Jet Ranger OLED Dual Stepper schematic

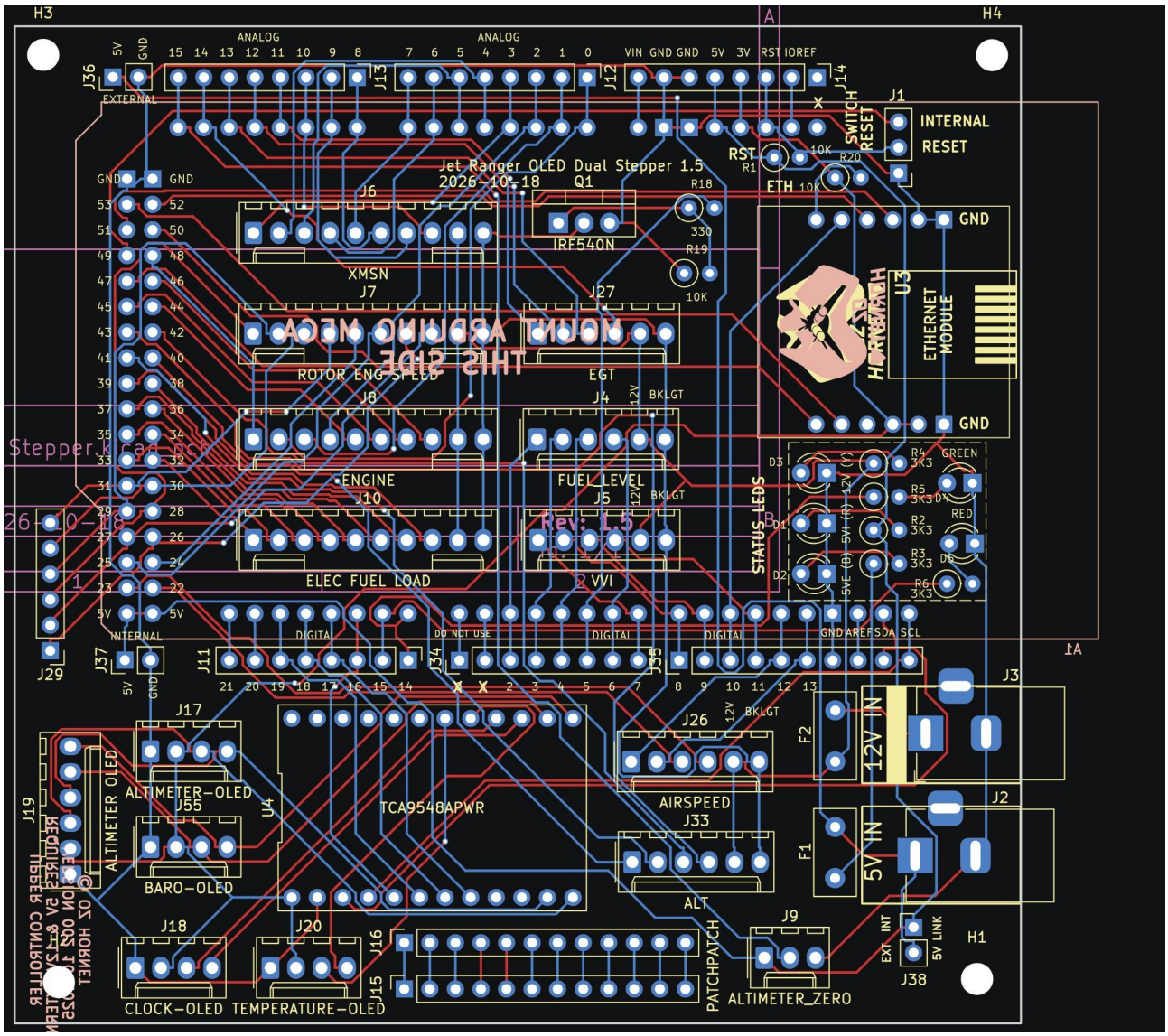


Figure 58 — Jet Ranger OLED Dual Stepper board layout

E.6 Jet Ranger X27 Small Stepper

Title block revision 2 (2022-08-04). One board for each stepper gauge. One X27 or VID29 stepper motor, two backlight LEDs with a 470 ohm resistor (the schematic value, see the backlight resistor table) and the six-pin connector to the controller.

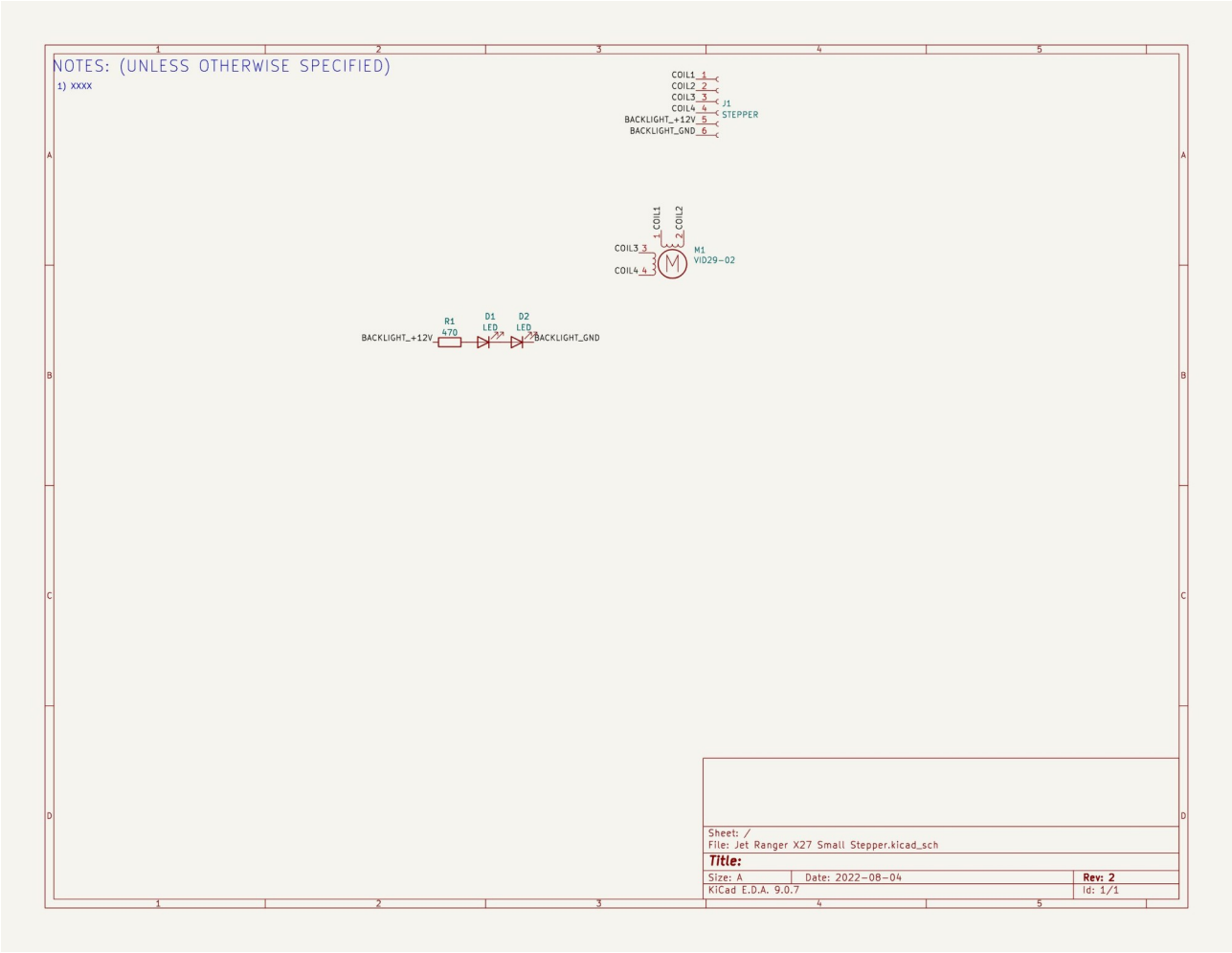


Figure 59 — Jet Ranger X27 Small Stepper schematic

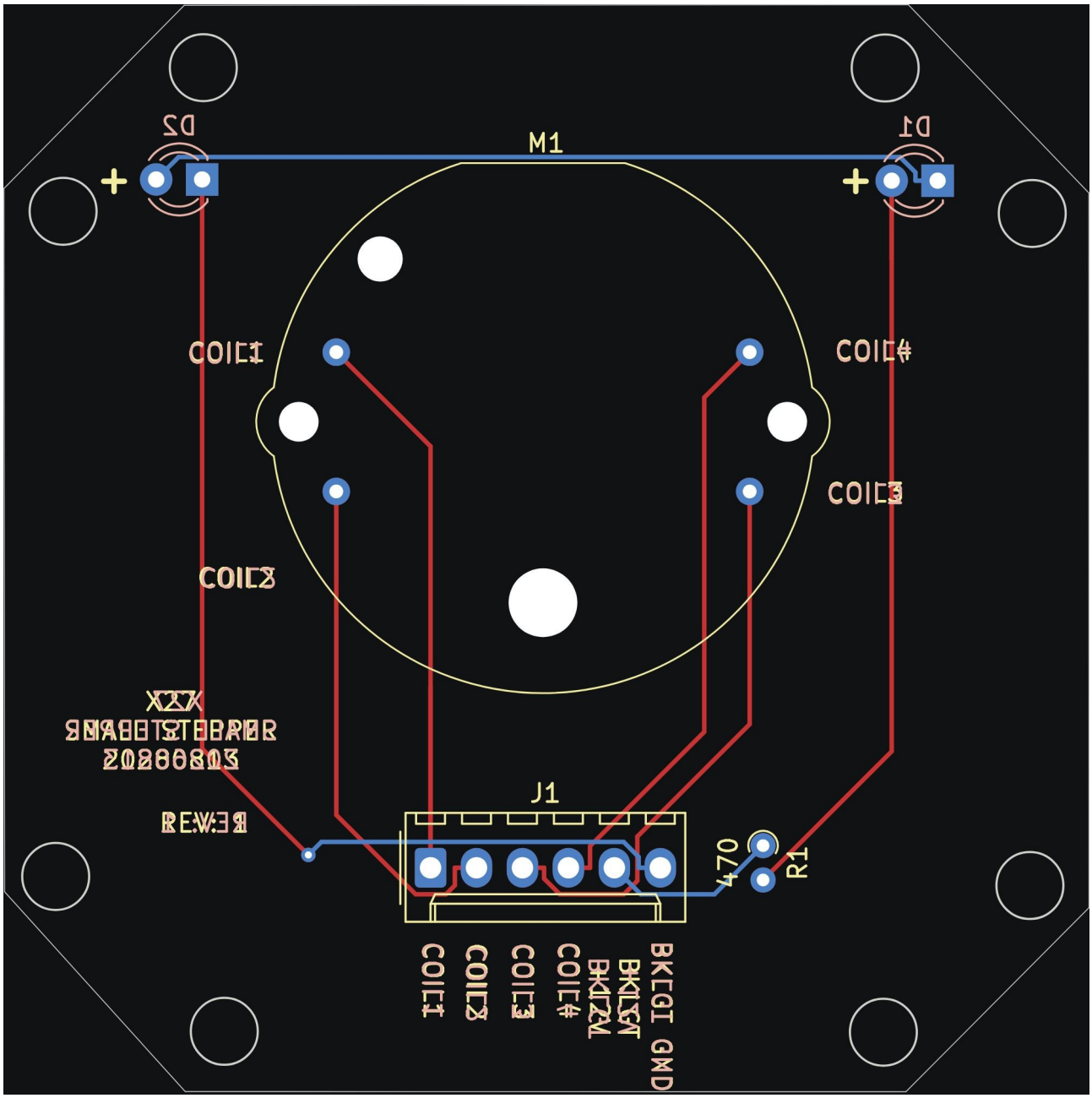


Figure 60 — Jet Ranger X27 Small Stepper board layout

E.7 Jet Ranger Upper ADF Controller

Revision 1.0 (2026-10-26). The combined ADF and Upper controller (172.16.1.107), the reference PCB for the ADF controller. It merges the Jet Ranger Upper Controller (1.4) and the Jet Ranger ADF Controller (1.6) onto one Mega, started from the ADF Controller. The ADF major and minor knobs (D6 to D9), the ADF keypad (A1 to A7, connector J41), the landing gear, flaps, speed brake and brake inputs (D31 to D34), the hyd boost, anti-ice and caution dim inputs (D35 to D37, J20), three landing gear lamps (D2 to D4), the I2C OLED, the Upper switch matrix (rows D22 to D30 on J30, columns D38 to D45 on J31 with 10 k pull-ups), the Upper keypad (A9 to A15, J32), the Warning Mute (J17) and TOT Test (J18) switches, the backlight MOSFET (D5, panel backlight on J33), a panel brightness pot on A0 and a 12 V monitor on A8 (47 k and 10 k divider). The 1x16 breakout headers J9 to J14 were removed to make room. The layout is routed (no unconnected items in KiCad's design rule check).

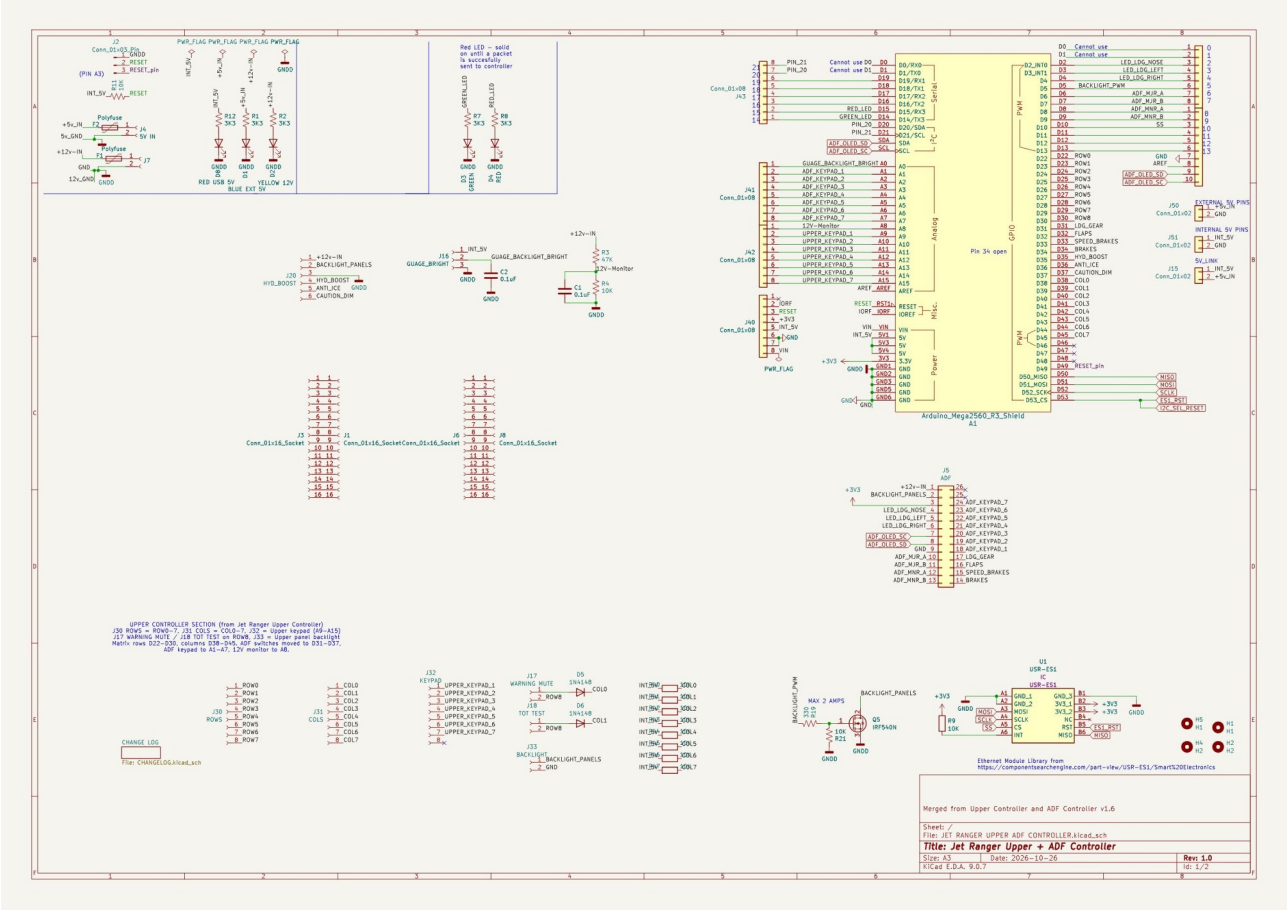


Figure 61 — Jet Ranger Upper ADF Controller schematic

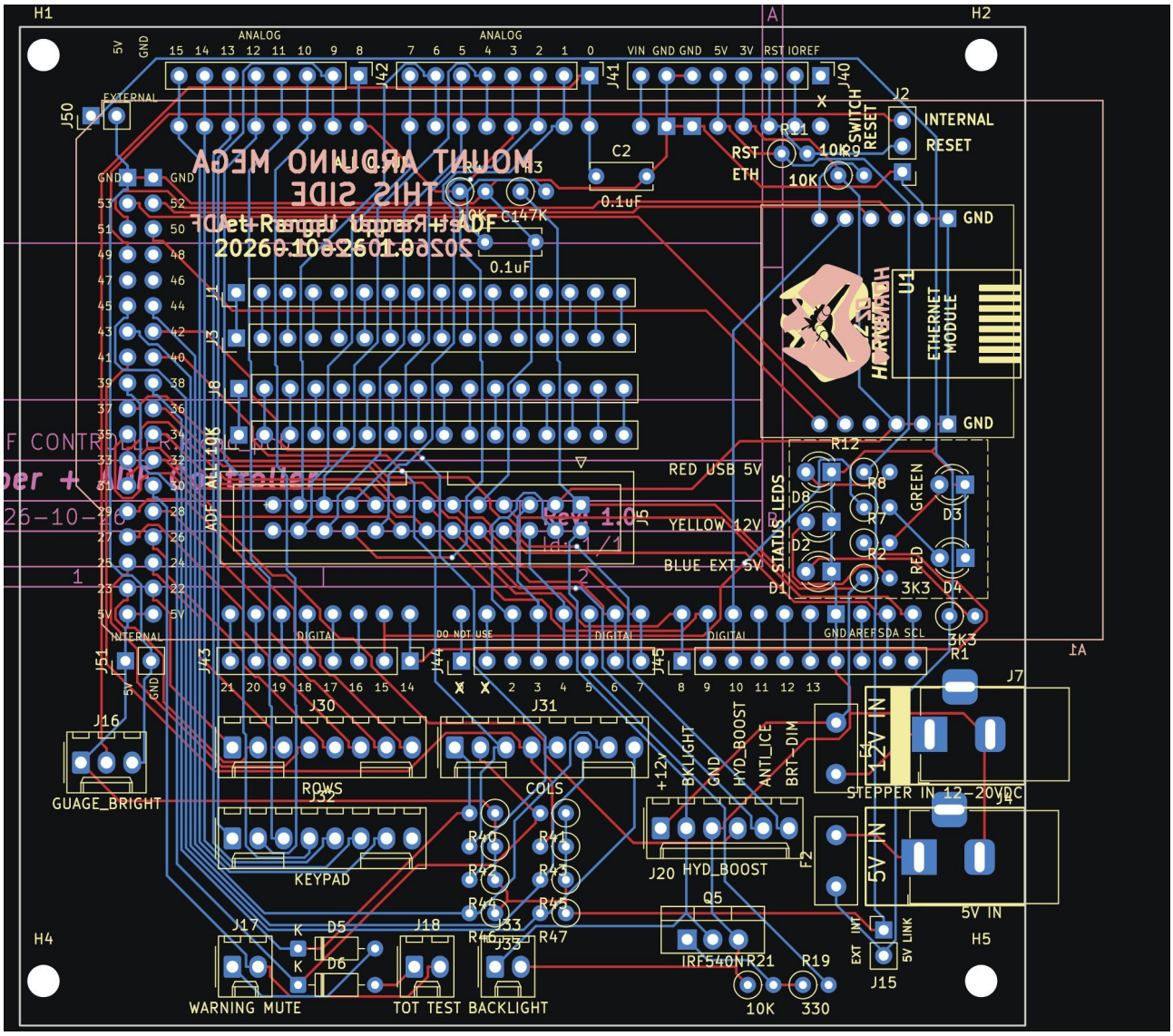


Figure 62 — Jet Ranger Upper ADF Controller board layout

E.8 Jet Ranger Audio Panel

Revision 1.1 (2026-10-17). Switch panel for the Radio Auto Pilot controller's `AUDIO` connector. Eight switches in a diode matrix on columns 3 and 4 (COM1, BOTH, NAV2, DME, COM2, NAV1, MKR, ADF), eleven LEDs on the MAX7219 digits 2 and 3, a connector for the Audio panel OLED and the backlight connector.

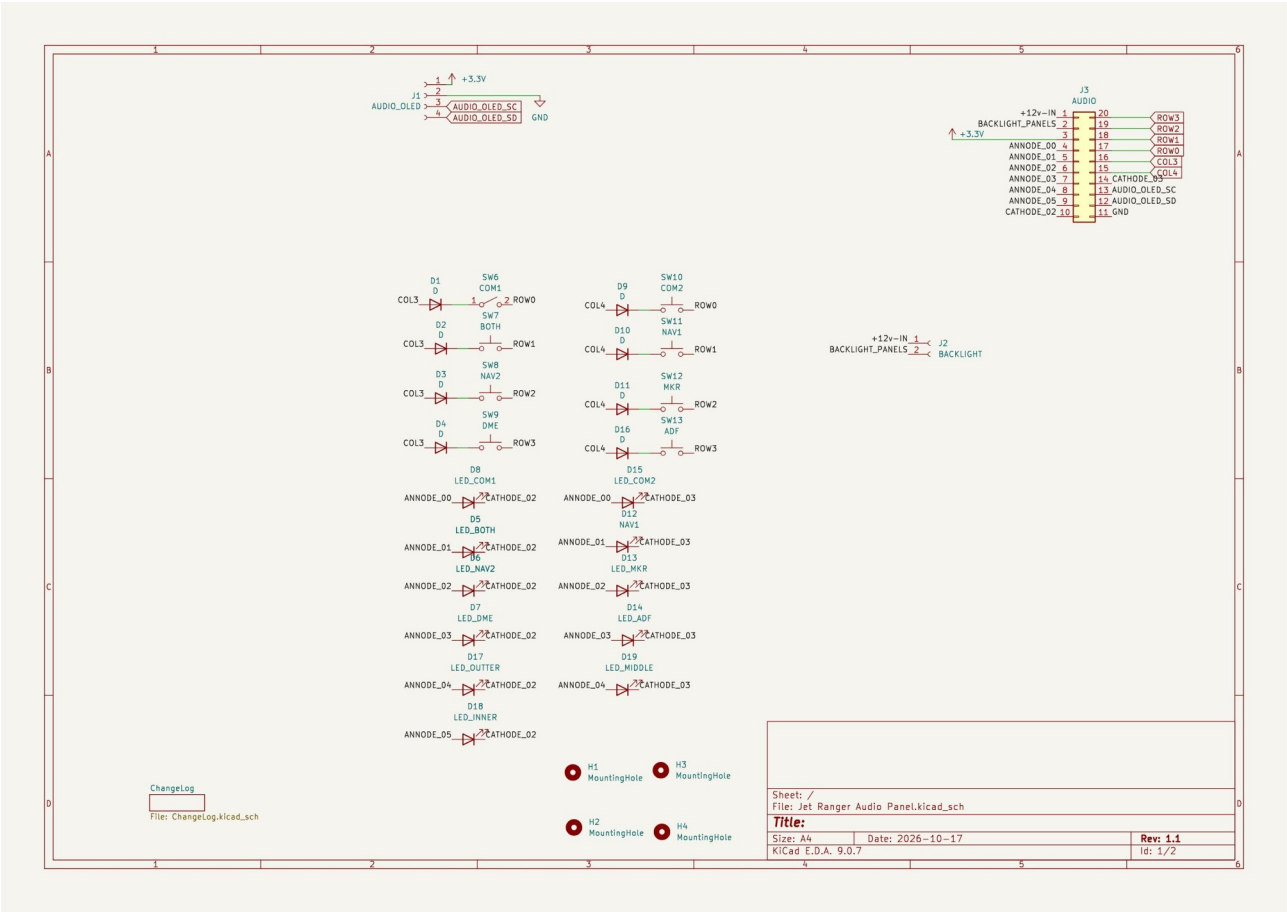


Figure 63 — Jet Ranger Audio Panel schematic

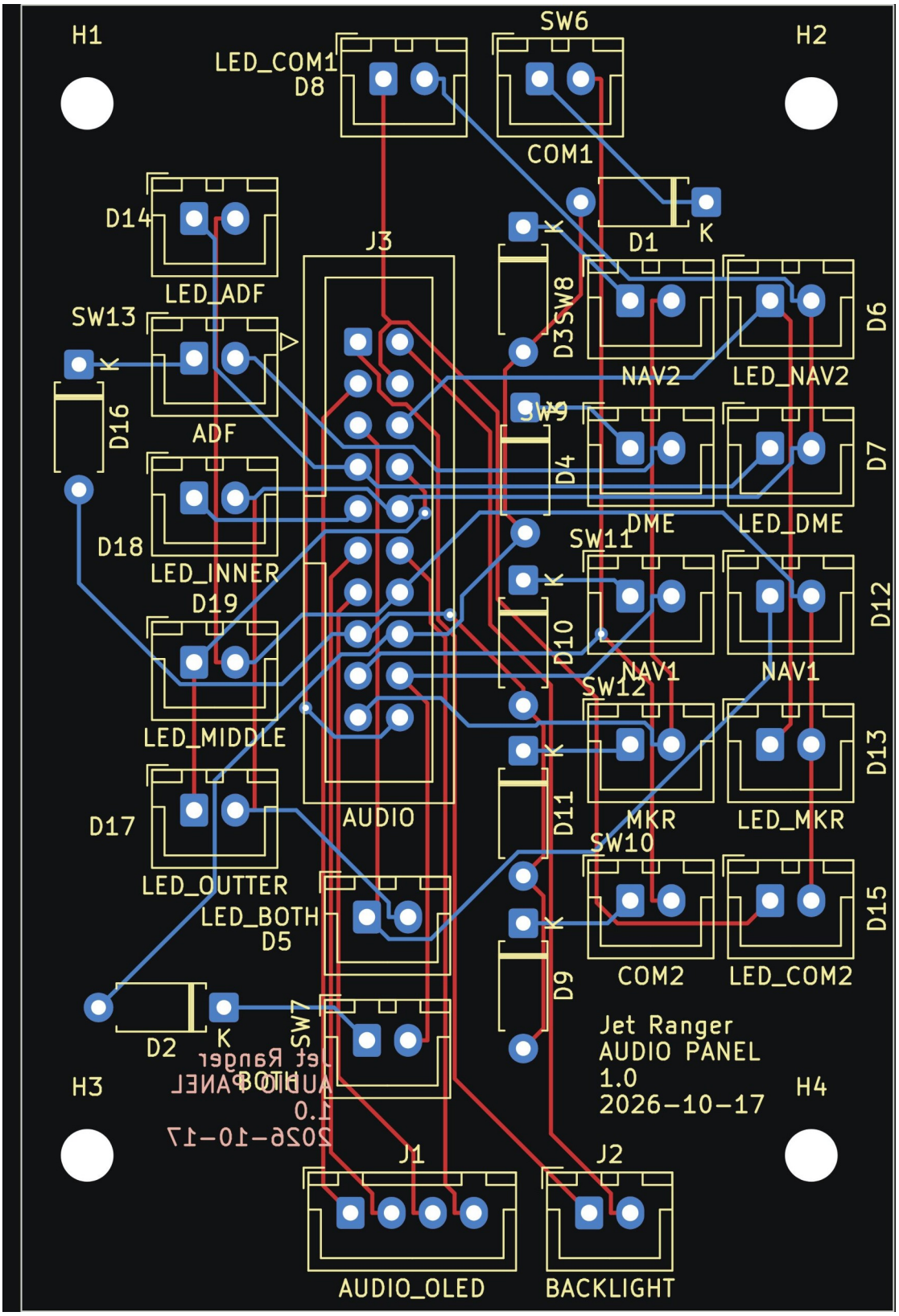


Figure 64 — Jet Ranger Audio Panel board layout

E.9 Jet Ranger Auto Pilot Panel

Revision 1.1 (2026-10-17). Switch panel for the Radio Auto Pilot controller's `AUTOPILOT` connector. Eight switches in a diode matrix on columns 1 and 2 (AP ENGAGE, HDG, CRS, ALT, APPR, FD, BC, IAS), five rotary encoders (CRS, ALT, VS, IAS, HDG), eight LEDs on the MAX7219 digits 0 and 1, the AP OLED connector, the autopilot magnet coil connection and the backlight connector.

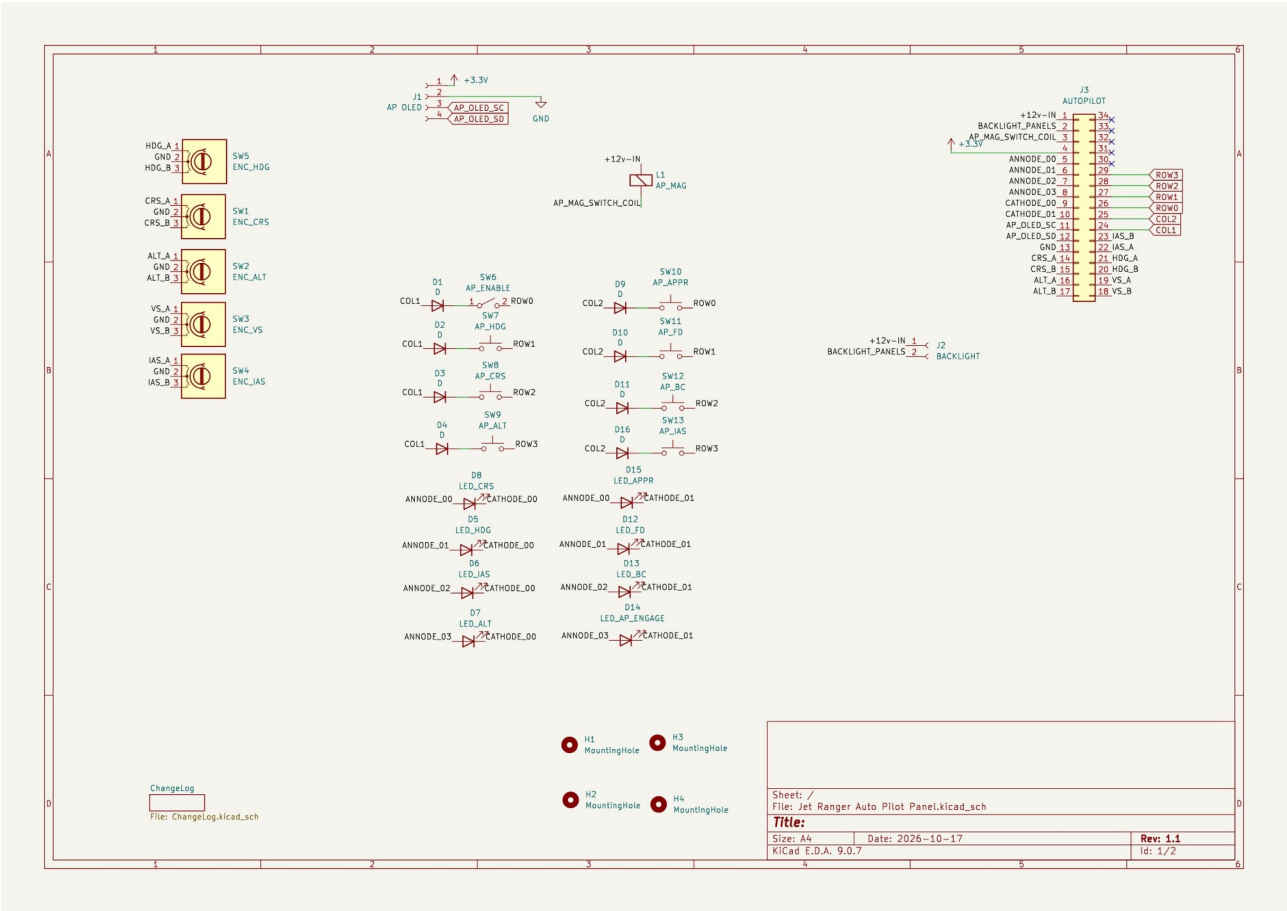


Figure 65 — Jet Ranger Auto Pilot Panel schematic

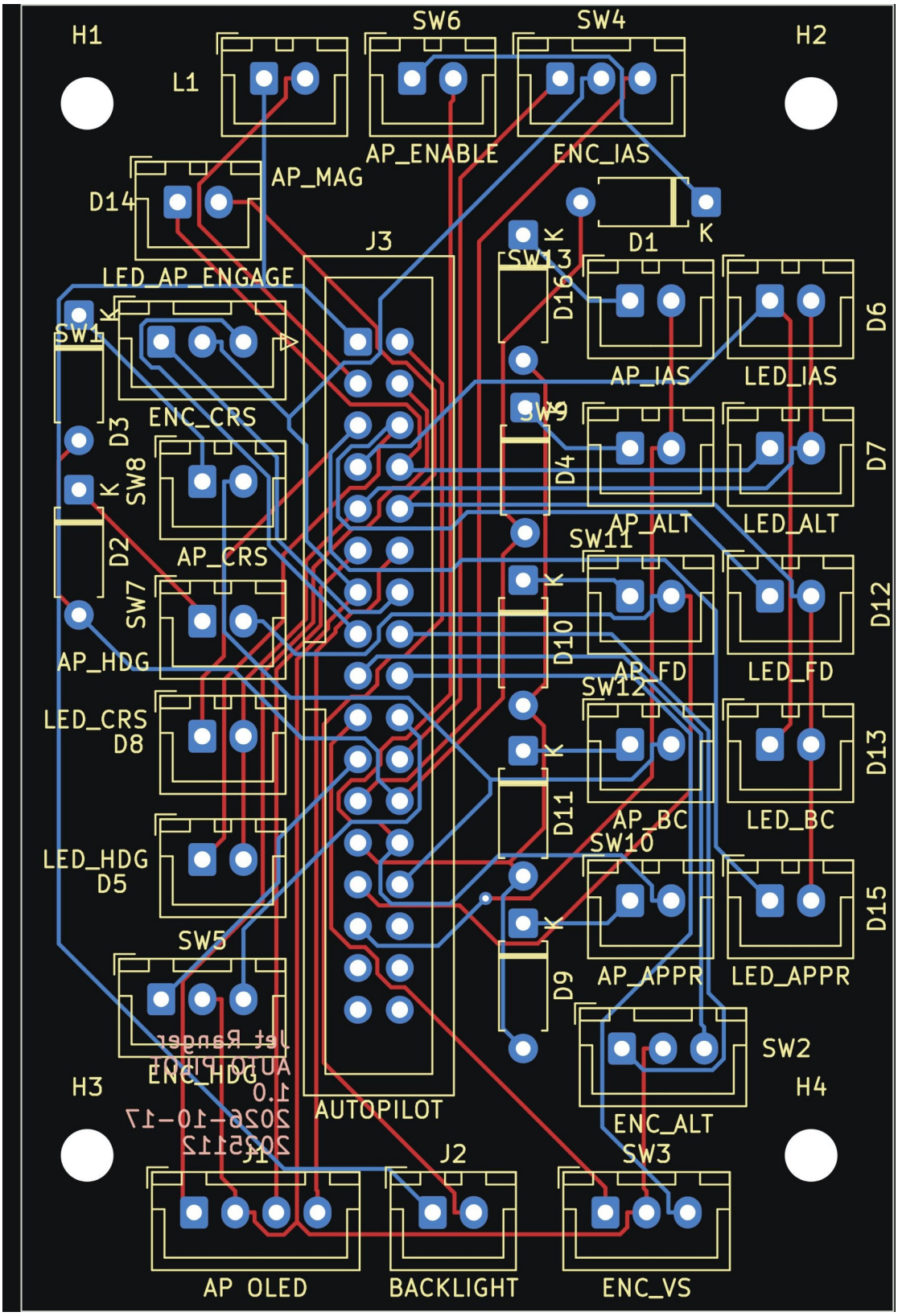


Figure 66 — Jet Ranger Auto Pilot Panel board layout

E.10 Jet Ranger ADF Panel

Revision 1.1 (2026-10-17). Panel for the ADF controller's ADF connector. The ADF major and minor encoders, the landing gear, flaps, speed brake and brakes switches, three landing gear lamps with 330 ohm resistors, the keypad connector, the ADF OLED connector and the backlight connector.

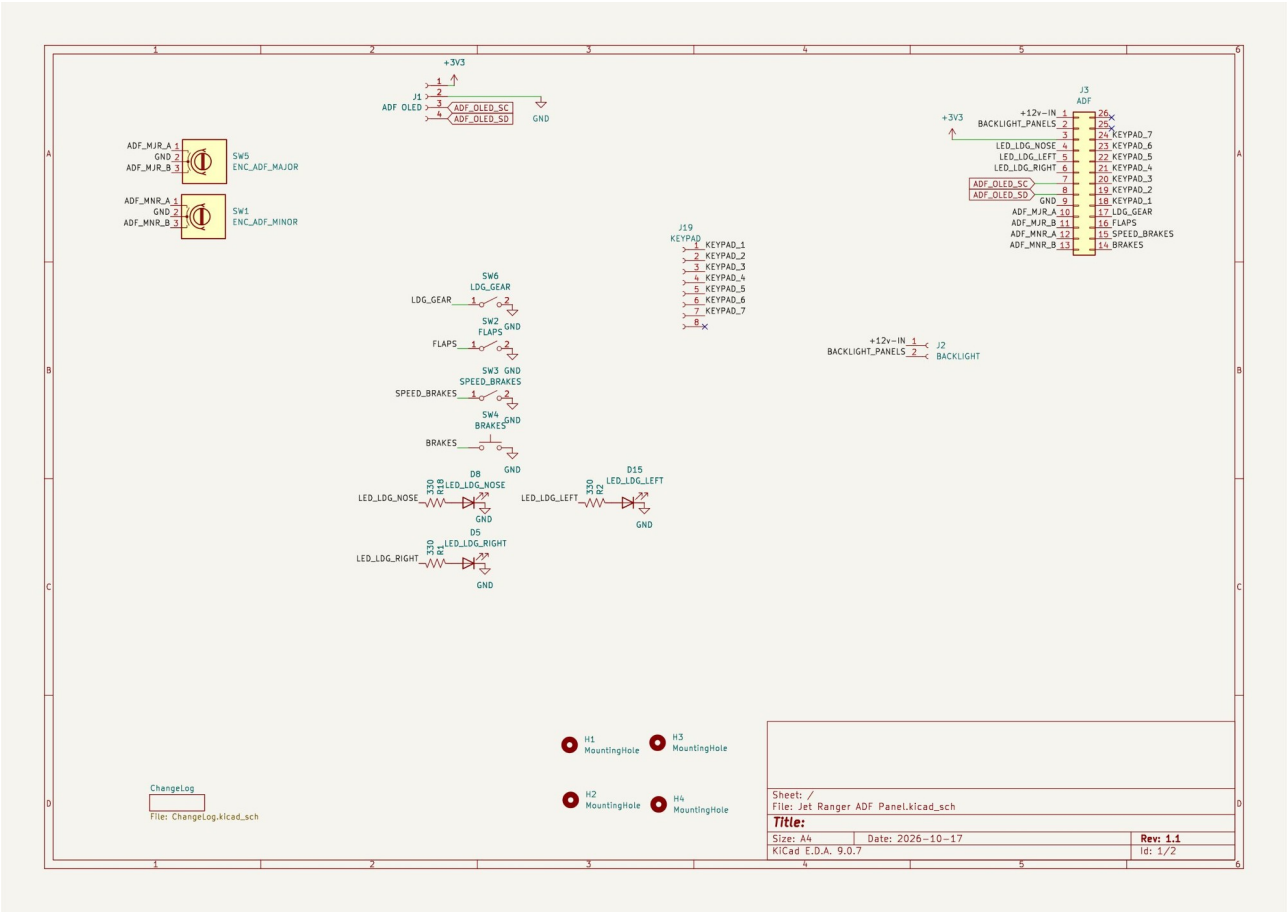


Figure 67 — Jet Ranger ADF Panel schematic

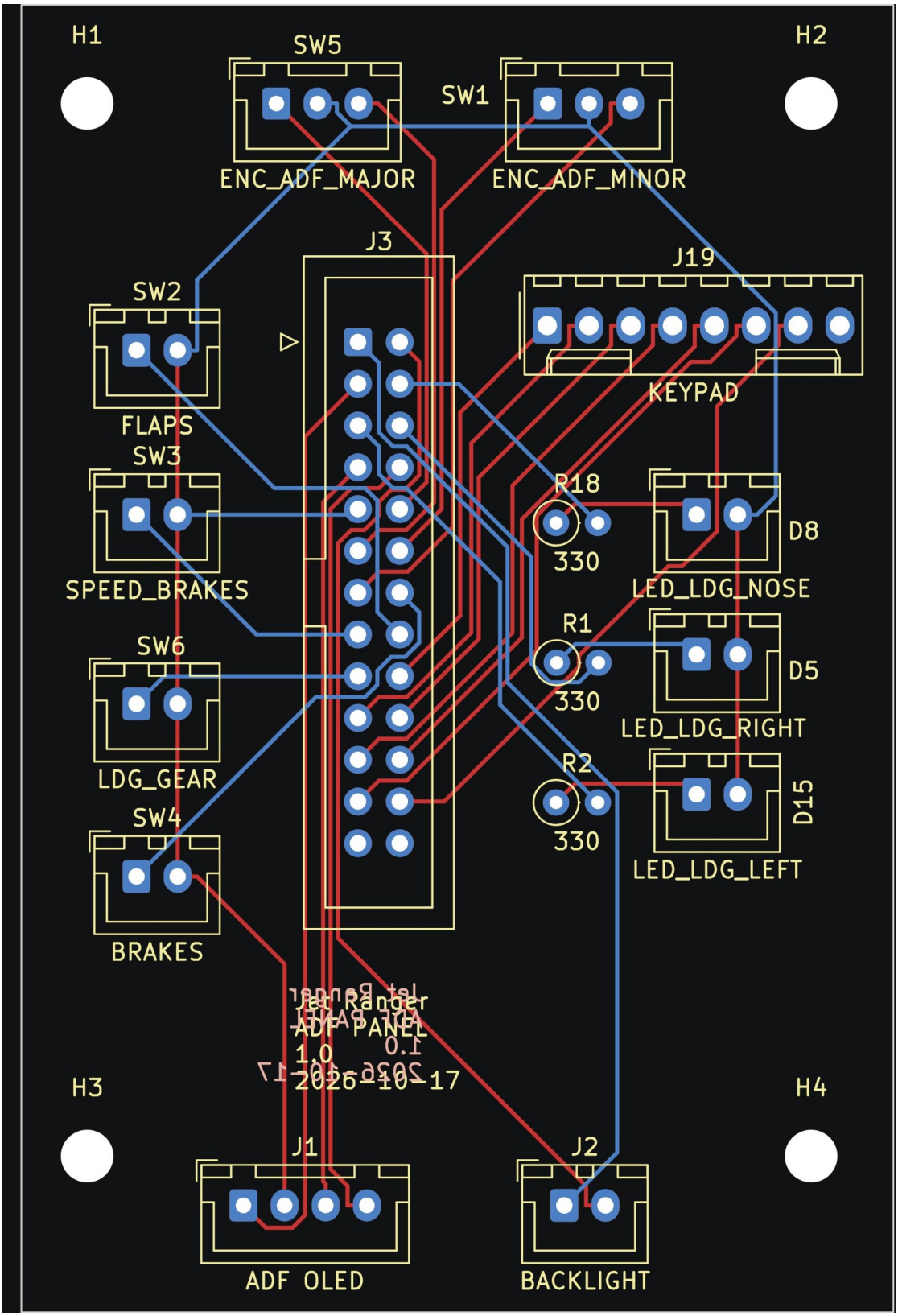


Figure 68 — Jet Ranger ADF Panel board layout

E.11 Annunciator

Schematic sheet only: the project has no board layout. The same sheet is used inside the Servo, 13 Stepper and Dual Stepper schematics. Warning lamp annunciator circuit.

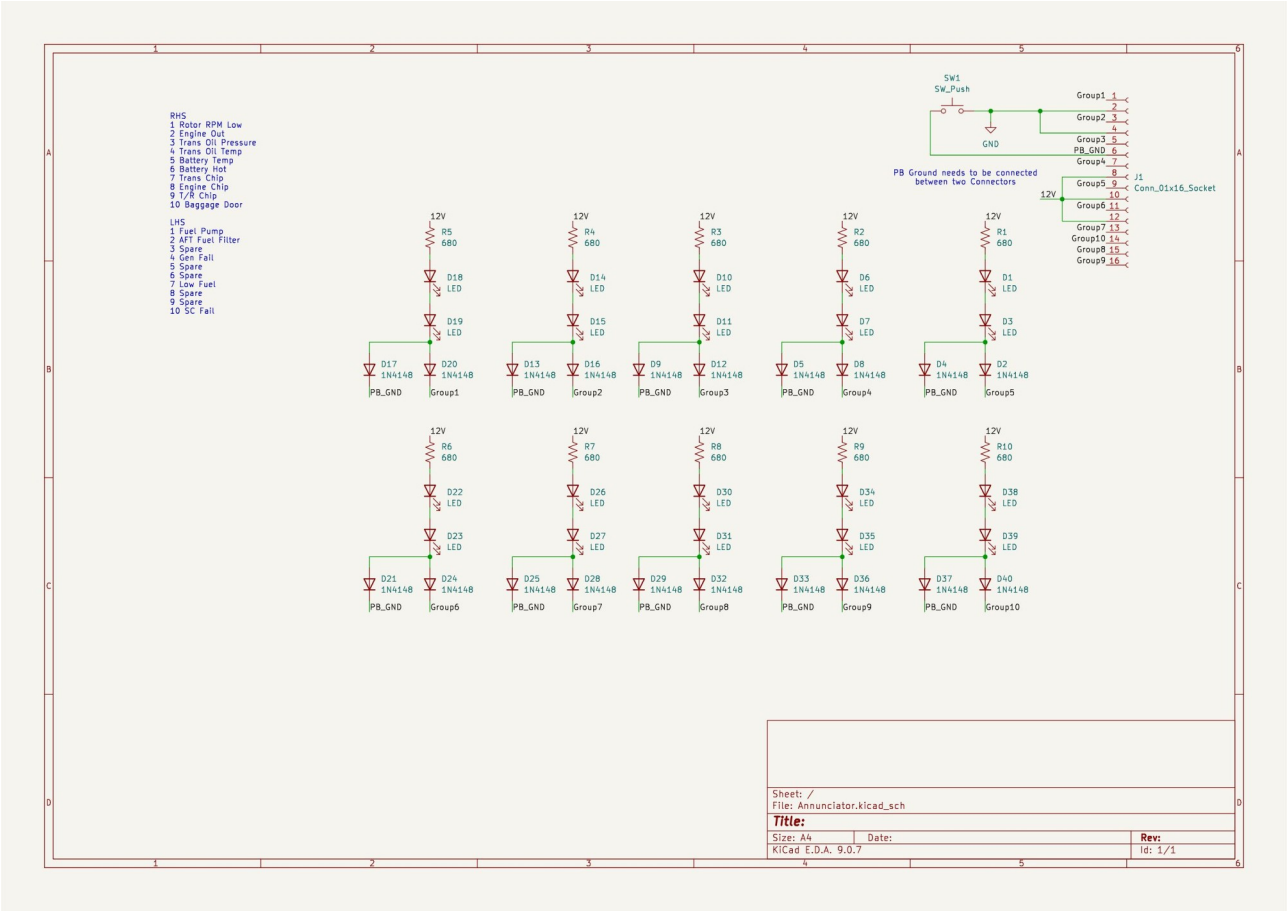


Figure 69 — Annunciator schematic

E.12 Jet Ranger Altimeter

Title block revision 2 (2022-08-04). Not covered by the sketches. Altimeter gauge board.

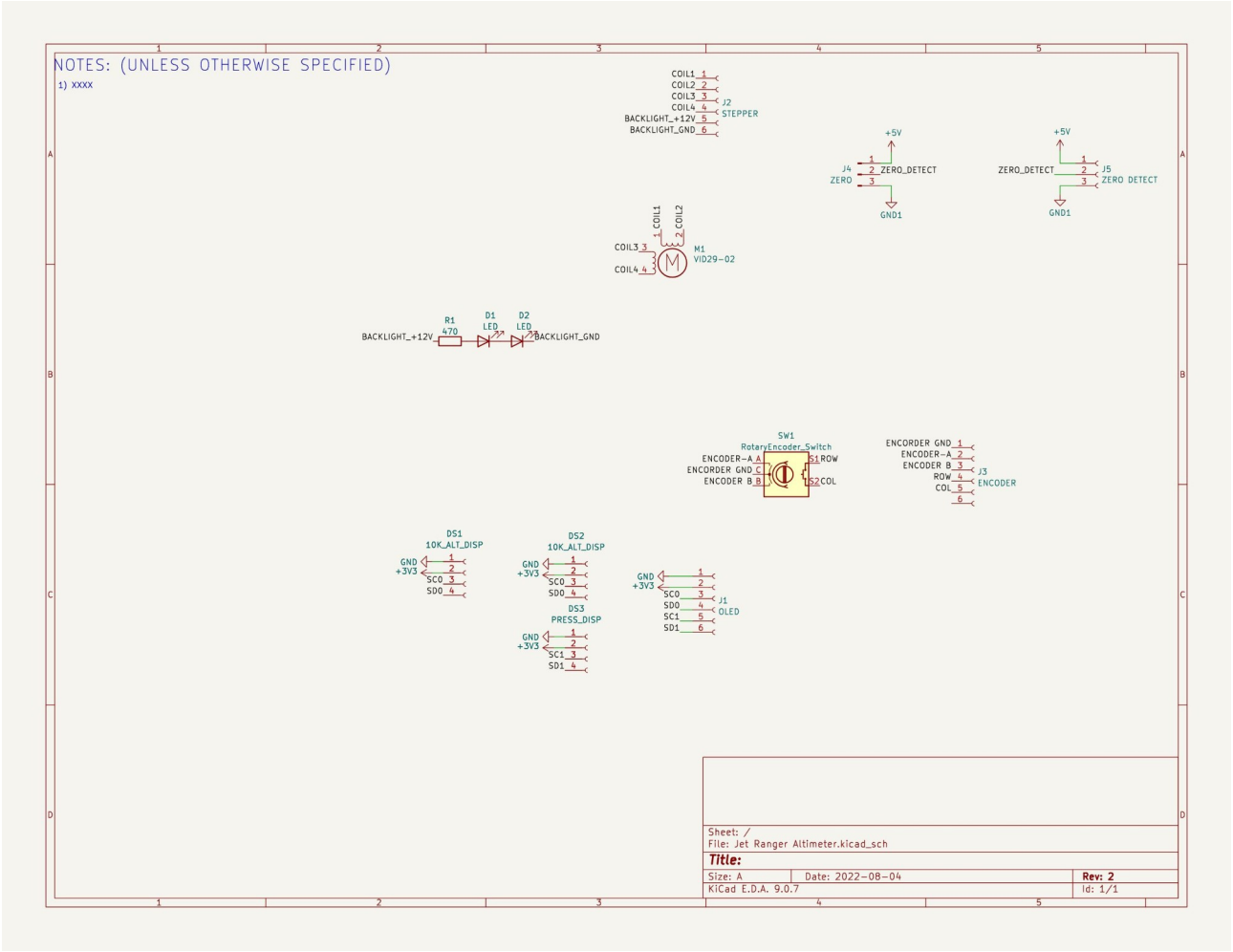


Figure 70 — Jet Ranger Altimeter schematic

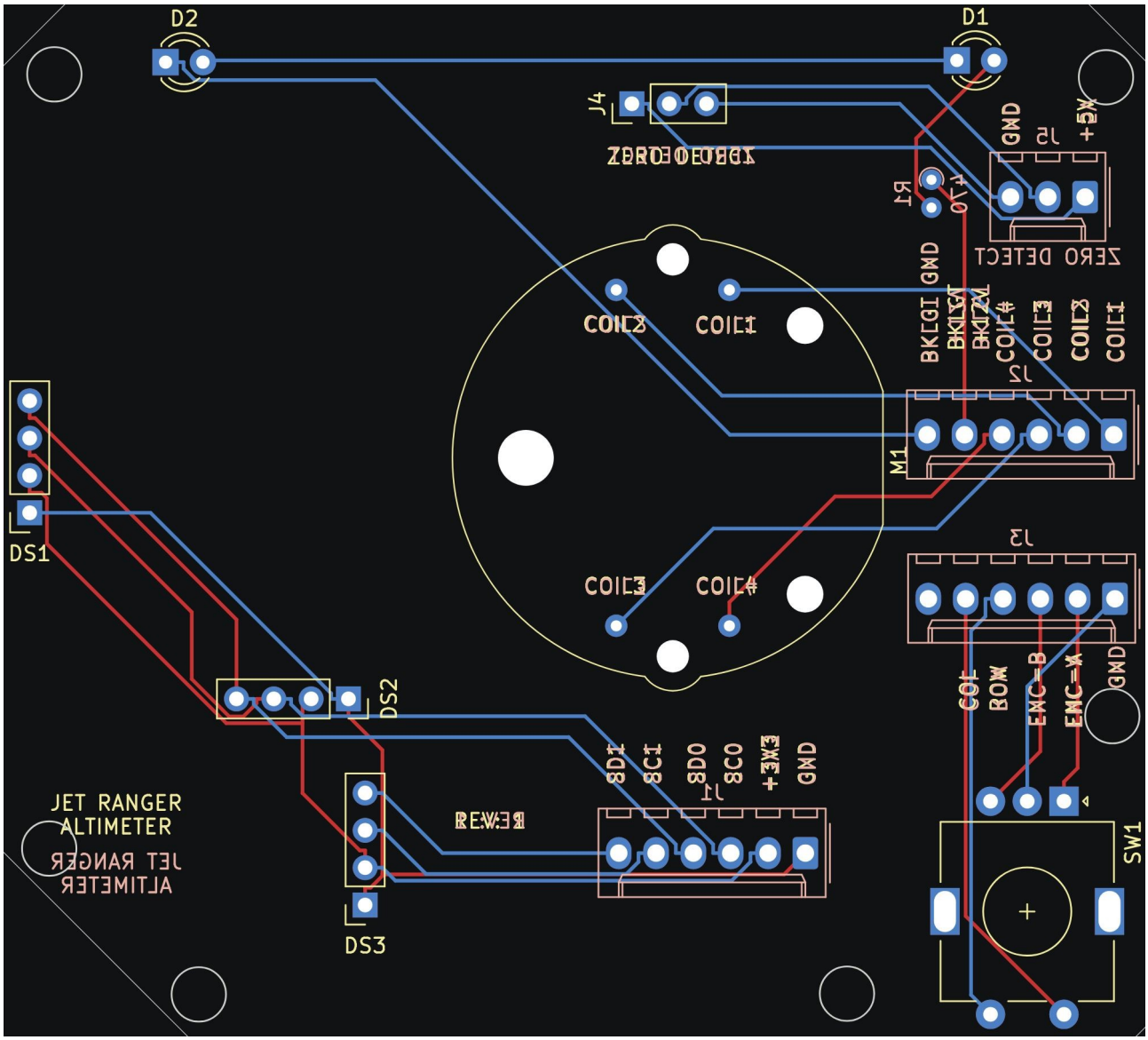


Figure 71 — Jet Ranger Altimeter board layout

E.13 Large Guage (Simmeters Large Guage Replacement)

Title block revision 2 (2022-08-04). Not covered by the sketches. Replacement board for a large Simmeters gauge.

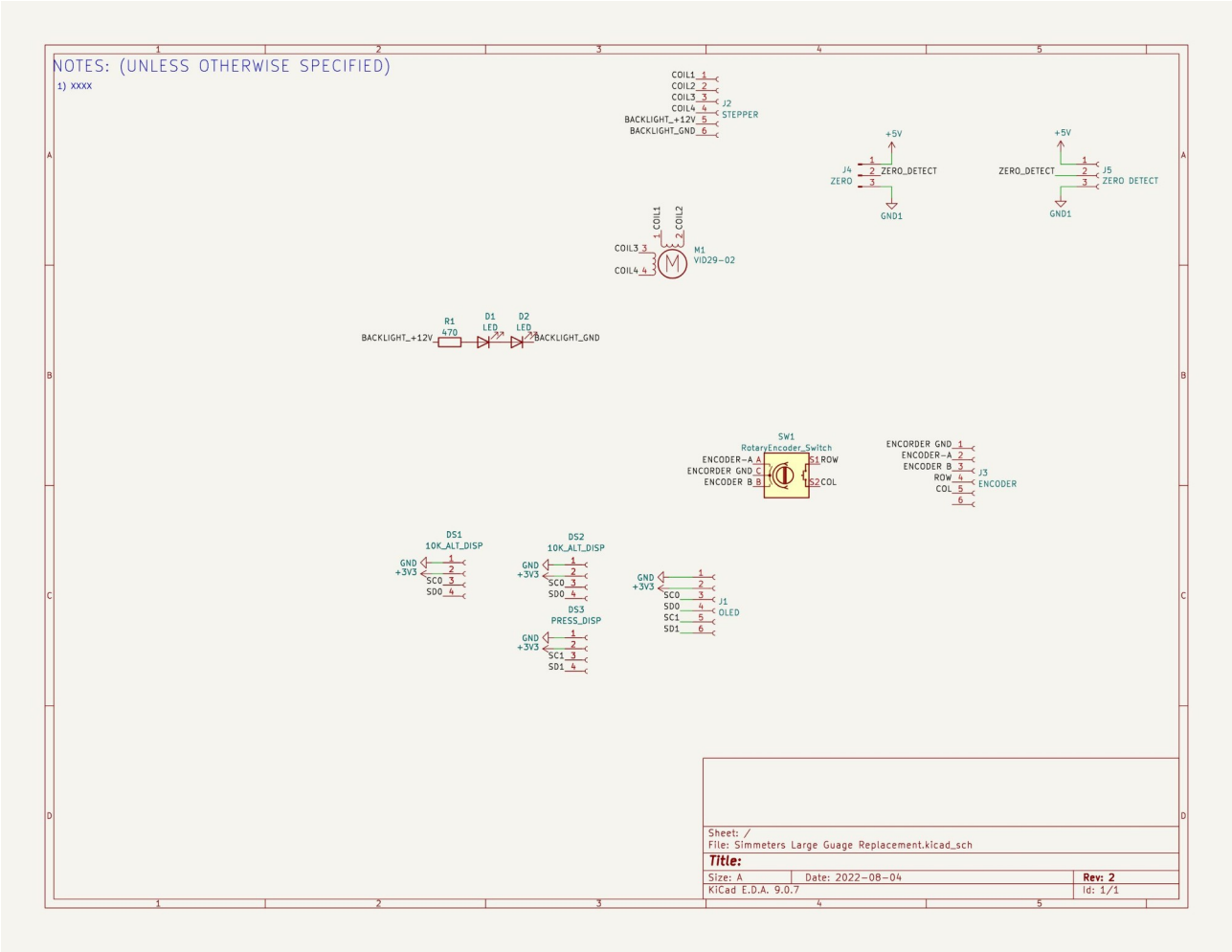


Figure 72 — Large Guage (Simmeters Large Guage Replacement) schematic

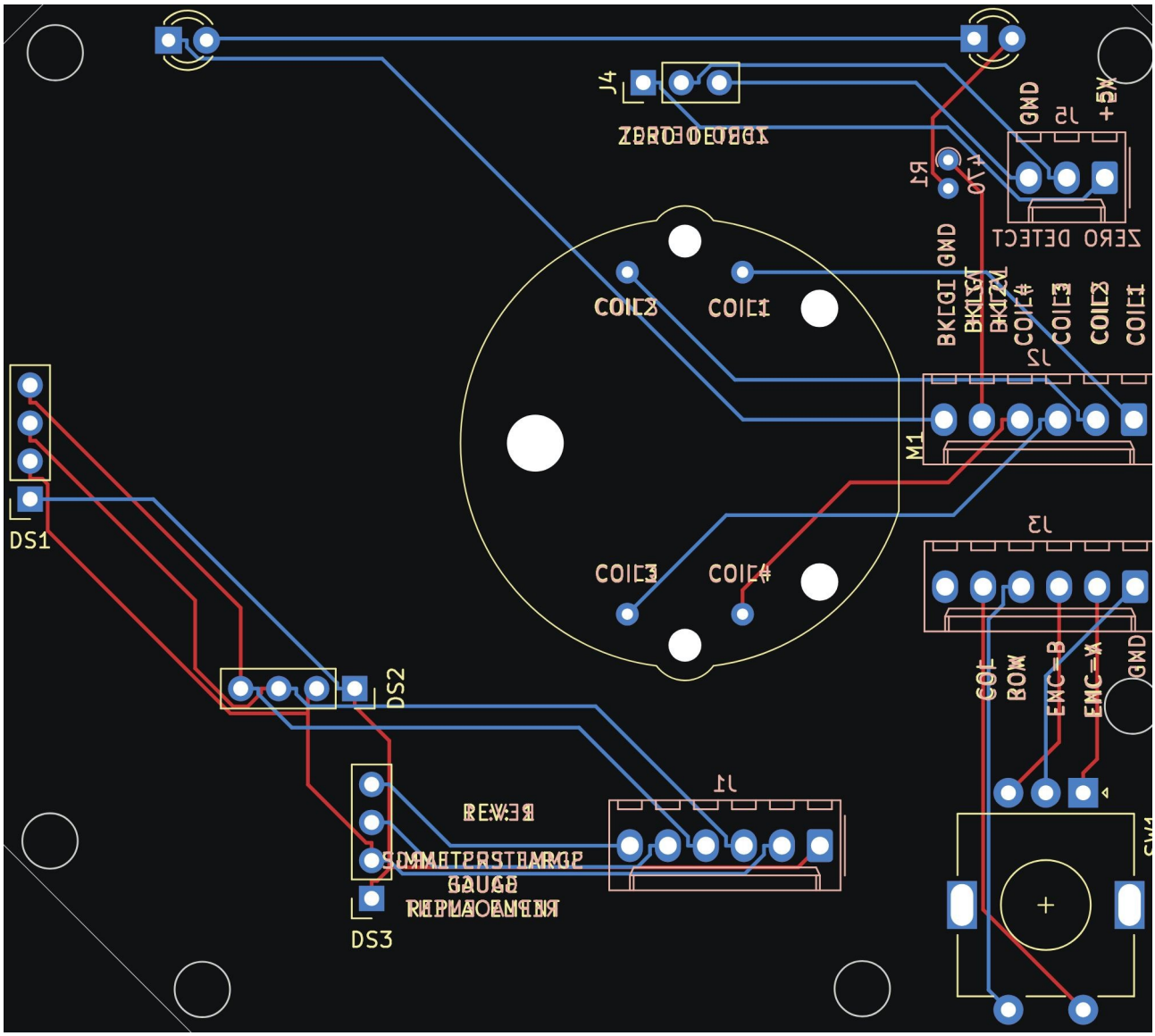


Figure 73 — Large Guage (Simmeters Large Guage Replacement) board layout

E.14 Simkits Large Dual Gauge

Title block revision 2 (2022-08-04). Not covered by the sketches. Replacement board for a large dual Simkits gauge.

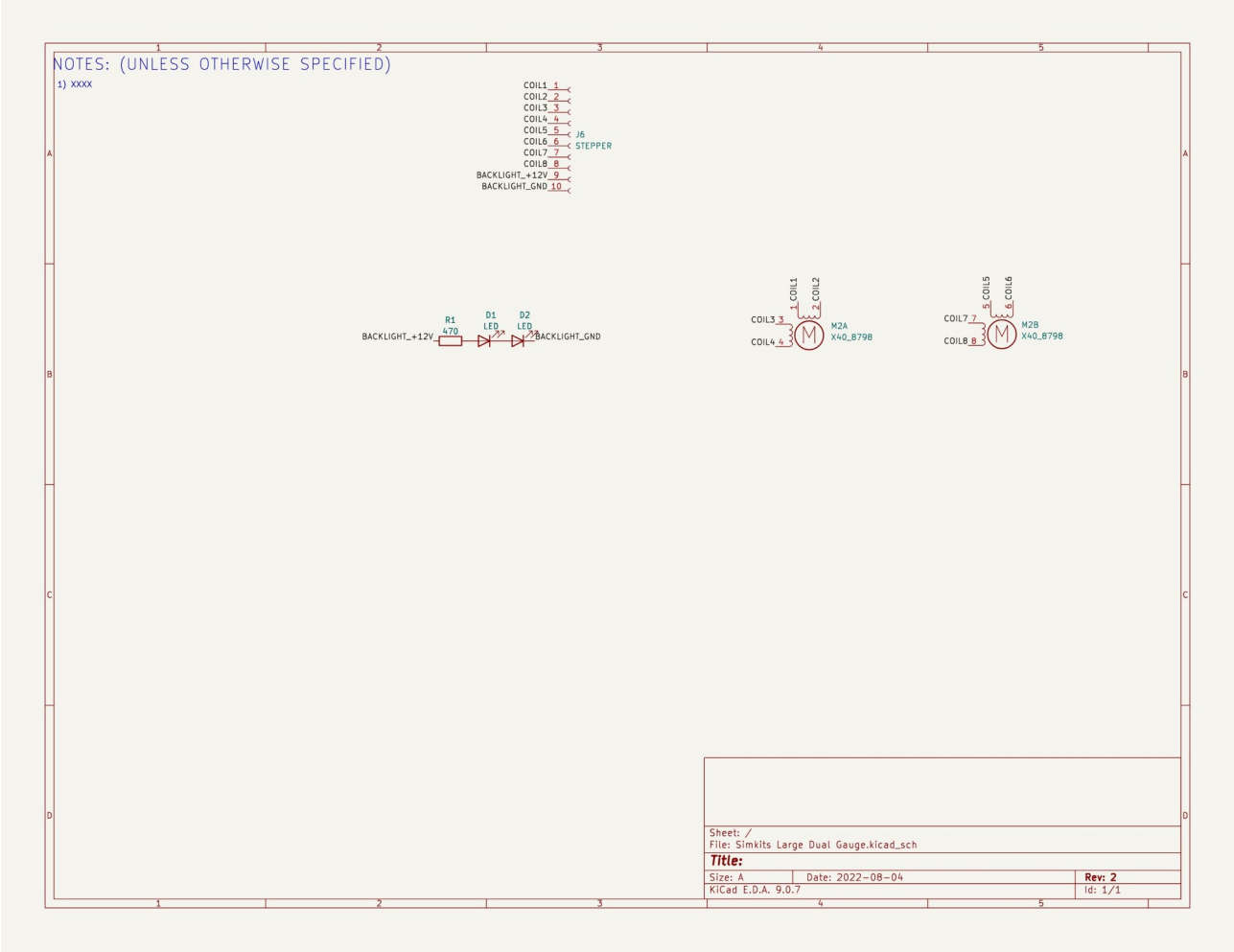


Figure 74 — Simkits Large Dual Gauge schematic

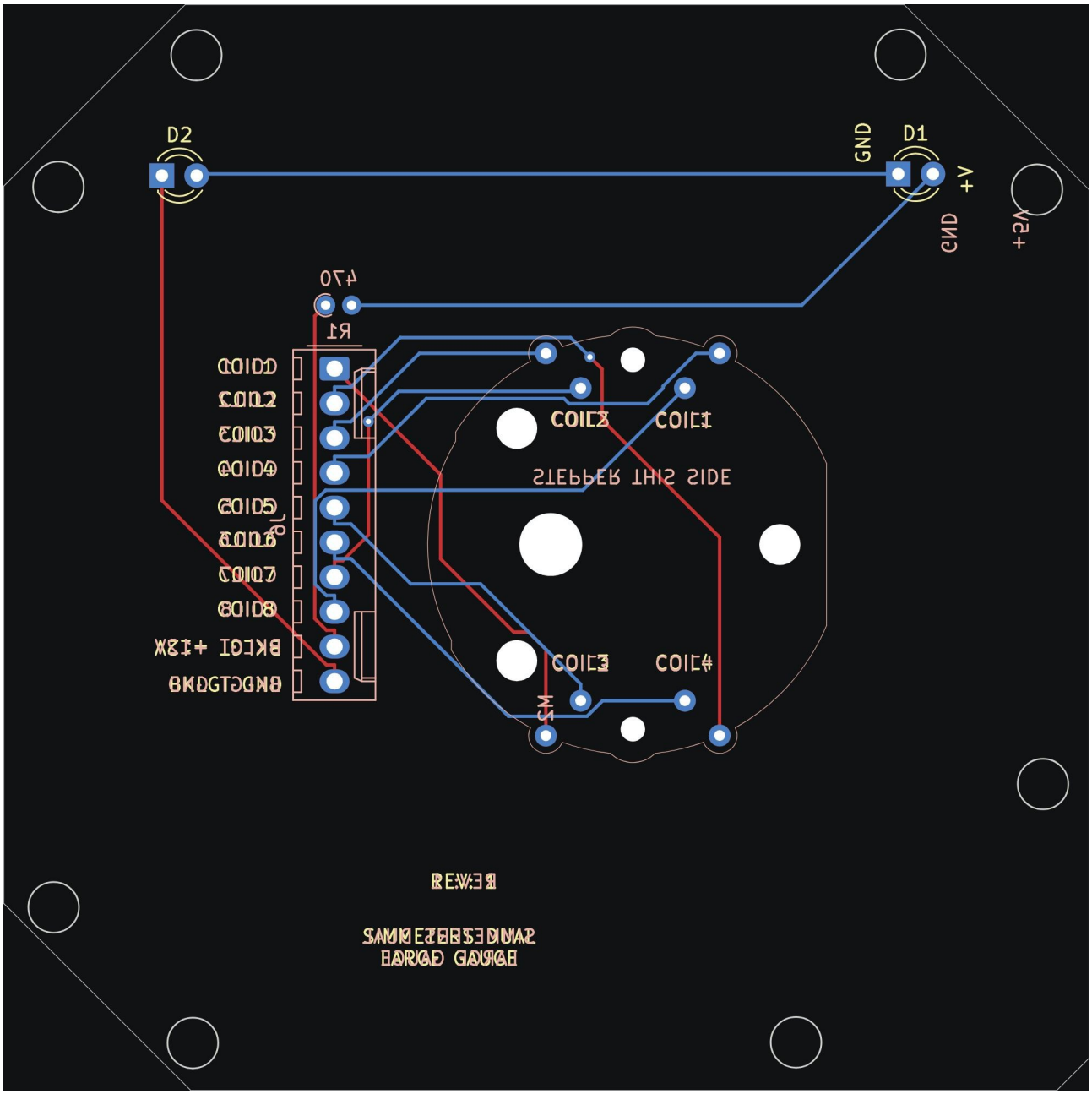


Figure 75 — Simkits Large Dual Gauge board layout

E.15 Simkits Large Guage

Title block revision 2 (2022-08-04). Not covered by the sketches. Replacement board for a large Simkits gauge.

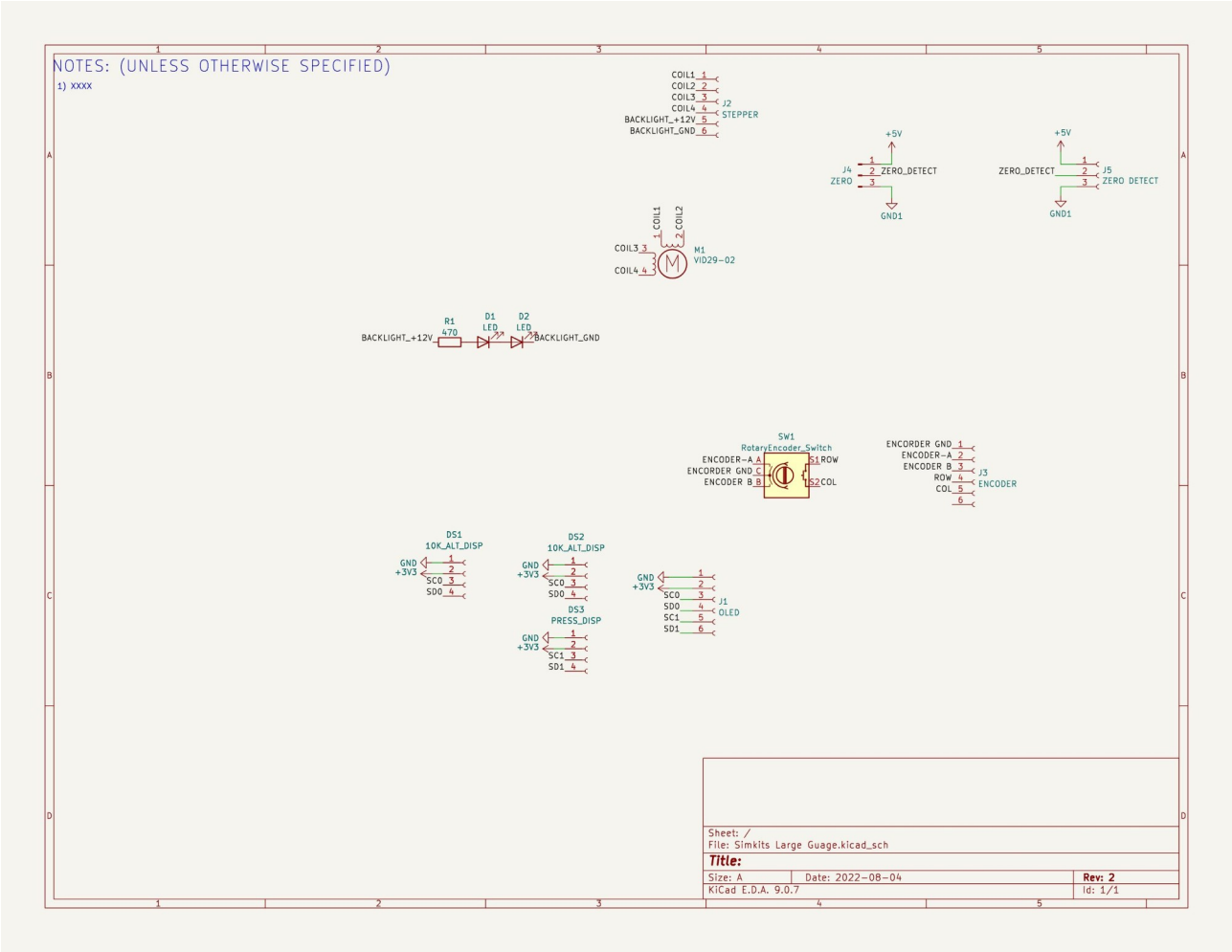


Figure 76 — Simkits Large Guage schematic

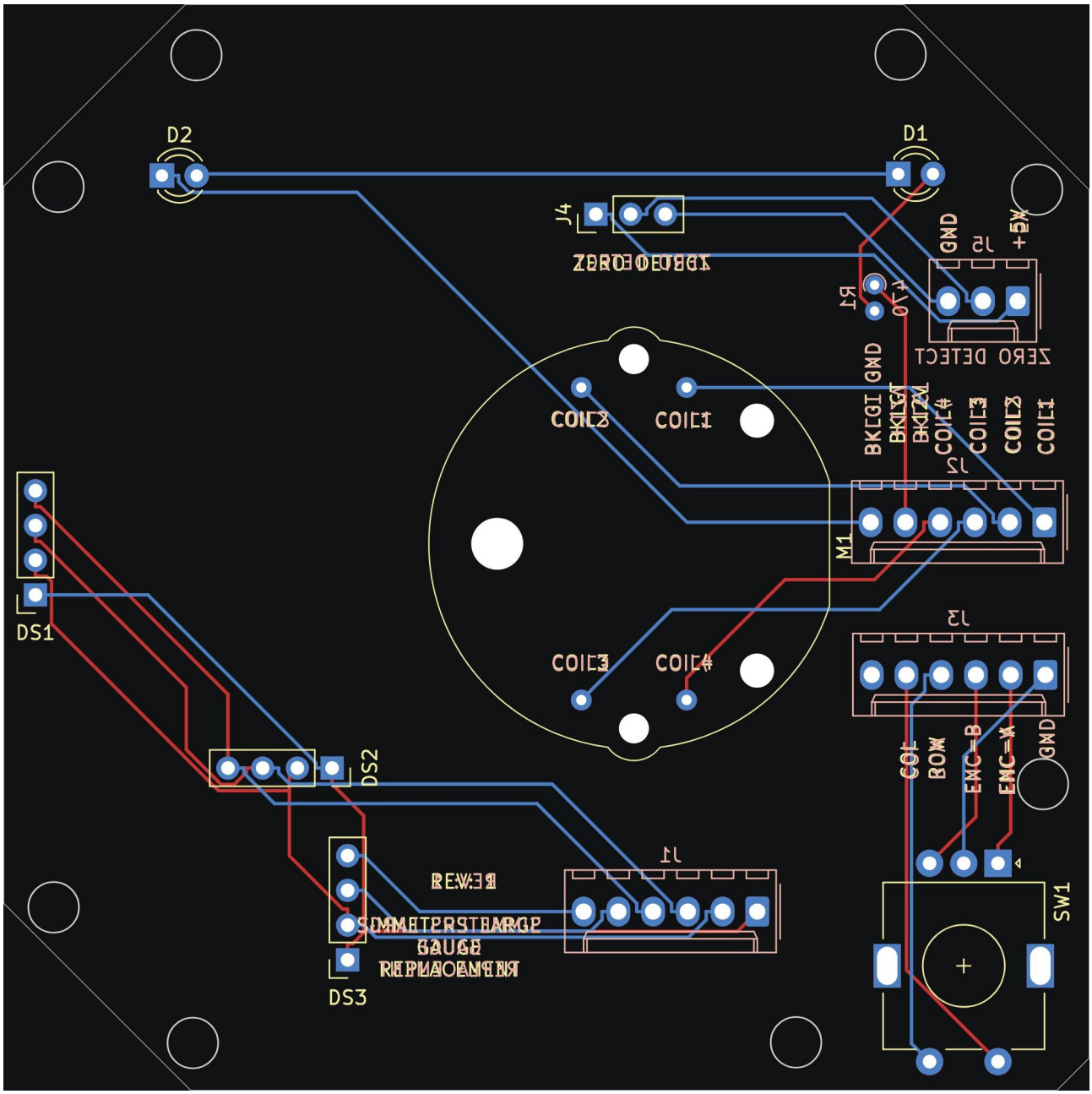


Figure 77 — Simkits Large Guage board layout

E.16 Simkits Small Dual Gauge

Title block revision 2 (2022-08-04). Not covered by the sketches. Replacement board for a small dual Simkits gauge.

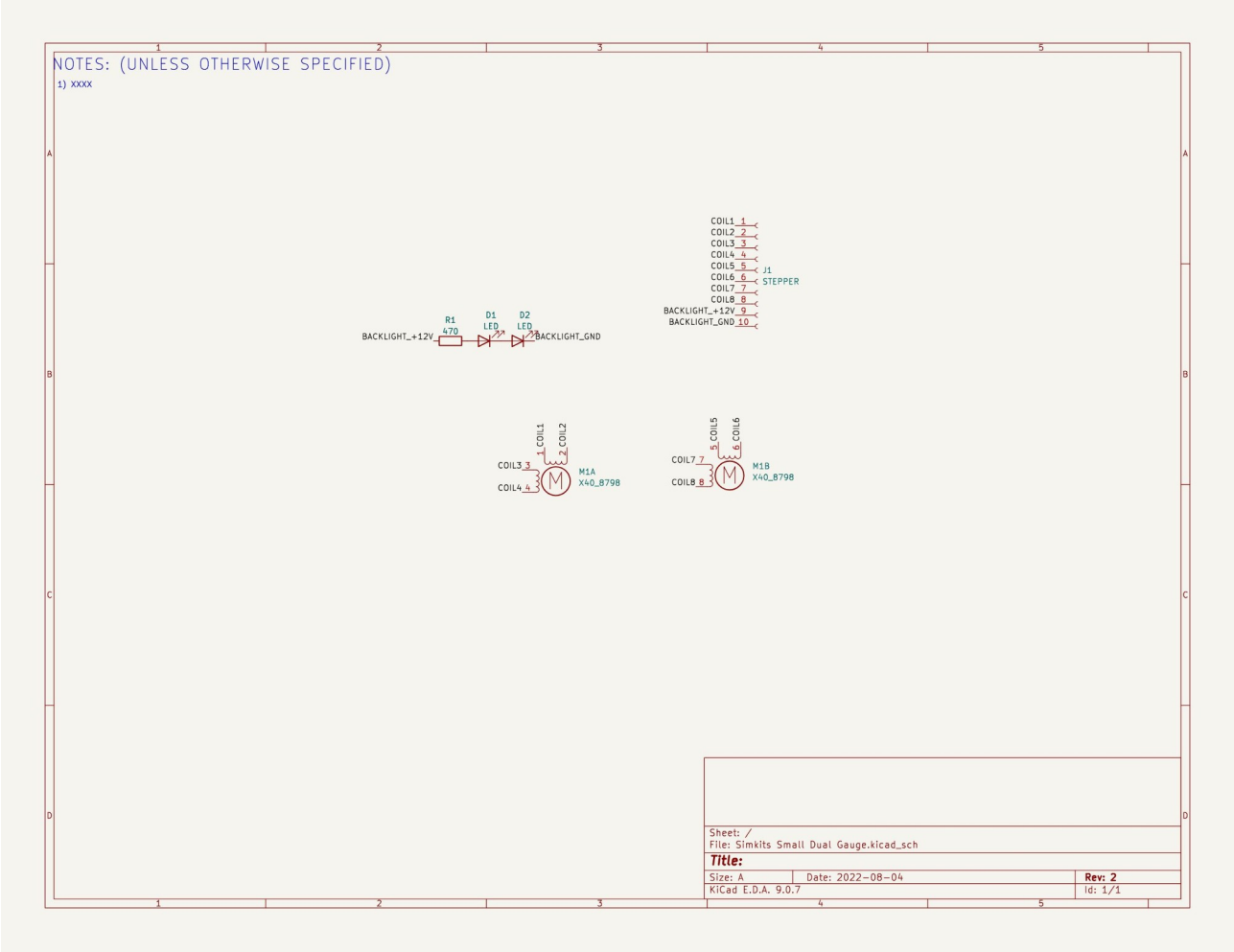


Figure 78 — Simkits Small Dual Gauge schematic

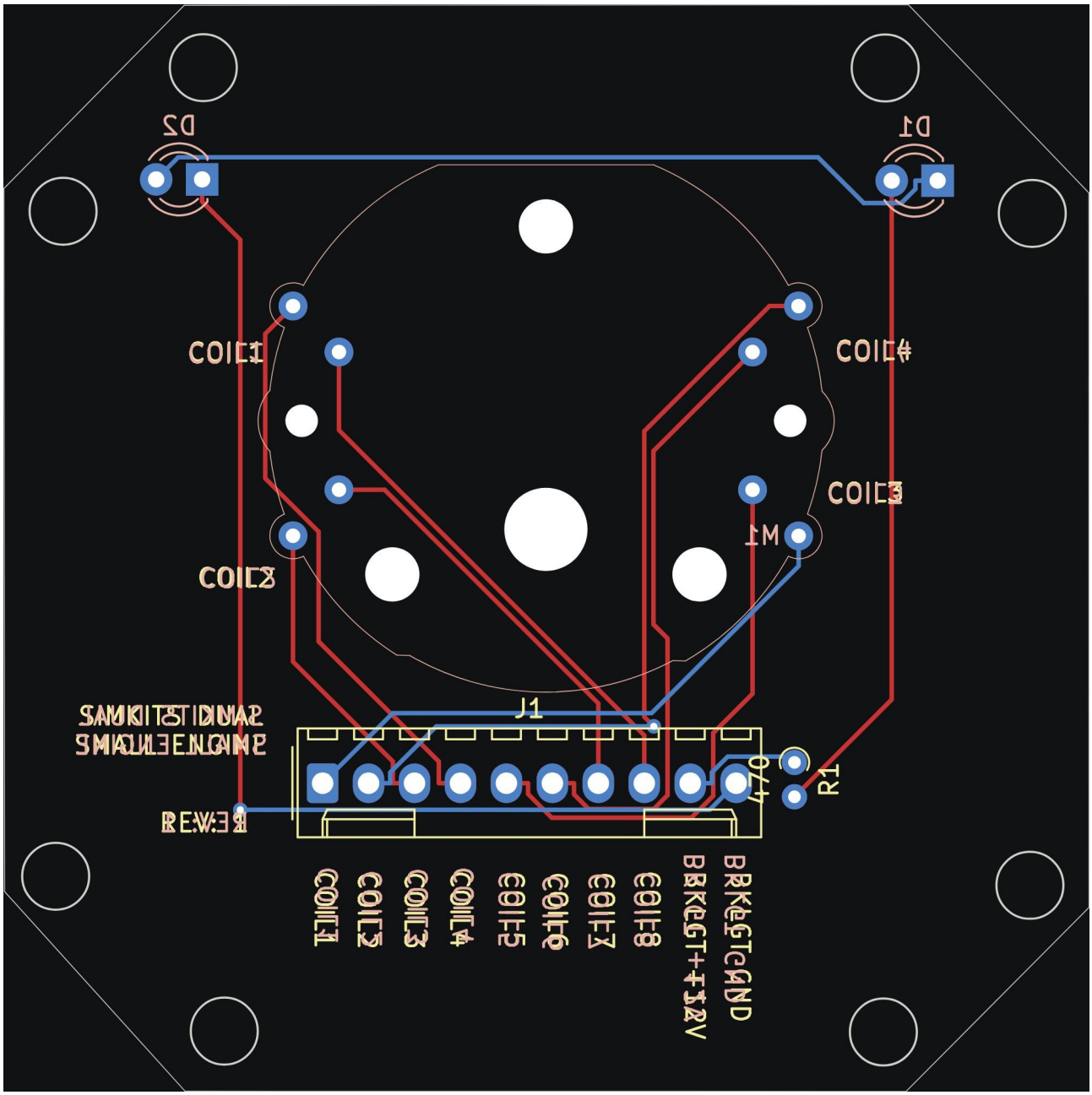


Figure 79 — Simkits Small Dual Gauge board layout

E.17 Simkits Small Guage

Title block revision 2 (2022-08-04). Not covered by the sketches. Replacement board for a small Simkits gauge.

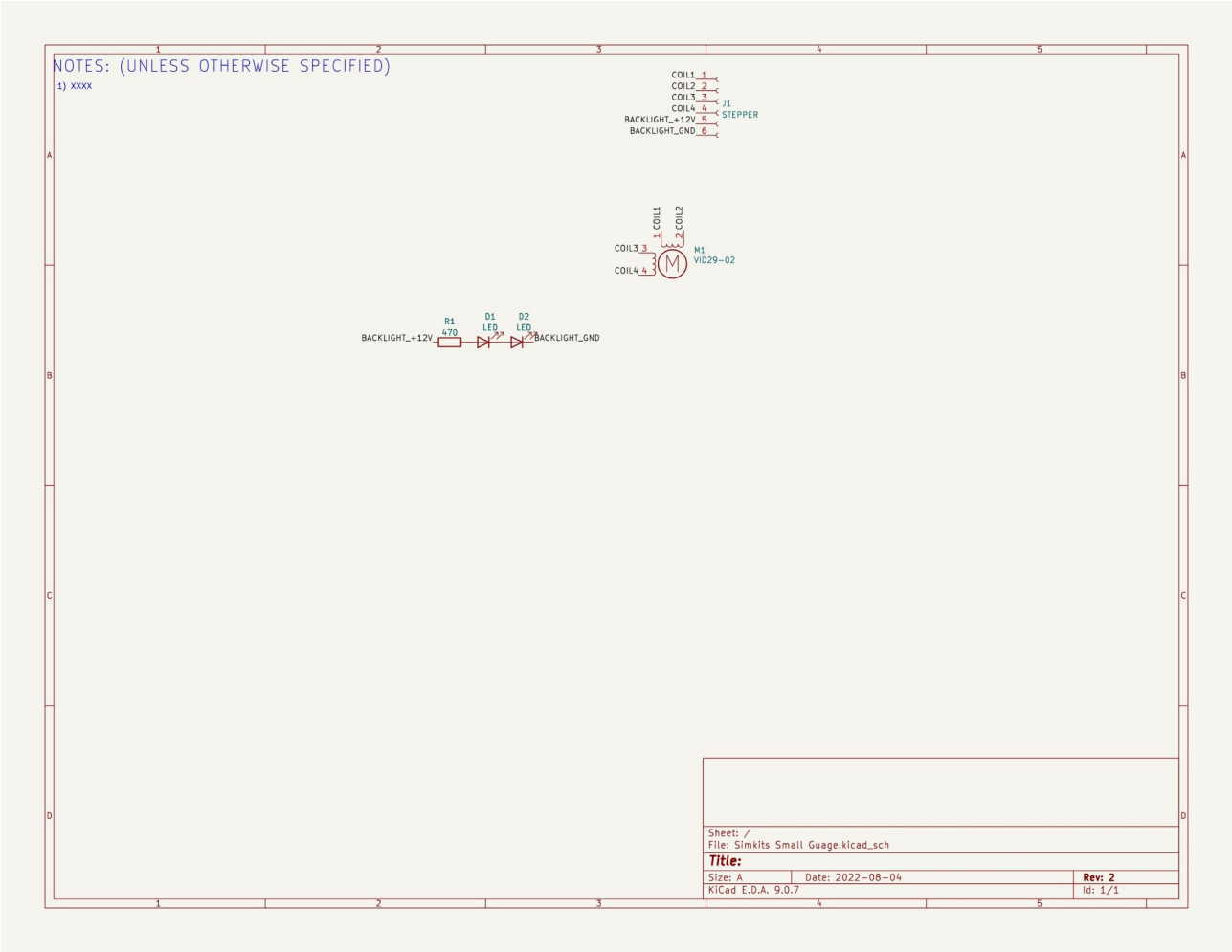


Figure 80 — Simkits Small Guage schematic

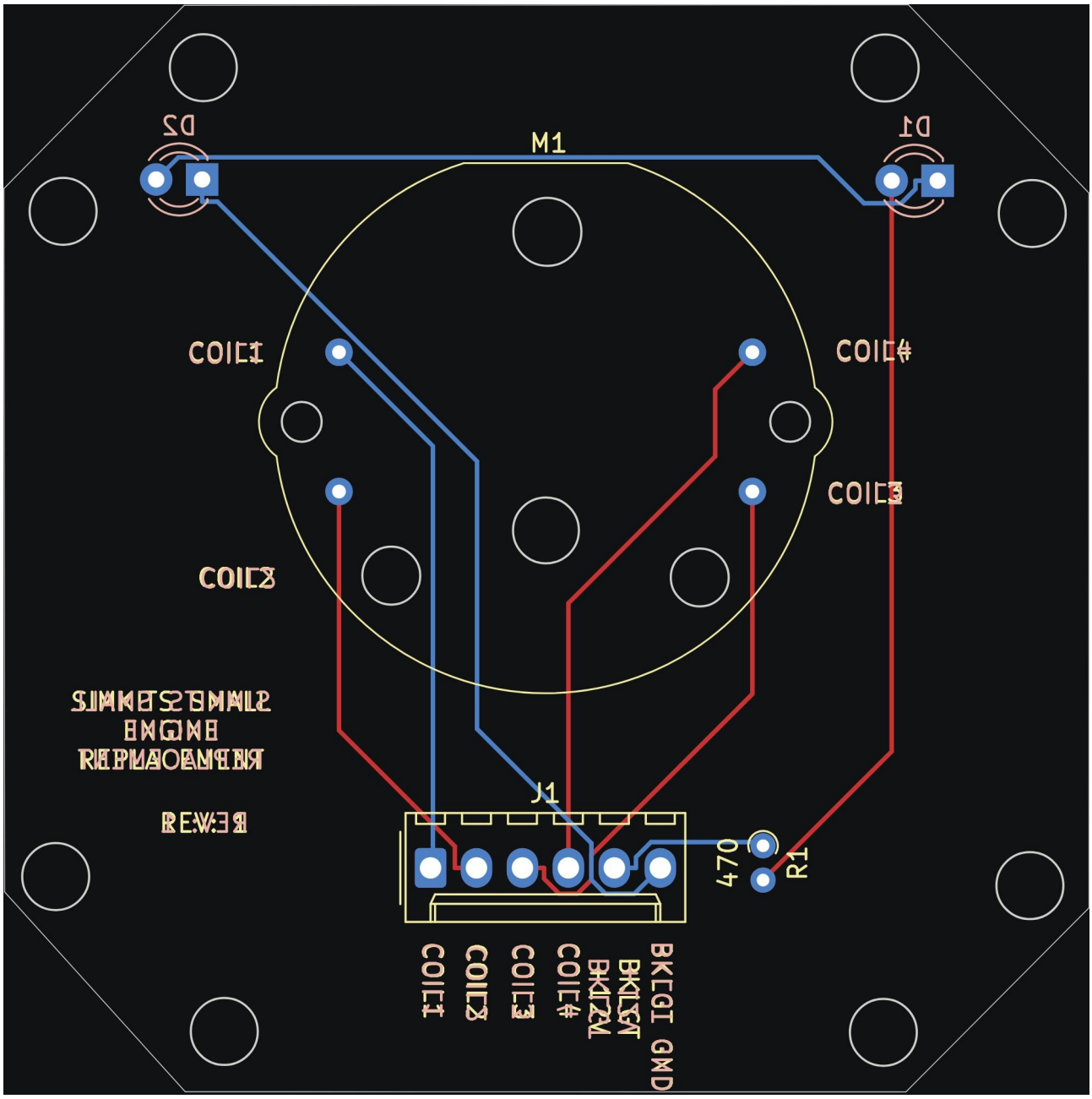


Figure 81 — Simkits Small Guage board layout

Appendix F: Installing the controller PCBs

These pictures are an aid for installing the controller boards. Each shows the **top (component) side** of a controller PCB as it is seen when the board is fitted, with only the board outline, the mounting holes, the pads and silkscreen outlines of the **external connectors**, the unlabelled round pads in rows and a double column around the edge (the Arduino Mega headers) and the outline of the Ethernet module. Every external connector is labelled with its reference (J5, SW2) and, where there is room, its name. **Pin 1 of each connector is the square pad**; on a ribbon-cable (IDC) header the small triangle also marks pin 1. Nothing else is printed, so the pictures are easy to read at the bench. They are drawn from the KiCad layout files by `export_install_views.py`, so they show the boards as they were on the day the document was built.

F.1 Jet Ranger Radio Auto Pilot

Revision 1.4 (31 October 2026, with the Huey Warning Panel connectors J20 to J23), for the Radio controller (172.16.1.101). Connectors with the name left off in the picture (a row of gauge sockets, for example) are listed in the table.

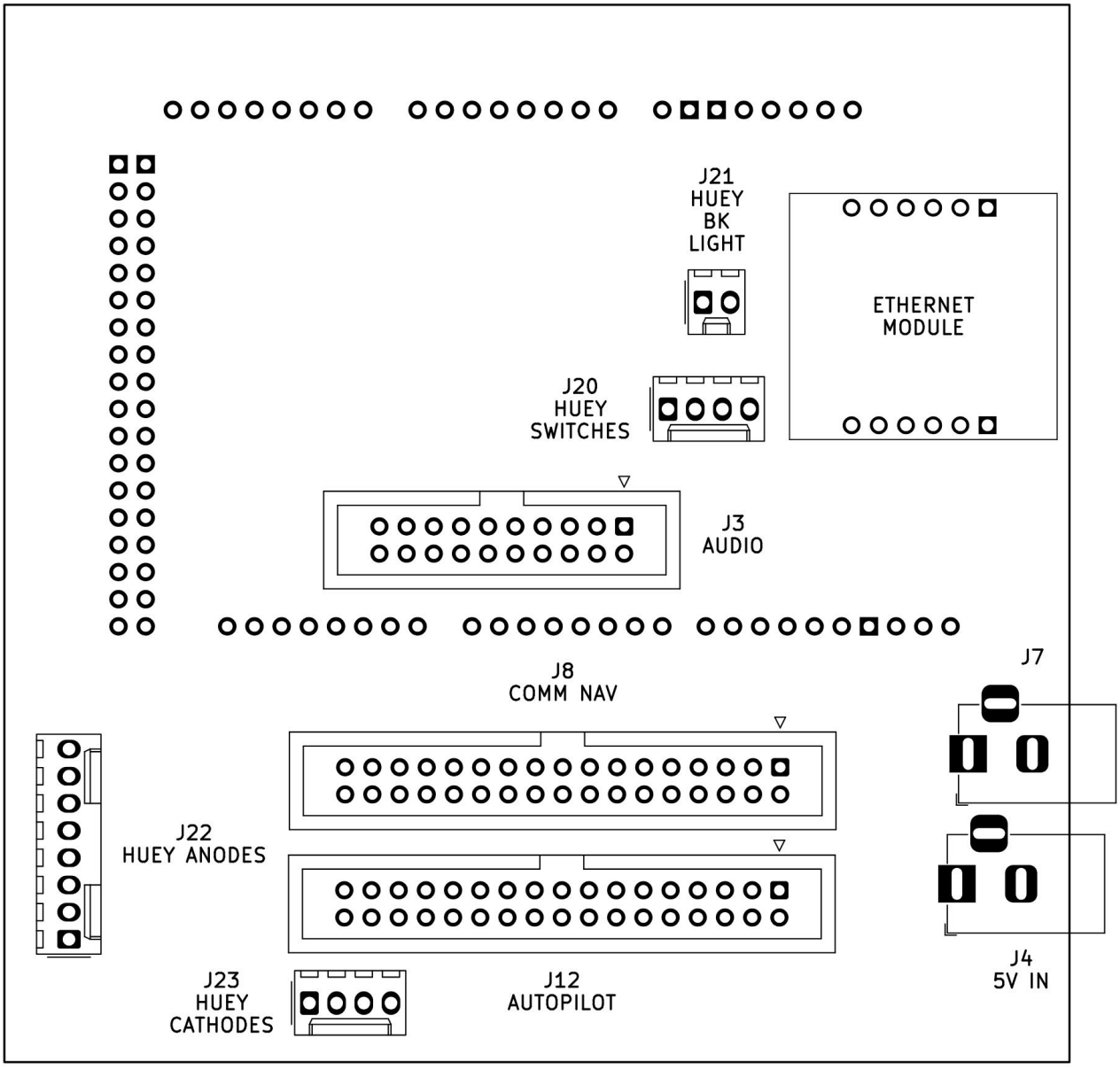


Figure 82 — Jet Ranger Radio Auto Pilot, top side with the external connectors labelled

Label	Name	Type	Pins	Notes
J3	AUDIO	IDC header	20 (2 x 10)	Audio panel
J4	5V IN	DC jack	3	5 V supply input (DC jack)
J7	STEPPER IN 12-20VDC	DC jack	3	12 to 20 V supply input (DC jack)
J8	COMM NAV	IDC header	34 (2 x 17)	COM and NAV panel
J12	AUTOPILOT	IDC header	34 (2 x 17)	Auto Pilot panel
J20	HUEY SWITCHES	Molex KK-254	4	Huey Warning Panel switches: GND, brightness / dim (A8), test (A9), reset (A10)
J21	HUEY BK LIGHT	Molex KK-254	2	Huey Warning Panel backlight: +12 V and the switched backlight
J22	HUEY ANODES	Molex KK-254	8	Huey Warning Panel LED anodes ANNODE_00 to ANNODE_07
J23	HUEY CATHODES	Molex KK-254	4	Huey Warning Panel LED cathodes CATHODE_04 to CATHODE_07 (MAX7219 digits 4 to 7)

F.2 Jet Ranger Front Panel Servo

Revision 1.4, for the Servo controller (172.16.1.102). Connectors with the name left off in the picture (a row of gauge sockets, for example) are listed in the table.

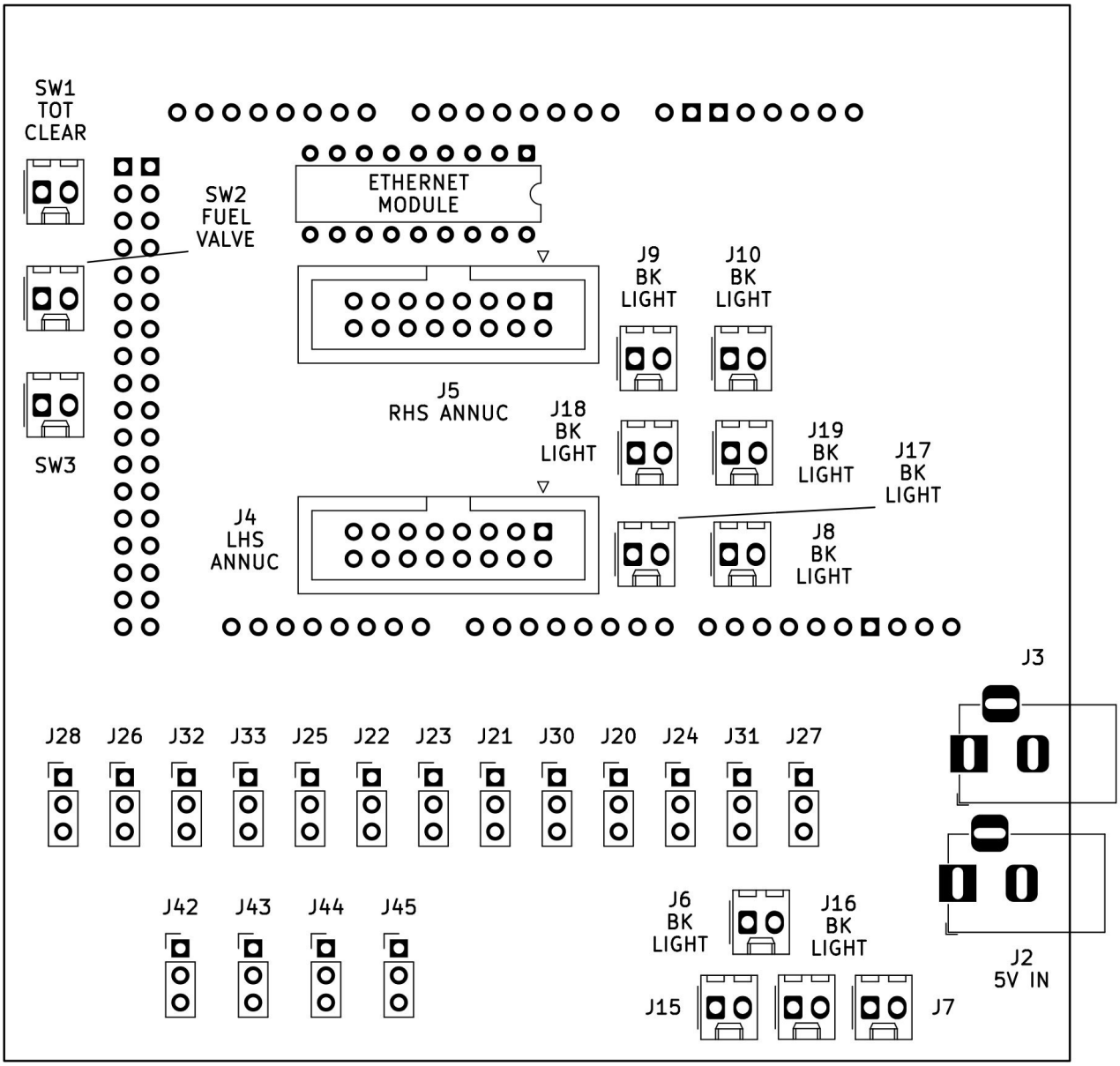


Figure 83 — Jet Ranger Front Panel Servo, top side with the external connectors labelled

Label	Name	Type	Pins	Notes
J2	5V IN	DC jack	3	5 V supply input (DC jack)
J3	STEPPER IN 12-20VDC	DC jack	3	12 to 20 V supply input (DC jack)
J4	LHS ANNUC	IDC header	16 (2 x 8)	Left hand annunciator panel
J5	RHS ANNUC	IDC header	16 (2 x 8)	Right hand annunciator panel
J6	BK LIGHT	Molex KK-254	2	
J7	BK LIGHT	Molex KK-254	2	
J8	BK LIGHT	Molex KK-254	2	
J9	BK LIGHT	Molex KK-254	2	
J10	BK LIGHT	Molex KK-254	2	

Label	Name	Type	Pins	Notes
J15	BK LIGHT	Molex KK-254	2	
J16	BK LIGHT	Molex KK-254	2	
J17	BK LIGHT	Molex KK-254	2	
J18	BK LIGHT	Molex KK-254	2	
J19	BK LIGHT	Molex KK-254	2	
J20	AIRSPEED	Pin header	3	
J21	EGT	Pin header	3	
J22	TURBINE SPEED	Pin header	3	
J23	RADAR ALT	Pin header	3	
J24	ENG TORQUE	Pin header	3	
J25	ROTOR SPEED	Pin header	3	
J26	VVI	Pin header	3	
J27	ENG OIL TEMP	Pin header	3	
J28	FUEL LEVEL	Pin header	3	
J30	GAS PRODUCER	Pin header	3	
J31	ENG OIL PRES	Pin header	3	
J32	XMSN OIL TEMP	Pin header	3	
J33	XMSN OIL PRESS	Pin header	3	
J42	PITCH	Pin header	3	Attitude indicator pitch servo
J43	ROLL	Pin header	3	Attitude indicator bank (roll) servo
J44	ELEC LOAD	Pin header	3	
J45	FUEL LOAD	Pin header	3	
SW1	TOT CLEAR	Molex KK-254	2	TOT clear switch (D43), to ground when pressed
SW2	FUEL VALVE	Molex KK-254	2	Fuel valve switch (D48), to ground when on
SW3	WARNING MUTE	Molex KK-254	2	Warning mute switch (D47), to ground when pressed

F.3 Jet Ranger 13 Stepper

Revision 1.6, for the Stepper controller (172.16.1.105). Connectors with the name left off in the picture (a row of gauge sockets, for example) are listed in the table.

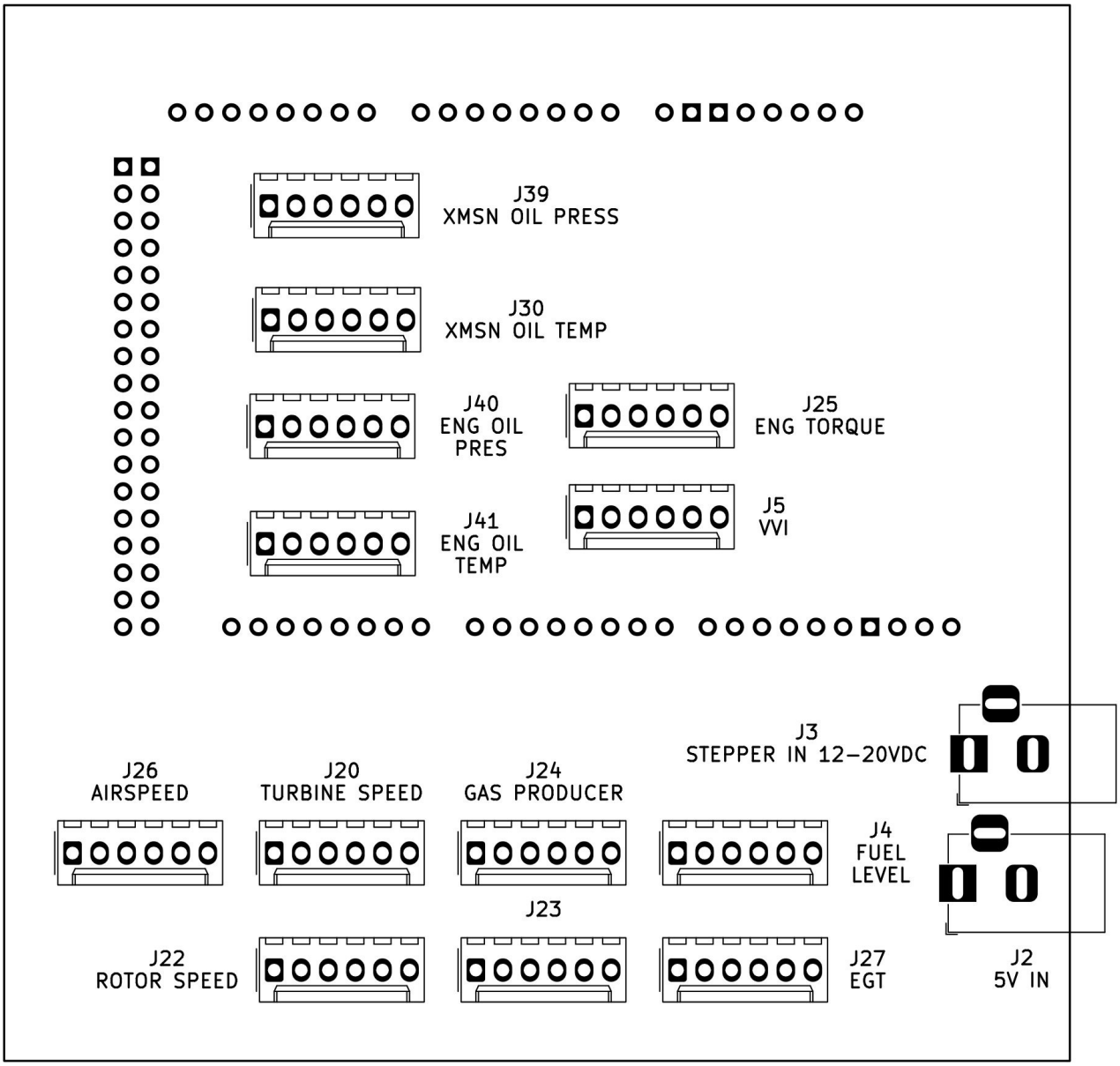


Figure 84 — Jet Ranger 13 Stepper, top side with the external connectors labelled

Label	Name	Type	Pins	Notes
J2	5V IN	DC jack	3	5 V supply input (DC jack)
J3	STEPPER IN 12-20VDC	DC jack	3	12 to 20 V supply input (DC jack)
J4	FUEL LEVEL	Molex KK-254	6	
J5	VVI	Molex KK-254	6	
J20	TURBINE SPEED	Molex KK-254	6	
J22	ROTOR SPEED	Molex KK-254	6	
J23	RADAR ALT	Molex KK-254	6	
J24	GAS PRODUCER	Molex KK-254	6	
J25	ENG TORQUE	Molex KK-254	6	
J26	AIRSPEED	Molex KK-254	6	

Label	Name	Type	Pins	Notes
J27	EGT	Molex KK-254	6	
J30	XMSN OIL TEMP	Molex KK-254	6	
J39	XMSN OIL PRESS	Molex KK-254	6	
J40	ENG OIL PRES	Molex KK-254	6	
J41	ENG OIL TEMP	Molex KK-254	6	

F.4 Jet Ranger OLED Dual Stepper

Revision 1.5, for the Dual Stepper controller (172.16.1.106). Connectors with the name left off in the picture (a row of gauge sockets, for example) are listed in the table.

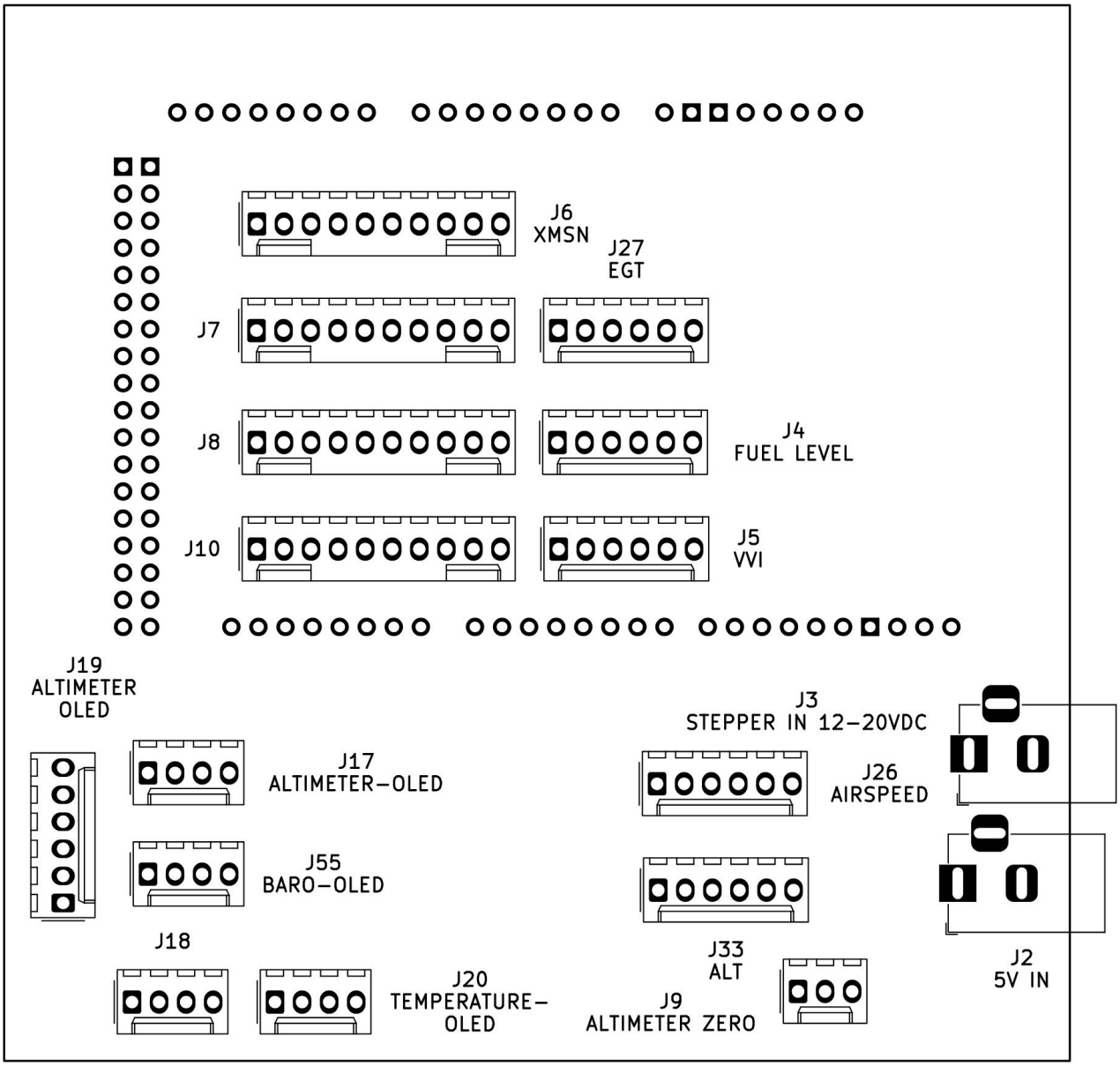


Figure 85 — Jet Ranger OLED Dual Stepper, top side with the external connectors labelled

Label	Name	Type	Pins	Notes
J2	5V IN	DC jack	3	5 V supply input (DC jack)
J3	STEPPER IN 12-20VDC	DC jack	3	12 to 20 V supply input (DC jack)
J4	FUEL LEVEL	Molex KK-254	6	
J5	VVI	Molex KK-254	6	
J6	XMSN	Molex KK-254	10	
J7	ROTOR ENG SPEED	Molex KK-254	10	
J8	ENGINE	Molex KK-254	10	
J9	ALTIMETER ZERO	Molex KK-254	3	Altimeter zero sensor
J10	ELEC FUEL LOAD	Molex KK-254	10	
J17	ALTIMETER-OLED	Molex KK-254	4	Altimeter OLED

Label	Name	Type	Pins	Notes
J18	CLOCK-OLED	Molex KK-254	4	Clock OLED
J19	ALTIMETER OLED	Molex KK-254	6	
J20	TEMPERATURE-OLED	Molex KK-254	4	Temperature OLED
J26	AIRSPEED	Molex KK-254	6	
J27	EGT	Molex KK-254	6	
J33	ALT	Molex KK-254	6	
J55	BARO-OLED	Molex KK-254	4	Barometer OLED

F.5 Jet Ranger Upper ADF Controller

Revision 1.0, for the ADF and Upper controller (172.16.1.107). Connectors with the name left off in the picture (a row of gauge sockets, for example) are listed in the table.

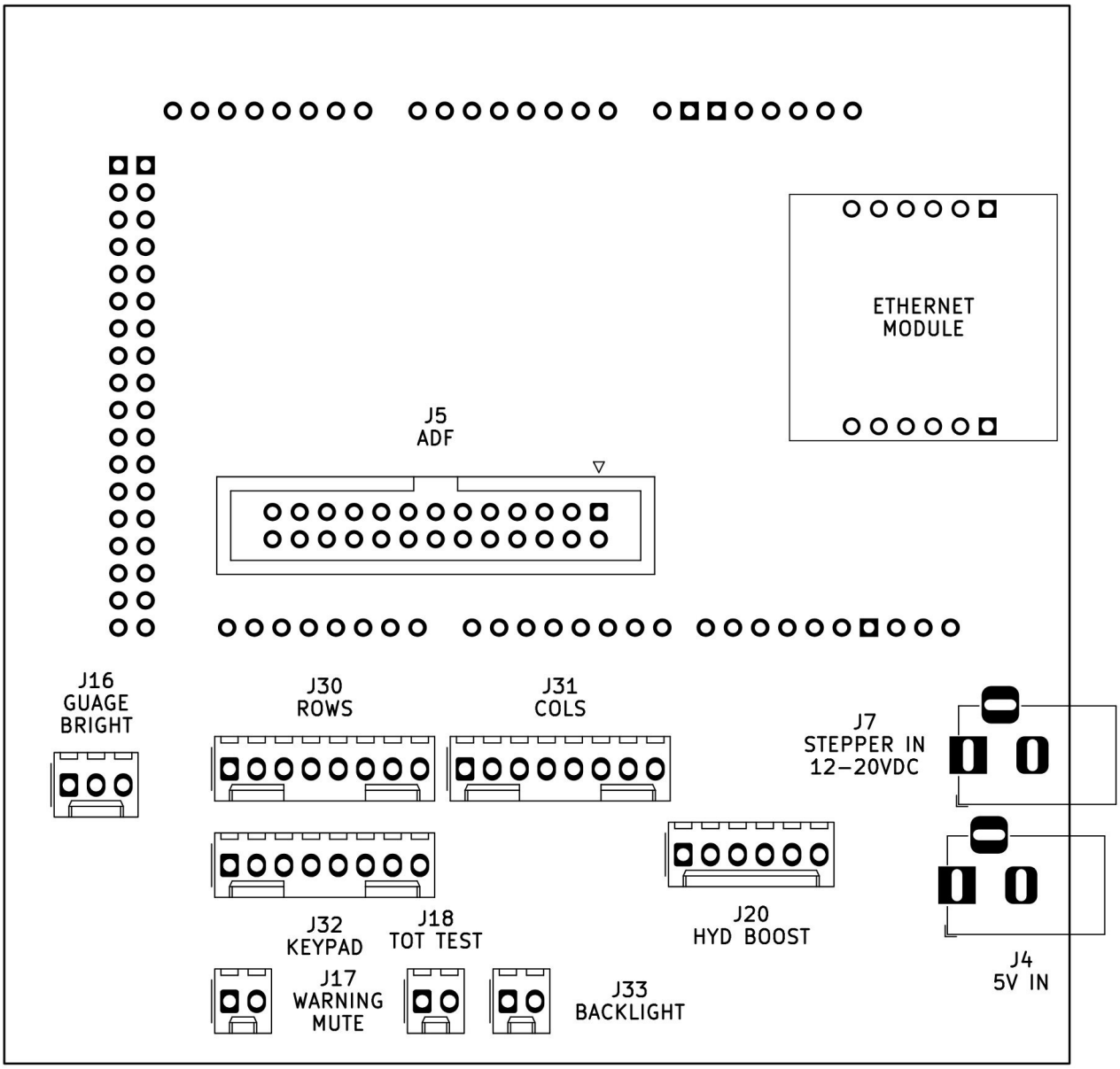


Figure 86 — Jet Ranger Upper ADF Controller, top side with the external connectors labelled

Label	Name	Type	Pins	Notes
J4	5V IN	DC jack	3	5 V supply input (DC jack)
J5	ADF	IDC header	26 (2 x 13)	ADF panel (knobs, keypad, switches, lamps)
J7	STEPPER IN 12-20VDC	DC jack	3	12 to 20 V supply input (DC jack)
J16	GUAGE BRIGHT	Molex KK-254	3	Panel brightness pot
J17	WARNING MUTE	Molex KK-254	2	Warning mute switch (matrix ROW8)
J18	TOT TEST	Molex KK-254	2	TOT test switch (matrix ROW8)
J20	HYD BOOST	Molex KK-254	6	Hyd boost, anti-ice and caution dim switches
J30	ROWS	Molex KK-254	8	Upper switch matrix rows ROW0 to ROW7
J31	COLS	Molex KK-254	8	Upper switch matrix columns COL0 to COL7
J32	KEYPAD	Molex KK-254	8	Upper numeric keypad

Label	Name	Type	Pins	Notes
J33	BACKLIGHT	Molex KK-254	2	Panel backlight

Appendix G: Instrument map

This appendix draws every gauge, lamp and display of the cockpit as the real thing (dial gauges with needles, the attitude indicator, annunciator and legend lamps, gear lights, OLED displays) and joins each one by a cable to the connector on the controller board that drives it. The connector references and names are the ones printed on the PCBs (see the installation views in Appendix F). Needles and lamps are drawn at illustrative readings and the lens colours are illustrative. Switches, knobs, keypads and the USB HID board are inputs and are not drawn. The full-size drawing is on one A3 landscape page, `Jet Ranger Instrument Map A3.pdf` (also an SVG and a PNG) in the documentation folder; it is made by `DocBuild/make_instrument_map.py`.

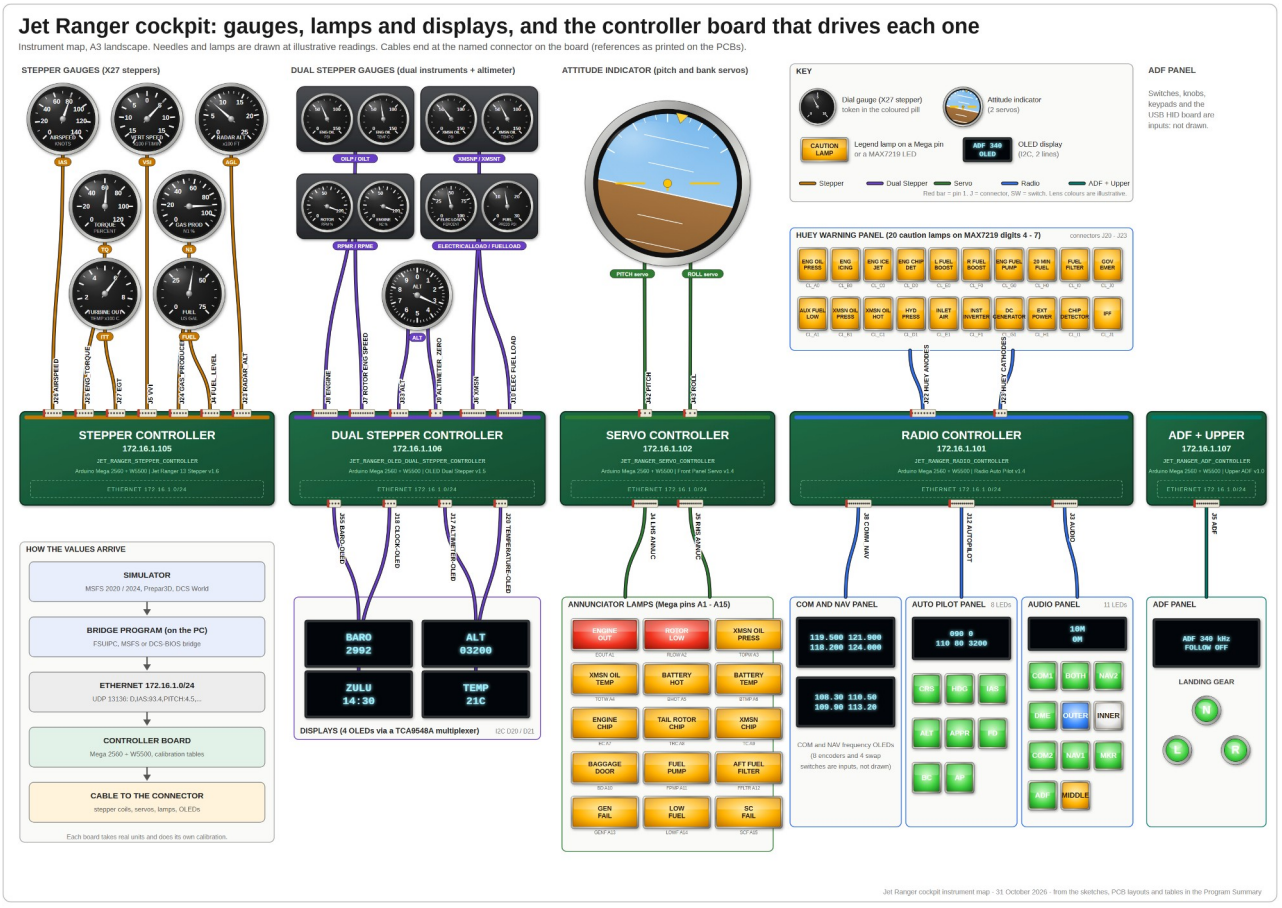


Figure 87 — The whole map (the A3 original is Jet Ranger Instrument Map A3.pdf; zoom in the PDF to read it)

G.1 Stepper gauges and the Stepper controller

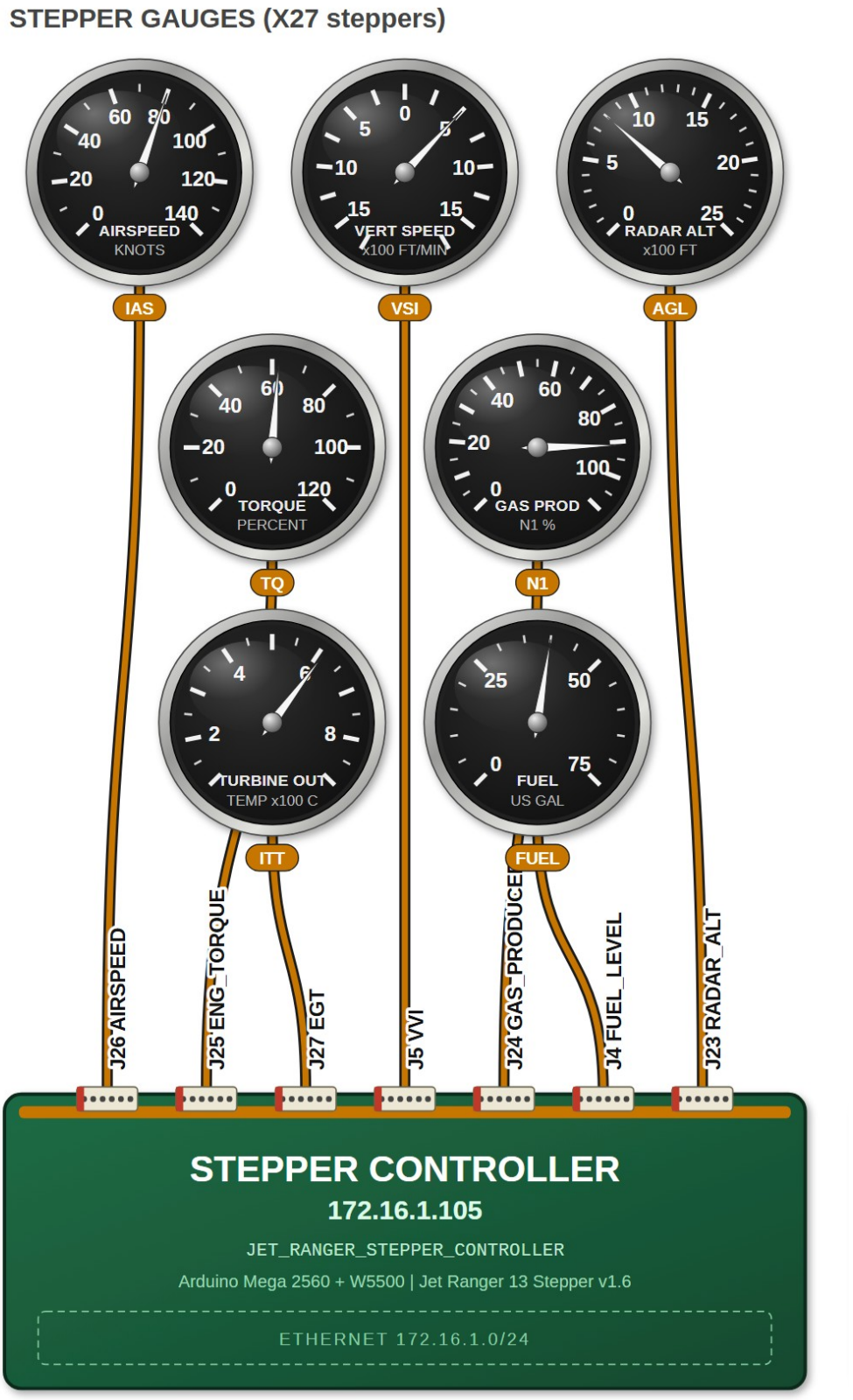


Figure 88 — Stepper gauges and the Stepper controller

G.2 Dual Stepper gauges and the Dual Stepper controller

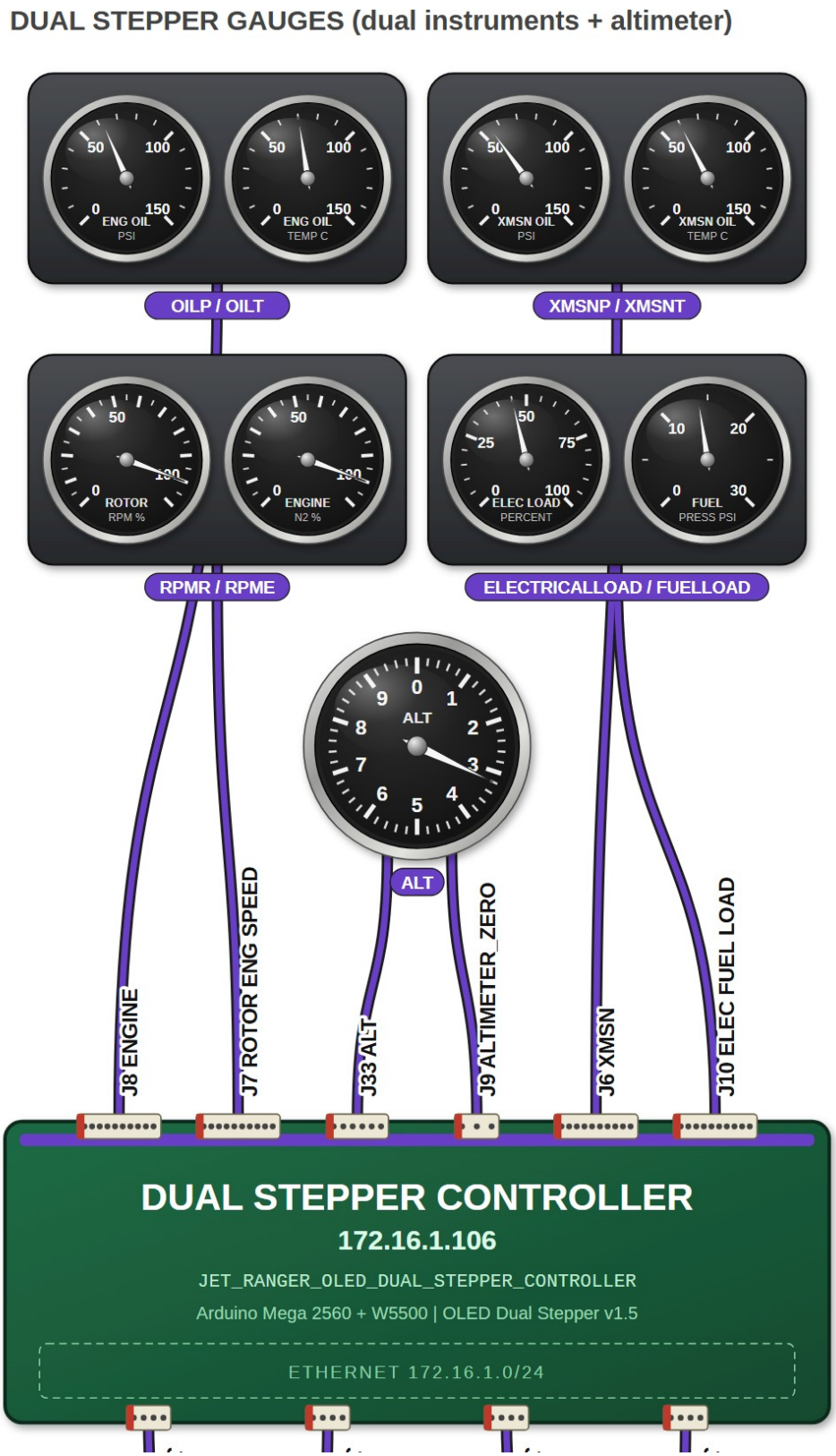


Figure 89 — Dual Stepper gauges and the Dual Stepper controller

G.3 Attitude indicator and the Servo controller

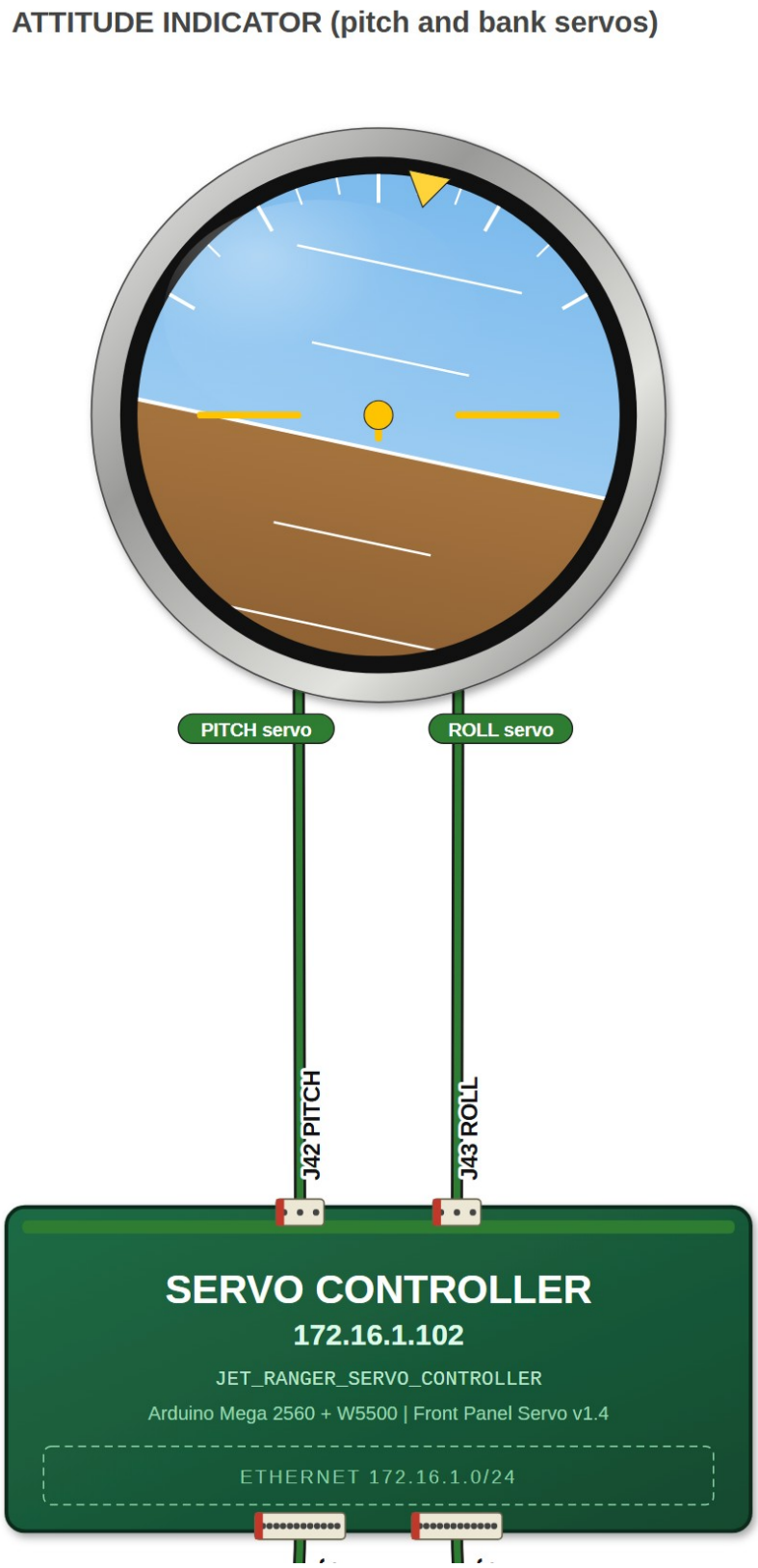


Figure 90 — Attitude indicator and the Servo controller

G.4 Huey Warning Panel, key, and the top of the Radio and ADF boards

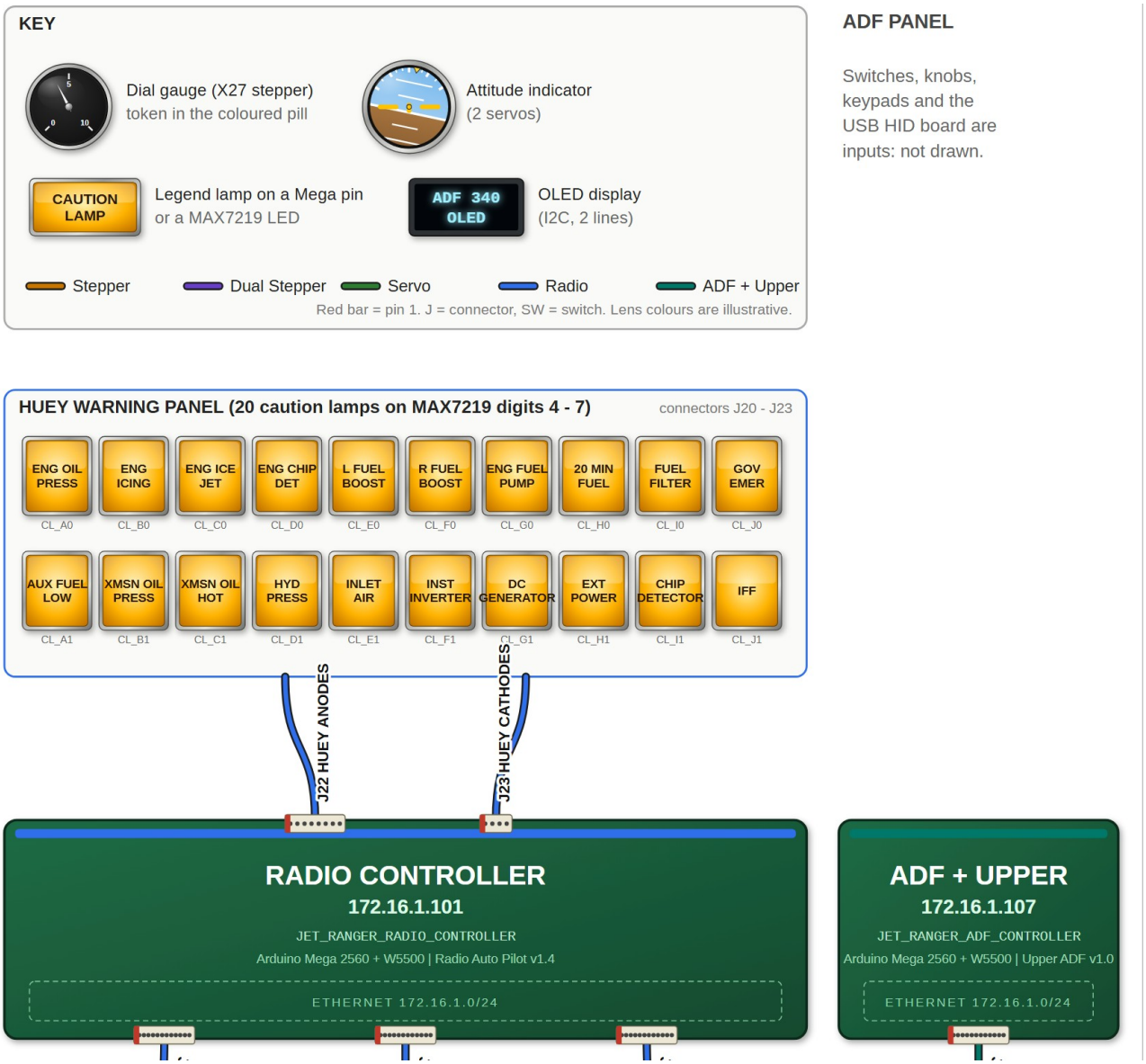


Figure 91 — Huey Warning Panel, key, and the top of the Radio and ADF boards

G.5 Dual Stepper displays and the Servo annunciator lamps

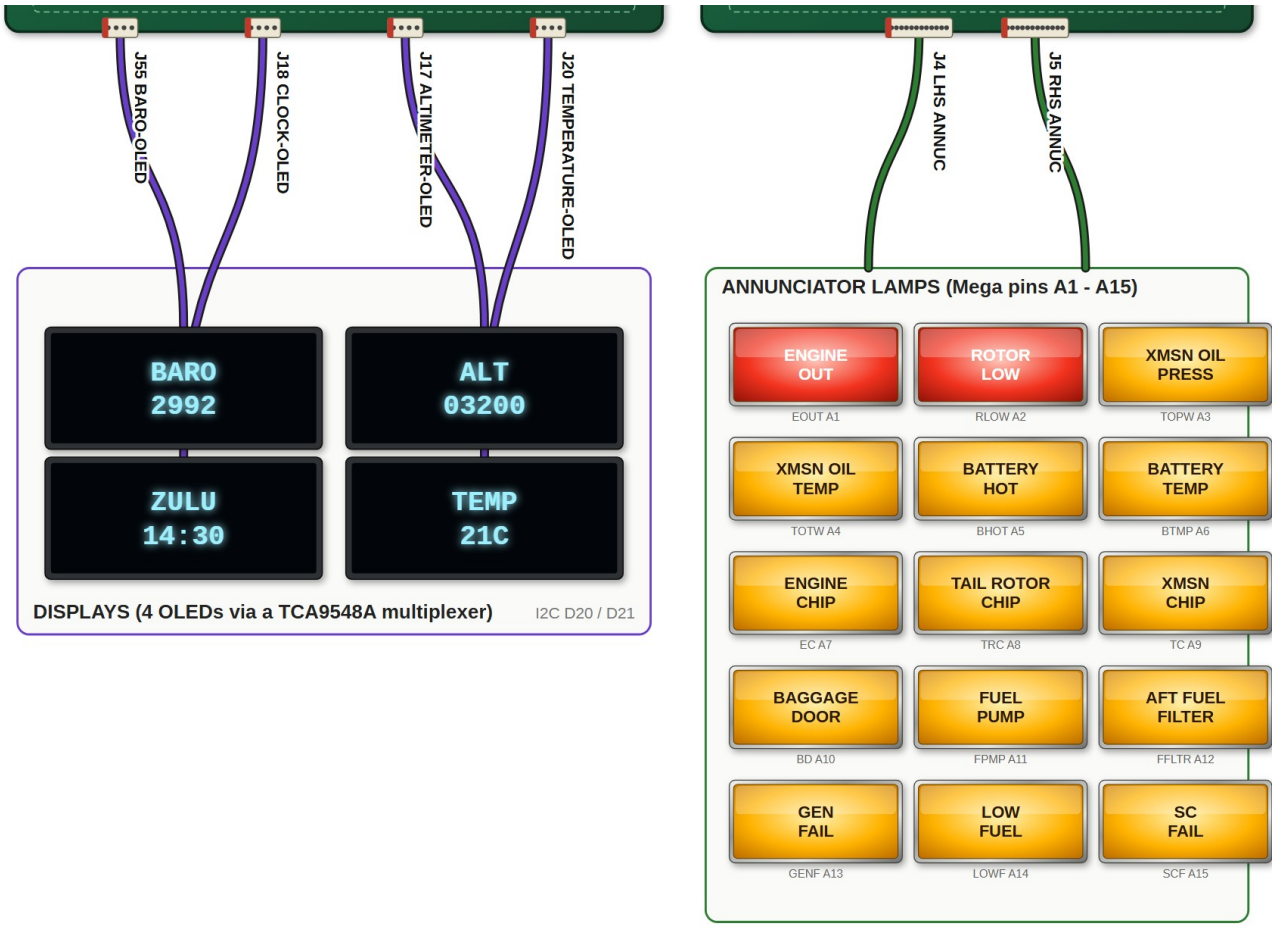


Figure 92 — Dual Stepper displays and the Servo annunciator lamps

G.6 Radio panels (COM and NAV, Auto Pilot, Audio) and the ADF panel

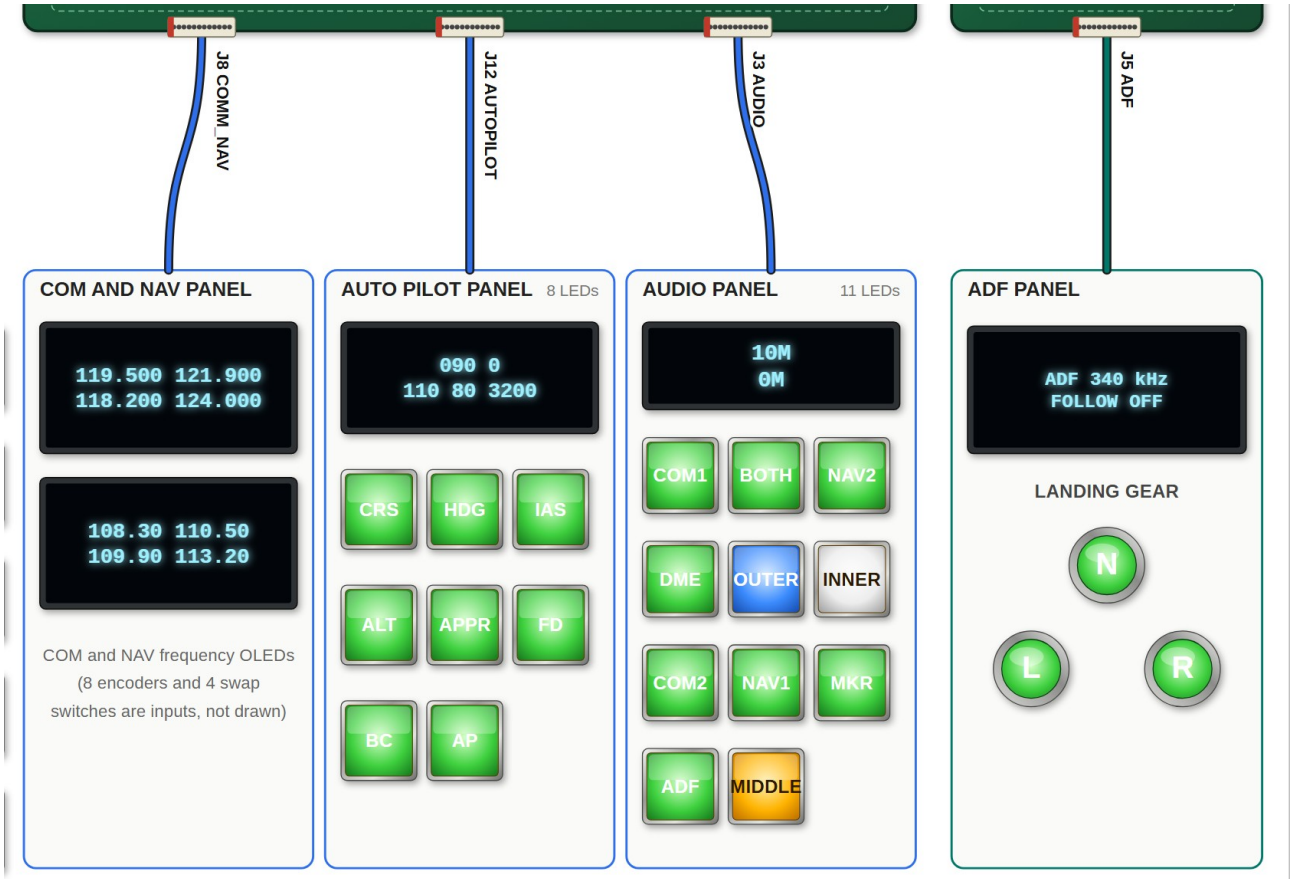


Figure 93 — Radio panels (COM and NAV, Auto Pilot, Audio) and the ADF panel

G.7 Gauges and the connectors they use

Instrument	Token	Connector on the board	Range
Airspeed	IAS	Stepper .105: J26 AIRSPEED	0 to 140 knots
Vertical speed	VSI	Stepper .105: J5 VVI	-1750 to +1750 ft/min
Radar altitude	AGL	Stepper .105: J23 RADAR_ALT	0 to 2500 ft
Torque	TQ	Stepper .105: J25 ENG_TORQUE	0 to 120 percent
Gas producer (N1)	N1	Stepper .105: J24 GAS_PRODUCER	0 to 103 percent
Turbine out temperature	ITT	Stepper .105: J27 EGT	100 to 900 C
Fuel quantity	FUEL	Stepper .105: J4 FUEL_LEVEL	0 to 75 US gallons
Engine oil pressure and temperature (dual instrument)	OILP, OILT	Dual Stepper .106: J8 ENGINE	0 to 150 psi, 0 to 150 C
Transmission oil pressure and temperature (dual instrument)	XMSNP, XMSNT	Dual Stepper .106: J6 XMSN	0 to 150 psi, 0 to 150 C
Rotor speed and engine speed N2 (dual instrument)	RPMR, RPME	Dual Stepper .106: J7 ROTOR ENG SPEED	0 to 110 percent
Electrical load and fuel load (dual instrument)	ELECTRICALLOAD, FUELLOAD	Dual Stepper .106: J10 ELEC FUEL LOAD	0 to 100 percent, 0 to 30 psi
Altimeter (and its zero sensor)	ALT	Dual Stepper .106: J33 ALT, J9 ALTIMETER_ZERO	one turn per 1000 ft
Attitude indicator pitch	PITCH	Servo .102: J42 PITCH	-30 to +30 degrees
Attitude indicator bank	BANK	Servo .102: J43 ROLL	-90 to +90 degrees

G.8 Lamps and displays and the connectors they use

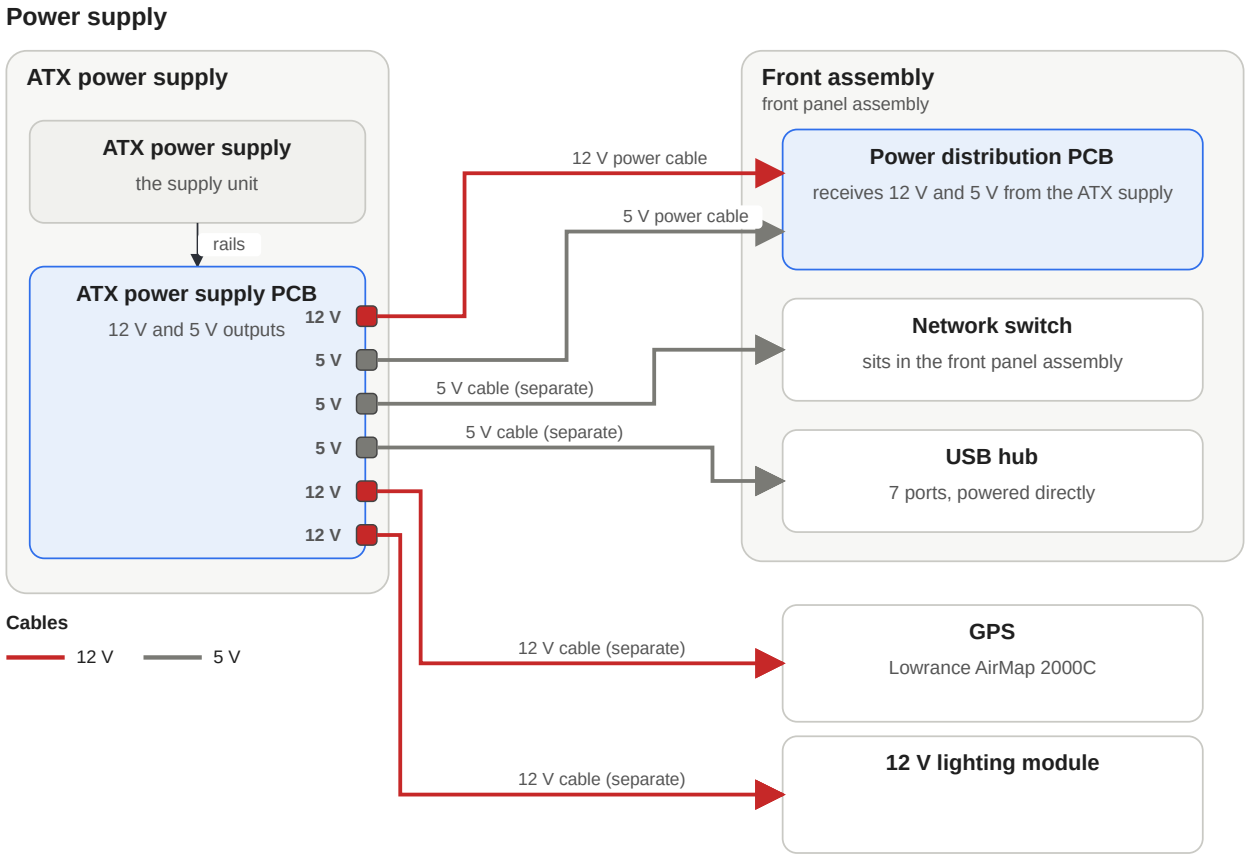
Lamps and displays	Tokens	Connector on the board
15 annunciator lamps (engine out, rotor low, transmission oil pressure and temperature, battery hot and temperature, engine chip, tail rotor chip, transmission chip, baggage door, fuel pump, aft fuel filter, generator fail, low fuel, SC fail)	EOUT, RLOW, TOPW, TOTW, BHOT, BTMP, EC, TRC, TC, BD, FPMP, FFLTR, GENF, LOWF, SCF on Mega pins A1 to A15	Servo .102: J4 LHS_ANNUC and J5 RHS_ANNUC (how the lamps divide between the two is not recorded)
COM and NAV frequency OLEDs	C1A, C1S, C2A, C2S	Radio .101: J8 COMM_NAV
Auto Pilot display (OLED) and 8 LEDs	APHDG, APCRS, APIAS, APVS, APALT; LED_CRS, LED_HDG, LED_IAS, LED_ALT, LED_APPR, LED_FD, LED_BC, LED_APENG	Radio .101: J12 AUTOPILOT
DME display (OLED) and 11 audio panel LEDs	LED_COM1, LED_BOTH, LED_NAV2, LED_DME, LED_OUTER, LED_INNER, LED_COM2, LED_NAV1, LED_MKR, LED_ADF, LED_MID	Radio .101: J3 AUDIO
20 Huey Warning Panel lamps	LED_H_* (MAX7219 digits 4 to 7)	Radio .101: J22 HUEY_ANODES and J23 HUEY_CATHODES (rev 1.4 layout; the panel is hand wired today)
4 Dual Stepper OLEDs (barometer, altimeter digits, clock, temperature)	ALT, ZULU, TEMP	Dual Stepper .106: J55 BARO-OLED, J17 ALTIMETER-OLED, J18 CLOCK-OLED, J20 TEMPERATURE-OLED
ADF display (OLED) and 3 landing gear lamps	LDGN, LDGL, LDGR (Mega pins D2, D3, D4)	ADF and Upper .107: J5 ADF

Which dual instrument uses which socket is taken from the names on the PCBs (J8 ENGINE, J6 XMSN, J7 ROTOR ENG SPEED, J10 ELEC FUEL LOAD); the Stepper's turbine out temperature gauge is shown on J27 EGT; and how the 15 annunciator lamps divide between J4 and J5 is not recorded, so the drawing joins the whole lamp panel to both.

Appendix H: Cockpit systems wiring

This appendix shows how the cockpit's power, USB, lighting, display and sound systems are wired; the diagrams are generated by `DocBuild/diagrams_e.py` (`power_supply`, `usb_wiring`, `ckas_platform`, `lighting`, `pc_displays` and `sound_system`). The controller boards and the network are in the chapters before the appendices. The diagrams show what is connected to what; they do not show cable lengths or the details of each piece of equipment.

H.1 Power supply



The ATX supply PCB feeds the power distribution PCB with its own 12 V and 5 V cables.
The GPS, the 12 V lighting module, the network switch and the USB hub each have a separate cable of their own.
The USB hub is powered directly, not through the power distribution PCB. Each cable is one line; ground returns are not drawn.

Figure 94 — Power supply

An **ATX power supply** with its own **ATX power supply PCB** powers the cockpit. The PCB is connected to the **power distribution PCB** in the front assembly by two cables, one for 12 V and one for 5 V. Four loads have a separate cable of their own from the ATX supply PCB:

- a separate **12 V cable to the GPS**;
- a separate **5 V cable to the network switch**, which sits in the front panel assembly;
- a separate **5 V cable to the USB hub**, which is also in the front assembly: the hub is powered directly and not through the power distribution PCB;
- a separate **12 V cable to the 12 V lighting module**.

In the diagram 12 V cables are red and 5 V cables are gray. Each cable is drawn as one line; the ground returns are not drawn.

H.2 USB wiring

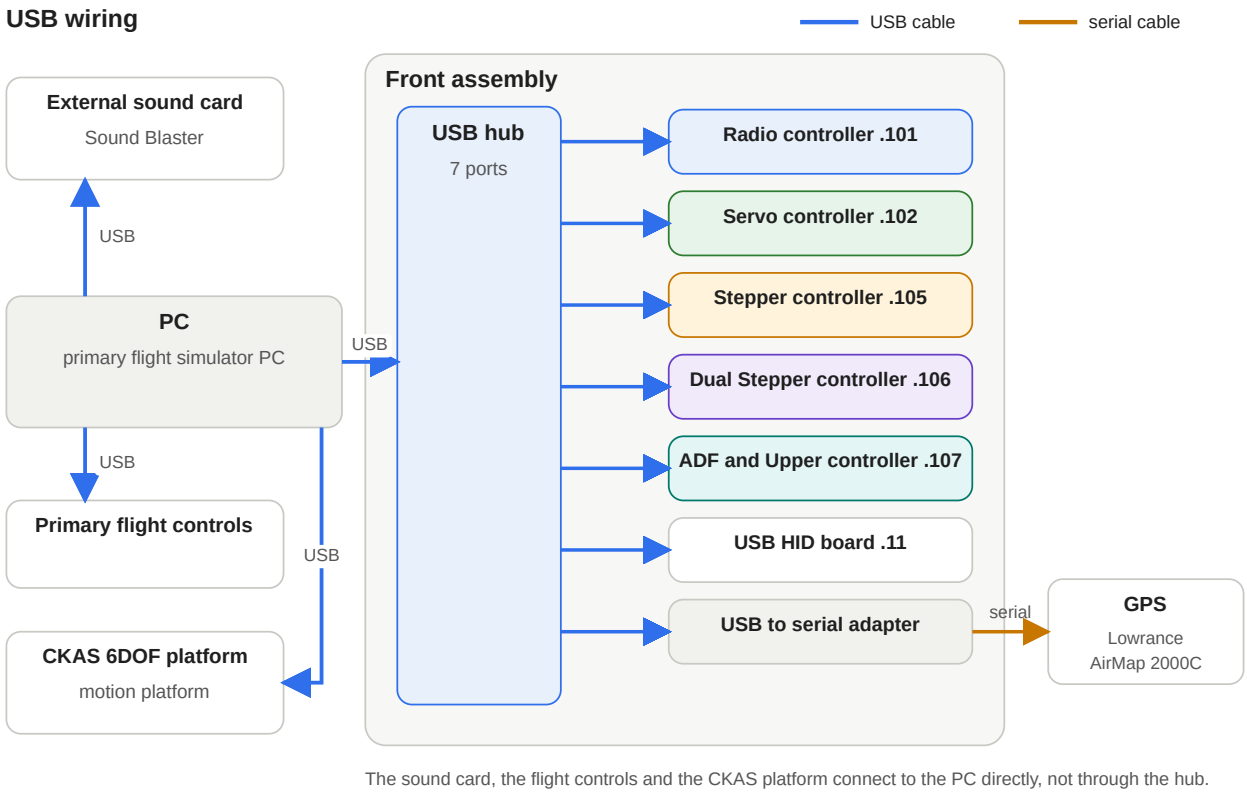


Figure 95 — USB wiring

A **7 port USB hub** in the front assembly (powered directly from the ATX power supply PCB, see Power supply) is connected to the PC. The hub has a connection to each controller PCB (Radio, Servo, Stepper, Dual Stepper, and ADF and Upper), to the USB HID board, and to a **USB to serial adapter**, which connects to the GPS by a serial cable.

Three things connect to the PC by their own USB cables and not through the hub: the external **Sound Blaster** sound card, the **primary flight controls** and the **CKAS 6DOF platform**.

H.3 CKAS 6DOF platform

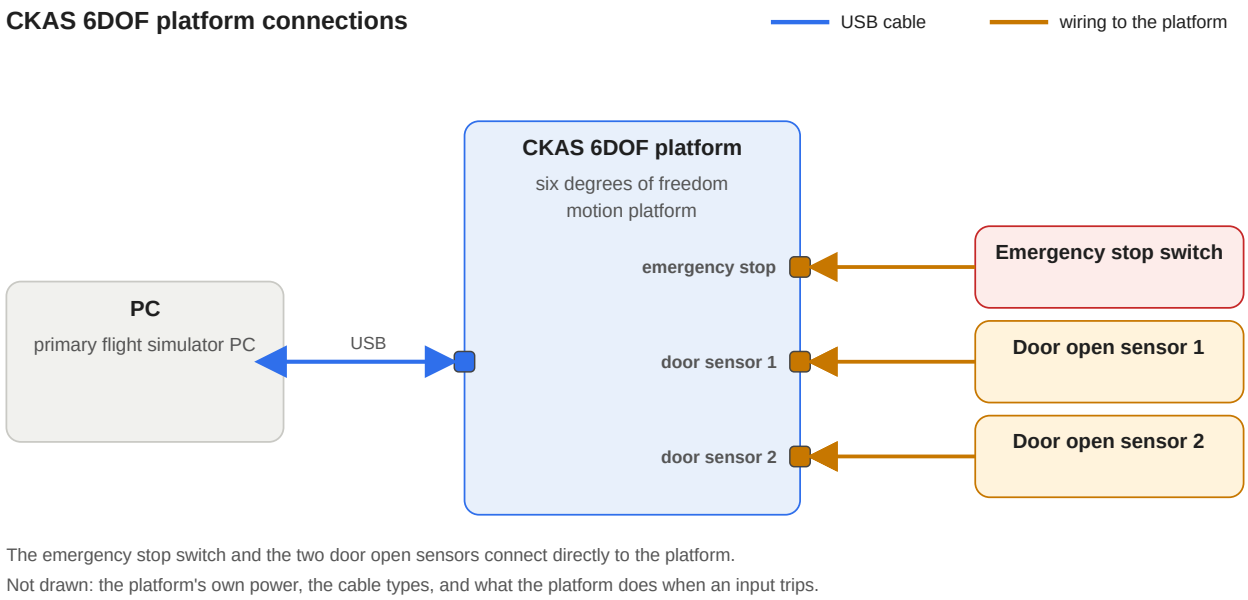
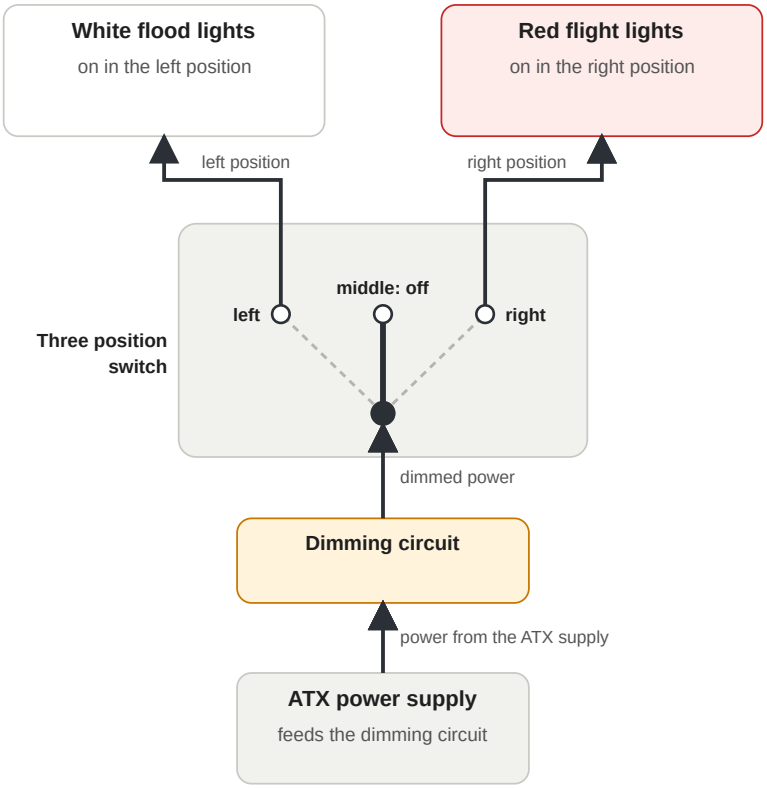


Figure 96 — CKAS 6DOF platform

The **CKAS 6DOF platform** is connected to the PC by USB. An **emergency stop switch** and two **door open sensors** (door sensor 1 and door sensor 2) are wired to the platform. The diagram does not show the platform's own power, the cable types, or what the platform does when one of these inputs trips.

H.4 Lighting

Lighting



The switch is drawn in the middle (off) position. The supply voltage, the dimming circuit's design and the wiring of the lights are not drawn.

Figure 97 — Lighting

The ATX power supply feeds a **dimming circuit**, and the dimming circuit feeds a **three position switch**:

Switch position	Result
Left	The **white flood lights** are on
Middle	Off
Right	The **red flight lights** are on

Because the dimming circuit is between the supply and the switch, it dims whichever lights are switched on. The diagram does not show the supply voltage, the design of the dimming circuit or the wiring of the lights.

H.5 PC displays

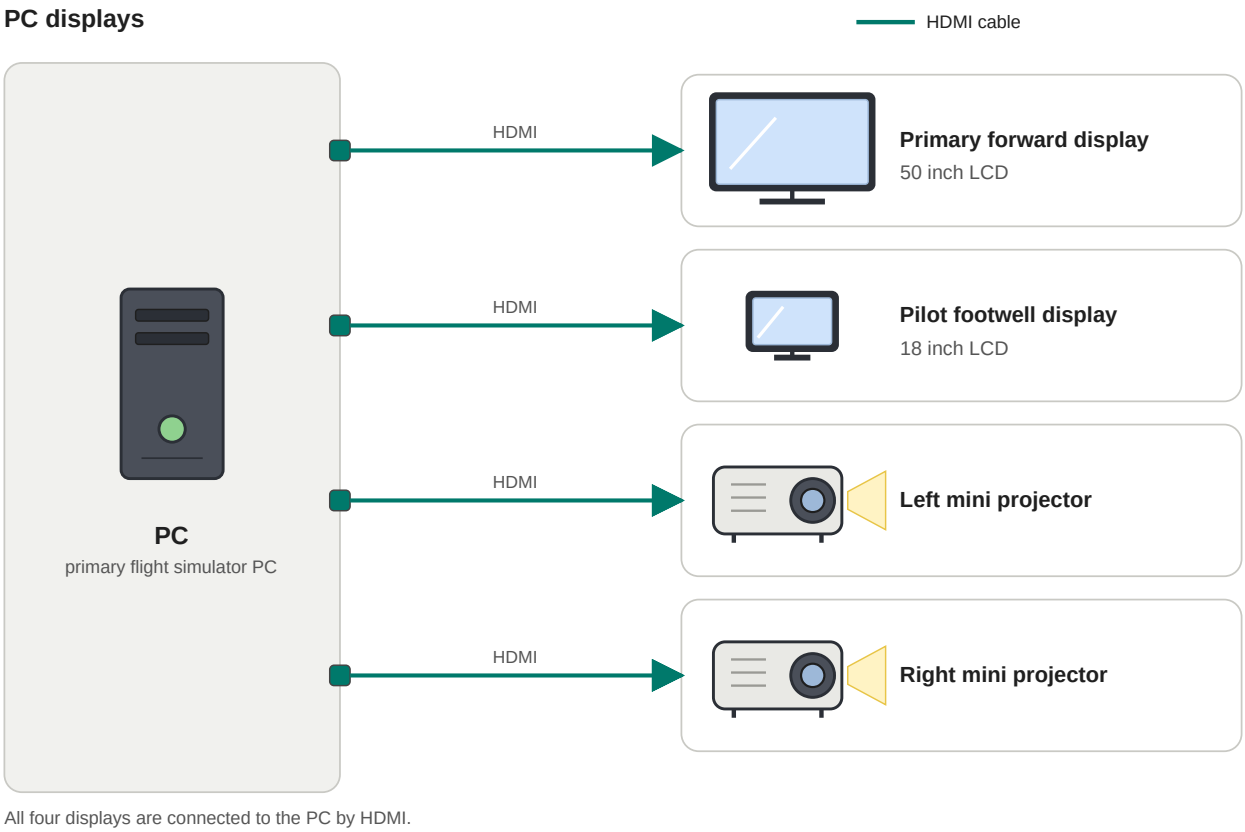
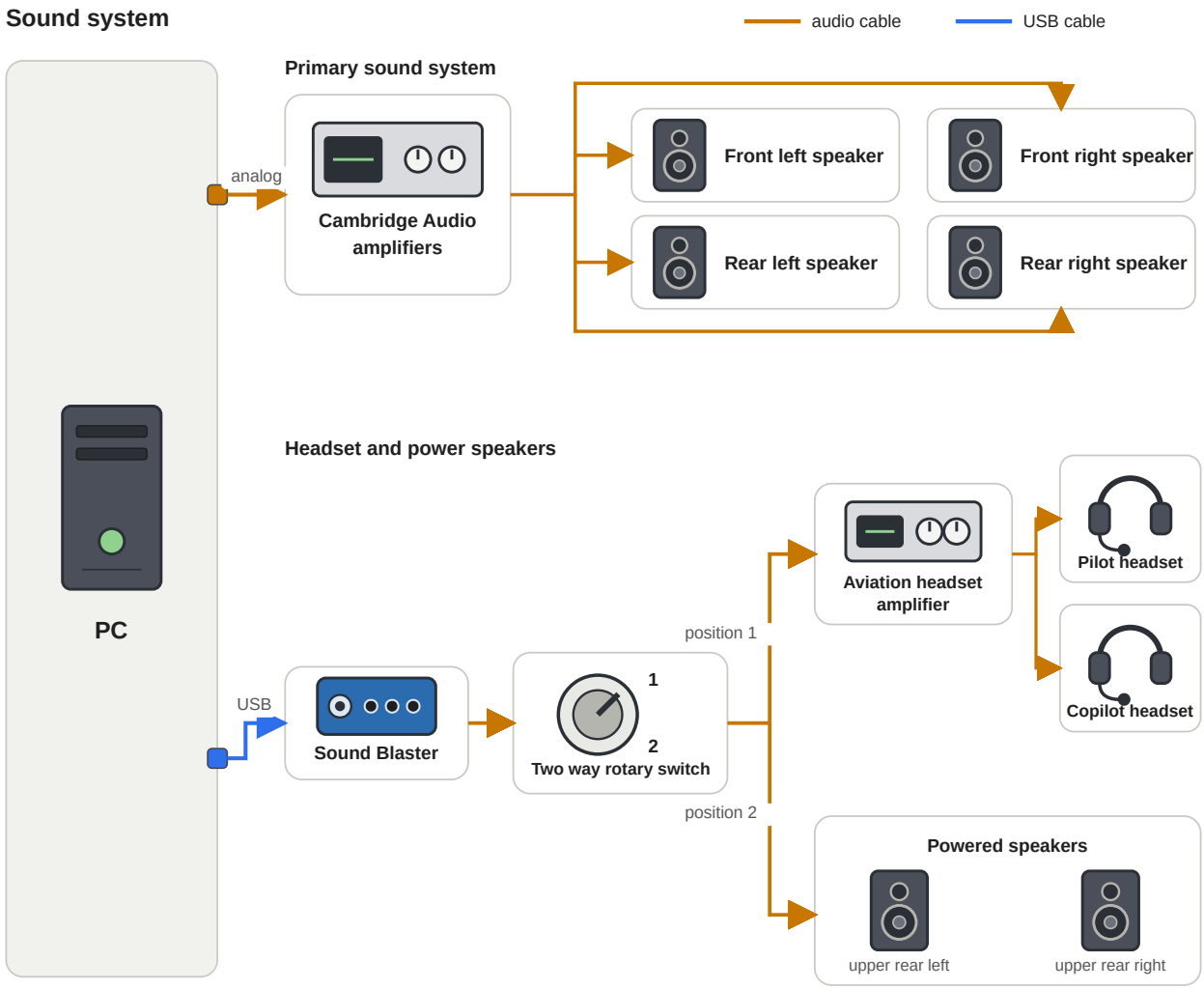


Figure 98 — PC displays

Four displays are connected to the PC by HDMI: the **primary forward display**, a 50 inch LCD; the **pilot footwell display**, an 18 inch LCD; and a **left mini projector** and a **right mini projector**.

H.6 Sound system



The sound card feeds either the headset amplifier or the powered speakers, never both at once.

Figure 99 — Sound system

There are two sound systems.

- **Primary sound system.** The Cambridge Audio amplifiers are connected to the PC by analog connections and drive four speakers: front left, front right, rear left and rear right.
- **Headset and powered speakers.** The USB-connected Sound Blaster feeds a **two way rotary switch**. In position 1 the sound goes to an **aviation headset amplifier**, to which the pilot headset and the copilot headset connect. In position 2 it goes to a set of **powered speakers** at the upper rear left and right of the pit. The sound card feeds one of the two at a time.